



SPRING

COURSE MATERIAL



NARESH

TM



Opp. Satyam Theatre, Ameerpet, Hyderabad - 16
Ph: 040-23746666, 23734842 Cell: 9000994007 / 08

technologies

B.O. Kaba Plaza, 3rd Floor, LB Road, Adyar, Chennai - 20. Phone : 044-6555 5557, 6555 5558



info@nareshit.com



www.nareshit.com



nareshit



nareshitech

SPRING INDEX

Fundamental -----	1-12
• Framework • Application Framework • Invasive Framework • Non Invasive Framework • Spring Modules and Definitions • Dependency Lookup • Dependency injection • POJO and POJI Model Programming • JavaBean	
Spring Core Module -----	13-123
• Dependency Injection • Constructor Injection • Setter Injection • Collaboration and tight coupling, loose coupling • Strategy pattern • Cyclic dependency injection • Difference between setter injection and constructor injection • Inner Bean • Collection Classes Injection • Bean inheritance • Collection Merging • Dependency lookup by using IOC container • FactoryMethod • Bean Scopes • Bean Wiring • Autowiring • Dependency -Check • Nested BeanFactory • P-NameSpace and C-NameSpace • ApplicationContext container • difference between BeanFactory and ApplicationContext container • Bean Life Cycle • <i>Bean lifecycle Annotation Approach</i> • Aware injection • FactoryBean • Method Replacement or Method Injection • Lookup method injection • BeanPostProcessor • BeanFactoryPostProcessors • Spring Core Module with Annotations	
Spring JDBC/DAO -----	124-176
• Introduction • Plain JDBC • Spring JDBC • Types Spring Jdbc Classes • Using JdbcTemplate • NamedParameterJdbcTemplate • Working with SimpleJdbcInsert, SimpleJdbcCall • Mapping SQL Operations as Subclasses (MappingSqlQuery,SqlUpdate)	
Spring AOP -----	177-221
• Introduction • Need of AOP • Aspect • JoinPoint • PointCut	

• Advice • Target • Weaving • Dynamic Proxy • Spring AOP XML based configuration • Around Advice • Before Advice • After Returning Advice • Throws Advice • Programatic Spring AOP • AspectJ AOP with Spring XML based configurations • AspectJ AOP with Spring Annotation based configurations	
Transaction Management	222-242
• Introduction • Transaction Models • Spring Transaction • Declarative Transaction management using spring AOP • Transactional properties	
Spring MVC	243-384
• Introduction • Difference between MVC1 and MVC2 • Spring MVC features • Controller Classes hierarchy • Controller • AbstractController • AbstractCommandController • SimpleFormController • Form validation with SimpleFormController • MultiActionController • AbstractWizardFormController • HandlerMapping • BeanNameUrlHanlderMapping • SimpleUrlHandlerMapping • ControllerClassNameHandlerMapping • DefaultAnnotationHandlerMapping • ViewResolver • UriBeanViewResolver • UriBasedViewReslover • ResolverBundleViewResolver • XmlViewResolver • BeanNameViewResolver • Annotation based Spring MVC • File Uploading and File downloading • Spring MVC with Tiles • Handling exceptions • Spring Security	
Spring ORM	385-397
• Introduction • Spring ORM Advantages • Spring with Hibernate Integration • HQL queries with named parameters • HQL query with positional parameters	
Spring JEE/Context Module	398
SPRING FAQs	399-427

Spring

Fundamental

-> C,C++, Java are programming languages.

-> JDBC, Servlet, EJB,JSP,JMS,java mail,jta,jpa and etc are java technologies.

-> Struts, SPRINGFRAMEWORK, Spring, Hibernate and etc are Java Frameworks.

Programming Language:

It is directly installed software having features to develop software applications.

It defines syntaxes (Rules), semantics (Structures) of programming.

These are base to create software technologies, frameworks, tools, DB software, operating systems and etc.

These are raw materials of software programming.

Eg: C,C++,C#,Java.

Software Technology:

It is software specification having set of rules and guidelines to develop software by using programming language support. Software technology is not installable but software created based on software technology is installable. Working with this software is nothing but working with software technologies. In Java environment, technology rules are nothing but interfaces and guidelines are nothing but classes. Abstract class represent both rules and guidelines.

Eg: JDBC,JSP, Servlet and etc....

JDBC technology gives rules and guidelines to develop driver using java language. Working with JDBC Driver is nothing but working with Jdbc technologies.

There are 2 types of software technologies.

1. Open Technologies
2. Proprietary Technologies.

Open Technologies:

Here the rules and guidelines of technology are open to all software vendor companies to develop software.

EX: All Java Technologies (JDBC, Servlet, JSP, etc..)

Proprietary Technologies.

Here the vendor company who has given technology is only allowed to develop software based on technologies.

EX: All Microsoft Technologies (Like ASP.NET)

Frameworks:

Framework is installable software that uses existing technologies internally to simplify the application deployment having ability to generate common logic of application internally.

Framework is installable software that provides abstraction layer on existing technologies to simplify the application development process.

EX: : Struts, Spring, Hibernate,...

While working with technologies the programmer has to develop both common logic and specific logics of application.

While working with frameworks the programmer has to develop only application specific logics because the common logics will be generated by the framework dynamically.

Every framework uses technology internally but it never make program to know about it. This is nothing but providing abstraction layer (hiding implementations/internals)

Main JDBC App:

Register JDBC Driver (load Driver class)
Establish the Connection
Create Statement Object
Send and execute SQL query in DB software
Fetch results and process results.
Close Jdbc Objects

Spring jdbc App:

Get JdbcTemplate Object.
Send and execute SQLQuery in DB software
Fetch results and process results.

While working with technology we need to write some common logics in every applications of that technology. This is called **boiler plate code** problem.

While working with frameworks we just need to develop application specific logics that means it solves **boiler plate code** problem.

Based on kinds of application and logics we develop, there are 3 types of java Frameworks.

Web Application Framework:

Provides Abstraction layer in servlets, jsp technology
Simplifies MVC Architecture based web application development
Springframework->From Sun MS (Oracle)
Struts-> From Apache
Webwork->from open symphony.
DF->from Oracle
SpringMvc->From interface

ORM Framework:

Provides Abstraction layer on Jdbc technology.
Simplifies Object relational mapping persistence logic development.
Allows to portability/DB independent persistence logic.
Hibernate -> from Softtraa (RedHat)
JPLink -> Oracle Cooperation
JB-> Apache
Jatis->from Apache
JDO->from adobe

Application Framework:

Provides abstraction layer on multiple technologies like jdbc, servlets, jsp, jndi, jms, jta, jaas, ESB, rmi and etc..
Allows to develop all kinds of logic like business logic, persistence logic, presentation logic, integration logic and etc..
Allows to develop all kinds of applications like standalone applications, web applications distributed applications and etc..
Spring-> from interfaces
Based on mode of development they allow, there are 2 kinds of frameworks.

Declarative Framework:

Where the classes of application should extend from framework api class or should implement framework api interface.

This process makes api classes as tightly bounded classes of frameworks.

We cannot change framework of the application/ project by changing the libraries.

Non Invasive Framework:

Here the class of application need not to extend from framework class or need not to implement framework application interfaces. So our application classes are not tightly bound with framework. We can change framework of application/ project by changing the libraries (jar files)

EX: Struts 2.x, Spring, Hibernate, and etc..

Applets-> For Gaming (1995)

JavaBean-> for B.logic/service logic development (1996)

(Disadvantage: Can't apply services, can't be accessed from remote clients)

Security, transactions, logging. Connection pooling and etc are called services/ middleware services.

EJB -> for Business logic and service logics development (1998)

Advantages:

Allows both local and remote clients

Allows to apply middleware services.

Disadvantages:

Heavy weight and complex.

EJB is suitable for banking projects.

Spring (2003)

Initially for b.logic development with ordinary classes allowing to apply middleware services

Now spring can be used to develop all kinds of logics and applications in light weight environment.

Before Frameworks:

Programmer directly uses technologies to develop both common and specific logics of application.

With Frameworks:

Programmers uses frameworks to develop only specific logics because the common logics will generated by framework internally by using technologies support.

Is the spring replacement for struts?

Ans: Using struts we can develop only web application. There is no provision in struts to develop and use middleware services / aspects (security, logging and etc..)

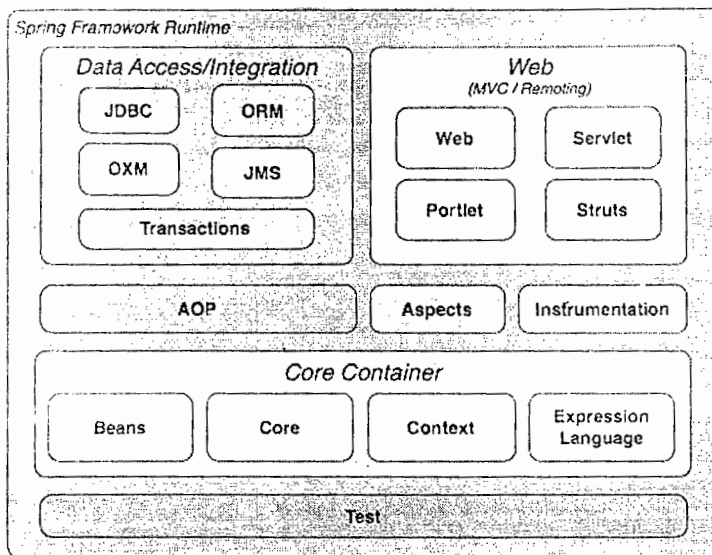
Using spring we can develop all kinds of applications and all kinds of logics including web applications. It also provides environment to develop aspects/middleware services.

Spring is more powerful than struts

Is the Spring replacement for JEE/JSE technologies?

Ans :No

Spring complement JEE/JSE technologies by using then internally in the application development process.

Spring 1.X Modules (7 Modules) :

Spring 2.x web module : Spring 1.x web + spring 1.x web mvc module.

Spring 2.x JEE modules : Spring 1.x context module.5

Spring Core:

This module is base for other modules, provides containers that required to manage various spring comps/resources.

Useful to develop standalone applications, Designed supporting dependency injection (injecting values to Objects dynamically)

Spring DAO/JDBC:

Provides abstraction layer on plain jdbc to simplify the jdbc style persistence logic. and **Exception Re-Throwing** concept is used to designed DAO Exception herirachy

Spring ORM:

Provides abstraction layer on multiple ORM framework (like Hibernate) and simplifies O-R mapping persistence logic.

Spring Web:

Provides plug-in to make spring applications communicatable from other framework applications like Struts, JSRINGFRAMEWORK and etc ...applications.

Spring Web MVC:

Spring it's own web framework to develop MVC architecture based web applications with plan classes by using orbiter methods of the classes.

Spring Context:

Provides abstraction layer on multiple JSE, JEE module technologies to simplify the development of the comman logics.

Mailing applications

Distributes applications

Messaging applications

Scheduling applications

and

Spring AOP:

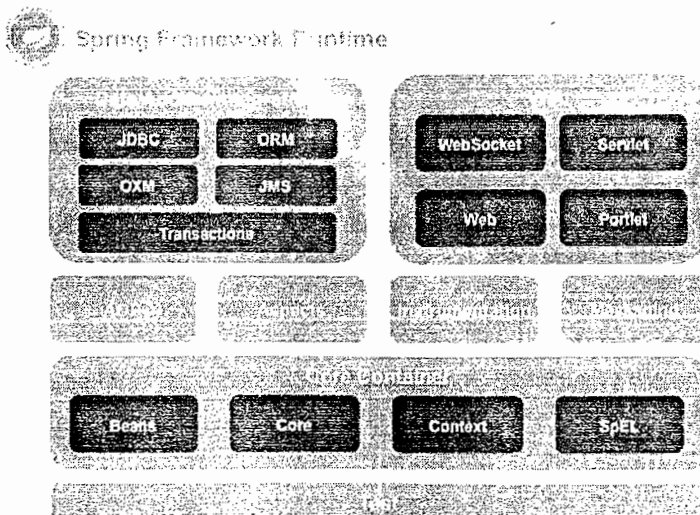
Provides a new methodology of programming to develop aspect/ middleware services logics and to apply then spring applications.

Before AOP:

```
class Bank {  
    public void withdraw (-,-){  
        Security logic  
        Logging logic  
        Transaction logic  
        withdraw logic (bal=bal-amt)  
    }  
    public void deposit(){  
        Security logic  
        Logging logic  
        Transaction logic  
        Deposite logic (bal=bal+amt)  
    }  
}
```

With AOP:

```
class Bank{  
    public void withdraw (-,-){  
        Withdraw logic(bal=bal-amt);  
    }  
    public void deposit(){  
        Deposit logic (bal=bal+amt);  
    }  
}
```

Spring 3.x Overview Diagram:

Spring 3.x is having 20 modules that are grouped into core container, AOP, instrumentation, Text, DataAccess/integration, MVC/ remoting groups.

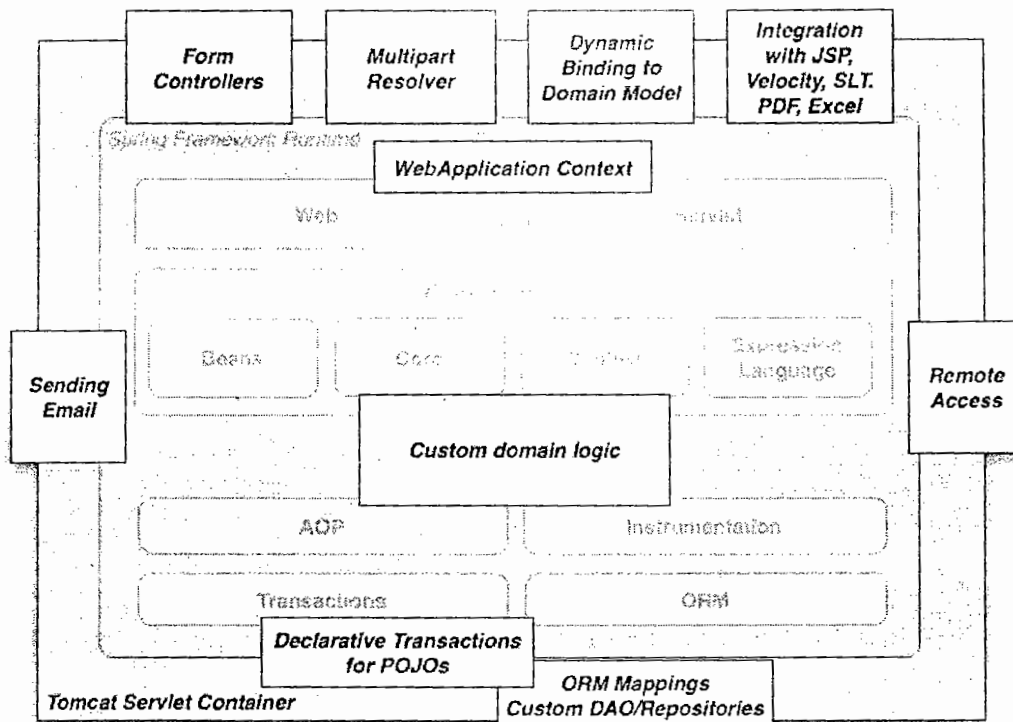
Test module allows to perform unit testing (programmers testing on his own piece of code) junit does not allow to mock object (dummy objs), where spring 'Test' module allows us to work mock object.

OXM (Object xml mapping) is given to convert java objs to xml tags and vice-versa (alternate to JAXB)

Spring 2.x JEE services they placed in DataAccess/integration, MVC/remoting groups.

Portlet supports allows us to develop "portal" applications multiple windows (web pages) in a webpage.

Spring 4.x override diagram:



Spring 3.x is compatible with jdk 1.5

Spring 4.x is compatible with jdk1.6 and support all jdk1.8 features. In this version all deprecated classes, methods are removed

In spring 4.x messaging a sub framework in spring that given given alternate to jms to get messages based asynchronous common bet components.

Asynchronous communication allows the client to given next request or to client side operations without given request related response from server.

Web socket is a web label protocol that is designed on the top of http protocol for full 2-way communication.

Industry uses "MVC" as defact architecture to develop java based medium/large scale web applications.

M-> Model ->Data + Business Logic+ persistence logic (like account officer)

V->View ->presentation logic (like Beautician)

C->Controller ->Integration logic (like Traffic police/supervisor)

Note:

Integration logic controls and monitors all the activities of application execution. It is responsible to get communication between view layer and model layer components.

Jsp->servlets-> java class/java Bean->DB software (1990-1998)
Jsp->servlets-> EJB Session Bean->java Bean->DB software (1990-1998)
Jsp->servlets-> EJB Session Bean->DAO Class (jdbc)->DB software.
Struts->EJB Session Bean->DAP Class (jdbc)->DB Software.

DAO: Data Access Object:

The java class that separates persistence logic from another logics and makes the persistence logic as reusable logic and flexible logic to modify is called DAO.

Jdbc is light weight but does not allows to develop Object based persistence logic. EJB entity Beans heavy weight but allows to develop Object based persistence logic. To overcome both problems we can use "Hibernate"

Struts->EJB session Bean-> hibernate->DB software

Struts->Spring-Hibernate->DB Software.

Struts->WebServices->Hibernate->DB Software.

Struts->Spring JEE/Bean->Spring ORM+AOP/DAO->DB Software.

Struts->Spring JEE/Bean->Spring ORM/DAO+AOP->DB Software.

Struts->Spring WebServices/Web services->Spring ORM+AOP/DAO->DB Software.

Spring:

|----->**Type:** Java application framework

|-->**Version:**4.x (Compactable with jdk 1.6+)
3.x (Compactable with jdk 1.5+)

|-->**vendor :** interface 21

|-->**Creator:** Mr.Rod Jonshon

|-->Springsoftware comes as set of jars

-->**website:** <http://spring.io>

-->**To downloadsoftware :** Download as zip file from spring.io

Spring-framework-4.1.6.release-di.zip

Spring-framework-3.2.Release-with-docs.zip.

| -> **To installsoftware:** Extract zip file.

Spring 4.1.6/3.2.6 software installation gives

libs->having jarfiles.

Docs-->apidocs, reference docs (PDF)

Schemas-->xsd file having schema rules.

Every DTD and XSD file contains rules to construct xml documents.

DTD-->document type definition

XSD-->xml schema definition.

Definition:

Spring is an opensource, light weight, loosely coupled aspect oriented, dependency injection based java application framework to develop various java applications.

Along with spring software installation we get its source code. This makes spring as open source.

Open source software are not only free software. They also expose their source code along with installation.

Spring is lightweight because

Spring framework comes as zip file having less size

Spring gives 2 lightweight containers which can activated anywhere in the application.

To work with spring container we donot need any application server/webserver software.

We can use specific modules of spring without having any link with other modules.

Eg: We can use spring core module alone or spring core and spring DAO module together without an link with other module.

If degree of dependency is less between modules, then they are called loosely coupled components.

Eg: Tv and remote.

If degree of dependency is more between modules then they are called tightly coupled components.

Eg: Keyboard and system.

In spring framework core module is base module for all modules i.e we should use every module along with core module. But remaining modules of spring can be used alone without having any dependency with other modules. The degree of dependency is very less between other modules of spring.

This makes spring as loosely coupled framework.

AOP is a new methodology of programming that compliments OOP to separate secondary middleware services from primary business logic and allows to link them at runtime dynamically through some configuration. This gives advantages of reusing middleware service logic and flexibility of enabling or disabling middleware services logic on primary logis (like security, transaction and etc....)

Dependency Lookup:

If the resource/ class/ Object is searching and getting is dependent values from other resources then it is called dependency lookup. Here the resource pulls the dependent values.

The way get Objects from Jndi registry is called "Dependency lookup"

The way we get Jdbc connection Object from jdbc Connection pool is called "Dependency Lookup"

If student is getting course material only after putting request for it is called dependency lookup.

Dependency injection (Inversion of control/IOC):

If underlying server/container/ framework/runtime (like JRE) assigns values to resources/class/objs dynamically at run time then it is called **dependency injection**.

Here dependency values will be pushed to the resource.

Eg: JVM calling constructor automatically to assign initial values of object when object is created.

Servlet container injects ServletConfig obj to our servlet class Object by calling init(ServletConfig config) method when servlet (Container creates our Servlet class obj)

Student gets course material the moment he register for course.

What is pojo class, POJI, Bean/Comonent class, Spring Bean?

POJO class (Plain old java Object class)

The java class without any specialities i.e it is an ordinary java class.

While developing this class we need not to follow any serious rule.

The java class that is not extending from technology or framework api class and not implementing technology/ framework api interfaces is called pojo class.

Eg:

```
class Demo
```

```
{.....
```

```
.....
```

```
}
```

It is pojo class

```
class Test implements Serializable{
```

```
.....
```

```
.....
```

```
}
```

Pojo class

Eg :

```
class Test extends Demo
```

```
{
```

```
.....
```

```
.....
```

```
}
```

```
class Demo{
```

```
.....
```

```
.....
```

```
}
```

Eg:

```
class Test extends Thread{
```

```
.....
```

```
.....
```

```
}
```

"Test" -> pojo class

Eg :

```
class Test extends HttpServlet{
```

```
.....
```

```
.....
```

```
}
```

"Test" is non pojo class.

Eg:

```
class Test implements java.sql.Connection{
```

```
.....
```

```
.....
```

```
}
```

"Test" is non pojo

Eg:

```
class Test {
```

```
public void m1(Statement st){
```

```
.....
```

```
.....
```

```
}
```

```
}
```

"Test" is pojo class.

Eg:

```
class Test extends Demo{
```

```
.....
```

```
.....
```

```
}
```

```
class Demo implements java.rmi.Remote{
```

```
.....  
.....  
}
```

"Demo", "Test" are not pojo class

Eg:

Entity

class Test{

```
.....  
.....  
}
```

Test is a pojo class.

No need to include annotation and internal methods of tech or framework.

Don't consider annotations and api's used inside the methods of a class to check whether the class is pojo class or not.

POJI (Plain Old Java Interface)

It indicates an interface without specialities i.e. an ordinary interface.

The interface that is not extending from technology/ framework api interface is called poji.

Eg:

```
interface Test{  
.....  
.....  
}
```

Eg:

```
interface Test extends Serializable{  
.....  
.....  
}
```

Test is poji

Eg:

```
interface Test extends java.rmi.Remote{  
.....  
.....  
}
```

"Test" is not poji

Eg:

```
interface Test extends Demo  
{  
.....  
.....  
}
```

```
interface Demo {  
.....  
.....
```

```
}
```

Test, Demo are pojis

The frameworks that support pojo/ poji model programming are called non invasive frameworks or light weight frameworks.

Eg:

Hibernate, Spring, Struts 2.x etc

Java Bean:

It is a java class that is developed with some standards

Must be public class and should not be final or abstract class.

Can be implement Serializable, if needed.

Must be direct or indirect 0-parameter constructor.

Member variables are called bean properties and they must be taken as private.

Every bean property should have setter and getter methods (public)

Eg:

```
public class StudentBean{  
    //bean properties  
    private int st;  
    private String name;  
    //setter and getter  
    .....  
    .....  
}
```

In real time javabean is used as helper class to represent data with multiple values in the form of object and to send those objects from one layer to another layer.

JavaBean object is acceptable format to send and receive data across the multiple layers irrespective of technologies or that are used in the layer to develop the logics.

The JavaBean class whose object holds i/p value given by client/enduser is called VO class.

The JavaBean class whose object holds data needed for persistence operation (DAO Class) is called as (Business class) BO class.

BO class: Business Object Class

VO class: Value Object class.

DTO class: DataTranspringframeworkerObject class.

Bean Class/Component Class:

It is a java class that contains member variables and methods having b.logic manipulating the data of member variables.

Eg:

```
class CardProcessor{  
    private int cardNo;  
    public int processcard (Int cardNo){  
    .....  
    ..... //b.logic  
    .....  
    }  
    public boolean blockcard (int cardNo)  
    {
```

```
.....  
..... //b.logic  
.....  
}  
}
```

The Bean/component class can extend or implement any class or interface.

The java class whose object can be managed by spring container is called springBean.

SpringBean can be pojo class, java bean class.

Bean/component class, but its object must be created and managed by spring container.

Spring Bean can be user defined class/pre-defined class /third party class.

We can't take abstract class/interface as springBean because it can't create object for abstract class or interface.

To make java class as spring Bean it must be configure spring bean configuration file (xml) to make spring container to create and manage that spring Bean class object.

Important points:

Spring Bean can be pojo class? (Yes)

Every javaBean is pojo class? (Yes)

Every pojo class is java Bean? (No)

Every springBean is java Bean? (No)

Spring Bean can be a java Bean (Yes)

Spring Bean cannot implement spring api interface? (No)

Any component/bean class can be taken spring Bean? (Yes)

Spring gives 2 light weight containers which can be started in any application by just creating object for container classes.

Any java class or application that can manage the life cycle of given resource (class) is called container.

Spring containers are

BeanFactory (Basic container)

(performs springBean management and dependency injection)

ApplicationContext (Advanced Container)

(internally uses BeanFactory container so can perform advanced operations along with Bean management and dependency injection)

To activate/ start BeanFactory container we should create object of a class that implements "BeanFactory"

EX:

Beanfactory Factory= new XmlBeanFactory (resource obj)

Points to spring bean config file (xml file)

Spring containers (light weight) are not replacement/ alternate for servlets, jsp containers (heavy weight)

Spring Core Module

Base module for other spring module

Gives two IOC containers/spring containers.

BeanFactory

ApplicationContext.

When it is used alone we can develop standard applications.

When it is used along other technologies we can develop model layer b.logic by take in the advantage of dependency injection.

Talks about springBean life cycle management and dependency injection.

Dependency Injection:

If the underlying server or container dynamically assigns the dependent values to our resource (class/object) then it is called dependency injection. There are multiple ways of performing dependency injection

Setter injection Container calls setTransaction management x(-) to inject dependent values.

Constructor injection (container uses parameterized constructor to create bean class object and to inject value to that object)

interface injection (Aware injection)

Method injection

Lookup Method injection and etc....

Setter injection sample code:

//WishGenerator.java (POJO class)

package com.nt.beans;

import java.util.Date;

public class WishGenerator {

//bean properties

private String name;

private Date date;

public WishGenerator(){

System.out.println("WishGenerator:0-param constructor");

}

public WishGenerator(String name, Date date) {

System.out.println("WishGenerator:2-param constructor");

this.name = name;

this.date = date;

}

//setter methods for setter injection

public void setName(String name) {

System.out.println("WishGenerator:setName(-)");

this.name = name;

}

public void setDate(Date date) {

System.out.println("WishGenerator:setDate(-)");

this.date = date;

}

// B.method

```
public String generateWishMsg(){  
    return "Good Morning:"+ name+"--->" +date;  
}  
} //class
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"  
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
xsi:schemaLocation="http://www.springframework.org/schema/beans  
http://www.springframework.org/schema/beans/spring-beans-2.0.xsd">  
    <bean id="dt" class="java.util.Date"/>  
    <bean id="wsg" class="com.nt.beans.WishGenerator" >  
        <property name="name" value="raja"/>  
        <!-- <property name="date" ref="dt"/> -->  
    </bean>  
</beans>
```

Any <file name>.xml can be taken as spring config file. But it is recommended to take applicationContext.xml in standalone spring applications.

In springBean config file we configure classes as springBeans

Dependency injection configurs

Spring bean lifecycle Configuration and etc

We must supply springBean configuration file while activating spring container.

Every spring Bean is identified with its bean id. <property> (tag) is used to configure bean property for **setter injection**.

In the above configurs spring container creates "Date", WishGenerator" class objects using o-param constructor and setName(-) and setDate (-) methods to perform setter injection

Constructor injection Sample Code:

```
//WishGenerator.java  
package com.nt.beans;  
public class WishGenerator {  
    //bean properties  
    private String name;  
    private Date date;  
    //param constructor for constructor injection  
    public WishGenerator (String name, Date date){  
        this.name=name;  
        this.date=date;  
    } //B.method  
    .....  
    .....  
}
```

applicationContext.xml:

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.0.xsd">
<bean id="dt" class="java.util.Date"/>
<bean id="wsg" class="com.nt.beans.WishGenerator>
<constructor-arg value="Raja"/>
<constructor-arg value="dt"/>
</beans>

```

Based on above configuration the spring container uses 0-param constructor to create Date class object and uses 2-param constructor to create wishGenerator class object. (because the <constructor-arg> tag is placed 2 times in the xml file). In this process injection also will be completed.

Procedure to develop spring core module app that performs setter injection:

Step 1: keep eclipse IDE ready

Step 2: Design the application

▲ IOC proj1 (Setter injection)

▲ src

▲ com.nt.beans

▷ WishGenerator.java

▲ com.nt.cfgs

▷ applicationContext.xml

▲ com.nt.test

▷ ClientApp.java

▷ JRE System Library [JavaSE-1.7]

▲ IOCLib

▷ spring-beans-4.1.6.RELEASE.jar - C:\Users\del

▷ spring-context-4.1.6.RELEASE.jar - C:\Users\del

▷ spring-context-support-4.1.6.RELEASE.jar - C:\

▷ spring-core-4.1.6.RELEASE.jar - C:\Users\del

✓ spring-expression-4.1.6.RELEASE.jar - C:\User

▷ commons-logging.jar - C:\Users\del\h\Desktop

Step3: Develop the above resources

(Workspace : N+Sp27)

//wishGenerator.java (pojo class)

package com.nt.beans;

import java.util.Date;

public class WishGenerator{

//bean properties

private String name;

private Date date;

//Getter methods for setter injection

public void setName(String name){

this.name=name;

}

public void setDate(Date date){

```
this.date=date;
}
//B.logic
public String generatorWishMsg (){
return "GoodMorning" +name+ "--->" +date;
}}
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.0.xsd">
<bean id="dt" class="java.util.Date"/>
<bean id="wsg" class="com.nt.beans.WishGenerator">
<property name="name" value="Raja"/>
<property name="date" ref="dt"/>
</bean>
</beans>
```

Note:

XSD, DTD are the two technologies that can be used to define building blocks or rules for constructing xml documents.....

XSD-->Xml Schema Definition

```
//ClientApp.java
package com.nt.test;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.FileSystemResource;
import com.nt.beans.WishGenerator;
public class ClientApp {
    public static void main(String[] args) {
        System.out.println("ClientApp:main(-)");
        //Locate Spring Bean configure file
        FileSystemResource res=new
        FileSystemResource("src/com/nt/configures/applicationContext.xml");
        //Activate BeanFactory container
        XmlBeanFactory factory=new XmlBeanFactory(res);
        // Get Bean class obj
        //WishGenerator bean=(WishGenerator)factory.getBean("wsg");
        WishGenerator bean=factory.getBean("wsg",WishGenerator.class);
        // Call B.method
        String result=bean.generateWishMsg();
        System.out.println(result);
    } //main
} //class
```


Step 4: Add the following jar files to Build path of project.

Right click on project-->Buildpath-->Config buildpath-->libraries--->Addexternal jars-->

Add external jars-->

Commons-logging.jar (collect from spring 2.5)

Spring-beans-<version>.jar

Spring-context-<vers>.jar

Spring-context-support-<version>.jar

Spring-core-<vers>.jar

Spring-expression-<version>.jar

Step5: Run the client App

FileSystemResource *res=new FileSystemResource*

("src/com/nt/configures/applicationContext.xml");

Gives resource object by using java.io.File (1);

Internally pointing to the path and filename of spring bean configure file.

XmlBeanFactory *factory=new XmlBeanFactory (res);*

or

BeanFactory *factory=new XmlBeanFactory(res);*

Based on the given "res" obj it locates the spring bean configure file (xml) from specified path of file system.

Checks whether that xml file is well formed and valid documents or not.

Note1: If Xml file satisfies springframework syntax rules then it is called well-formed document.

Note:2 If xml file satisfies the xsd and DTD rules then it is called valid document.

Creates an empty logical IOC container in jvm memory.

Uses the SAXpraser (xml parser) to xml file (Bean configure file) data and maintain inmemory metadata or inmemory configure.

Returns XmlBeanFactory class obj ref representing IOC container.

WishGenerator *bean=(WishGenerator)factory.getBean("wsg");*

Loads "com.nt.beans.WishGenerator" class from inmemory metadata based on the given "wsg" and creates object for it.

Class.forName("com.nt.beans.Wish").newInstance();

Observes "name" with <property> tag and calls setName("Raja") method on WishGenerator class obj.

Observes "date" with <property> tag and load "java.util.Date" class to create the Object.

Class.forName("java.util.Date").newInstance();

Also setDate(Date Class obj) on WishGenerator obj to perform setter injection.

Keeps Date class object and WishGenerator class obj in IOC container as elements of HashMap.

Key

Value

Dt

Date Class object (dt)

Wsg

WishGenerator class obj (wsg)

(bean id)

(Bean class object)

Returns WishGenerator class object reference back to ClientApp

If we configure both setter injection and constructor injection on bean property the values given by setter injection will remain final values of injection because setter(-) method executes after constructor executes and overrides the values injected by the constructor execution.

If spring bean class is configured without any configuration or only with setter injection (one or more properties) then container uses O-param constructor to create beans class object. If any

property is configured for constructor injection then it uses parameterized constructor to create bean class object.

In spring 3.x bean properties and its setter methods can be taken as static. But in spring 2.x we cannot take setter methods as static.

```
WishGenerator bean=(WishGenerator)Factory.getBean("wsg");
```

(Typecasting mandatory)

```
WishGenerator bean=Factory.getBean("wsg", WishGenerator.class)
```

(we can avoid type casting)

Prototype:

```
public Object getBean (String id);
```

What is the difference between FileSystemResource and ClassPathResource?

FileSystemResource can locate given spring bean configure file from the specified path of file system. Here we can pass either absolute path or relative path.

```
FileSystemResource res=new FileSystemResource ("src/con/nt/configures/applicationContext.xml")
```

ClasspathResource can locate given spring bean configure file from directories or jarfiles that are added to classpath/buildpath.

```
ClassPathResource res=new ClassPathResource("applicationContext.xml");
```

(But add com.nt.configures pkg to build path)

Right click on project->buildpath-->library-->Add class folder---->.....

Understanding "Collaboration" and tight coupling, loose coupling:

In real project development we can't develop entire logic in one class. We need to take multiple classes and these classes will maintain dependency with each other.

The Class that uses other class logics/method is called target class and the class that acts as helper class is called dependent class and we can say both classes are in collaboration.

Filpkart and DTDC classes must be there in collaboration. i.e Filpkart class is target class/main class and DTDC class is dependent class.

To make target class/main class using logics of dependent class we can use the following approaches.

Approaches 1:

Create Dependent class object in target class.

Approaches 2:

Create dependent class obj in Factory class and make target class using that factory class.

Approaches 3:

Target class gets Dependent class obj from jndi registry.

Approaches 4:

Target class extends from dependent class.

Approaches 5:

Make underlying container injecting dependent class object to target class.

Approaches 1:

Here target class creates the object of dependent class user that object.

This is nothing but composition (classes containing HAS -A relation)

Target class	Dependent Class
<pre> class Filpkart { DTDC dtdc=new DTDC (); public void purchase(); dtdc.deliver(); } </pre>	<pre> class DTDC { deliver(); } </pre>

Limitations :

Instead of DTDC courier, if flipkart want to use BuleDart/FirstFlight courier, we need to modify target class (Flipkart).

If deliver is changed to supply () method in DTDC class we need to modify the filpkart class.

The above limitation indicates target and dependent classes are tightly coupled.

Approaches 2:

The class that can create and return one or more objs is called Factory class.

<pre> public class DTDCFactory public static DTDC getInstance() { return new DTDC(); } } class Filpkart { public void purchase() { DTDC dtdc=DTDCDFactory.getInstance(); dtdc.deliver(); } } </pre>	<pre> class DTDC{ public void deliver() { } } </pre>
---	--

Limitations:

If filpkart wants to uses BuleDart then it need BuleDartFactory class instead DTDCFactory class
Same as approach 2

Note: Here tightlycopuoling observes.

Approaches 3:

If you want to provide global visibility obj or obj ref then it should be placed in jndi registry.

Eg: rmi registry, cos registry and etc.....

Note: Every webserver/application server gives one built-in jndi registry.

```

class Filpkart{
    public void purchase(){
        InitialContext ic=new InitialContrext();
        DTDC dtdl=(DTDC)ic.lookup("cr");
        dtdl.deliver();
    }
}

```

Advantage:

Because jndi registry DTDC obj gets global visibility.
Filpkart class can use already available DTDC class obj.

Limitation:

If Filpkart wants BuleDart obj instead DTDC obj we need to place that obj in jndi registry.
Same as approach 1.
It also gives tight coupling.

Approach 4:

Make target class extending from dependent class.

```
class DTDC
{
    deliver ()
    {
        .....
        .....
    }
}

class Filpkart extends DTDC
{
    purchase()
    {
        deliver();
    }
}
```

Limitations:

Filpkart class cannot extends more than one dependent class.
In order to change dependency from DTDC to BuleDart, we need to make Filpkart extending from BuleDart class.
Same as approach 1

Note: Show tightly coupling.

Approach 5:

Use dependency injection (setter injection/constructor injection)

```
class Filpkart
{
    private DTDC dtdc;
    public void setDtdc(DTDC dtdc)
    {
        this.dtdc=dtdc;
    }
    public void purchase ()
    {
        .....
        .....
        dtdc.deliver();
    }
}

class DTDC
{
    public void deliver()
    {
        .....
        .....
    }
}
```

This approach 5 is much similar to approach 1 or approach 2. only difference is the underlying container is managing dependency in approach 5 where as programmer is managing dependency in approach 1 and 2.

Limitations:

Same as approach 1

Note: shows tightly coupling.....

Approach 5 is good towards managing dependency. But in order to achieve loose coupling we shouldn't directly work with dependent classes in target class. It is recommended to make all dependent classes implementing common interface having common method declaration. So that we can start using this common interface in the target class.

```
class Flipkart
{
    private Courier courier;
    public void setCourier(Courier courier)
    {
        this.courier=courier;
    }
    public void purchase(){
        courier.delivery();
        .....
        .....
    }
}
```

The above Flipkart is not working with implement class name. It is working with interface. So we can make underlying container injecting any implement class object to FlipKart (target class). All dependent classes are implementing common interfaces courier, so there is no possibility of changing method name "deleiver()"

Note:

The above code shows lossely coupling between target class (Flipkart) and dependent classes. To achieve lossely coupling in spring use.

Dependency injection
interface model programming (poji model)

2nd project:

```
package com.nt.beans;
public interface Courier {
    public String deliver(int orderid);
}

package com.nt.beans;
public class BlueDart implements Courier {
    public BlueDart(){
        System.out.println("BlueDart:0-param constructor");
    }
    @Override
    public final String deliver(int orderid) {
        return "BlueDart is ready to deliver products of "+orderid;
    }
}
```

```
}  
}  
  
package com.nt.beans;  
public class DTDC implements Courier {  
    public DTDC(){  
        System.out.println("DTDC:0-param constructor");  
    }  
    public final String deliver(int orderid) {  
        return "DTDC is ready to deliver products of "+orderid;  
    }  
}  
  
} //BlueDart.java
```

```
//Flipkart.java  
package com.nt.beans;  
import java.util.Random;  
public class FlipKart {  
    private Courier courier;  
    public FlipKart(){  
        System.out.println("FlipKart:0-param constructor");  
    }  
    //setter method for setter injection  
    public void setCourier(Courier courier){  
        System.out.println("FlipKart:setCourier(-)");  
        this.courier=courier;  
    }  
    public String purchase(String items[]){  
        //generate order id  
        Random rad=new Random();  
        int orderid=rad.nextInt(700000);  
        // deliver order  
        String status=courier.deliver(orderid);  
        return status+ "Bill Amt for order id"+orderid+" is 7000";  
    }  
} //method  
} //class
```

applicatioContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"  
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
xsi:schemaLocation="http://www.springframework.org/schema/beans  
http://www.springframework.org/schema/beans/spring-beans.xsd">  
    <bean id="dtdc" class="com.nt.beans.DTDC" scope="prototype"/>  
    <bean id="blueDart" class="com.nt.beans.BlueDart" scope="singleton"/>  
    <bean id="fpk" class="com.nt.beans.FlipKart" scope="singleton"/>
```

```

<property name="courier">
  <ref bean="dtdc"/>
</property>
</bean>
</beans>

```

```

package com.nt.test;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.FileSystemResource;
import com.nt.beans.FlipKart;
public class LCTestApp {
    public static void main(String[] args) {
        // Create BeanFactory obj to create IOC container
        BeanFactory factory=new XmlBeanFactory
                                (new
        FileSystemResource("src/com/nt/configures/applicationContext.xml"));
        // Get Bean obj
        FlipKart bean=factory.getBean("fpk",FlipKart.class);
        factory.getBean("fpk");
        String billmsg=bean.purchase(new String[]{"shirt","mobile","books"});
        System.out.println(billmsg);
    }
}

```

In spring bean config file:-

```

<property name="msg" value="Hello"/>
Can be written as
<property name="msg">
  <value>Hello</value>
</property>
<property name="date" ref="dt"/>
Can be written as
<property name="date">
  <ref bean="dt"/> or <ref local="dt"/>
</property>

```

Attaching source code and api docs (Spring) with eclipse ide:

Right click on project --->buildpath--->configurebuildpath
 Select any jar file that contains the classes that you are using
 Source attachment--->edit--->external location----> path:--->external file-->select
 <spring_home>/libs/spring_beans_<x.x>
 Release_sources jar.file
 Java doc location
 Edit---->javadoc in archive-->external file--->archive path (Select spring software zip file)
 Path in archive : select..docs/javadoc_api folder in the above zip file --->validate-->ok

Creating user-defined library (SpringIOC) in eclipse IDE:

Right click project-->build path---->configure build path---->library---->Add library-->user library----
 >next---->userlibrary
 --->new-->IOCLibrary-->Add external jars--->select required jar files.

Design pattern:

Design patterns are set of rules which come as best solutions for reoccurring problem of application development.

Design patterns are best practices for utilizing software languages, technologies f/w's effertly in application development.

Ex: Singleton, factory and etc....

BeanFactory IOC container is designed based on the factory pattern....

It is always recommended to develop spring apps based on strategy pattern.

Stratgery pattern:

While developing multiple classes having dependency.

It is recommended to follow this design pattern.

This pattern is not spring pattern. It can be used anywhere since spring manages dependency between classes, it is recommended to use this pattern while developing spring application.

✓Strategy pattern is all about implementing 3 principles.

✓Prefer composition(HAS-A) over inheritance.

✓Always code to interface and never code with implementations(classes)

Code should be open to extension and must be closed for modification

Prefer composition over inheritance:

If one class creates the objects of other class to use its logics then it is called composition

(Has -A Declarationship)

```
class A{
```

```
.....
```

```
.....
```

```
}
```

```
class B
```

```
{
```

```
A a=new A();
```

```
}
```

If one class extends from another class then it is called inheritance.

```
class A
```

```
{
```

```
.....
```

```
.....
```

```
}
```

```
class B extends A
```

```
{
```

```
.....
```

```
.....
```

```
}
```

Inheritance is having following limitation in java.

It doesn't support multiple inheritance

Code becomes easily breakable

```
class C extends A,B
```

```
{
```

```
.....
```

```
.....
```

```
}
```



```
class C
{
    A a=new A();
    B b=new B();
}
```

Code is easily breakable:**EX:**

```
class Test{
    public int x()
    {
        .....
        .....
        return 100;
    }
}
class Demo extends Test {
    public int x(){
        .....
        .....
        return 1000;
    }
}
```

If change return type or param type of x() in Test we can not compile Demo class because overriding rules (let us "Test", "Demo" class given by 2 different developers)

Solution:

```
class A{
    public int x(){
        .....
        return 100;
    }
}
class B
{
    A a=new A();
    public int y(){
        int res=a.x();
    }
}
```

If x() method return type will change in "A", then there is no need of doing modifications major by in 'B'.

If x() return type is changed to float then we need to write y() in 'B' as shown below

```
public int y(){
    Float res=a.x();
    .....
    .....
}
```

Always code to interface and never code to implementation:

```
class A{  
.....  
}  
class B{  
.....  
}  
class Test{  
A a=new A();  
.....  
}
```

Assigning class "B" object instead of class "A" object in Test class is not possible with minimum modification.

Eg:

```
class A{  
.....  
}  
interface x{  
.....  
}  
class B implements x{  
.....  
}  
class Test{  
X x=new A();  
Or  
X x=new B();  
.....  
}
```

When we code with interfaces we can achieve loosely coupling between target class and dependent class.

Eg:

```
class Test{  
X x;  
public void setTransactionManagement (X x)  
{  
this.x=x;  
}  
}
```

```
Test t=new Test();
t.setTransactionManagement (new A());
t.setTransactionManagement (new B());
```

Note:

Without modifying the code "Test" class we can assign either the object for class A or class B. This gives loosely coupling.

Code must be open for extension and must be closed for modification.

If we follow second principle perfectly by taking the methods of implementation classes A,B (as method) our code becomes closed for modification.

Similarly it allows to more implementation classes for interface x having new logics. This makes our code open for extension.

Observing strategy pattern in Application no:2 :

Flipkart class wants the logics of DTDC or BuleDart classes. But it is not inheriting from those classes, but getting object of those classes through composition. (principle 1)

BlueDart, DTDC classes are implementing "Courier (i)" or their objects are assigned to FlipKart class through Courier (i). (principle 2)

By making the deliver method of BuleDart,DTDC classes as final methods, we can keep that code class for modification and we can develop new implements class for "Courier (i)" if new Courier service (like firstflight) is required for Flipkart class (principle-3)

Resolving/identifying params in constructor injection:-

We generally configure the dependent values of constructor injection in the order the params 0* constructor are available. But we can mismatch this order during configure if we can identify the params by using index or type or name.

EX:**Resource based parameter type:**

- IOCProj3(Constructor Injection)
 - src
 - com.nt.beans
 - Faculty.java
 - Machine.java
 - Marks.java
 - Student.java
 - com.nt.cfgs
 - applicationContext.xml
 - com.nt.test
 - ClientApp.java
 - JRE System Library [JavaSE-1.7]
 - IOCLib
 - spring-beans-4.1.6.RELEASE.jar - C:\Users\...
 - spring-context-4.1.6.RELEASE.jar - C:\Users\...
 - spring-context-support-4.1.6.RELEASE.jar - C:\Users\...
 - spring-core-4.1.6.RELEASE.jar - C:\Users\...
 - spring-expression-4.1.6.RELEASE.jar - C:\Users\...
 - commons-logging.jar - C:\Users\...\Desktop

```
package com.nt.beans;
public class Student {
    private int sno;
```

```
private String sname;
private float avg;
public Student(int sno, String sname, float avg) {
    System.out.println("Student:3-param constructor");
    this.sno = sno;
    this.sname = sname;
    this.avg = avg;
}
@Override
public String toString() {
    return "Student [sno=" + sno + ", sname=" + sname + ", avg=" + avg + "]\n";
}
}
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="St" class="Pkg.Student">
<constructor-arg value="1001" type="int"/>
<constructor-arg value="89.45F" type="float"/>
<constructor-arg value="raja" type="java.lang.String"/>
</bean>
</beans>
```

Limitation:

If multiple params are having same type then this approach can't be used.

EX2: Resolving based on index:

```
package com.nt.beans;
public class Marks{
    private int m1,m2,m3;
    public Marks(int m1,int m2,int m3){
        this.m1=m1;
        this.m2=m2;
        this.m3=m3;
    }
    //Place toString()
    @Override
    public String toString() {
        return "Marks [m1=" + m1 + ", m2=" + m2 + ", m3=" + m3 + "]\n";
    }
}
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="mk" class="Pkg.Marks">
  <constructor-arg value="30" index="2"/>
  <constructor-arg value="20" type="1"/>
  <constructor-arg value="10" index="0"/>
</bean>
</beans>
```

EX:3

```
package com.nt.beans;
import java.util.Date;
public class Machine {
    private int id;
    private String name;
    private Date dom;
    public Machine(int id, String name, Date dom) {
        this.id = id;
        this.name = name;
        this.dom = dom;
    }
    @Override
    public String toString() {
        return "Machine [id=" + id + ", name=" + name + ", dom=" + dom + "]";
    }
}
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="dt" class="java.util.Date"/>
  <bean id="m1" class="com.nt.beans.machine">
    <constructor-arg ref="dt"/>
    <constructor-arg value="Ramesh"/>
    <constructor-arg value="101" index="0"/>
  </bean>
</beans>
```

EX4:**Resolving configure params through name:**

```
//Faculty.java
package com.nt.beans;
import java.beans.ConstructorProperties;
public class Faculty {
    private int id;
    private String name,subject;
    @ConstructorProperties({"no","fname","sub"})
```

```
public Faculty(int no, String fname, String sub) {
    this.id = no;
    this.name = fname;
    this.subject = sub;
}
public String toString() {
    return "Faculty [id=" + id + ", name=" + name + ", subject=" + subject + "];"
}
} //class
```

applicationContext.xml :

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="st" class="com.nt.beans.Student">
<constructor-arg value="1001" type="int"/>
<constructor-arg value="89.45f" type="float"/>
<constructor-arg value="raja" type="java.lang.String"/>
</bean>
</beans>
```

Note:

```
<bean id="mk" class="com.nt.beans.Marks">
<constructor-arg value="40" index="2"/>
<constructor-arg value="20" index="1"/>
<constructor-arg value="10" index="0"/>
</bean>
```

Note:

```
<bean id="dt" class="java.util.Date"/>
<bean id="m1" class="com.nt.beans.Machine">
<constructor-arg ref="dt"/>
<constructor-arg value="ramesh"/>
<constructor-arg value="101" index="0"/>
</bean>
```

Note:

```
<bean id="f1" class="com.nt.beans.Faculty">
<constructor-arg name="sub" value="java"/>
<constructor-arg name="fname" value="rajesh"/>
<constructor-arg name="no" value="678"/>
</bean>
```

In Spring 3.x the IOC container can't get the parameter names of constructor directly until the parameter names are configured by using "@ConstructorProperties(-)"

In Spring 4.x the IOC container can recognize constructor parameters directly.

Cycle dependency injection:

If 2 beans are dependent to each other then we can say they are in cyclic dependency. Spring container can't perform cyclic dependency injection through constructor injection but can perform through setter injection.

Problem:

- ▲ IOCProj4 (Cyclic Injection)
 - ▲ src
 - ▲ com.nt.beans
 - ▷ A.java
 - ▷ B.java
 - ▲ com.nt.cfgs
 - ▷ applicationContext.xml
 - ▲ com.nt.test
 - ▷ ClientApp.java
 - ▷ JRE System Library [JavaSE-1.7]
 - ▲ IOCLib
 - ▷ spring-beans-4.1.6.RELEASE.jar - C:\Users\del\
 - ▷ spring-context-4.1.6.RELEASE.jar - C:\Users\del\
 - ▷ spring-context-support-4.1.6.RELEASE.jar - C:\
 - ▷ spring-core-4.1.6.RELEASE.jar - C:\Users\del\
 - ▷ spring-expression-4.1.6.RELEASE.jar - C:\Users\
 - ▷ commons-logging.jar - C:\Users\del\Desktop

```
class A{
    B b;
    public A(B b){
        this.b=b;
    }
}
```

```
class B{
    A a;
    public B(A a){
        this.a=a; }
}
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframewrk.org/schema/beans/spring-beans.xsd">
    <bean id="a" class="A">
        <constructor-arg ref="b"/>
    </bean>
    <bean id="b" class="B">
        <constructor-arg ref="a"/>
    </bean>
</beans>
```

Solution: (Using Setter injection)

```
class A{
    B b;
    this.b=b;
}
class B {
    A a;
    public void setA(A a){
        this.a=a;
    }
}
```

applicationContext.xml (Supports)

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="a" class="A">
<property name="b" ref="b"/>
</bean>
<bean id="b" class="B">
<property name="a" ref="a"/>
</bean>
</beans>

```

When should we go for setter injection and when should we go for constructor injection?

Ans: If bean class contains only one property or if all the property of bean class should participate in dependency injection then go for constructor injection, because construct injection is very faster than setter injection.

If Bean class contains more than one property and there is no need of making all properties participate in dependency injection, then we need to go for setter injection.

Eg: If bean class contains four properties participating in dependency injection, then we need to write max of four setter methods to perform setter injection. But we need to write four factorial (24) constructors to support constructor injection in all angles.

Difference between setter injection and constructor injection:**Setter injection**

Perform injection after creating Bean class obj so it is bit delayed injection.
 Supports cyclic dependency injection
 Property tag is required.
 To perform setter injection in n properties
 In all angles n setter methods are required.

If all properties are configured for setter Injection then container creates Bean class object using O-param constructor.

Constructor injection

Performs injection while creating Bean class object so no delay in injection
 Doesn't supports cyclic injection
 constructor -arg tag is required.
 To perform constructor injection on n
 Properties in all angles n! constructors are required.

If any property is configured for constructor injection container Bean class object using Parameterized constructor.

Inner Bean:

This is no way related with inner classes of java. It is all about configure one bean as inner bean of another bean by placing <bean> tag inside the <property> or <constructor-arg> tag.

EX:

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="fpk" class="com.nt.beans.Flipkart">
<property name="courier">
<bean id="dtdc" class="com.nt.beans.DTDC"/>
<!--.....inner bean-->

```



```
</property>
</bean>
</beans>
```

Use inner bean configure if want to make bean as dependent bean to another bean only for one time.

Limitations with inner bean:

Inner bean cannot be used as dependent bean of multiple other beans.

Inner bean object cannot be accessed from client applications.

Note: Bean id is optional for inner beans.

Can we configure spring bean class without?

Ans: Yes possible, because the container generates default bean id in the following notation.

```
<pkg>.<BeanClass>#n
```

```
--->0,1,2,3.....
```

Ex: `<bean class="com.nt.beans.FlipKart">`

(bean id is "com.nt.beans.Flipkart#0")

```
</bean>
```

```
<bean class="com.nt.Flipkart"> (bean id is "com.nt.beans.Flipkart#1)
```

```
</bean>
```

Spring supports dependency injection on the following properties.

Simple properties (primitive data types, String)

Reference/Object type properties

Collection properties (Array, List, Set, Map, Properties, and etc.....)

Property type

Simpler property

Reference property

Array property

Tag for dependency injection configuration

value attribute or `<value>` tag

Reference attribute or `<ref>` tag

```
<list>
```

Property type

Java.util.List

Java.util.Set

Java.util.Map

Java.util.Properties

Tag for dependency injection tag

```
<list>
```

```
<set>
```

```
<map>
```

```
<props>
```

- IOCProj5(Collection Injection)
 - src
 - com.nt.beans
 - College.java
 - EmployeeProfile.java
 - Student.java
 - User.java
 - User1.java
 - com.nt.cfgs
 - applicationContext.xml
 - com.nt.test
 - CollectionTest.java
 - JRE System Library [JavaSE-1.7]
 - IOCLib
 - spring-beans-4.1.6.RELEASE.jar - C:\Users\...
 - spring-context-4.1.6.RELEASE.jar - C:\Users\...
 - spring-context-support-4.1.6.RELEASE.jar - C:\Users\...
 - spring-core-4.1.6.RELEASE.jar - C:\Users\...
 - spring-expression-4.1.6.RELEASE.jar - C:\Users\...
 - commons-logging.jar - C:\Users\delin\Desktop\...

```

package com.nt.beans;
import java.util.Arrays;
import java.util.List;
public class Student {
    private int marks[];
    private List<String> names;
    public void setNames(List<String> names){
        this.names=names; }
    public void setMarks(int[] marks) {
        this.marks = marks;
    }
    @Override
    public String toString() {
        return "Student [marks=" + Arrays.toString(marks) + ", names=" + names
            + " ]";
    }
}

```

applicationContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="st" class="com.nt.Student">
<property name="colors">
<list>
<value>red</value>
<value>green</values>
</list>
</property>
</bean>
</beans>

```

Internal Code:

```
Student st=(Student)Class.forName("Student").newInstance();
String colors[]=new String[]{"red","green"};
St.setColors(colors);
```

Injecting List:

```
public class Student{
private List<String> names;
public void setNames (List<String> names){
this.names=names;
}
//toString
.....
.....
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="St" class="com.nt.beans.Student">
<property name="names">
<list>
<value>Raja</value>
<value>Ravi</value>
<value>Raju</value>
</list>
</property>
</bean>
</beans>
```

Note: java.util.List allows duplicates. So we can inject duplicates values.

Internal code:

```
Student st=(Student)Class.forName("com.nt.beans.Student").newInstance();
List <String>names=new ArrayList <String>();
names.add("Raja");
names.add("Ravi");
names.add(Raju);
st.setName(names);
```

Injecting Set:

Java.util.Set type collection doesn't allows duplicate vales.

Eg:

```
package com.nt.beans;
import java.util.Set;
public class User{
```

```
private Set<String> phones;
public void setPhones(Set<String> phones){
    this.phones=phones;
}
// toString()
@Override
public String toString() {
    return "User [phones=" + phones + "]";
}
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="user" class="com.nt.beans.User">
<property name="phones">
<set>
<value>+91-9859922331</value>
<value>+91-892149322</value>
</set>
</property>
</bean>
</beans>
```

Injecting Map:

Each element of map allows one key, one value use map tag to configure injection.

EX:

```
package com.nt.beans;
import java.util.Map;
public class College{
    private Map<String,String> facultySubjects;
    private Map<?,?> facultyPhones;
    public void setFacultySubjects(Map<String,String> facultySubjects){
        this.facultySubjects=facultySubjects;
    }
    public void setFacultyPhones(Map<?, ?> facultyPhones) {
        this.facultyPhones = facultyPhones;
    }
    @Override
    public String toString() {
        return "College [facultySubjects=" + facultySubjects
            + ", facurtyPhones=" + facultyPhones + "];"
    }
}
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```

xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="clg" class="com.nt.beans.College">
<property name="facultySubjects">
<map>
<entry key="Raja" value="java"/>
<entry>
<key><value>Ravi</value></key>
<value>.NET</value>
</entry>
<entry key="Ramesh">
<value>SAP</value>
</entry>
</map>
</property>
</bean>
</beans>

```

Map collections elements allow any objects as key and any object as values, but keys cant be duplicated and values can be duplicated.

EX:

```

public class College{
private Map<?,?> facultyPhones;
public void setFacultyPhones(Map<?,?> facultyPhones){
This.facultyPhones=facultyPhones;
}
//toString ()
.....
.....
}

```

applicationContext.xml:

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="id" class="java.util.Date"/>
<bean id="clg" class="com.nt.beans.college">
<property name="facultyPhones">
<map>
<entry kay="Raja" value-ref="user"/>
<entry key-ref="dt" value-ref="user"/>
<!--<entry>
<key><ref bean="dt"/> </key>
<ref bean="user"/>
</entry> ----->
</map>
</property>

```

```
</bean>
</beans>
```

Injecting java.util.Properties:

Java.util.Properties allows only *String* as the key-value pairs i.e key must be *String* and value must be *String*.

EX:

```
package com.nt.beans;
import java.util.Properties;
public class EmployeeProfile{
    private Properties empDesgs;
    public void setEmpDesgs(Properties empDesgs){
        this.empDesgs=empDesgs;
    }
    // toString()
    @Override
    public String toString() {
        return "EmployeeProfile [empDesgs=" + empDesgs + "]";
    }
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="emp" class="com.nt.beans.employeeProfile">
<property name="empDesgs">
<props>
<prop key="Anil">Clerk</prop>
<prop key="Jani">SE</prop>
</props>
</property>
</bean>
</beans>
```

Injecting Null <null>

Given to injection null value to reference type bean property.

Useful in constructor injection to inject null value to certain reference type parameter when we cannot supply value.

In constructor injection we need to supply values to all the params. If we cannot supply values to certain parameters then we can <null /> tag to inject null value.

EX:

```
public class User{
    private int id;
    private String name;
    private Date doj;
    public user(int id, String name, Date dob){
        this.id=id;
```

```

this.name=name;
this.dob=dob;
}
//toString()
.....
.....
}

```

applicationContext.xml:

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="user" class="pkg.user">
<constructor-arg value="100"/>
<constructor-arg value="Raja"/>
<constructor-arg><null/></constructor-arg>
<!--.....null injection-->
</bean>
</beans>

```

Bean inheritance:

- ▲ IOCProj6(Bean Inheritance)
 - ▲ src
 - ▲ com.nt.beans
 - ▶ Bike.java
 - ▲ com.nt.cfgs
 - ▶ applicationContext.xml
 - ▲ com.nt.test
 - ▶ ClientTest.java
 - ▶ JRE System Library [JavaSE-1.7]
 - ▲ IOCLib
 - ▶ spring-beans-4.1.6.RELEASE.jar - C:\Users\del
 - ▶ spring-context-4.1.6.RELEASE.jar - C:\Users\del
 - ▶ spring-context-support-4.1.6.RELEASE.jar - C:\Users\del
 - ▶ spring-core-4.1.6.RELEASE.jar - C:\Users\del
 - ▶ spring-expression-4.1.6.RELEASE.jar - C:\Users\del
 - ▶ commons-logging.jar - C:\Users\del\Deskt

```

//Bike.java
package com.nt.beans;
public class Bike{
    private String id;
    private String make;
    private String engineCC;
    //write setTransaction management xx(-) methods
    public void setId(String id) {
        this.id = id;
    }
    public void setMake(String make) {
        this.make = make;
    }
}

```

```
public void setEngineCC(String engineCC) {  
    this.engineCC = engineCC;  
}  
//write toString()  
@Override  
public String toString() {  
    return "Bike [id=" + id + ", make=" + make + ", engineCC=" + engineCC + "];"  
}  
}
```

We use inheritance for reusability and extensibility. Bean inheritance is no way related with class level physical inheritance. It is all about about xml file level bean configures inheritance.

When we want configure same class for multiple times as multiple spring beans we can use reuse of bean properties configure across the multiple beans configuration by using bean inheritance.

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"  
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
xsi:schemaLocation="http://www.springframework.org/schema/beans  
http://www.springframework.org/schema/beans/spring-beans.xsd">  
    <bean id="pulsor1" class="com.nt.beans.Bike">  
        <property name="id" value="TS081111"/>  
        <property name="maxe" value="Bajaj"/>  
        <property name="engineCC" value="150"/>  
    </bean>  
    <bean id="pulsor2" class="com.nt.beans.Bike">  
        <property name="id" value="TS082222"/>  
        <property name="maxe" value="Bajaj"/>  
        <property name="engineCC" value="150"/>  
    </bean>  
</beans>
```

Problem:

Maxe, engineCC property values configure is not reusable even though they are same in multiple bean configures.

Solution: (Bean inheritance)

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"  
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
xsi:schemaLocation="http://www.springframework.org/schema/beans  
http://www.springframework.org/schema/beans/spring-beans.xsd">  
    <bean id="pulsor1" class="com.nt.beans.Bike">  
        <property name="id" value="TS081111"/>  
        <property name="maxe" value="Bajaj"/>  
        <property name="engineCC" value="150"/>  
    </bean>  
    <bean id="pulsor2" class="com.nt.beans.Bike" parent="pulsor1">  
        <property name="id" value="TS082222"/>  
    </bean>  
</beans>
```


Solution2: (improved Solution)

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="basepulso" class="com.nt.beans.Bike" abstract = "true">
<property name="maxe" value="Bajaj"/>
<property name="engineCC" value="150"/>
</bean>
<bean id="pulso1" class="com.nt.beans.Bike" parent="basepulso">
<property name="id" value="TS021111"/>
</bean>
<bean id="pulso2" class="com.nt.beans.Bike" parent="basepulso">
<property name="id" value="Ts022222"/>
</bean>
</beans>

```

Important Points:

It is not class level physical inheritance.

Parent bean configuration and child bean configuration can refer same class or different classes.

Note:

When abstract = "true" it doesn't make bean class as abstract class, but it makes bean configuration as abstract bean configuration i.e we can't call factory.getBean(-) having that abstract bean configuration bean id (throws Exception)

Collection Merging: —

It allows to merge the value Set to Base bean configure collection property with the values set to child bean configuration collection property by using merge="true" attribute.

▲ IOCProj/(CollectionMerging)

- ▲ src
 - ▲ com.nt.beans
 - ▷ EnggCourse.java
 - ▲ com.nt.cfigs
 - ▷ applicationContext.xml
 - ▲ com.nt.test
 - ▷ ClientApp.java
- ▶ JRE System Library [JavaSE-1.7]
- ▲ IOCLib
 - ▷ spring-beans-4.1.6.RELEASE.jar - C:\Users\de
 - ▷ spring-context-4.1.6.RELEASE.jar - C:\Users\de
 - ▷ spring-context-support-4.1.6.RELEASE.jar - C:
 - ▷ spring-core-4.1.6.RELEASE.jar - C:\Users\deli
 - ▷ spring-expression-4.1.6.RELEASE.jar - C:\User
 - ▷ commons-logging.jar - C:\Users\deli\Desktop

EnggCourse.java:

```

package com.nt.beans;
import java.util.List;
public class EnggCourse{
    private List <String> subjects;
    public void setSubjects(List <String>subjects){
        this.subjects=subjects;
    }
}

```

```
// toString()
@Override
public String toString() {
    return "EnggCourse [subjects=" + subjects + "];"
}
}
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="base1styear" class="com.nt.beans.EnggCourse" abstract="true">
<property name="subjects">
<list>
<value>C</value> ✓
<v>Math</v> ✓
</list> ✓
</property>
</bean>
<bean id="ECE1styear" class="com.nt.beans.EnggCourse" parent="base1styear">
<property name="subjects"> ✓
<list>
<value>digital elect</value> ✓
<value>C</value>
<value>Math</value>
</list>
</property>
</bean>
</beans>
```

Problem: Writing common Subjects (C, Maths) again in child bean config indicates duplication.

Approach2 :**applicationContext.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="base 1st year" class="com.nt.beans.EnggCourse" parent="base1styear">
<property name="Subjetcs">
<list>
<value>digital value</value>
</list> <prop></bean></beans>
```

Problem:

Child bean configuration overrides the value of parent bean configuration.

Approach3:**applicationContext.xml:**

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="ECE1styear" class="EnggCourse" parent="Base1styear">
<property name="Subject">
<list merge="true">
<value>Dig EC</value>
</list>
</property>
</bean>

```

Client App:

```

EnggCourse EcEngg=factory.getBean("ECE1styear", EnggCourse.class);
System.out.println (ECEEngg);

```

BeanFactory Aliasing :-

Alias name is nick name or petname. It is all about providing more alternative name/ids to use bean.

Prior to spring 2.0, "name" attribute is given to provide aliasname. From spring2.0, <alias> tag is introduced for the same.

We can use both name attribute, <alias> tags to provide alias names from spring2.0

Eg:

```

<bean id="system" name="pc, desktop" alias="com.nt.beans.computer">
.....

```

```

</bean>

```

```

<alias name="system" alias="machine"/>

```

```

<alias name="system" alias="box"/>

```

Note1: <alias> tag is recommended to use because it allows to provide alias names to multiple bean in single place.

Note2: aliasing is useful in the maintenance mode of bean configurations where we can short alias name for lengthy beans ids.

In ClientApp

```

factory.getBean("box");
factory.getBean("PC");
factory.getBean("System");

```

Performing dependency lookup by using IOC container:

If target class/main class writes the code to search and gather dependency values/objects is called dependency lookup of dependency pool. By creating IOC container in target class or main class we can perform dependency lookup/pull operation to get dependent objects.

If target class is wants to use dependent object in all the methods of target class then go for dependent class object only in specific method then go for dependency lookup or pool.

- IOCProj8(Dependency Lookup-IDREF)
 - src
 - com.nt.beans
 - Engine.java
 - Viechle.java
 - com.nt.cfgs
 - applicationContext.xml
 - com.nt.test
 - ClientApp.java
 - JRE System Library [JavaSE-1.7]
 - IOCLib
 - spring-beans-4.1.6.RELEASE.jar - C:\Users\del
 - spring-context-4.1.6.RELEASE.jar - C:\Users\del
 - spring-context-support-4.1.6.RELEASE.jar - C
 - spring-core-4.1.6.RELEASE.jar - C:\Users\del\
 - spring-expression-4.1.6.RELEASE.jar - C:\User
 - commons-logging.jar - C:\Users\del\Desktop

Engine.java:

```
package com.nt.beans;
public class Engine{
    public Engine(){
        System.out.println("Engine:0-param constructor");
    }
    public void start(){
        System.out.println("Engg started");
    }
}
```

Viechle.java:

```
package com.nt.beans;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.cglib.proxy.Factory;
import org.springframework.core.io.FileSystemResource;
public class Viechle{
    private String enggld;
    public Viechle(){
        System.out.println("Viechle:0-param constructor");
    }
    public void setEnggld(String enggld){
        this.enggld=enggld;
    }
    public void move(){
        BeanFactory factory=new XmlBeanFactory(new
            FileSystemResource("src/com/nt/configures/applicationContext.xml"));
        Engine engine=factory.getBean(enggld,Engine.class);
        engine.start();
        System.out.println("Viechle moved for journey");
    }
}
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="engg1" class="com.nt.beans.Engine"/>
<bean id="viechle" class="com.nt.beans.Viechle">
  <property name="engglid">
    <idref bean="engg1"/>
  </property>
</bean>
</beans>
```

Client Application:

```
package test;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.FileSystemResource;
import com.nt.beans.Viechle;
public class ClientApp {
    public static void main(String[] args) {
        BeanFactory factory=new XmlBeanFactory(new
        FileSystemResource("src/com/nt/configures/applicationContext.xml"));
        Viechle viechle=factory.getBean("viechle",Viechle.class);
        viechle.move();
    }
}
```

In the above code:

<property name="engglid" value="engg"/> tag injects dependent class bean id (engine class) to target class object (vehicle class) as an ordinary String value without checking whether bean class is configured or not in this given bean id. To overcome this problem and to inject beanid only when bean class is configured with that beanid need to use '<idref>' tag.

EX: <bean id="engg" class="com.nt.beans.Engine"/>
<bean id="Vehicle" class="com.nt.beans.vehicle">
 <property name="engglid">
 <idref bean="engg"/>
 </property>
</bean>

Eg: String name="test" ---->"test" class is maintained as String content.

Class name=Test.class-->"Test" class() maintained as class in the object of java.lang.Class

FactoryMethod:

The method that's capable of creating either same object or different class object is called FactoryMethod.

Two types of factory method

1) Instance factory method

Eg1: `String S1=new String ("Hello");`

`String S2=S1.concat("123");` //hello 123 (return same class object)

Eg2: `StringBuffer sb=new StringBuffer("Hello how are you?");`

`String S1=sb.substring(0,4);` //given hello (return different class object)

2) Static Factory method:

Eg1: `Class e= Class.forName("java.util.Date");` (returns same class object)

Eg2: `Console cons=System.console();` (returns different class object)

We can make IOC container to create spring bean class.

Object in 4 ways

- 1) Using O-param constructor.
- 2) Using parameterized constructor.
- 3) Using instance factory method.
- 4) Using static factory method.

What is bean writing?

What is auto wiring?

We can make the IOC container to create our spring bean object by using

O-param constructor

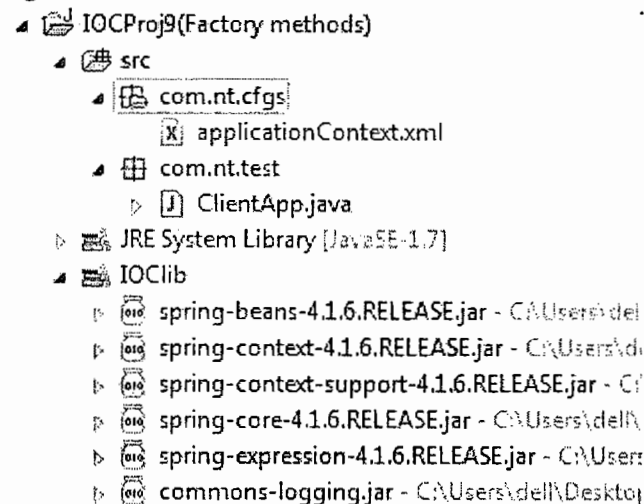
(When no properties are configure for dependency injection and when only setter injection enable)

Parameterized constructor(When atleast 1 property is configure for constructor)

Instance factory method (When factory-method attribute is specified)

Static factory method (when factory-bean attribute is specified)

Eg:



applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<!-- Using static factory methods -->
<bean id="c1" class="java.lang.Class" factory-method="forName">
```

```
<constructor-arg value="java.util.Date"/>
</bean>
<bean id="c2" class="java.util.Calendar" factory-method="getInstance"/>
<!-- Using instance factory methods-->
<bean id="s1" class="java.lang.String">
  <constructor-arg value="hello"/>
</bean>
<bean id="s2" factory-bean="s1" factory-method="concat">
  <constructor-arg value="how are u?"/>
</bean>
<bean id="sb" class="java.lang.StringBuffer">
  <constructor-arg value="hello how are u?"/>
</bean>
<bean id="s3" factory-bean="sb" factory-method="substring">
  <constructor-arg value="0"/>
  <constructor-arg value="5"/>
</bean>
</beans>
```

ClientApp.java

```
package test;
import java.util.Calendar;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.FileSystemResource;
public class ClientApp {
    public static void main(String[] args) {
        // Create IOC Container
        BeanFactory factory=new XmlBeanFactory(new
        FileSystemResource("src/com/nt/configures/applicationContext.xml"));
        // get Bean
        Class c1=factory.getBean("c1",Class.class);
        System.out.println("c1 data"+c1+" c1 class name"+c1.getClass());
        //get Bean
        Calendar c2=factory.getBean("c2",Calendar.class);
        System.out.println("c2 data"+c2+" c2 class name"+c2.getClass());
        // get Bean
        String s2=factory.getBean("s2",String.class);
        System.out.println("s2 data"+s2+" s2 class name"+s2.getClass());
        //get Bean
        String s3=factory.getBean("s3",String.class);
        System.out.println("s3 data"+s3+" s3 class name"+s3.getClass());
    }
}
```

Singleton java class:

The java class that allows us to create only one object per jvm is called *singleton* java class.

Take singleton java class in the following situations

If class is not having any state

If class is having only sharable read only state (final variable)

If class is having huge amount of state and performing write operations on that state in synchronized manner (1 thread at a time)

Instead of creating multiple objects with same data or no data, create one object and access it for multiple times for this take the class as singleton java class.

If we just create one object for java class even though class allows us to create multiple objects then that class is not called Singleton java class.

Eg: our servlet class is not singleton class because it allows us to create multiple objects but most of the times servlet container creates only 1 object.

Bean Scopes:

We can make IOC container keeping spring bean class objects in different scopes. They are

Spring 1.x

singleton

prototype

Spring 2.x

Singleton and prototype

session

request

globalsession

Note: GlobalSession is removed in spring 3.x and y.x version

Default scope is "singleton".

Singleton:

Returns same bean class object for all factory.getBean(-) method calls with same bean id.

Prototype:

Returns separate bean class object for every factory.getBean(-) method call.

Request:

Bean class object is specific to each request.

Session:

Bean class object is visible through out session

GlobalSession:

Specific to portlets env.....

Use "scope" attribute of <bean> tag to specify bean scope.

Note: request, session scopes can be used only in spring MVC applications.

Eg: applicationContext.xml

```
<bean id="v1" class="com.nt.beans.vehicle" scope="prototype">
```

```
.....
```

```
.....
```

```
</bean>
```

```
Vehicle v1=factory.getBean("v1", vehicle.class);
```

```
Vehicle v2=factory.getBean("v1", vehicle.class);
```



```
System.out.println (v1.hashCode()+" "+v2.hashCode());  
System.out.println (v1=v2);
```

Note:

The Bean scope 'singleton' never makes our bean class as singleton java class. But gives singleton behavior while creating object.

If Bean scope is singleton then IOC container creates the bean class object and keeps in HashMap element as value by having bean id as key and uses that object across the multiple factory.getBean(-) method calls.

If scope is scope then IOC contains doesn't keep the created bean class object in HashMap so it returns new object for every factory.getBean(-) call

If scope is "request" then bean object will be maintained as request attribute.

If scope is "Session" then the bean object will be maintained as a session attribute.

Bean Wiring:

Configuring bean and their dependency injection in spring bean configuration file is called wiring.

Explicit wiring: Here we use <property>, <constructor-arg> tags for dependency injection configuration.

Auto Wiring: Here IOC container automatically detects dependents and injects them to target objects. Here no <property>, <constructor-arg> are required.

Autowiring/implicit wiring limitations:

Can be used as only to inject objects but not the simple values.

If IOC container have multiple dependencies to inject then ambiguity problem may raise.

Kills the readability of xml file, so the bug fixing becomes very complex.

Note: Use autowiring only in POC projects or PILOT project for faster/Rapid application development.

By using "autowire" attribute of <bean> tag are can enable autowiring.

There are 4 modes of performing autowiring

byName

byType

constructor

autodetect (removed from spring 3.x)

autowire="byName"

Performs setter injection

For all target class property name and dependent class bean id must match.

Here there is no possibility of getting ambiguity problem

```
└─ IOCProj10(Autowire)  
  └─ src  
    └─ com.nt.beans  
      ├── TourPlan.java  
      └── TravelAgent.java  
    └─ com.nt.cfgs  
      └─ applicationContext.xml  
    └─ com.nt.test  
      └─ ClientApp.java  
  └─ JRE System Library [JavaSE-1.7]  
  └─ IOCLib  
    ├── spring-beans-4.1.6.RELEASE.jar - C:\Users\de  
    ├── spring-context-4.1.6.RELEASE.jar - C:\Users\de  
    ├── spring-context-support-4.1.6.RELEASE.jar - C  
    ├── spring-core-4.1.6.RELEASE.jar - C:\Users\de  
    ├── spring-expression-4.1.6.RELEASE.jar - C:\User  
    └── commons-logging.jar - C:\Users\de\Deskto
```

```
package com.nt.beans;
public class TravelAgent {
    private TourPlan tp;
    public TravelAgent(){
        System.out.println("TravelAgent:0-param constructor");
    }
    public TravelAgent(TourPlan tp){
        System.out.println("TravelAgent:1-param constructor");
        this.tp=tp;
    }
    public void setTp(TourPlan tp){
        System.out.println("TravelAgent:setTp(-)");
        this.tp=tp;
    }
    // toString()
    @Override
    public String toString() {
        return "TravelAgent [tp=" + tp + "]";
    }
}
```

TourPlan.java (dependent class)

```
package com.nt.beans;
import java.util.Arrays;
public class TourPlan {
    private String[] places;
    //setter method
    public void setPlaces(String[] places) {
        this.places = places;
    }
    // toString()
    @Override
    public String toString() {
        return "TourPlan [places=" + Arrays.toString(places) + "]";
    }
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="tp" class="com.nt.beans.TourPlan">
    <property name="places"> <!-- explicit wiring -->
        <list>
            <value>Delhi</value>
            <value>simla</value>
        </list>
    </property>
</bean>
```

```
</property>
</bean>
<bean id="tp2" class="com.nt.beans.TourPlan">
  <property name="places"> <!-- explicit wiring -->
    <list>
      <value>NewYork</value>
      <value>Goa</value>
    </list>
  </property>
</bean>
<bean id="tagent" class="com.nt.beans.TravelAgent" autowire="constructor">
  <constructor-arg ref="tp2"/>
</bean>
</beans>
```

ClientApp.java

```
package test;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.FileSystemResource;
import com.nt.beans.TravelAgent;
public class ClientApp {
    public static void main(String[] args) {
        //Create IOC Container
        BeanFactory factory =new XmlBeanFactory(new
        FileSystemResource("src/com/nt/configures/applicationContext.xml"));
        // Get Bean
        TravelAgent agent=(TravelAgent)factory.getBean("tagent");
        System.out.println(agent);
    } //main
} //class
```

autowire="byType":

perform setter injection

for this target class property type and dependent class must be match or must be compatible with each other.

There is possibility of getting ambiguity problem.

Eg:

```
<bean id="tplan" class="com.nt.beans.Tourplan">
.....
.....
<bean id="target" class="com.nt.beans.TravelAgent" autowiring="byType"/>
```

Note: If we configure "TourPlan" class one more time in spring bean configure file with different bean id then we get ambiguity problem.

autowire="constructor"

uses parameterized constructor to perform constructor injection.

There is a possibility of getting ambiguity problem. If multiple dependencies are found, but it doesn't throw exception if target class contains O-param constructor otherwise exception will be thrown.

Eg: `<bean id="tp1" class="com.nt.beans.TravelAgent" autowire="constructor">`

`autowire="autodetect" // (Try in spring 2.x)`

If target class contains o-param constructor then it performs "byType" mode autowiring.

If target class contains parameterized constructor then it performs "constructor" mode auto wiring.

Removed from spring 3.0 onwards

Eg: `<bean id="tp1" class="com.nt.beans.TourPlan">`

.....

.....

`</bean>`

`<bean id="target" class="com.nt.beans.TravelAgent" autowire="autodetect">`

autowire="no"

Disable autowiring.

What happens if we enable both auto wiring and explicit wiring on same bean property?

Ans: If both wirings are performing the same constructor injection or same setter injection then explicit wiring value will be injected as final values.

If both wiring are performing two different injections then setter injection values will be injected as final value.

Dependency –Check:

While performing constructor injected by using certain parameterized constructor, all the params of that constructor must be configured for wiring, if not configured, exception will be raised.

The above restriction is not there for setter injection configuration. To enable such restriction while working with setter injection go for dependency check by adding "dependency-check" attribute in <bean> tag.

Dependency-check="simple":

Properties are configured for setter injection or not, if not then exception will be raised.

Dependency-Check="objects":

Checks whether all reference type properties are configuration for setter injection or not.

Dependency-check="all":

Checks whether both simple and object properties are configured for setter injection or not.

Dependency-check="none":

Disables dependency check. Use Spring-beans 2.0 xsd in applicationContext.xml.

- ▲ IOCProj11(Dependency-check)
 - ▲ src
 - ▲ com.nt.beans
 - ▶ Engine.java
 - ▶ Viechle.java
 - ▲ com.nt.cfigs
 - ▶ applicationContext.xml
 - ▲ com.nt.test
 - ▶ ClientApp.java
 - ▶ JRE System Library [JavaSE-1.7]
 - ▲ IOCLib
 - ▶ spring-beans-4.1.6.RELEASE.jar - C:\Users\Adi\...
 - ▶ spring-context-4.1.6.RELEASE.jar - C:\Users\Adi\...
 - ▶ spring-context-support-4.1.6.RELEASE.jar - C:\Users\Adi\...
 - ▶ spring-core-4.1.6.RELEASE.jar - C:\Users\Adi\...
 - ▶ spring-expression-4.1.6.RELEASE.jar - C:\Users\Adi\...
 - ▶ commons-logging.jar - C:\Users\Adi\...

EX:

Vehicle.java

```
package com.nt.beans;
public class Viechle {
    private int id;
    private Engine engine;
    public void setId(int id) {
        this.id = id;
    }
    public void setEngine(Engine engine) {
        this.engine = engine;
    }
    @Override
    public String toString() {
        return "Viechle [id=" + id + ", engine=" + engine + "];"
    }
}
```

Engine.java

```
package com.nt.beans;
public class Engine {
    private String type;
    public void setType(String type) {
        this.type = type;
    }
    @Override
    public String toString() {
        return "Engine [type=" + type + "];"
    }
}
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.0.xsd">
  <bean id="eng" class="com.nt.beans.Engine">
    <property name="type" value="diesel"/>
  </bean>
  <bean id="viechle" class="com.nt.beans.Viechle" dependency-check="all">
    <!-- <property name="id" value="1010"/> -->
    <property name="engine" ref="eng"/>
  </bean>
</beans>
```

ClientApp.java:

```
package test;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.FileSystemResource;
import com.nt.beans.Viechle;
public class ClientApp {
    public static void main(String[] args) {
        //Create IOC Container
        BeanFactory factory = new XmlBeanFactory(new
        FileSystemResource("src/com/nt/configures/applicationContext.xml"));
        //Get Bean
        Viechle v = factory.getBean("viechle", Viechle.class);
        System.out.println(v);
    } //main
} //class
```

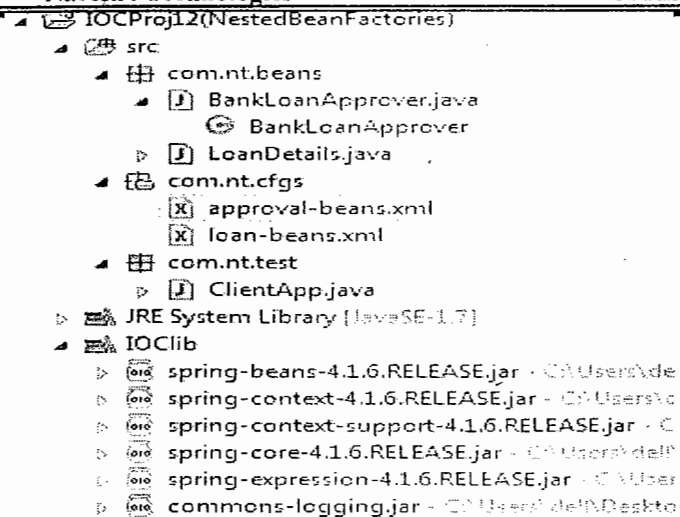
Note: "dependency-check" is removed from spring 2.5 because it allow to apply restriction on group of properties not on specified properties. "Required" is given as an alternate.

Nested Beanfactory:

In real time project contains multiple layers. Instead of configuring all the beans of these multiple layers in single IOC container, it is recommended to take multiple IOC containers on one per layer basis.

To use the beans of 1 IOC container in anther IOC container it is recommended to make them as Nested containers like parent container and child container.

The beans of parent container can be used in child container beans but reserve is not possible. Generally service layer beans want to use the beans of persistence layer, so we configure beans of persistence layer in parent container and the beans of service layer in child container.

**LoanDetails.java :**

```

package com.nt.beans;
public class LoanDetails {
    private int loanId;
    private String loanType;
    private String customer;
    public LoanDetails(){
        System.out.println("LoanDetails:0-param constructor");
    }
    //setters and getters
    public void setLoanId(int loanId) {
        this.loanId = loanId;
    }
    public void setLoanType(String loanType) {
        this.loanType = loanType;
    }
    public void setCustomer(String customer) {
        this.customer = customer;
    }
    public int getLoanId() {
        return loanId;
    }
    public String getLoanType() {
        return loanType;
    }
    public String getCustomer() {
        return customer;
    }
    //toString()
    @Override
    public String toString() {
        return "LoanDetails [loanId=" + loanId + ", loanType=" + loanType
            + ", customer=" + customer + "];"
    }
}

```

BankLoanApprover.java

```
package com.nt.beans;

public class BankLoanApprover {
    private LoanDetails details;
    public BankLoanApprover(){
        System.out.println("BankLoanApprover:0-param constructor");
    }
    //setters
    public void setDetails(LoanDetails details) {
        this.details = details;
    }
    // toString()
    @Override
    public String toString() {
        return "BankLoanApprover [details=" + details + "]";
    }
    public String approveLoan(){
        if(details.getLoanType().equals("two-wheeler")){
            return "Loan is approved";
        }
        else{
            return "Loan in rejected";
        }
    }
}
```

loan-beans.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="ld" class="com.nt.beans.LoanDetails">
        <property name="loanId" value="11"/>
        <property name="loanType" value="two-wheeler"/>
        <property name="customer" value="raja"/>
    </bean>
</beans>
```

approval-beans.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="ld" class="com.nt.beans.LoanDetails">
        <property name="loanId" value="11"/>
        <property name="loanType" value="3-wheeler"/>
        <property name="customer" value="raja"/>
    </bean>
    <bean id="bla" class="com.nt.beans.BankLoanApprover">
```



```
<property name="details" ref="id"/>
</bean>
</beans>
```

ClientApp.java:

```
package test;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.FileSystemResource;
import com.nt.beans.BankLoanApprover;
public class ClientApp {
    public static void main(String[] args) {
        //Create parent IOC container (Parent BeanFactory)
        BeanFactory pfactory=new XmlBeanFactory(new
        FileSystemResource("src/com/nt/configures/loan-beans.xml"));
        //create child IOC container (child BeanFactory)
        BeanFactory cfactory=new XmlBeanFactory(new
        FileSystemResource("src/com/nt/configures/approval-beans.xml"),pfactory);
        // get Bean
        BankLoanApprover approver=cfactory.getBean("bla",BankLoanApprover.class);
        System.out.println(approver.approveLoan());
    } //main
} //class
```

While working with nested BeanFactory/NestedContainers the springBean configure file associated with child container is called child configure is called parent configure file.

Can you explain various attributes of <ref>tag?

Ans: <ref local="<bean id">":

Locates given bean id based spring bean configure from current/local spring bean configure (Child configure File)

Eg: <property name="details">

<ref local="id"/>

</property>

<ref parent="<bean id">

Locates the given bean id based spring bean configure from parent spring bean configure file.

Eg: <property name="details">

<ref parent="id"/>

</property>

<ref bean="<beanid">":

Frist tries to locate from current/local bean configure if file not available then it tries to locate from parent bean configure file

Eg: <property name="details">

<ref bean="id"/>

</property>

<property name="details" ref="id"/> is equal to ref bean="id"/>

P-Name space and C-Name space:

In xml schema "name space" is a library that contains set of xml tags. Every name space is identified with its name space url.

In spring 3.0 p-name space is introduced to perform setter injection configuration without using lengthy <property> tags. It allows to place attribute directly in <bean> tags having the following syntax.

P:<property name>="Value"

P:<property name>-ref="<bean id>"

P-name space url is: <http://WWW.springframework.org/schema/p>

Similarly c-name space introduced from spring 3.1.1 version for constructor injection configures by avoiding the lengthy <constructor-arg>tags.

It allows to place attributes directly in <bean> tag having following syntax.

C:<property name>="value"

C:<property name>-ref="<bean id>"

C-name space uri is: <http://www.springframework.org/schema/c> (Collect from spring reference)

**Emp.java**

```
package com.nt.beans;
public class Emp {
    private int eno;
    private String ename;
    private Dept dept;
    public int getEno() {
        return eno;
    }
    public void setEno(int eno) {
        this.eno = eno;
    }
    public String getEname() {
        return ename;
    }
    public void setEname(String ename) {
        this.ename = ename;
    }
    public Dept getDept() {
        return dept;
    }
}
```

```
}  
public void setDept(Department dept) {  
    this.dept = dept;  
}  
@Override  
public String toString() {  
    return "Emp [eno=" + eno + ", ename=" + ename + ", dept=" + dept + "];"  
}
```

Dept.java

```
package com.nt.beans;  
import java.util.Date;  
public class Dept {  
    private int deptno;  
    private String name;  
    private Date dop;  
    public Dept(int deptno, String name, Date dop) {  
        System.out.println("Dept:3-param constructor");  
        this.deptno = deptno;  
        this.name = name;  
        this.dop = dop;  
    }  
    @Override  
    public String toString() {  
        return "Dept [deptno=" + deptno + ", name=" + name + ", dop=" + dop  
            + "];"  
    }  
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"  
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
xmlns:c="http://www.springframework.org/schema/c"  
xmlns:p="http://www.springframework.org/schema/p"  
xsi:schemaLocation="http://www.springframework.org/schema/beans  
http://www.springframework.org/schema/beans/spring-beans.xsd">  
    <bean id="dt" class="java.util.Date" p:year="115"  
        p:month="10"  
        p:date="20"/>  
    <bean id="dept" class="com.nt.beans.Department"  
        c:deptno="8001"  
        c:name="Accounts"  
        c:dop-ref="dt"/>  
    <bean id="emp" class="com.nt.beans.Employee"  
        p:eno="101"  
        p:ename="raja"  
        p:dept-ref="dept"/>  
</beans>
```

Client.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.Emp;
public class ClientApp {
    public static void main(String[] args) {
        // create IOC container
        ApplicationContext cTransactionmanagement = new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //BeanFactory factory = new XmlBeanFactory(new
        FileSystemResource("src/com/nt/configures/applicationContext.xml"));
        //Get Bean
        Emp emp = cTransactionmanagement.getBean("emp", Emp.class);
        System.out.println(emp);
    } //main
} //class
```

Working with ApplicationContext container (IOC container):

BeanFactory container means it is the IOC container that is created by creating BeanFactory object.
ApplicationContext container means it is the IOC container that is created by creating
ApplicationContext Object.

ApplicationContext object is the object of a java class that implements
org.springframework.context.ApplicationContext(i).

(It is sub interface of BeanFactory (i));

There are 3 popular implementation classes for ApplicationContext(i) among which we can create
ApplicationContext container (IOC container)

FileSystemXmlApplicationContext:

Creates ApplicationContext container by locating spring bean configure file from the specified path
of FileSystem.

Eg: `ApplicationContext cTransaction management = new
FileSystemXmlApplicationContext("src/com.nt/configures/applicationContext.xml");`

ClassPathXmlApplication Context:

Create ApplicationContext container by locating spring bean configure file from the directories and
jar files added to classpath.

Eg: `ApplicationContext cTransaction management = new ClassPathXmlApplicationContext
("applicationContext.xml");`

XmlWebApplicationContext:

Creates ApplicationContext container in web applications

ApplicationContext container can perform all the operations of BeanFactory container and also
perform these additional operations.

Pre-installation of singleton spring Beans.

Ability to work with properties file.

Support for i18n internalized

Support for event handling /processing. And etc..

While working with BeanFactory container the container instantiates spring bean only when
factory.getBean(-) method is called (for any scope bean).

While working with Application-context container the singleton scope beans will be instantiated the moment the container is started. This is called Pre-instantiation of singleton Beans.

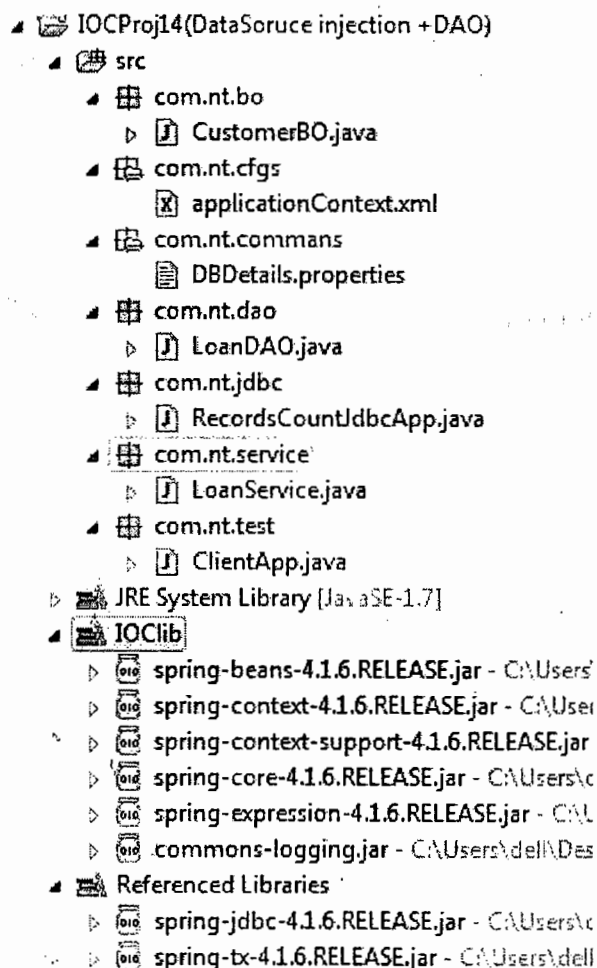
If prototype scope is dependent to singleton scope then prototype scope bean will also be preinstantiated to support dependency injection on singleton scope bean. But it doesn't mean prototype scope bean has changed scope to "Singleton".

Bean pre-instantiation is very much similar to enabling <load-on-startup> on servlet.

We have spring bean configuration file with 10 beans. How can we enable pre-instantiation only on 6 beans?

Ans: Take 6 beans as singleton scope beans and other 4 beans as "prototype" scope beans and don't make 'prototype' scope beans as dependent to singleton scope beans. The java class that contains persistence logic is called DAO. The java class that contains business logic is called service/Business service class. Generally service class needs DAO class object. JDBC DataSource object represents jdbc connections pool and we can use this jdbc DataSource object to get connection object from jdbc connection pool. Spring api gives

org.springframework.jdbc.datasource.DriverManagerDataSource" class to give dataSource object pointing to the IOC container managed jdbc connection pool. It creates the connection object in connection pool based on the bean property values configuration. They are driverClassName, url, username, password.



applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:c="http://www.springframework.org/schema/c"
xmlns:p="http://www.springframework.org/schema/p"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="ppc"
class="org.springframework.beans.factory.config.PropertyPlaceholderConfigurer">
<property name="location" value="com/nt/commons/DBDetails.properties"/>
</bean>
<bean id="drds" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
<property name="driverClassName" value="{jdbc.driver}"/>
<property name="url" value="{jdbc.url}"/>
<property name="username" value="{db.user}"/>
<property name="password" value="{db.pwd}"/>
</bean>
<bean id="dao" class="com.nt.dao.LoanDAO">
<property name="ds" ref="drds"/>
</bean>
<bean id="service" class="com.nt.service.LoanService">
<property name="dao" ref="dao"/>
</bean>
</beans>
```

Customer.java

```
package com.nt.bo;
public class CustomerBO {
    private int cno;
    private String cname;
    private float pamt,intramt;

    public int getCno() {
        return cno;
    }
    public void setCno(int cno) {
        this.cno = cno;
    }
    public String getCname() {
        return cname;
    }
    public void setCname(String cname) {
        this.cname = cname;
    }
    public float getPamt() {
        return pamt;
    }
    public void setPamt(float pamt) {
```

```
        this.pamt = pamt;
    }
    public float getIntramt() {
        return intramt;
    }
    public void setIntramt(float intramt) {
        this.intramt = intramt;
    }
}
```

DBDetails.properties

jdbc properties

db.user=scott

db.pwd=tiger

jdbc.driver=oracle.jdbc.driver.OracleDriver

jdbc.url=jdbc:oracle:thin:@localhost:1521:xe

LoanDAO.java

```
package com.nt.dao;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import javax.sql.DataSource;
import com.nt.bo.CustomerBO;
public class LoanDAO {
    private static final String INSERT_QRY="INSERT INTO CUSTOMER_LOANINFO
VALUES(?,?,?,?)";
    private DataSource ds;
    public void setDs(DataSource ds) {
        this.ds = ds;
    }
    public int insert(CustomerBO cbo){
        Connection con=null;
        PreparedStatement ps=null;
        int result=0;
        try{
            // get Connection obj from jdbc con pool
            con=ds.getConnection();
            //create PreparedStatement obj
            ps=con.prepareStatement(INSERT_QRY);
            //set values to Query param
            ps.setInt(1,cbo.getCno());
            ps.setString(2,cbo.getCname());
            ps.setFloat(3,cbo.getPamt());
            ps.setFloat(4,cbo.getIntramt());
            //execute the Query
            result=ps.executeUpdate();
        }
        catch(SQLException se){
```



```
int cnt=0;
if(rs.next()){
    cnt=rs.getInt(1);
}
System.out.println("Records count"+cnt);

//close jdbc objs
rs.close();
st.close();
con.close();
} //main
} //class
```

LoanService.java

```
package com.nt.service;
import com.nt.bo.CustomerBO;
import com.nt.dao.LoanDAO;
public class LoanService {
    private LoanDAO dao;
    public void setDao(LoanDAO dao) {
        this.dao = dao;
    }
    public float calcIntrAmt(int cno,String cname,float pamt, float rate,float time){
        //write b.logic
        float intramt=pamt*rate*time/100.0f;
        //use DAO class persistence logic
        CustomerBO cbo=new CustomerBO();
        cbo.setPamt(pamt);
        cbo.setCno(cno);
        cbo.setCname(cname);
        cbo.setIntramt(intramt);
        int result=dao.insert(cbo);
        return intramt;
    } //method
} //class
```

Client.java

```
package test;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.config.BeanFactoryPostProcessor;
import org.springframework.beans.factory.config.ConfigurableListableBeanFactory;
import org.springframework.beans.factory.config.PropertyPlaceholderConfigurer;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.FileSystemResource;
import com.nt.service.LoanService;
public class ClientApp {
    public static void main(String[] args) {
        // Create IOC container
```

```
//ApplicationContext cTransaction management =new
FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
BeanFactory factory=new XmlBeanFactory(new
FileSystemResource("src/com/nt/configures/applicationContext.xml"));
//get PropertyPlaceholderConfigurer obj (pphc)
BeanFactoryPostProcessor
bfpp=factory.getBean("ppc",PropertyPlaceholderConfigurer.class);
bfpp.postProcessBeanFactory(((ConfigurableListableBeanFactory) factory);
//get Bean
LoanService service=factory.getBean("service",LoanService.class);
// call b.method
float intramt=service.calcIntrAmt(1009,"Raja",9000,2,20);
System.out.println("Intr Amount"+intramt);
} //main
} //class
```

The java class with standards (minimum getter & setter methods) is called javaBean.

In realtime the data between 2 component/resources/layer will not be sent as individual values. They will be placed in java bean class object and will be sent to from source layer to destination layer.

The javaBean whose object holds the enduser suppliers i/p values (from data) is called VO class (Value Object).

The javaBean whose object holds the data required for persistence logic or the data generated by persistence logic is called Bo class. (Business Object)

The javaBean whose object holds data to transpiring frameworker from 1 layer to another layer is called DTO class. This class generally implements "Serializable" interface .

DTO--->DataTranspiringframeworker object

Generally the DAO Class contains 2 parts

Query Part ()

Code Part()

Working with properties file:

The text file that maintains the entries as key value pairs is called properties file.

We generally use properties file to maintain jdbc properties like driverClassName, url, username and password and make them as flexible to modify without touching the source code.

Spring application can achieve this flexibility through xml file (spring bean configure file)

If spring application is added as new app to the existing project and if existing applications of that project are getting jdbc properties from properties file then are should make sure that spring application also get jdbc properties from properties file.

In ApplicationContext container environment . we need to configure

"org.springframework.beans.factory.config.propertyPlaceholderConfigure" class as spring bean to make container to locate given properties file and to recognize place holders in spring bean configure file.

Place holders---->\${-----}

Each placeholder represents 1 key of the property file to get values from properties file.

I18n (Internationalization):

Locate means country+language

Eg: en-us

en-Br (English as it speaks in Britan)

fr-FR

hi-IN

Making our application working for different locales is called enabling I18n (internationalization). It deals with displaying presentation labels.

Displaying numbers, JEE application we use ResourceBundle, Locale classes of java.util.package to work with I18n.

To apply I18n we need to take multiple properties file having presentation labels on 1 per locale basis. All these files must have same keys and different values.

Eg: app.properties (Base file)

APP_de_DE.properties (for German)

And etc....

If no mataning file is found the base file will be automatically used.

We can use google translator labels in other language based on the given English word.

We can also place Unicode numbers in the properties files to render non English character labels.

For that we need to use third party tools like Unicode editor and jdks "native2ascii" tool.

Procedure-1:

Download Unicode editor from www.higopi.com and type hindi words (use index.html)

Copy hindi words from Unicode editors and place them in notepad.

(save the file by taking "unicode" as encoding type)

Input.Transaction management t:

Write hindi words.

Use native2Ascii to get Unicode numbers.

--->native2Ascii encoding=Unicode inputs.Transaction management t output.Transaction management t (contains Unicode numbers)

Limitations of JEE,JSE envinorment while working with I18n:

In a web application if multiple Servlets/jsp program wants to display o Labels collected from properties file then we need to create "ResourceBundle" class object in multiple servlet/jsp programs or we need to place "ResourceBundle" object as request attribute value.

Note: Some plan has to be decided to use ResourceBundle object.

If we want to display values of same key belonging to multiple locale properties file them we need to create multiple resource bundle class objects.

We can overcome these problems in spring envinorment by using application context container with the support of "ResourceBundleMessageSource" Class.

I18n in spring:

Only application context container supports I18n.

We can use "ResourceBundleMessageSource" class as spring Bean by specifying base property file name as the value of "basename" property.

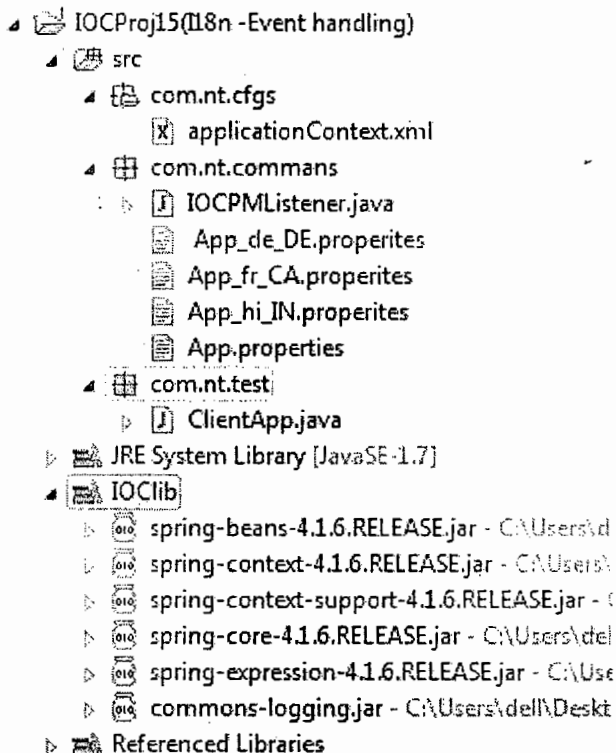
Since we configure the above class as singleton scope SpringBean that the container will creates one object and uses it for multiple times. But the above bean must be configure with fixed bean id "messageSource".

We can call cTransaction management .getMessage() to read message from properties file by supplying appropriate locate object. This cTransaction management .getMessage () method make container to locate properties file based on locale object data with support of bean whose id="messageSource".

Advantage:

Just by creating one object for "ResourceBundleMessageSource" we can use properties file in multiple locations.

Allows to read same message in different languages with one object of above class.

**EX:****Event Handling:**

Event is an object/action that is raised on the component/object.

Event handling means executing logic when event is raised for this we need the support for EventListeners.

There are four actors in Even processing/handling.

Source object (Eg: Button)

Event Class (Eg: ActionEvent)

EventListener (Eg: ActionListener)

Eventhandling method (Eg: Actionperformance (-))

We can perform event handling on applicationContext container to keep track of when container is started and stopped. This allows programmer to evaluate the performance of spring code/logic.

In spring environment for event handling

Source object: ApplicationContext container

Event class: ApplicationEvent

Event Listner: ApplicationListner

Event handling methods: onApplicationEvent(-) //executes when container started or stopped.

Ex-app:

Keep any application ready having the support of Application context container.

Develop the javaclass for event handling

Configure the above Listner class in applicationContext.xml

```
<bean class="com.nt.event.IOCAMOListner"/>
```

Create ApplicationContext container and stop that in the client application.

IOCPOMListner.java:

```

package com.nt.event;
public class IOCPOMListner implements ApplicationListner{
    private Long end;
    private Long start;
    public void onApplicaitonEvent (ApplicationEvent ae){
        If (ae.toString().indexOf("Refreshed")!=-1){
            start=System.currentTimeMillis();
        }
        Else
            end=System.currentTimeMillis();
        System.out.println("Container acting time" +(end-start) +"ms");
    }
}

```

ClientApp.java:

```

FileSystemXmlApplicationContext cTransactionmanagement =new
FileSystemXmlApplicationContext ("src/com/nt/configures/applicationContext.xml");
.....
.....
cTransactionmanagement .close(); //close the container

```

Note1: cTransactionmanagement .getBean(-)/Factory.getBean(-) gives Bean class object based on given bean id.

Note2: cTransactionmanagement .getMessage (-,-) gives locale specific message collected from properties based on given "Key".

Where to use BeanFactory? Where to use ApplicationContext?

Ans: If we are developing small scale applications likes mobile applications, EmbeddedSystem Applications having the support of spring then use BeanFactory (Here few extra KBs are also matters towards application size)

If we are developing enterprise applications (like web applications, Distributed applications) then use ApplicationContext (always prefer working with ApplicationContext)

Similarities and difference between bean BeanFactory and ApplicationContext container?

Operation container	BeanFactor	ApplicationContext
Bean instantiation	Yes	Yes
Bean wiring	Yes	Yes
Auto Wiring	Yes	Yes
Properties File Support	Yes	Yes
l18n	no	Yes
Eventhandling	no	Yes
Automatic Registration of bean postprocessor	no	Yes
Automatic Registration of bean preprocessor	no	Yes
Stop/closing Container	no	Yes

Bean Life Cycle:

It talks about spring bean Class object creation to destruction.

In servlet lifecycle we use following lifecycle method

init(-)

service (-,-)

destroy ()

In ordinary class we use constructor, finalize () methods to place initialization and uninitialization logic.

SpringBean allows two lifecycle methods.

init() method:

Executes after all dependency inject. So can be used to verify whether injection are done with properly or not with appropriate value.

destroy() Method:

Executes where IOC container is about to destroy our bean class object. Useful to place uninitialization logics.

(Where these methods can be custom name)

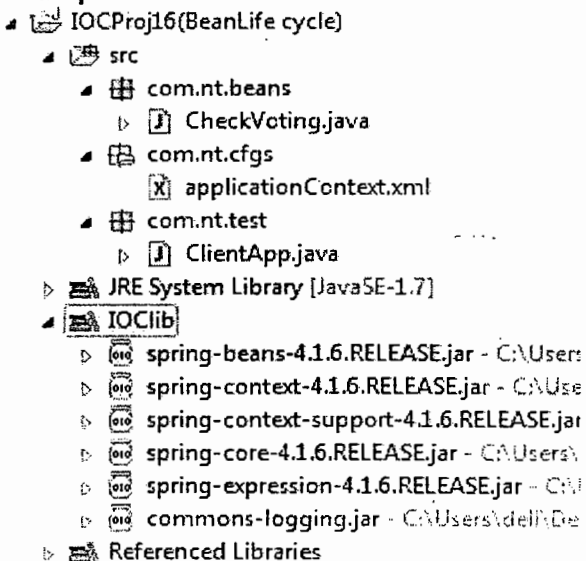
In spring we can perform spring lifecycle operations in three ways.

- 1) Declarative Approach
- 2) Programming Approach
- 3) Annotation Approach

Declarative Approach (xml):

Allows to configure two user-defined methods as bean lifecycle methods. These methods are called custom init methods, custom destroy methods. These methods return type should be void and must not contain parameters.

We must configure the above methods as lifecycle methods using init-method, destroy-method attributes of <bean> tag.

Example:**applicationContext.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:c="http://www.springframework.org/schema/c"
```

```
xmlns:p="http://www.springframework.org/schema/p"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="cv" class="com.nt.beans.CheckVoting" init-method="check" destroy-
method="myDestroy">
<property name="name" value="ravi"/>
<property name="addrs" value="hyd"/>
<property name="age" value="30"/>
</bean>
</beans>
```

CheckVoating.java

```
package com.nt.beans;
import org.springframework.beans.factory.DisposableBean;
import org.springframework.beans.factory.InitializingBean;
public class CheckVoting implements InitializingBean, DisposableBean {
    private String name, addrs;
    private int age;
    public void setName(String name) {
        System.out.println("setName(-)");
        this.name = name;
    }
    public void setAddrs(String addrs) {
        System.out.println("setAddrs(-)");
        this.addrs = addrs;
    }
    public void setAge(int age) {
        System.out.println("setAge(-)");
        this.age = age;
    }
    public void check(){
        System.out.println("Bean:custom-init()");
        if(name==null || age<=0)
            throw new IllegalArgumentException("name, age must set with values");
    }
    public String checkVotingElgibility(){
        if(age>=18){
            return name+" u r elgible to vote ";
        }
        else{
            return name+" u r not elgible vote";
        }
    }
    @Override
    public void destroy() throws Exception {
        System.out.println("Bean:destroy()");
        name=null;
        age=0;
    }
}
```

```

        addrs=null;
    }
    @Override
    public void afterPropertiesSet() throws Exception {
        System.out.println("Bean:AfterPropertiesSet()");
        if(name==null || age<=0)
            throw new IllegalArgumentException("name, age must set with values");
    }
    public void myDestroy(){
        System.out.println("Bean:custom Destroy()");
        name=null;
        age=0;
        addrs=null;
    }
}

```

Client.java

```

package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.CheckVoting;
public class ClientApp {
    public static void main(String[] args) {
        // Create IOC container
        ApplicationContext cTransactionmanagement =new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //get Bean
        CheckVoting cv=(CheckVoting)cTransactionmanagement.getBean("cv");
        System.out.println(cv.checkVotingEligibility());
        ((FileSystemXmlApplicationContext)cTransactionmanagement).close();
        /* BeanFactory factory=new XmlBeanFactory(new
        FileSystemResource("src/com/nt/configures/applicationContext.xml"));
        CheckVoting cv=(CheckVoting)factory.getBean("cv");
        System.out.println(cv.checkVotingEligibility());
        ((XmlBeanFactory)factory).destroySingletons(); */
    }
}

```

In custom init method we generally don't write initialization logic because it won't execute after Bean instantiation it executes after compiling all injection.

While working with ApplicationContext container when we close the container by calling cTransactionmanagement.close(). The container destroys bean class objects destroy bean class objects when these task is about to happen custom destroy method will execute. While working with BeanFactory container we can use factory.destroySingletons() method to destroy all singleton scope bean class objects.

Limitations of Declarative approach of bean Lifecycle:

If you forget to configure lifecycle method in xml file, they will not be executed.

Identifying lifecycle method is very complex while dealing with predefined, third party supplied classes as spring beans.

Bean LifeCycle programmatic approach:

Here our spring bean class must implement `InitializingBean(i)` and `DisposableBean(i)` given by spring api (Bean class becomes api dependent ---->(invasive))

`InitializingBean(i)` gives `afterPropertySet()`, `DisposableBean(i)` gives `destroy()` method.

We use `afterPropertySet()` method as alternative to custom `init()` and `destroy()` as custom destroy method.

Sample Code:

```
public class checkVoting implements InitializingBean, DisposableBean
{
    private int age;
    private String name, address;
    //setters
    .....
    .....
    public void afterPropertySet()
    {
        If (age<=0 || name==null)
        {
            Throw new IllegalArgumentException("name age must be set");
        }
        public void destroy () {
            age=0;
            name=null;
            address=null;
        }
        .....
        .....
    }
}
```

Note:

There is no need of configure any life cycle methods in xml spring bean configure.

If we place Bean LifeCycle configures in both programmatic approach and declarative approach then what happen?

Ans: Custom `init()` method executes after `DisposableBeans destroy ()` method.

Note1: It is recommended to use 1 approach of bean lifecycle at a time.

Note2: Programmatic approach makes our springBean class as spring api dependent most of spring api supplied class are given based on programmatic approach of bean lifecycle.

Note3: Custom `init()` and `InitializingBean's afterPropertySet()` is useful to set weather imp bean properties are injected with values or not.

Custom `destroy ()`, `DisposableBean's destroy ()` is useful to nullfy bean properties or to releases non-java resources that are associated with Bean.

Bean lifecycle Annotation Approach:

```
package com.nt.beans;
import javax.annotation.PostConstruct;
import javax.annotation.PreDestroy;
import javax.inject.Named;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Scope;
@Named("vt")
@Scope("singleton")
public class Voter {
    @Value("raja")
    private String name;
    @Value("30")
    private int age;
    @PostConstruct
    public void myInit(){
        System.out.println("Voter:myInit()");
        if(name==null || age<=0){
            throw new IllegalArgumentException(" name,age must be set");
        }
    }
    @PreDestroy
    public void myDestroy(){
        System.out.println("Voter:myDestroy()");
        name=null;
        age=0;
    }
    public String checkVotingEligibility(){
        if(age<18)
            return name+" u r not eligible to vote";
        else
            return name+"u r eligible to vote";
    }
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
    <context:component-scan base-package="com.nt.beans"/>
</beans>
```

client.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.Voter;
public class ClientApp {
    public static void main(String[] args) {
        // create IOC Container
        ApplicationContext cTransaction management =new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //get Bean
        Voter vt=cTransaction management .getBean("vt",Voter.class);
        String status=vt.checkVotingElgibility();
        System.out.println("Result::"+status);
        ((FileSystemXmlApplicationContext)cTransaction management ).close();
    }
}
```

Conclusion on Bean LifeCycle:

While working with user-defined spring bean prefer declarative approach or annotation approach.
While working with third party API supplied classes as spring bean use Declarative approach.
While working with spring api supplied as spring bean 1st check for programmatic approach implementation, if not found the go for declarative approach.

Aware injection/interface injection/contextual dependency lookup:

If underlying container assigns or injects objects/value to target class only when target class implementa/extends certain interface/class is called contextual dependency lookup.

Servlet container assigns config object to our servlet class only when our servlet class implements javax.servlet.Servlet 1). Directly or indirectly 2)Contextual dependency lookup

If one class needs another class then we go for either dependency lookup or dependency injection. If target class wants to use dependency object in all the methods then go for dependency injection. If target class wants to use dependent object in specific methods then go for dependency lookup.

If we perform dependency lookup in regular fashion we need to create one more IOC container in target class to get Dependency class object, so instead of creating separate IOC container ("Client Application started") to inject BeanFactory or applicationContext object by implementing BeanFactory aware or we can use that injected object to get dependent class object.

Here underlying container is injecting object/value only when target class implements xxxAware interface. So it is called Awareinjection or interface injection or contextualdependency lookup (CDL).

XxxAware interface are:

BeanNameAware----->To injection current beanid

BeanFactoryAware----->To inject BeanFactory object

ApplicationContextualAware----->To inject ApplicationContext object.

 IOCProj17(Aware Injection or InterfaceInjection)

- src
 - com.nt.beans
 - Engine.java
 - Viechle.java
 - com.nt.cfgs
 - applicationContext.xml
 - com.nt.test
 - ClientApp.java
- JRE System Library (JavaSE-1.7)
- IOCLib
 - spring-beans-4.1.6.RELEASE.jar - C:\Users\dell
 - spring-context-4.1.6.RELEASE.jar - C:\Users\dell
 - spring-context-support-4.1.6.RELEASE.jar - C
 - spring-core-4.1.6.RELEASE.jar - C:\Users\dell
 - spring-expression-4.1.6.RELEASE.jar - C\Use
 - commons-logging.jar - C:\Users\dell\Desktop

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="engg1" class="com.nt.beans.Engine"/>
<bean id="viechle" class="com.nt.beans.Viechle">
  <property name="engglId">
    <idref bean="engg1"/>
  </property>
</bean>
</beans>
```

Viechle.java

```
package com.nt.beans;
import org.springframework.beans.BeansException;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.BeanFactoryAware;
import org.springframework.beans.factory.BeanNameAware;
import org.springframework.context.ApplicationContext;
import org.springframework.context.ApplicationContextAware;
public class Viechle implements ApplicationContextAware{
  private String engglId;
  private ApplicationContext cTransactionmanagement ;
  public void setApplicationContext(ApplicationContext cTransactionmanagement )
    throws BeansException {
    System.out.println("Viechle:setApplicationContext(-)");
    this.cTransaction management =cTransactionmanagement ;
  }
  public Viechle(){
    System.out.println("Viechle:0-param constructor");
  }
}
```

```
}  
public void setEnggId(String enggId){  
    System.out.println("Viechle:setEnggId");  
    this.enggId=enggId;  
}  
public void move(){  
    Engine engine=cTransaction management .getBean(enggId,Engine.class);  
    engine.start();  
    System.out.println("Viechle moved for journey");  
}  
}
```

Engine.java

```
package com.nt.beans;  
public class Engine{  
    public Engine(){  
        System.out.println("Engine:0-param constructor");  
    }  
    public void start(){  
        System.out.println("Engg started");  
    }  
}
```

Client.java

```
package test;  
import org.springframework.context.ApplicationContext;  
import org.springframework.context.support.FileSystemXmlApplicationContext;  
import com.nt.beans.Viechle;  
public class ClientApp {  
    public static void main(String[] args) {  
        ApplicationContext cTransactionmanagement =new  
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");  
        Viechle viechle=cTransactionmanagement .getBean("viechle",Viechle.class);  
        viechle.move();  
    }  
}
```

Note: The IOC container that is created by creating BeanFactory object. Internally maintain bean id's BeanFactory object.

The IOC container that created by creating ApplicationContext object internally maintains bean ids, BeanFactory object, ApplicationContext objects and etc..

All these objects can be injected to any class through Aware injection.

FactoryBean:

The class that is capable of creating and returning objects is called Factory class.

In spring FactoryBean is "selfless" bean i.e it doesn't gives its own class object, it always gives one resultant object.

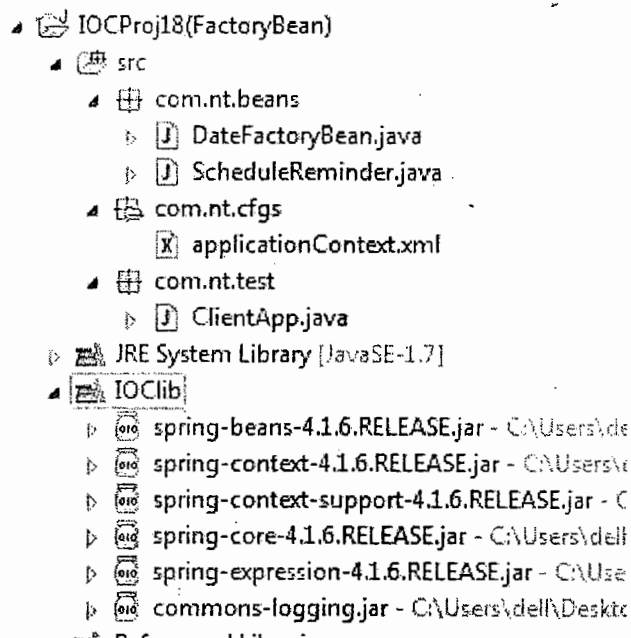
To develop class as FactoryBean class the class must implement the spring API's FactoryBean (i) and should provide definition for three methods.

getObject ()---->create and returns resultant object.

getObjectType()----->returns java.lang.Class object holding resultant object class name.

isSingleton ()----->Specifies whether resultant object should be taken as singleton object (True) or prototype object (false)

The Bean that gets objects from jndi registry based on given jndi name will be developed as Factory Bean.



DateFactoryBean.java

```
package com.nt.beans;
import java.util.Date;
import org.springframework.beans.factory.FactoryBean;
public class DateFactoryBean implements FactoryBean<Date>{
    private int year;
    private int month;
    private int date;
    public DateFactoryBean(int year, int month, int date) {
        System.out.println("DateFactoryBean:3-param constructor");
        this.year = year;
        this.month = month;
        this.date = date;
    }
    //@Override
    public Date getObject() throws Exception {
        System.out.println("DFB:getObject()");
        return new Date(year,month,date);
    }
    //@Override
    public Class<?> getObjectType() {
```

```
        System.out.println("DFB:getObjectType()");
        return Date.class;
    }

    // @Override
    public boolean isSingleton() {
        System.out.println("DFB:isSingleton()");
        return true;
    }
}
```

ScheduleReminder.java

```
package com.nt.beans;
import java.util.Date;
public class ScheduleReminder {
    private Date date;
    public void setDate(Date date) {
        this.date = date;
    }
    @Override
    public String toString() {
        return "ScheduleReminder [date=" + date + "]";
    }
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="dfb" class="com.nt.beans.DateFactoryBean">
        <constructor-arg value="110"/>
        <constructor-arg value="10"/>
        <constructor-arg value="20"/>
    </bean>
    <bean id="reminder" class="com.nt.beans.ScheduleReminder">
        <property name="date" ref="dfb"/>
    </bean>
</beans>
```

Client.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.ScheduleReminder;
public class ClientApp {
    public static void main(String[] args) {
```

```
//create IOC Container
ApplicationContext cTransaction management =new
FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
// get Bean
ScheduleReminder reminder=cTransaction management
.getBean("reminder",ScheduleReminder.class);
System.out.println(reminder);
}
```

How FactoryBean is able to give another object (Resultant Object)?

IOC Container creates FactoryBean class object normally based on configurations done in spring bean configuration file, but before keeping that object in a scope, IOC container checks whether the class of the object is implementing FactoryBean() to get Resultant object and calls isSingleton() to keep Resultant object as singleton scope or prototype scope object with the bean id given for FactoryBean (Xmlfile).

Method Replacement:

In normal classes if you want to replace existing logic of B. method with new logics, we have 2 options.

Comment the existing logic and write the new logic in the same b.method.

Develop the sub class and override b.method with new logics.

In the above model we need to modify source code to revert back to original logics if we write new logics on experimental basis or if new logics the caution is failed in production environment at client side.

The another problem with above approach is reverting back to original b.logic is complex when source code is not exposed (.java code).

In retail industry offers will be placed for certain period, once the offer is over standard prices will be charged. To execute new logics during offer period and to revert back to original logics once the offer is over without touching the source code can't use the above models.

To overcome the above problems use method replace/replacement concept of spring. For this we need to develop separate class implementing MethodReplacer (i) and we need to write new logics in reimplement (-,-,-) method.

Public object reimplement (object obj, Method method, Object [] args)

Object obj---->gives the access to original bean class object.

Method method---->gives the access to original method that we want to replace.

Object [] args---->gives access original method args.

We need to configure method replace class as spring bean and we need the IOC container to execute reimplement (-,-,-) when original method is called, for this we need to use

<replaced-method name="....." (Original Method) replacer="....." (bean id method replacer class)/>

- ▶ IOCProj19(Method Replacer)
 - ▶ src
 - ▶ com.nt.beans
 - ▶ Bank.java
 - ▶ IntrRateReplacer.java
 - ▶ com.nt.cfgs
 - ▶ applicationContext.xml
 - ▶ com.nt.test
 - ▶ ClientApp.java
 - ▶ JRE System Library [JavaSE-1.7]
 - ▶ IOCLib
 - ▶ spring-beans-4.1.6.RELEASE.jar - C:\Users\...
 - ▶ spring-context-4.1.6.RELEASE.jar - C:\User...
 - ▶ spring-context-support-4.1.6.RELEASE.jar
 - ▶ spring-core-4.1.6.RELEASE.jar - C:\Users\Ad...
 - ▶ spring-expression-4.1.6.RELEASE.jar - CAU...
 - ▶ commons-logging.jar - C:\Users\del\NDer...

Bank.java

```
package com.nt.beans;
public class Bank {
    public float calcIntrAmt(float pamt,float time){
        System.out.println("Bank:calcIntrAmt(-,-)");
        return pamt*time*2.0f/100;
    }
    public float calcIntrAmt(){
        return 123.0f;
    }
}
```

IntrRateReplacer.java

```
package com.nt.beans;
import java.lang.reflect.Method;
import java.util.Arrays;
import org.springframework.beans.factory.support.MethodReplacer;
public class IntrRateReplacer implements MethodReplacer {
    private float rate;

    public IntrRateReplacer() {
        System.out.println("IntrRateReplacer:0-param constructor");
    }
    public void setRate(float rate) {
        this.rate = rate;
    }
    @Override
    public Object reimplement(Object bean, Method method, Object[] args)
        throws Throwable {
        System.out.println("IntrRateReplacer:reimplement(-,-,-)");
        System.out.println("bean class"+bean.getClass());
    }
}
```

Naresh i Technologies, Opp. Satyam Theatre, Ameerpet, Hyd, Ph: 040-23746666, www.fb.com/nareshit ::82::

Internals:

IOC container creates sub class for main Bean class (Bank) like "Bank\$\$CGLIB....." By using CGLib library internally, In this class business method (calcIntrAmt()) will be overridden in this method definition the object of method replacer class (IntrRateReplacer) will be accessed and reimplement method will be called.

Due to this new logic will be executed when client application cTransaction management .getBean() or factory.getBean () the above generated subclass object will be returned.

Sample Code:

```
public class Bank$$CGLIB extends Bank implements AppliactionContextAware
{
    private ApplicationContexte Transactionmanagement ;
    public void setApplicationContext (ApplicationContext eTransaction management )
    {
        This.eTransaction management =eTransaction management ;
    }
    public float calcIntrAmt (-,-,-)
    {
        IntrRateReplacer ir=eTransaction management .getBean ("mr", IntrRateReplcer.class);
        float retval=ir.reimplement (-,-,-);
        return retval;
    }
}
```

Note Important :

The method that you want to replace should not be final, static (this methods can't be overridden in sublcass)

Eventhough you can write replacement logics for multiple method in single reimplement method by using if conditions is recommended to take separate method replacer for every class.

Explain various injections supported by spring?

Setter injection

Constructor injection

Aware injection/context dependency lookup/ Interface injection

Method injection (method replacer)

Lookup method injection:

Problem with Lookup method injection:

- ▲  IOCProj20(LMInjection-problem)
 - ▲  src
 - ▲  com.nt.beans
 - ▷  MyServletContainer.java
 - ▷  RequestHandler.java
 - ▲  com.nt.cfgs
 - ▷  applicationContext.xml
 - ▲  com.nt.test
 - ▷  ClientApp.java
 - ▷  JRE System Library [JavaSE-1.7]
 - ▲  IOCLib
 - ▷  spring-beans-4.1.6.RELEASE.jar - C:\Users\c
 - ▷  spring-context-4.1.6.RELEASE.jar - C:\Users\c
 - ▷  spring-context-support-4.1.6.RELEASE.jar -
 - ▷  spring-core-4.1.6.RELEASE.jar - C:\Users\de
 - ▷  spring-expression-4.1.6.RELEASE.jar - C:\Us
 - ▷  commons-logging.jar - C:\Users\de\I\Deskl

MyServletContainer.java

```
package com.nt.beans;

public class MyServletContainer {
    private RequestHandler handler;
    public MyServletContainer() {
        System.out.println("MyServletContainer:0-param constructor");
    }
    public void setHandler(RequestHandler handler) {
        this.handler = handler;
    }

    public void processRequest(String data){
        handler.handle(data);
    }
}
```

RequestHandler.java

```
package com.nt.beans;

public class RequestHandler {
    private static int count;
    public RequestHandler() {
        count++;
        System.out.println("RequestHandler: 0-param constructor");
    }
    public void handle(String data){
        System.out.println("Handling Request "+count+" with data "+data);
    }
}
```

applicationContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="rh" class="com.nt.beans.RequestHandler" scope="prototype"/>
    <bean id="container" class="com.nt.beans.MyServletContainer" scope="singleton">
        <property name="handler" ref="rh"/>
    </bean>
</beans>

```

Client.java

```

package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.MyServletContainer;
public class ClientApp {
    public static void main(String[] args) {
        // IOC Container
        ApplicationContext cTransaction management =new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //get Bean
        MyServletContainer container=cTransaction management
        .getBean("container",MyServletContainer.class);
        container.processRequest("hello");
        container.processRequest("hai");
    }
}

```

When prototype scope bean is dependent to singleton scopeBean somehow the prototype scope bean also acts as singleton scope bean.

Eg: All request coming to webApplication will be handled by single servlet container/ web container (Singleton).

This container uses separate RequestHanlder to process each request so servlet container is target class and RequestHandler is dependent class, moreover ServletContainer should be taken as singleton scopebean and request handler should taken as prototype scope, but because of above problem "RequestHandler" also acts as the singleton scope bean.

```

<bean id="container" class="com.nt.beans Myservletcontainer" scope="singleton">
<property name="handler" ref="rh"/>
</bean>
<bean id="rh" class="com.nt.beans.RequestHandler" scope="prototype"/>

```

Target Bean Scope	Dependent Bean Scope	Resultant Scope of Dependent Bean
Prototype	Singleton	Singleton (OK)
Singleton	Singleton	Singleton (OK)
Prototype	Prototype	Prototype (OK)
Singleton	Prototype	Singleton (Not OK) (use dependency lookup or -Lookup method injection)

We should not think about dependency injection (constructor/setter) in the following situations.

If dependent bean class object is required in TargetBean only in specific methods

If target Bean scope is singleton and Dependent Bean scope is prototype (refer above table)

In the above situation prefer using Dependency lookup

If we go for traditional dependency lookup code we need to create extra IOC container in the above methods of target class (which is bad practice)

By using aware injection we can avoid that extra IOC container towards dependency lookup operational. But it kills the non-invasive behavior of spring App because target bean should implement XxxAware injection.

To overcome the above two problems we can use Lookup method Injection (LMI) which says performs "dependency lookup" but container should write code for it (not the programmer).

To perform Lookup Method Injection we need to consider the following points:

Target bean class should have abstracted method returning Dependent Bean class (for this target bean class must be taken as abstract class)

We must configure the above abstract method with targetBean class configure by using <lookup-method> tag

Note: We can configure abstract class as spring bean only when <lookup-method> tag is placed for lookup method injection otherwise not possible.

Spring IOC container generates subclass for targetBean class and implements the abstract method that class to get and Return dependent bean class object. For this it internally uses CGLIB Library.

In spring 2.x,3.x we need to supply CGLIB Library separately where as in spring 4.x, not required.

Lookup method injection use cases

servletContainer (Singleton)<----->Request Handler (prototype)

Manager(Singleton)<----->Customer (prototype)

LoanApprver (Singleton)<----->LoanDetails (prototype)

Final solution application Program

- ▲  IOCProj20(LMInjection-problem)
 - ▲  src
 - ▲  com.nt.beans
 - ▷  MyServletContainer.java
 - ▷  RequestHandler.java
 - ▲  com.nt.cfgs
 - ▷  applicationContext.xml
 - ▲  com.nt.test
 - ▷  ClientApp.java
 - ▷  JRE System Library [JavaSE-1.7]
 - ▲  IOClib
 - ▷  spring-beans-4.1.6.RELEASE.jar - C:\Users\c
 - ▷  spring-context-4.1.6.RELEASE.jar - C:\Users\c
 - ▷  spring-context-support-4.1.6.RELEASE.jar -
 - ▷  spring-core-4.1.6.RELEASE.jar - C:\Users\de
 - ▷  spring-expression-4.1.6.RELEASE.jar - C:\Us
 - ▷  commons-logging.jar - C:\Users\del\Deski

MyServletContainer.java

```
package com.nt.beans;
import org.springframework.beans.BeansException;
import org.springframework.context.ApplicationContext;
import org.springframework.context.ApplicationContextAware;
public abstract class MyServletContainer{
    public MyServletContainer() {
        System.out.println("MyServletContainer:0-param constructor");
    }
    public void processRequest(String data){
        //get Dependent Bean
        RequestHandler handler=getRequestHandler();
        handler.handle(data);
    }
    public abstract RequestHandler getRequestHandler();
}
```

RequestHandler.java

```
package com.nt.beans;
public class RequestHandler {
    private static int count;
    public RequestHandler() {
        count++;
        System.out.println("RequestHandler: 0-param constructor");
    }
    public void handle(String data){
        System.out.println("Handling Request "+count+" with data "+data); } }
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="rh" class="com.nt.beans.RequestHandler" scope="prototype"/>
  <bean id="container" class="com.nt.beans.MyServletContainer" scope="singleton">
    <lookup-method name="getRequestHandler" bean="rh"/>
  </bean>
</beans>
```

Client.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.MyServletContainer;
public class ClientApp {
    public static void main(String[] args) {
        // IOC Container
        ApplicationContext cTransaction management =new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.
xml");
        //get Bean
        MyServletContainer container=cTransaction management
        .getBean("container",MyServletContainer.class);
        container.processRequest("hello");
        container.processRequest("hai");
        System.out.println(container.getClass());
    }
}
```

BeanPostProcessor (Bpp):

Assigning some values to bean properties after injection is called Post processing. We can do this post processing inside the bean class and specific o bean class by using customInit method or InitializingBean's after propertySet() of bean class.

If we want to apply common post processing logic for multiple beans that to being from outside the bean class then we can use BeanPostProssor class.

The Bean post processor class executes for all beans that are configured in spring bean configure file.

To develop Beanpost processor we need to take a class that implements BeanPostProcessor (i) and should provide implementation for 2 methods.

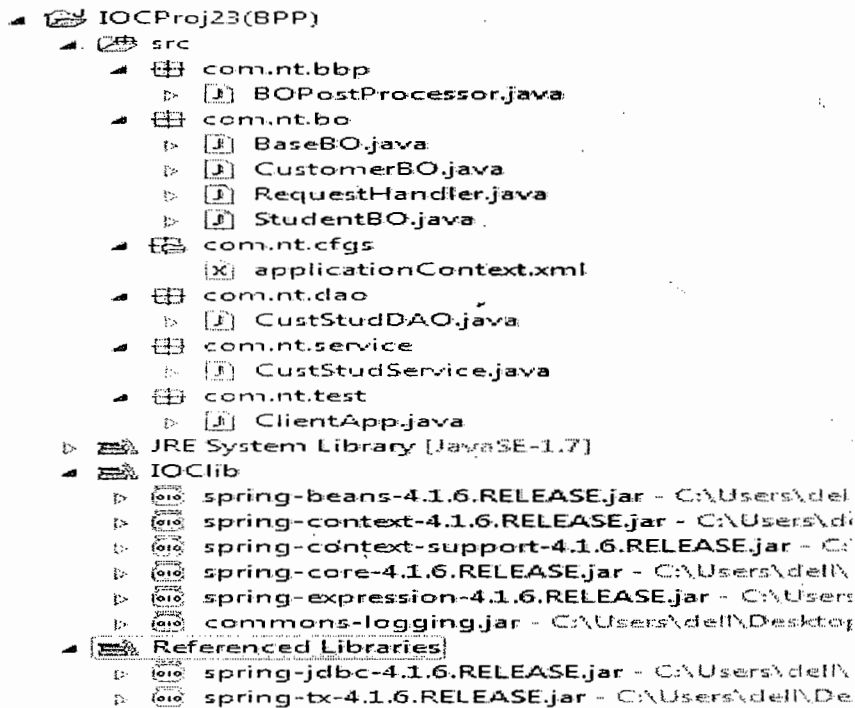
PostProcessorBeforeInitialization (Object bean, Spring Beanid)

Executes before afterPropertySet() fro every beans class.

PostProssorAfterInitialization (Object bean, Spring bean id)

Executes after custom init() for every bean class.

The concept is useful to develop custom post process/initialization logic for all beans in one class.

**BaseBO.java**

```

package com.nt.bo;
import java.util.Date;
public abstract class BaseBO {
    private int id;
    private String name;
    private Date doj;
    public BaseBO() {
        System.out.println("BaseBO:0-param constructor");
    }
    public void setDoj(Date doj) {
        this.doj = doj;
    }
    public void setId(int id) {
        this.id = id;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getId() {
        return id;
    }
    public String getName() {
        return name;
    }
    public Date getDoj() {
        return doj;
    }
}

```

CustomerBO.java

```
package com.nt.bo;
public class CustomerBO extends BaseBO {
    private float billAmt;
    public CustomerBO() {
        System.out.println("CustomerBO:0-param constructor");
    }
    public float getBillAmt() {
        return billAmt;
    }
    public void setBillAmt(float billAmt) {
        this.billAmt = billAmt;
    }
}
```

StudentBO.java

```
package com.nt.bo;
public class StudentBO extends BaseBO {
    private String course;
    public StudentBO() {
        System.out.println("StudentBO:0-param constructor");
    }
    public String getCourse() {
        return course;
    }
    public void setCourse(String course) {
        this.course = course;
    }
}
```

BOPostProcessor.java

```
package com.nt.bpp;
import java.util.Date;
import org.springframework.beans.BeansException;
import org.springframework.beans.factory.config.BeanPostProcessor;
import com.nt.bo.BaseBO;
public class BOPostProcessor implements BeanPostProcessor {
    public BOPostProcessor() {
        System.out.println("BPP: 0-param constructor");
    }
    @Override
    public Object postProcessBeforeInitialization(Object bean, String bid)
        throws BeansException {
        System.out.println("BPP:BeforeInitialization(-,-)");
        if(bean instanceof BaseBO){
            ((BaseBO) bean).setDoj(new Date());
        }
    }
}
```

```
        return bean;
    }

    return bean;
}

@Override
public Object postProcessAfterInitialization(Object bean, String bid)
    throws BeansException {
    System.out.println("BPP:AfterInitialization(-,-)");
    return bean;
}
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="stBO" class="com.nt.bo.StudentBO" scope="prototype"/>
    <bean id="custBO" class="com.nt.bo.CustomerBO" scope="prototype"/>
    <bean id="dao" class="com.nt.dao.CustStudDAO" scope="singleton"/>
    <bean id="service" class="com.nt.service.CustStudService" scope="singleton">
        <property name="dao" ref="dao"/>
        <lookup-method name="getStudentBO" bean="stBO"/>
        <lookup-method name="getCustomerBO" bean="custBO"/>
    </bean>
    <bean id="bpp" class="com.nt.bpp.BOPostProcessor"/>
</beans>
```

CustStudDAO.java

```
package com.nt.dao;
import com.nt.bo.CustomerBO;
import com.nt.bo.StudentBO;
public class CustStudDAO {
    public CustStudDAO() {
        System.out.println("DAO:0-param constructor");
    }

    public void insertData(StudentBO stbo){
        System.out.println("Student Data is");
        System.out.println(stbo.getId()+" "+stbo.getName()+" "+stbo.getCourse()+"
stbo.getDoj());
    }
    public void insertData(CustomerBO custbo){
        System.out.println("Customer Data is");
        System.out.println(custbo.getId()+" "+custbo.getName()+" "+custbo.getBillAmt()+"
custbo.getDoj());
    }
}
```

CustStudService.java

```
package com.nt.service;
import com.nt.bo.CustomerBO;
import com.nt.bo.StudentBO;
import com.nt.dao.CustStudDAO;
public abstract class CustStudService {
    private CustStudDAO dao;

    public CustStudService() {
        System.out.println("Service : 0-param constructor");
    }

    public abstract StudentBO getStudentBO();
    public abstract CustomerBO getCustomerBO();
    public void setDao(CustStudDAO dao) {
        this.dao = dao;
    }

    public void processStudent(int id,String name,String course){
        System.out.println("Procesing Student Info");
        StudentBO stbo=getStudentBO();
        stbo.setId(id); stbo.setName(name); stbo.setCourse(course);
        //use dao
        dao.insertData(stbo);
    }

    public void processCustomer(int id,String name,float billAmt){
        System.out.println("Procesing Customer Info");
        CustomerBO custbo=getCustomerBO();
        custbo.setId(id); custbo.setName(name); custbo.setBillAmt(billAmt);
        //use dao
        dao.insertData(custbo);
    }
}
```

Client.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.CustStudService;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext cTransaction management =new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        /*      BeanFactory factory=new XmlBeanFactory(new
        FileSystemResource("src/com/nt/configures/applicationContext.xml"));
        //register BeanPostProcessor
        BOPostProcessor bpp=new BOPostProcessor();
        ((ConfigurableListableBeanFactory)factory).addBeanPostProcessor(bpp); */
    }
}
```

```

        //get Bean
        CustStudService service=cTransaction management
        .getBean("service",CustStudService.class);
        //call b.methods
        service.processCustomer(1001,"raja",9000);
        service.processStudent(101,"karan","java");
        service.processCustomer(1002,"raja",9000);
        service.processStudent(102,"karan","java");
    }
}

```

In real time beanpost processor will be use to place common post process initialization logic that is required for multiple Beans like assigning system date as date of joining or last modification date, date of updation and etc.....

This concept is very useful towards auditing activity.

While working with Application Context container if we just configure BeanPostProcessor class in Xml file as spring Bean the container automatically recognize and use that BeanpostProcessor.

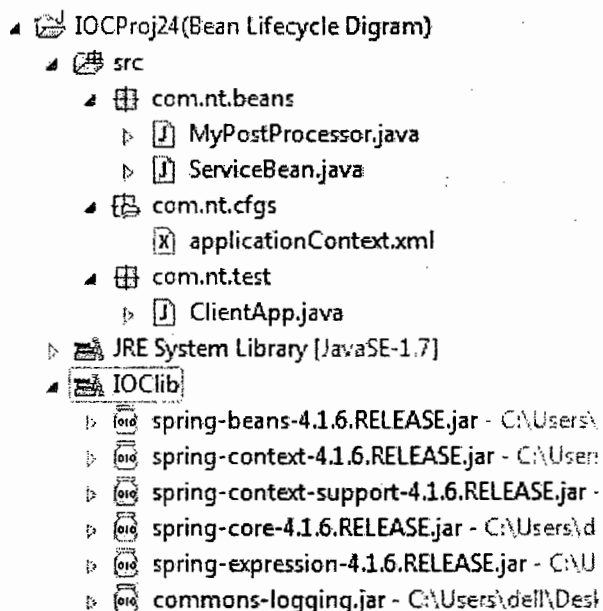
While working with BeanFactory container we must have to register BeanPostProcessor explicitly as shown below

```

BOPostProcessor bpp=new BoPostProcessor (); //Bpp Lass
((ConfigurableListableBeanFactory)Factory).addBeanPostProcessorcbpp);

```

Bean Life Cycle diagram with respect to BeanFactoryContainer:



```

package com.nt.beans;
import org.springframework.beans.BeansException;
import org.springframework.beans.factory.config.BeanPostProcessor;
public class MyPostProcessor implements BeanPostProcessor {
    @Override
    public Object postProcessAfterInitialization(Object bean, String id)
        throws BeansException {
        System.out.println("BPP:postProcessAfterInitialization");
        return bean;
    }
}

```

```
}  
@Override  
public Object postProcessBeforeInitialization(Object bean, String id)  
    throws BeansException {  
    System.out.println("BPP:postProcessBeforeInitialization");  
    return bean;  
}  
}
```

```
package com.nt.beans;  
import org.springframework.beans.BeansException;  
import org.springframework.beans.factory.config.BeanPostProcessor;  
public class MyPostProcessor implements BeanPostProcessor {  
    @Override  
    public Object postProcessAfterInitialization(Object bean, String id)  
        throws BeansException {  
        System.out.println("BPP:postProcessAfterInitialization");  
        return bean;  
    }  
    @Override  
    public Object postProcessBeforeInitialization(Object bean, String id)  
        throws BeansException {  
        System.out.println("BPP:postProcessBeforeInitialization");  
        return bean;  
    }  
}
```

```
<beans xmlns="http://www.springframework.org/schema/beans"  
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
xsi:schemaLocation="http://www.springframework.org/schema/beans  
http://www.springframework.org/schema/beans/spring-beans.xsd">  
    <bean id="service" class="com.nt.beans.ServiceBean" init-method="myInit" destroy-  
method="myDestroy">  
        <constructor-arg value="hello"/>  
        <property name="msg" value="hello"/>  
    </bean>  
    <bean id="bpp" class="com.nt.beans.MyPostProcessor"/>  
</beans>
```

package test;

```
import org.springframework.context.ApplicationContext;  
import org.springframework.context.support.FileSystemXmlApplicationContext;  
import com.nt.beans.ServiceBean;  
public class ClientApp {  
    public static void main(String[] args) {  
        //create IOContainer  
        ApplicationContext ctx=new  
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");  
        //get Bean
```

```
/*ServiceBean service=ctx.getBean("service",ServiceBean.class);
System.out.println(service.generateWish());*/

((FileSystemXmlApplicationContext)ctx).close();
}
}
```

Bean Life Cycle diagram with respect to ApplicationContext container:

Diagram: class room diagram

Complete Internal flow of core module application:

Programmers creates BeanFactory object.

Given springBean configure file will be located and it will be verified whether documents well formed document or void document .

Creates Inmemory logical IOC container in the jvm's memory.

Reads springBean configure file entries and creates an inmemory metadata having all the configuration.

ClientApp class factory.getBean("id") method.

Loads Bean class from inmemory meta data based on given bean id and instantiates that bean based on constructor injection configures.

Keep Bean class objects in specified scope (if cyclic dependency is found Bean Instantiation fails)

Performs setter injection based on configures.

Performs Aware injection based on the XxxAware interface that are implement.

Calls postProcessBeforeInitialization (-) if BeanPostProcessor is added.

Calls afterPropertySet (-) if initializationBean (i) implemented.

Calls custom init method if "init-method" attribute is specified.

Calls PostProcessAfterInitialization ()

Returns Bean class object to client application.

Note: If ApplicationContext object is create (f) to (n) also takes place without calling getBean(-) for singleton scope beans.

Note: if <lookup-method> or <replaced-method> tags are configure inside <bean> tag then container does not create object given bean class. It creates object for the sub class of the bean class.

BeanFactoryPostProcessors:

BeanPostProcessor is useful to write postProcess initialization logics with multiple beans.

BeanFactoryPostProcessor is useful to add extra configuration by modifying inmemory metadata after the creation of IOC container and before accessing bean object by calling getBean (-)

It acts as extension look/anchor for IOC container to add extra configuration without touching the xml file. (Springbean configuration file) i.e it modifies the inmemory metadata of IOC container dynamically at runtime after IOC container creation and before bean class object is accessed.

To develop custom BeanFactoryProcessor we need to take a class implementing

BeanFactoryPostProcessor (i)

Spring has given lots of pre-define BeanFactoryPostProcessor like propertyPlaceholder configure.

This class modifier in memory metadata by replacing placeholder \${--} with the values created from properties File.

ApplicationContext object based IOC container automatically recognizes the

BeanFactoryPostProcessor that is configured in xml file But for BeanFactory object based IOC container we need to register explicitly.

By default the IOC container of "BeanFactory" can't work with properties file and place holder. But we can make it working by registering "propertyPlaceholderConfigurer" as beanFactory Post processor.

Example:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="pcc" class="org.gpr.prk.beana.Factory.config.PlaceholderConfigure">
<Property name="location" value="com/nt/commons/DBdetails.properties"/>
</bean>
</beans>
```

ClientApp:

```
BeanFactory factory=new XmlBeanFactory (new FileSystemResource
("src/com/nt/configures/applicationContext.xml"));
//get propertyplaceholder configure object (pphc)
BeanFactoryPostProcessor bfpp=factory.getBean ("PPC" PropertyPlaceholderConfigurer.class);
//register
Bfpp.postProcessBeanFactory((configurableList-BeanFactory)Factory);
```

PropertyEditor:

In spring we use setter or constructor injection to assign values to bean properties. In spring Bean configuration file we configure these dependent values as String values, but they will converted into int,long,float,date and etc. values as required for bean properties automatically. For this spring internally uses PropertyEditors to converts spring values to appropriate values.

All property editors are the classes implementing java.beans.propertyEditor (i).

Spring has given multiple built-in PropertyEditors like StringArrayPropertyEditors, ByteArrayPropertyEditor, DateFormatterPropertyEditor and etc....

Understanding Built-in property editors.

-  IOCProj25(PropertyEditor1)
 -  src
 -  com.nt.beans
 - ▷  AadharDetails.java
 -  com.nt.cfgs
 - ▷  applicationContext.xml
 -  com.nt.test
 - ▷  ClientApp.java
 - ▷  JRE System Library [JavaSE-1.7]
 -  IOCLib
 - ▷  spring-beans-4.1.6.RELEASE.jar - C:
 - ▷  spring-context-4.1.6.RELEASE.jar - C:
 - ▷  spring-context-support-4.1.6.RELEASE.jar - C:
 - ▷  spring-core-4.1.6.RELEASE.jar - C:\
 - ▷  spring-expression-4.1.6.RELEASE.jar - C:\
 - ▷  commons-logging.jar - C:\Users\di


```
package com.nt.beans;
import java.io.File;
import java.util.Arrays;
import java.util.Date;
public class AadharDetails {
    private int uid;
    private String name;
    private Date dob;
    private File photo;
    private String[] verifiers;
    public void setUid(int uid) {
        this.uid = uid;
    }
    public void setName(String name) {
        this.name = name;
    }
    public void setDob(Date dob) {
        this.dob = dob;
    }
    public void setPhoto(File photo) {
        this.photo = photo;
    }
    public void setVerifiers(String[] verifiers) {
        this.verifiers = verifiers;
    }
    @Override
    public String toString() {
        return "AadharDetails [uid=" + uid + ", name=" + name + ", dob=" + dob
            + ", photo=" + photo + ", verifiers="
            + Arrays.toString(verifiers) + "]";
    }
}
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="aadhar" class="com.nt.beans.AadharDetails">
<property name="uid" value="101"/>
<property name="name" value="Raja"/>
<property name="dob" value="12/11/1990"/>
<property name="Photo" value="C:/abd/picd.gif"/>
<property name="verifiers" value="sahin,Ramesh,Ravi"/>
</bean>
</beans>
```

Note: In both container (BF,AC) Built-in property editors are automatically registered. Internally uses DateFormatter propertyEditor to convert given date value to java.util.Date class object.

Spring allows to create custom property editor class either by implementing java.beans.PropertyEditor (i) or by extending from PropertyEditorSupport (i)

IntrAmtDetails.java

```
public class IntrAmtDetails
```

```
{
```

```
private float p,t,r;
```

```
//setters
```

```
}
```

IntrAmtCalculator.java

```
public class IntrAmtCalculator
```

```
{
```

```
privator IntrAmtDetails details;
```

```
//setters
```

```
.....}
```

In applicationContext.xml

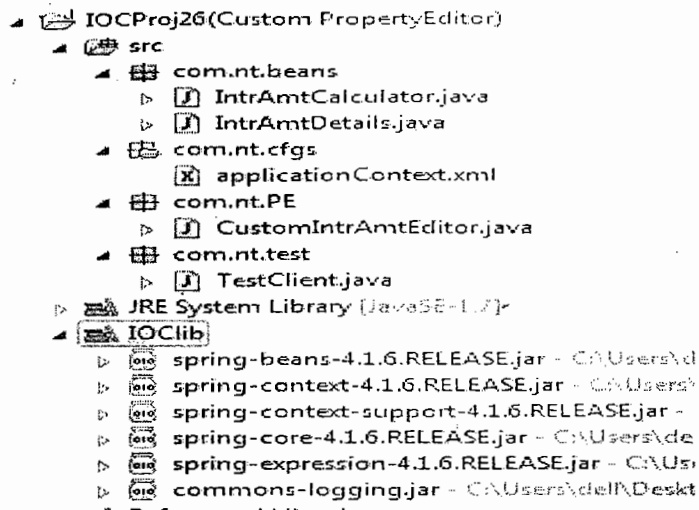
```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="amtdetails" class="pkg.IntrAmtDetails"/>
<property name="P" value="1000"/>
<property name="t" value="10"/>
<property name="r" value="2"/>
</bean>
<bean id="intrCalculator" class="pkg.IntrAmtCalculator">
<property name="details" ref="amtdetails"/>
</bean>
</beans>
```

But we want like this

applicationContext.xml

```
<bean id="intrCalculator" class="pkg.IntrAmtCalculator">
<property name="details" values="1000,10,2"/>
</bean>
```

Here "details" to reference type property but we want to perform simple values based injection which is not possible directly. So must go for custom property editor development.



```
package com.nt.beans;
public class IntrAmtCalculator {
    private IntrAmtDetails details;
    public void setDetails(IntrAmtDetails details) {
        this.details = details;
    }

    public float calcIntrAmt(){
        return (details.getPrinciple()*details.getRate()*details.getTime())/100.0f;
    }
}

package com.nt.beans;
public class IntrAmtDetails {
    private int principle;
    private int time;
    private int rate;
    public void setPrinciple(int principle) {
        this.principle = principle;
    }
    public void setTime(int time) {
        this.time = time;
    }
    public void setRate(int rate) {
        this.rate = rate;
    }
    public int getPrinciple() {
        return principle;
    }
    public int getTime() {
        return time;
    }
    public int getRate() {
        return rate;
    }
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- <bean id="amtDetails" class="com.nt.beans.IntrAmtDetails">
        <property name="principle" value="1000"/>
        <property name="time" value="10"/>
        <property name="rate" value="2"/>
    </bean>
    <bean id="intrCalculator" class="com.nt.beans.IntrAmtCalculator">
        <property name="details" ref="amtDetails"/>
    </bean> -->
    <bean id="intrCalculator" class="com.nt.beans.IntrAmtCalculator" scope="prototype">
        <property name="details" value="1000,10,2"/>
    </bean>
</beans>

package com.nt.PE;
import java.beans.PropertyEditorSupport;
import com.nt.beans.IntrAmtDetails;
public class CustomIntrAmtEditor extends PropertyEditorSupport {
    public CustomIntrAmtEditor() {
        System.out.println("CustomIntrAmtEditor:0-param constructor");
    }
    @Override
    public void setAsText(String text) throws IllegalArgumentException {
        System.out.println("CustomIntrAmtEditor:setAsText(-)");
        // spilt the given value
        int principle=Integer.parseInt(text.substring(0,text.indexOf(", ")));
        int time=Integer.parseInt(text.substring(text.indexOf(",")+1,text.lastIndexOf(", ")));
        int rate=Integer.parseInt(text.substring(text.lastIndexOf(",")+1,text.length()));
        IntrAmtDetails details=new IntrAmtDetails();
        details.setPrinciple(principle); details.setTime(time); details.setRate(rate);
        setValue(details);
    }
}

package test;
import org.springframework.beans.PropertyEditorRegistrar;
import org.springframework.beans.PropertyEditorRegistry;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.config.ConfigurableListableBeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.FileSystemResource;
import com.nt.PE.CustomIntrAmtEditor;
import com.nt.beans.IntrAmtCalculator;
```

```
import com.nt.beans.IntrAmtDetails;
public class TestClient {
    public static void main(String[] args) {
        //Create IOC container
        BeanFactory factory=new XmlBeanFactory(new
        FileSystemResource("src/com/nt/cfgs/applicationContext.xml"));
        ((ConfigurableListableBeanFactory)factory).addPropertyEditorRegistrar(new MyCustomPE());
        //get Bean
        IntrAmtCalculator iCalc=factory.getBean("intrCalculator",IntrAmtCalculator.class);
        System.out.println("Intr Amt"+iCalc.calcIntrAmt());
    } //main
} //class

class MyCustomPE implements PropertyEditorRegistrar{
    @Override
    public void registerCustomEditors(PropertyEditorRegistry registry) {
        registry.registerCustomEditor(IntrAmtDetails.class,new CustomIntrAmtEditor());
    }
}
```

Every IOC container internally maintains one builtin object called propertyEditorRegistry and IOC container expose that object to a class that implements PropertyEditorRegistrar interface as the argument of RegisterCustomEditors() method so that object to register custom property editor by specifying property type and the object of customPropertyEditor.

PropertyEditorRegistrar is an interface that makes IOC container it exposes the internally maintained PropertyEditorRegistry object to its implementation class.

Spring Core Module with Annotations:

Annotations: Data about data and code about code is called Metadata. Annotations are java statements to perform metadata operations in .java files.

Annotations are alternate for xml files to perform metadata, resource configure operations.

Xml files gives flexibility of modification but gives bad performance because the xml parses are heavy to process the xml documents.

Annotations do not provide flexibility of modification but provide good performance.

Syntax: @<Annotation name> (param1=val1....)

Annotations for documentation are introduced from jdk 1.0 and annotations for programming are introduced from jdk 1.5

We can apply annotations at different level like class level, Method level, field level. Param level and etc...

The annotations will be used by compilers, containers, AnnotationProcessors, JRE and etc....

Some annotations provides instructions in the generation of api documents.

Eg: @author, @since and etc...

Some annotations will provide instruction to compiler but no effect will be there at runtime.

Eg: @override (Tells compiler it is inherited method of class)

Some annotations are useful to mark the classes/interfaces as special items.

@Entity makes ordinary JavaBean as Hibernate domain class.

@WebServlet makes class as servlet comp.

Some annotations are useful to generate new classes using tools dynamically based on given class code.

Eg: WebServices.

All the java technologies, Frameworks that are designed based on jdk1.5 are supporting annotations.

Eg: Servlet 3.X+, hibernate 3.2X+, Spring2.X+, EJB3.0+, and etc...

If we perform certain settings by using annotations and xml file the settings done in xml file will override the settings done through annotations.

Spring annotations: (For core module)

Spring 2.0 onwards there is a support for annotation and more annotations are added incrementally in next version.

In spring programming we need to use both xml and annotations together because we can instruct IOC container to recognize the use annotations only through xml file.

We cannot use annotations to make predefined classes as the spring beans because we can't add annotations by opening the source code in those class. To configure such classes as spring beans we need to work with xml support.

B.Note: in annotation based programming

1st copy the extra .xsd file.

Add 1AOP jar

Add <context.annotation-config/tag> (common for all)

Register in RequiredAnnotationBeanPostProcessor class (individual)

Spring 2.0

@Configuration

@Required

@Repository

Spring 2.5

@Autowired

@Qualifier

@Scope

StereoType annotations:

@Component

@Service

@Controller

Spring 3.0

@Bean

@Depends on

@Lazy

@Value

Java config annotations (can be used only with spring3.X)

@PostConstruct

@PreDestruct

@Resource

@Inject

@Named

@Required:

To make dependency injection (setter/injection) mandatory on properties we can use dependency check activity. But this is removed from spring2.5 because it does not allow the programmer to enable the restriction on specific properties. It allows the programmer to enable restriction on group of properties like "simple", object, all"

To overcome this problem we can use @Required

Ex: Robot.java



```
package com.nt.beans;
import org.springframework.beans.factory.annotation.Required;
public class Robot{
    private int id;
    private String type;
    public void setId(int id){
        this.id=id;
    }
    @Required
    public void setType(String type){
        this.type=type;
    }
    //toString()
    @Override
    public String toString() {
        return "Robot [id=" + id + ", type=" + type + "]";
    }
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
<bean id="robot" class="com.nt.beans.Robot">
    <property name="id" value="1001"/>
    <!-- <property name="type" value="Army"/> -->
</bean>
```

```
</bean>
<context:annotation-config/>
<!-- <bean
class="org.springframework.beans.factory.annotation.R
equiredAnnotationBeanPostProcessor"/>-->
</beans>
```

Or

Makes IOC container to recognize the annotation and to provide functionality for the @Required annotations.

ClientApp.java:

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.Robot;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC Container
        ApplicationContext cTransaction management =new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.
xml");
        //getBean
        Robot robo=cTransaction management .getBean("robot",Robot.class);
        System.out.println(robo);
    }
}
```

The BeanPostPorcessor of every annotation contains logic to recognize the annotation and to provide functionality for the annotation. But configuring the Bean postprocessor of every annotation is complex process. To overcome this problem use <context:annotation-config/> tag To work with this tag import schema namespace in the xml.

@Autowired:

Given to perform byType, byName, constructor mode of autowiring. This annotation can be applied at field level, setter method level, constructor level and orbitary method level.

- IOCProj28(@AutoWired ,@Qualifier)
 - ▲ src
 - ▲ com.nt.beans
 - Engine.java
 - Viechle.java
 - ▲ com.nt.cfgs
 - applicationContext.xml
 - ▲ com.nt.test
 - TestClient.java
 - JRE System Library [JavaSE-1.7]
 - ▲ IOCLib
 - spring-beans-4.1.6.RELEASE.jar - C:\U
 - spring-context-4.1.6.RELEASE.jar - C\
 - spring-context-support-4.1.6.RELEASE
 - spring-core-4.1.6.RELEASE.jar - C:\Use
 - spring-expression-4.1.6.RELEASE.jar -
 - commons-logging.jar - C:\Users\dell\

Engine.java

```
package com.nt.beans;
public class Engine{
    private int id;
    private String type;
    //setters
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getType() {
        return type;
    }
    public void setType(String type) {
        this.type = type;
    }
    //toString()
    @Override
    public String toString() {
        return "Engine [id=" + id + ", type=" + type + "]";
    }
}
```

Vehicle.java

```
package com.nt.beans;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
public class Viechle{
    @Autowired
    @Qualifier("e")
```

```
private Engine engg;
/* @Autowired(required=true)
public Viechle(Engine engg){
    System.out.println("1-param constructor");
    this.engg=engg;
}*/
/* @Autowired(required=true)
public void setEngg(Engine engg){
    System.out.println("setEngg");
    this.engg=engg;
}
    @Autowired(required=true)
public void assign(Engine engg){
    System.out.println("assign");
    this.engg=engg;
}*/
@Override
public String toString() {
    return "Viechle [engg=" + engg + "];"
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
    <bean id="eng" class="com.nt.beans.Engine">
        <property name="id" value="11"/>
        <property name="type" value="CRDI"/>
        <qualifier value="e"/>
    </bean>
    <bean id="eng1" class="com.nt.beans.Engine">
        <property name="id" value="12"/>
        <property name="type" value="DDIS"/>
        <qualifier value="e1"/>
    </bean>
    <bean id="viechle" class="com.nt.beans.Viechle"/>
    <context:annotation-config/>
</beans>
```

ClientApp

```

package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.Viechle;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC Container
        ApplicationContext cTransactionmanagement =new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //getBean
        Viechle viechle=cTransactionmanagement.getBean("viechle",Viechle.class);
        System.out.println(viechle);
    }
}

```

While working with <context:annotation-cnfig> tag we need to add spring-app-<version>.Release.jar file as an extra jar file to build path.

Vehicle.java (at settermethod level)

```

public class vehicle
{
    private Engine engg;
    @Autowired (required=true)
    public void setEngg (Engine engg)
    {
        this.engg=engg;
    }
    .....
    .....
}

```

Autowired (required=true)

If dependency bean is not available to perform dependency injection through autowiring exception will be raised when required is true. Default value is "true". Null will be assigned if required="false" and dependent bean configuration is not available. (i.e no exception will raise)

Autowiring by using arbitrary method:

```

public class vehicle
{
    private Engine engg;
    @Autowitred (required=true)
    public void assign (Engg engg) //arbitrary method (your choice method)
    {
        this.engg=engg;
    }
    //toString()
    .....
    .....
}

```

Autowiring by using constructor:

```
public class Vehicle
{
    private Engine engg;
    @Autowired(required=true)
    public Vehicle(Engine engg)
    {
        this.engg=engg;
    }
    //toString()
    .....
    .....
}
```

We cannot apply @Autowired on the top of O-param constructor

At max only 1 constructor can have @Autowired annotation.

When @Autowired is placed on the top of field or setter method or arbitrary method. Then "byType" mode autowiring is takes place. When the same annotation is placed on top of constructor then constructor mode autowired will take place.

@Qualifier:

While performing "byType" mode autowiring there is a possibility of getting ambiguity if multiple dependent are found to inject to target class. In order to overcome this problem we need to use "@Qualifier" annotation which makes IOC container to perform "byName" mode of auto-wiring.

@Component, @Service, @Repository, @Controller:

The above annotations are called setter type annotation, because all the 4 type annotations are having same purpose with minor changes.

We can use all the 4 annotations to configure java classes as the spring beans.

@Component: Generic purpose setter type annotation to configure any java class as spring bean. i.e exposes given java class as spring bean to IOC container.

@Service: Specification of @Component, useful to configure the java classes of service layer. As of now no special behavior so it is same as @Component

@Repository: Specialization of @Component, useful to config the DAO classes as the Spring bean. Capable of translating one form of exception to another form of exceptions.

@Controller: Specialization of @Component, use to configure classes as the controller of spring mvc app to process request given by clients.

EX:

- ▲ IOCProj29(@Component,@Value)
 - ▲ src
 - ▲ com.nt.beans
 - Engine.java
 - Viechle.java
 - ▲ com.nt.cfgs
 - applicationContext.xml
 - ▲ com.nt.test
 - TestClient.java
 - JRE System Library [JavaSE-1.7]
 - ▲ IOClb
 - spring-beans-4.1.6.RELEASE.jar - C:\Use
 - spring-context-4.1.6.RELEASE.jar - C:\U
 - spring-context-support-4.1.6.RELEASE.j
 - spring-core-4.1.6.RELEASE.jar - C:\User
 - spring-expression-4.1.6.RELEASE.jar - C
 - commons-logging.jar - C:\Users\deli\PC

Vehicle.java:

```
package com.nt.beans;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
@Component
public class Viechle {
    @Autowired
    private Engine engg;
    //toString
    @Override
    public String toString() {
        return "Viechle [engg=" + engg + "]";
    }
}
```

Engine.java

```
package com.nt.beans;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
@Component("eng")
public class Engine {
    @Value("1001")
    private int id;
    @Value("DDIS")
    private String type;
    //setters
    public void setId(int id) {
        this.id = id;
    }
}
```

```
public void setType(String type) {  
    this.type = type;  
}  
//toString()  
@Override  
public String toString() {  
    return "Engine [id=" + id + ", type=" + type + "]";  
}  
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"  
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
xmlns:context="http://www.springframework.org/schema/context"  
xsi:schemaLocation="http://www.springframework.org/schema/beans  
http://www.springframework.org/schema/beans/spring-beans.xsd  
http://www.springframework.org/schema/context  
http://www.springframework.org/schema/context/spring-context.xsd">  
    <context:component-scan base-package="com.nt.beans"/>  
</beans>
```

ClientApp.java

```
package test;  
import org.springframework.context.ApplicationContext;  
import org.springframework.context.support.FileSystemXmlApplicationContext;  
import com.nt.beans.Viechle;  
public class ClientApp {  
    public static void main(String[] args) {  
        //create IOC Container  
        ApplicationContext cTransaction management =new  
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");  
        //getBean  
        Viechle viechle=cTransaction management .getBean("viechle",Viechle.class);  
        System.out.println(viechle);  
    }  
}
```

@Value:

It is given to configure dependent values for simple properties of dependency injection.

Java config annotations:

@Componet, @Autowired and etc. are spring supplied annotations, if we use those annotations the code of spring bean becomes tightly coupled code to spring framework. This kills **non-invasive**

features of spring.

To overcome this problem use java config annotations given by JSE and JEE modules for these annotations functionality will be given by the containers and frameworks that we are using.

For Example: For @Inject annotation spring provides it's functionalities and EJB provides it's functionalities. So if our code moves from 1 framework to another framework we can still continue with the same old java config annotations.

These annotations are part of JSE and J2EE specifications but implementation will be provided by different servers and frameworks. If we change runtime libraries of App we can still use same annotations in the classes.

Ex: @Inject, @Named, @Resource, @PostConstruct, @PostDestory and etc..

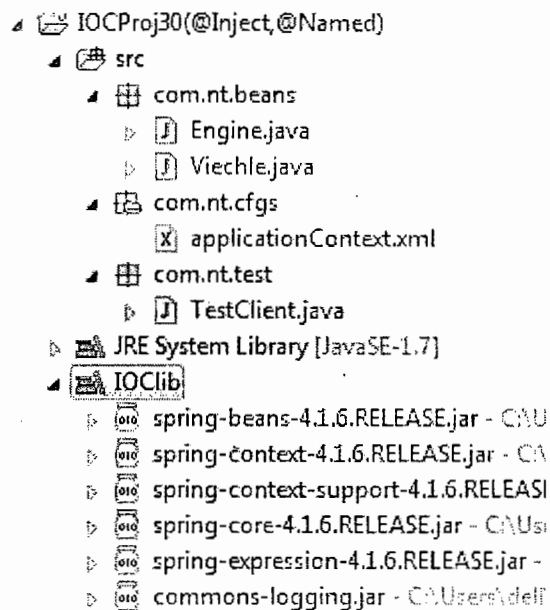
@Inject:

It is given in JEE specification in javax.inject package having ability of performing byname/byType/constructor mode of autowiring.

This is similar to @Autowire but the differences are

No required parameter in @inject

@Inject is java config annotation so non invasive spring application can be developed.



Vehicle.java

```
package com.nt.beans;
import javax.annotation.Resource;
import javax.inject.Inject;
import javax.inject.Named;
@Named("viechle")
public class Viechle{
    private Engine engg;
    /* @Inject
    public Viechle(Engine engg){
        this.engg=engg;
    }*/
    @Inject
    @Named("eng1")
    public void assign(Engine engg){
        this.engg=engg;
    }
}
```

```
/* @Inject
public void setEngg(Engine engg){
    this.engg=engg;
}*/
//toString
@Override
public String toString() {
    return "Viechle [engg=" + engg + "]\n";
}
}
```

Engin.java

```
package com.nt.beans;
import javax.inject.Named;
import org.springframework.beans.factory.annotation.Value;
@Named("eng")
public class Engine{
    @Value("1001")
    private int id;
    @Value("DDIS")
    private String type;
    //setters
    public void setId(int id) {
        this.id = id;
    }
    public void setType(String type) {
        this.type = type;
    }
    //toString()
    @Override
    public String toString() {
        return "Engine [id=" + id + ", type=" + type + "]\n";
    }
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
<!-- <bean id="eng1" class="com.nt.beans.Engine">
    <property name="id" value="1001"/>
    <property name="type" value="DDIS"/>
</bean>
<bean id="eng2" class="com.nt.beans.Engine">
```



```
<property name="id" value="1002"/>
<property name="type" value="CRDI"/>
</bean> -->
<context:component-scan base-package="com.nt.beans"/>
</beans>
```

Client.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.Viechle;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC Container
        ApplicationContext cTransaction management =new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //getBean
        Viechle viechle=cTransaction management .getBean("viechle",Viechle.class);
        System.out.println(viechle);
    }
}
```

@Inject can be applied at field/method (setter or arbitrary) method/constructor level.

To work with @Inject, @Named kind of java config annotations we need to add javax.inject.jar files.

Total jarfiles:

IOCLab+Spring-aop-<version>.jar+javax.inject.jar

@Named:

It is a java config annotation of javax.inject.pkg (JEE specification). This annotation can be used in 2 ways

To resolve ambiguity problems that raised while performing byType mode of autowiring (alternate to @Qualifier) can be used along with @Inject, @Resource, @Autowired

To configure/mark java classes as spring beans as alternate to spring specific annotations like @Component and etc...

@Resource

It is jdk supplied java config annotations of javax.annotation pkg

Can be used to perform byType, byname mode of Autowiring.

Can be applied at field/method (setter/arbitrary) level.

By using name param of this annotation we can perform byname mode of autowiring.

- ▲ IOCProj31(@Resource, @Scope)
 - ▲ src
 - ▲ com.nt.beans
 - ▷ Engine.java
 - ▷ Viechle.java
 - ▲ com.nt.cfigs
 - ▷ applicationContext.xml
 - ▲ com.nt.test
 - ▷ TestClient.java
 - ▷ JRE System Library [JavaSE-1.7]
 - ▲ IOCLib
 - ▷ spring-beans-4.1.6.RELEASE.jar - C:\Users\...
 - ▷ spring-context-4.1.6.RELEASE.jar - C:\Users\...
 - ▷ spring-context-support-4.1.6.RELEASE.jar - C:\Users\...
 - ▷ spring-core-4.1.6.RELEASE.jar - C:\Users\de...
 - ▷ spring-expression-4.1.6.RELEASE.jar - C:\Us...
 - ▷ commons-logging.jar - C:\Users\del\Desk...

Vehicle.java

```

package com.nt.beans;
import javax.annotation.Resource;
import javax.inject.Named;
import org.springframework.context.annotation.Scope;
@Named("viechle")
@Scope("prototype")
public class Viechle{
    @Resource
    private Engine engg;
    public Viechle(){
        System.out.println("Viechle:0-param constructor");
    }
    /* @Resource
    public void assign(Engine engg){
        this.engg=engg;
    }*/
    //@Resource(name="eng1")
    public void setEngg(Engine engg){
        this.engg=engg;
    }
    //toString
    @Override
    public String toString() {
        return "Viechle [engg=" + engg + "]";
    }
}

```

Engin.java

```

package com.nt.beans;
import javax.inject.Named;
import org.springframework.beans.factory.annotation.Value;

```

```
public class Engine{
    private int id;
    private String type;
    //setters
    public void setId(int id) {
        this.id = id;
    }
    public void setType(String type) {
        this.type = type;
    }
    //toString()
    @Override
    public String toString() {
        return "Engine [id=" + id + ", type=" + type + "]";
    }
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
    <bean id="eng1" class="com.nt.beans.Engine">
        <property name="id" value="1001"/>
        <property name="type" value="DDIS"/>
    </bean>
    <context:component-scan base-package="com.nt.beans"/>
</beans>
```

Client.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.Viechle;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC Container
        ApplicationContext cTransaction management =new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //getBean
        Viechle viechle=cTransaction management .getBean("viechle",Viechle.class);
        System.out.println(viechle);
        cTransaction management .getBean("viechle");
        cTransaction management .getBean("viechle");
    }
}
```

Difference between @Autowired, @Inject and @Resource:

@Autowired	@Inject	@Resource
Spring Annotation	JEE java configuration JDK	java config annotation
Support byType, byName,constructor mode of autowiring	Same	Supports byType, byname mode of autowiring.
Makes spring class as invasive	non-invasive class	non-invasive class
Param required is Available	non-param	param name is available.
Use @Qualifier or resolve of Resource annotation.	same	We can use name attribute @Named to Ambiguity problem

@Scope

This is spring annotation which can be applied on class level to specify springbean scope.

@Named("vehicle")

@Scope("Singleton")

public class vehicle

{

.....

.....

}

If the annotation param name is "value" then we can place the context of that param without specifying paramname.

@Named(Value="Vehicle") is equal to @Named ("vehicle")

Note: As of spring is providing functionality only to few java config annotations.

@PostConstruct, @PreDestroy:

Bean lifecycle configuration can be done in 3 ways.

Declarative Approach

(By using init-method, destroy-method attribute of <bean> tag)

Programatic Approach

(By implementing InitializationBean, DisposableBean interface)

Annotation Approach

(By using javaconfig annotations @PostConstruct, @PreDestroy)

@PostConstruct:

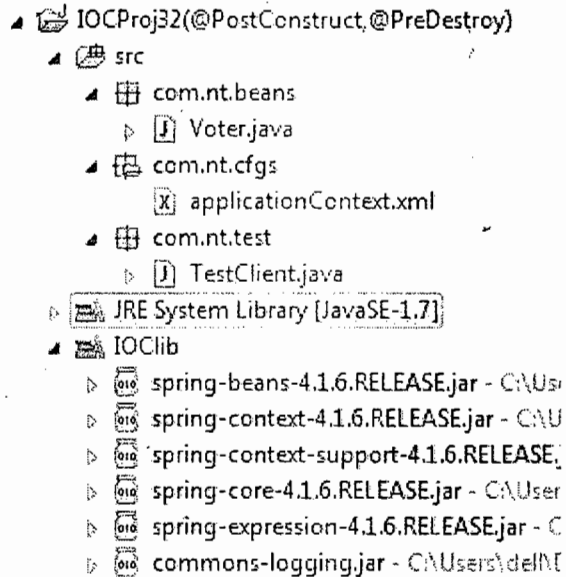
To config normal method as custom init method

@PreDestroy:

To configure normal method as custom destroy method

Note: Both method should have the following signature

```
public void <method> ()
{
```



Voter.java

```
package com.nt.beans;
import javax.annotation.PostConstruct;
import javax.annotation.PreDestroy;
import javax.inject.Named;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Scope;
@Named("vt")
@Scope("singleton")
public class Voter {
    @Value("raja")
    private String name;
    @Value("30")
    private int age;
    @PostConstruct
    public void myInit(){
        System.out.println("Voter:myInit()");
        if(name==null || age<=0){
            throw new IllegalArgumentException(" name,age must be set");
        }
    }
    @PreDestroy
    public void myDestroy(){
        System.out.println("Voter:myDestroy()");
        name=null;
        age=0;
    }
    public String checkVotingElgibility(){
        if(age<18)
            return name+" u r not eligible to vote";
    }
}
```

```
else
```

```
    return name+"u r eligible to vote";
```

```
    }
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
    <context:component-scan base-package="com.nt.beans"/>
</beans>
```

Client.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.Voter;
public class ClientApp {
    public static void main(String[] args) {
        // create IOC Container
        ApplicationContext cTransaction management =new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //get Bean
        Voter vt=cTransaction management .getBean("vt",Voter.class);
        String status=vt.checkVotingElgibility();
        System.out.println("Result:."+status);
        ((FileSystemXmlApplicationContext)cTransaction management ).close();
    }
}
```

Working with properties file in annotations environment:

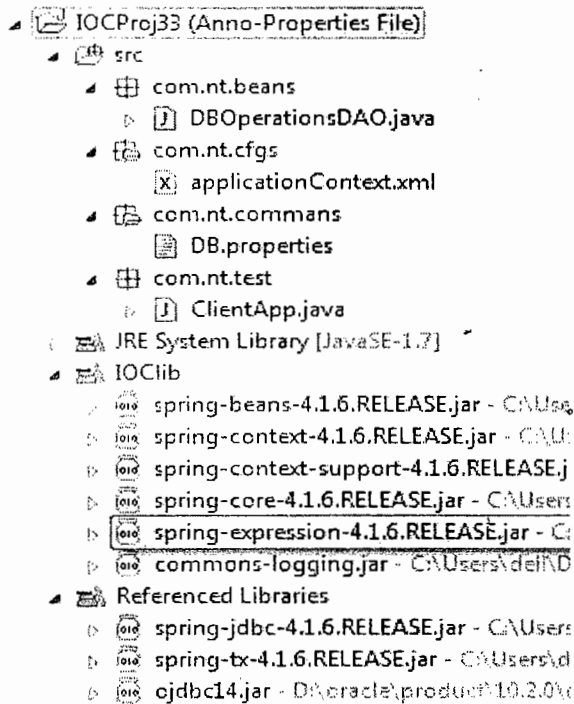
To detect the classes and to recognize then as spring beans through annotations we can use

<context:component-scan.....> tag

To detect the specified properties File that is configure through annotation we can use

<context:property-placeholder location=""> tag

To inject the values of properties files to bean properties we need to specify the keys of properties file through placeholder \${<key>}



DBOperations.java

```
package com.nt.beans;
import javax.inject.Named;
import org.springframework.beans.factory.annotation.Value;
@Named("dao")
public class DBOperationsDAO {
    @Value("${jdbc.driver}")
    private String driver;
    @Value("${jdbc.url}")
    private String url;
    @Value("${jdbc.user}")
    private String username;
    @Value("${jdbc.pwd}")
    private String password;
    @Override
    public String toString() {
        return "DBOperationsDAO [driver=" + driver + ", url=" + url
            + ", username=" + username + ", password=" + password + "];"
    }
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
    <context:component-scan base-package="com.nt.beans"/>
</beans>
```

```
<context:property-placeholder location="src/com/nt/commons/DB.properties"/>
</beans>
DB.properties
jdbc.driver=oracle.jdbc.driver.OracleDriver
jdbc.url=jdbc:oracle:thin:@localhost:1521:xe
jdbc.user=scott
jdbc.pwd=tiger
Client.java
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.DBOperationsDAO;
public class ClientApp {
    public static void main(String[] args) {
        //Create IOC Container
        ApplicationContext cTransaction management =new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //get Bean
        DBOperationsDAO dao=cTransaction management
        .getBean("dao",DBOperationsDAO.class);
        System.out.println(dao);
    }
}
```

Note: working with <context: property-placeholder....> tag is equal to working with PropertyPlaceholderConfigurer Bean class of Spring API.

Developing spring core module application without xml file:

For this we need to use "AnnotationConfigApplicationContext" to create core IOCcontainer by specifying "configuration"

The java class that contains methods to create and return bean class objects is called "Configuration class"

We use @Configuration to configure ordinary class as configuration class and we use @Bean on the top the configuration class method which produces objects (to configure those object as bean object use @Bean)

IOCProj34(@Configuration,@Bean,@Lazy)

- src
 - com.nt.beans
 - BeanConfigurator.java
 - Student.java
 - WishGenerator.java
 - com.nt.test
 - ClientApp.java
- JRE System Library [JavaSE-1.7]
- IOCLib
 - spring-beans-4.1.6.RELEASE.jar - C:\Users\del...
 - spring-context-4.1.6.RELEASE.jar - C:\Users\del...
 - spring-context-support-4.1.6.RELEASE.jar - C:\Users\del...
 - spring-core-4.1.6.RELEASE.jar - C:\Users\del...
 - spring-expression-4.1.6.RELEASE.jar - C:\Users\del...
 - commons-logging.jar - C:\Users\del\Deskto...

BeanConfigurator.java

```
package com.nt.beans;
import java.util.Date;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.DependsOn;
import org.springframework.context.annotation.Lazy;
import org.springframework.context.annotation.Scope;
@Configuration
public class BeanConfigurator {

    public BeanConfigurator() {
        System.out.println("BeanConfigurator: 0-param constructor");
    }
    @Bean(name="st")
    @Lazy(value=true)
    @Scope("prototype")
    public Student createStudent(){
        System.out.println("createStudent()");
        return new Student();
    }
    @Bean
    @DependsOn("st")
    public Date createDate(){
        System.out.println("createDate()");
        return new Date();
    }
    @Bean
    public WishGenerator createWishGenerator(){
        System.out.println("createWishGenerator()");
        return new WishGenerator();
    }
}
```

Student.java

```
package com.nt.beans;
public class Student {
    public Student() {
        System.out.println("Student:0-param constructor");
    }

    @Override
    public String toString() {

        return "101 --> raja";
    }
}
```

WishGenerator.java

```
package com.nt.beans;
public class WishGenerator {
    public WishGenerator() {
        System.out.println("WishGenerator:0-param constructor");
    }
    @Override
    public String toString() {
        return "Good Morning";
    }
}
```

clientApp.java

```
package test;
import java.util.Date;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import com.nt.beans.BeanConfigurator;
import com.nt.beans.Student;
import com.nt.beans.WishGenerator;
public class ClientApp {
    public static void main(String[] args) {
        //Create IOC Container
        ApplicationContext cTransaction management =new
        AnnotationConfigApplicationContext(BeanConfigurator.class);
        //getBeans
        /*    WishGenerator wg=cTransaction management .getBean(WishGenerator.class);
            System.out.println(wg);

            Student st=cTransaction management .getBean("st",Student.class);
            System.out.println(st);
            cTransaction management .getBean("st");
                Date d=cTransaction management .getBean(Date.class);
            System.out.println(d); */
        } } //main & Class
```

Limitations with above approach:

Kills non-invasive behavior of SpringBean

Kills loosely coupling behavior of spring bean.

Configure pre-defined, third party supplied classes as spring beans is very complex.

The class that is specified with `@Configuration` is called configuration class. In that class we add `@Bean` on the methods that produce bean class object. These methods will be executed automatically the moment IOC container started.

All the methods of configuration class that are annotated with `@Bean` will be executed automatically, the moment IOC container is created by specifying configuration class. If you want to delay certain method execution until cTransaction management .Bean(-) is called, then use `@Lazy` annotation

In order to create Bean class object only after creating another bean class object then we can go for `@DependsOn` annotation.

Bean Installation takes place when IOC container is created is called Eager Instantiation.

Bean Installation that takes place when cTransaction management .getBean() is invoked is called Lazy instantiation.

Spring JDBC/DAO

Where should we use JDBC and where should we use hibernate?

Ans: While working with jdbc if you want to process huge amount of records then we need to deal with single jdbc ResultSet object. While working with hibernate, if we want to process huge amount of records then we need to take multiple objects representing multiple records. This is a problem and it may crash the system, while creating huge amount of objects. Use JDBC offline applications that deal with huge amount of data where we don't need database portability feature. (having dB software without modifying the code)

Eg: Storing and managing census details, political party membership details by performing data validation and also correction. While developing online application (Internet applications) that deal with limited amount of personalized data then use Hibernate.

Eg: Online shopping mobile recharge online etc...

While developing internet application that performs batch processing of huge amount of data then use jdbc technology.

Eg: Courier Application that takes all orders and process at a time.

Eg: mobile number activation application that activates set of numbers at a time.

Spring jdbc provides abstraction layer on plain jdbc and simplifies persistence logic development. Instead of using plain jdbc in offline applications, Intranet applications that deal with huge amount of data through batch processing it is recommended to use spring jdbc.

Plain jdbc code:

Register jdbc driver

Establish the connection

Create statement

Send and execute SQL query

Gather Results and process Results

Exception handling

Close jdbc objects

Here we need to write both common logics and applications specification logics in the application, so it raises boiler plate code problem.

Spring jdbc App:

Configure and get jdbc Template class object

Send and execute SQL query.

Gather results and process results

Note: Here we need to take care of only application specific logics development.

Limitations with plain JDBC:

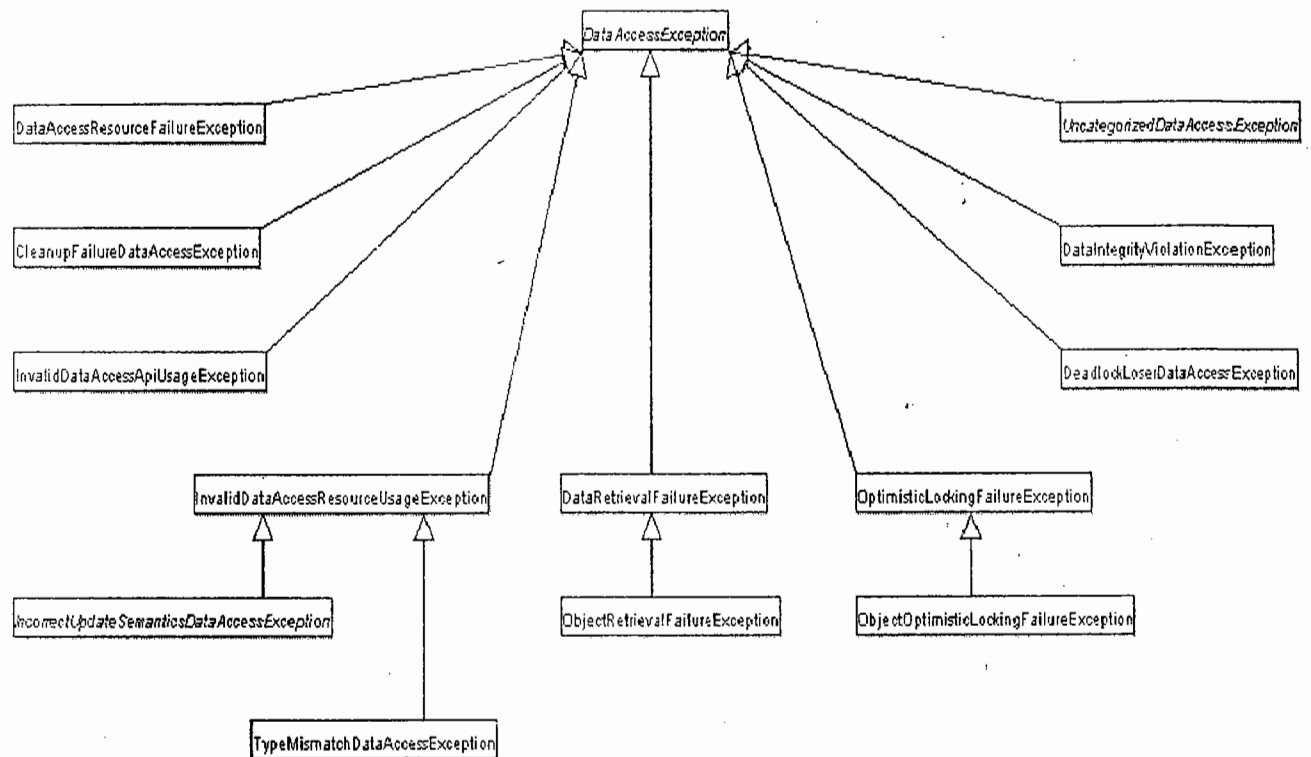
Boiler plate code problem

Exception handling is mandatory

Processing Resultset to get result in desired type is complex process.

No support for proper transaction management.

Spring JDBC/DAO Advanatges:



Solves Boiler plate code problem

Makes programmer to take care of only application specific logics as shown above.

Raises unchecked Exception (Sun classes of DataAccessException or DataAccessException class) so the exception is optional to catch and handle. It internally uses **exception rethrowing concepts** to convert checkedException (SQLException) into unchecked exception.

Gives various methods to get the records of ResultSet in desired types.

Takes care of opening connection and close connection activities.

Support both Named and positional param in SQL query.

Allows to use callback interface to develop more more customized persistence logic by using plain jdbc code.

Action	Programmer	Spring JDBC
Configure DataSource	Yes	
Creating/getting connection		Yes
Creating Statement		Yes
Preparing SQL query with Params	Yes	
Sending executing SQL query in DB software	Yes	
Iterating through ResultSet (if needed)		Yes
Exception Handling		Yes
Transaction management		Yes
Closing jdbc objects		Yes

Spring jdbc api pkgs

org.springframework.jdbc and its sub pkgs

org.springframework.dao and its sub pkgs

(this package contains only Exceptions)

Note: Spring jdbc/ORM module internally uses multiple other technologies/Frameworks to develop persistence logics but translate the exception of underlying technology/framework to spring's `DataAccessException` hierarchy.

There are 5 methodologies/technologies to develop persistence logic in spring jdbc.

JdbcTemplate

- 1) `NamedParameterJdbcTemplate`
- 2) `SimpleJdbcTemplate` (removed spring 4.0)
- 3) `SimpleJdbcInsert`, `SimpleJdbcCall`
- 4) **Modeling JDBC operations as Java objects**(Mapping SQL operations as subclass using `SQLQuery`, `SqlUpdate` classes. (JDO Style))

While working with any technique of spring JDBC we need to use JDBC connection to interact with database software.

It is recommended to use jdbc connection pool based jdbc connection instead of direct jdbc connection.

Do not use spring's built-in classes like `DriverManagerDataSource`, `SingletonConnectionDataSource` and etc. to work with spring jdbc connection pool because those classes actually doesn't pool the connection objects more ever creates jdbc connection object on demand.

What is the jdbc connection pool that should be used in our spring application?

Ans: If our spring api is standalone app running outside the server then use third party jdbc con pool like c3po or apache DBCP and etc..

If our spring app is deployable in server then use jdbc connection pool

Using JdbcTemplate:

This is central class of spring jdbc provides abstraction layer on plain jdbc and simplifies persistence logic development.

This class takes care of common workflow activities of jdbc programming like opening connection, creating statement, closing connectin and etc..... but makes the programmer to just provide SQL query and to extract results.

This class translates jdbc generated `SQLExceptions` into spring specific unchecked exception like `DataAccessException`.

This allows programmer to avoid common errors in application development like forgetting to close jdbc objects.

To create `JdbcTemplate` class object we need jdbc `DataSource` object.

We generally configure `JdbcTemplate` in spring bean configure file and use inject that object to DAO class.

persistence –beans.xml:

```
<bean id="drds" class="org.springframework.jdbc.dataSource.DriverManagerDataSource">
```

```
.....
```

```
.....
```

```
</bean>
```

```
<bean id="template" class="org.springframework.jdbc.Support.jdbcTemplate">
```

```
<property name="dataSource" ref="drds"/>
```

```
</bean>
```

There are 2 approaches of working with JdbcTemplate

Using direct methods (No need of jdbc code)

Using callback interfaces (Need to write plain jdbc code to place customized persistence logic) :

Using direct method:

Working with direct methods of JdbcTemplate class.

Select-Operator: (query, queryForXxx(-) methods)

queryForInt(), queryForLong()

To execute query that gives single numeric value as result.

Eg: Select count(*) from student;

queryForMap():

To execute the query that gives single record.

Eg: Select * from student where sno=1001;

queryForObject()

To execute the query and to get result into predefined or Bo class object.

Eg: Select * from student where sno=1001;

queryForList()

To execute query that gives multiple records.

Eg: Select * from student;

queryForRowSet()

To execute query that gives jdbc RowSet object and etc...

Note: If the above methods are called with one argument they internally uses jdbc simple statements object.

Note: If the above methods are called with 2 or 3 arguments the internally uses jdbc PreparedStatement object.

Non-Select operations

Update (-) method

batch updation

batchUpdate (-) and many more

- ▲  DAOProj1(JdbcTemplate-Directmethods)
 - ▲  src
 - ▲  com.nt.cfgs
 -  applicationContext.xml
 - ▲  com.nt.dao
 -  DBOperations.java
 -  DBOperationsDAO.java
 - ▲  com.nt.service
 -  DBOperationService.java
 - ▲  com.nt.test
 -  ClientApp.java
 - ▶  JRE System Library [JavaSE-1.7]
 - ▲  DAOlib
 - ▶  ojdbc14.jar - D:\oracle\product\10.2.0\db_1
 - ▶  spring-beans-4.1.6.RELEASE.jar - C:\Users\de
 - ▶  spring-context-4.1.6.RELEASE.jar - C:\Users\de
 - ▶  spring-context-support-4.1.6.RELEASE.jar -
 - ▶  spring-core-4.1.6.RELEASE.jar - C:\Users\de
 - ▶  spring-expression-4.1.6.RELEASE.jar - CAUs
 - ▶  spring-jdbc-4.1.6.RELEASE.jar - C:\Users\de
 - ▶  spring-tx-4.1.6.RELEASE.jar - C:\Users\del\
 - ▶  commons-logging.jar - C:\Users\del\Desktop
 - ▶  c3p0-0.9.1.jar - C:\Users\del\Desktop
 - ▶  commons-collections-3.1.jar - C:\Users\de
 - ▶  commons-dbcp-1.2.1.jar - C:\Users\del\De
 - ▶  commons-pool-1.2.jar - C:\Users\del\Desl

DBOperations.java

```
package com.nt.dao;
import java.util.List;
import java.util.Map;
public interface DBOperations {
    public int insert(int no,String name,String job,long sal);
    public int getSalary(int no);
    public Map<String,Object> listEmpDetails(int no);
    public List<Map<String,Object>> listAllEmpDetails();
    public int UpdateEmpSalary(int no,int newsal);
    public int deleteEmp(int no);
}
```

DBOperationsDAO.java

```
package com.nt.dao;
import java.util.List;
import java.util.Map;
import org.springframework.jdbc.core.JdbcTemplate;
public class DBOperationsDAO implements DBOperations {
    private static final String INSERT_QRY="insert into emp(empno,ename,job,sal)
    values(?,?,?,?)";
```



```
private static final String GET_EMP_SALARY="select sal from emp where empno=?";
private static final String GET_EMP_DETAILS_BY_NO="select empno,ename,job,sal from
emp where empno=?";
private static final String GET_ALL_EMP_DETAILS="select empno,ename,job,sal from Emp";
private static final String UPDATE_EMP_SALARY="update emp set sal=? where empno=?";
private static final String DELETE_EMP="delete from emp where empno=?";

    private JdbcTemplate jt;
    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }

    @Override
    public int insert(int no, String name, String job, long sal) {
        int result=jt.update(INSERT_QRY, no,name,job,sal);
        return result;
    }

    @Override
    public int getSalary(int no) {
        int salary=jt.queryForInt(GET_EMP_SALARY,no);
        return salary;
    }

    @Override
    public Map<String, Object> listEmpDetails(int no) {
        Map<String, Object> map=jt.queryForMap(GET_EMP_DETAILS_BY_NO,no);
        return map;
    }

    @Override
    public List<Map<String, Object>> listAllEmpDetails() {
        List<Map<String, Object>> list=jt.queryForList(GET_ALL_EMP_DETAILS);

        return list;
    }

    @Override
    public int UpdateEmpSalary(int no, int newsal) {
        int result=jt.update(UPDATE_EMP_SALARY,newsal,no);
        return result;
    }

    @Override
    public int deleteEmp(int no) {
        int result=jt.update(DELETE_EMP,no);
        return result; } }
```

DBOperationService.java

```
package com.nt.service;
import java.util.List;
import java.util.Map;
import com.nt.dao.DBOperationsDAO;
public class DBOperationService {
```

```
private DBOperationsDAO dao;
public void setDao(DBOperationsDAO dao) {
    this.dao = dao;
}

public String registerEmp(int no,String name,String job,int sal)
{
    int result=dao.insert(no, name, job, sal);
    if(result==0)
        return no+" Emp is not registered";
    else
        return no+" Emp is registered";
}

public int showSalary(int no){
    int salary=dao.getSalary(no);
    return salary;
}

public String hikeSalary(int no){
    //write B.logic
    //get Salary
    int salary=showSalary(no);
    int newsal=0;
    if(salary>=50000)
        newsal=Math.round(salary+(salary*0.1f));
    else
        newsal=Math.round(salary+(salary*0.2f));
    //use DAO
    int result=dao.UpdateEmpSalary(no, newsal);
    if(result==0)
        return no+" Emp Salary not hiked";
    else
        return no+"Emp Salary hiked";
}

public Map<String,Object> getEmpDetails(int no)throws Exception{
    Map<String,Object> map=dao.listEmpDetails(no);
    if(map!=null)
        return map;
    else
        throw new Exception(no+" Emp not found");
}

public List<Map<String,Object>> getAllEmpDetails()throws Exception{
    List<Map<String,Object>> list=dao.listAllEmpDetails();
    if(list!=null)
        return list;
    else
        throw new Exception("Employees not found");
}

public String fireEmp(int eno){
    int result=dao.deleteEmp(eno);
```

```
        if(result==0)
            return eno+" Employee not found";
        else
            return "Employee deleted";
    }
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.0.xsd">
    <!-- <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
        <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
        <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
        <property name="username" value="scott"/>
        <property name="password" value="tiger"/>
    </bean> -->
    <bean id="dataSource" class="com.mchange.v2.c3p0.ComboPooledDataSource">
        <property name="driverClass" value="oracle.jdbc.driver.OracleDriver"/>
        <property name="jdbcUrl" value="jdbc:oracle:thin:@localhost:1521:xe"/>
        <property name="user" value="scott"/>
        <property name="password" value="tiger"/>
        <property name="initialPoolSize" value="10"/>
        <property name="maxPoolSize" value="20"/>
    </bean>
    <bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
        <property name="dataSource" ref="dataSource"/>
    </bean>
    <bean id="DBODao" class="com.nt.dao.DBOperationsDAO">
        <property name="jt" ref="template"/>
    </bean>
    <bean id="DBOService" class="com.nt.service.DBOperationService">
        <property name="dao" ref="DBODao"/>
    </bean>
</beans>
```

Client.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.DBOperationService;
public class ClientApp {
    public static void main(String[] args) throws Exception{
        //Create IOC Container
```

```
ApplicationContext cTransaction management =new
FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
//get Service Bean
DBOperationService service=(DBOperationService)cTransaction management
.getBean("DBOService");
// Call B.methods
System.out.println("7499 Emp Details"+service.getEmpDetails(7499));
System.out.println("All Emp Details"+service.getAllEmpDetails());
System.out.println("7499 Emp Salary(Before hike)+"service.showSalary(7499));
System.out.println("7499 Emp Salary hiking..."+"service.hikeSalary(7499));
System.out.println("7499 Emp Salary(After hike)+"service.showSalary(7499));
//System.out.println(service.registerEmp(9001,"Raja","progammer",8999));
//System.out.println("Fire Emp"+service.fireEmp(9001));
} //main
} //class
```

Jar files: IOCLib jar files+object 14/6.jar+org.springframework.jdbc<ver>.jar

Commons-pool.jar+commons-dbcp.jar+org.springframework.+n<ver>.jar (from spring 2.5 software) or c3o-0.9.1.2.jar (To work with c3po pool)

To provide global visibility to object or object reference it will be placed in jndi registry having nickname/jndi name.

Every webserver/App server gives one built-in jndi registry.

In server managed jdbc connection pool environment the jdbc con pool will be represented by DataSource object and its reference will be placed in jndi registry of server for global visibility having jndi name.

The apps deployed in server or running from outside server can access DataSource object reference from jndi registry to get jdbc conn. Object from jdbc connection Pool.

In one server we can create multiple jdbc conn pool belonging to same on diff database softwares.

All conn object in conn pool represents connectivity with same database software.

The client application that wants to use connection pool can be there outside the server as standalone application and can be there inside the server as deployable servlet component, jsp component, Ejb component.

With respect to diagram:

TL or PL or Admin makes server to use typ1/2/4 jdbc driver to interact with database software and to create servermanaged jdbc connection pool having connection objects.

TL/PL/Admin creates DataSource object representing jdbc connection pool places that dataSource object reference in jndi registry having jndi name for global visibility.

In weblogic software we can create multiple domains and each domain acts as 1 domain server.

Multiple projects of a company uses multiple domains servers of weblogics software 1 per project basis i.e the weblogic software will be installed only for 1 in a common computer and it that software multiple domains will be created for multiple projects on 1 per project basis.

Procedure to create user-defined domains server in weblogic 12c:

Start---->Oracle---->oracle home----->weblogic server12c----->tools---->conf-wizard---->create new domain (domain location:..... sp27Domian)----->next----->next

Name: javaboss

Pass: javaboss1

Confo-pass:javaboss1

Next--->next--->select admin server---->next--->proj:7080----->create--->next----->finish.

Procedure to create jdbc DataSource pointing jdbc con pool for oracle in "sp27Domain" server of weblogic:

start sp27Domain server

D:\weblogic 12c\user-project\Domains\sp27Domain\Startweblogic.cmd

Open admin console of that domain server

http://localhost:7080/console----->

username: javaboss

password: javanoss1

login

Create a jdbc DataSource

Adminconsole--->services----->datasources--->new---->generic datasource

Name ds1

Jndi name: dsjndi

DatabaseType: Oracle

Driver: Oracle's Driver (thin) for service connection

Next--->next

Database name:xe

Hostname : localhost

Port: 1521

Db username: Scott

Password:tiger

Conform pass: tiger ----->Test Configuration---->next

Select admin Server----->Finish

Launch

Connection pool tab--->Advanced----->

Min:3

Max: 30

Save

Advanced:

Shrink frequency : 900 (specifies the idel connection onjects of jdbc connection pool will be destroyed automatically for every 900sec)

----->save

Spring gives 1 built-in FactoryBean class to get object from jndi registry based on given jndi name.

This Factory bean class name is

Org.springframework.jndi.JndiObjectFactoryBean

Eg: in bean Configure file.

<bean id="joFb" class="org.springframework.jndi.JndiObjectFactoryBean">

<property name="JndiName" value="DSJndi"/>

</bean>

<bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">

<property name="dataSource" ref="joFb"/>

</bean>

Since the above bean is FactoryBean the gathered dataSource object will be injected here.

Web App by using:

The above application is web application so it is deployed in server so use server managed jdbc con pool.

- DAOEmpSearchApp2
 - JAX-WS Web Services
 - Java Resources
 - src
 - com.nt.cfigs
 - applicationContext.xml
 - com.nt.controller
 - MainServlet.java
 - com.nt.dao
 - EmpDao.java
 - com.nt.service
 - EmpService.java
 - Libraries
 - DAOlib
 - EAR Libraries
 - JRE System Library [JavaSE-1.7]
 - Web App Libraries
 - JavaScript Resources
 - build
 - WebContent
 - META-INF
 - WEB-INF
 - lib
 - web.xml
 - Main.html
 - Result.html
 - Search.html

EmpDAO.java

```

package com.nt.dao;
import java.util.List;
import java.util.Map;
import org.springframework.jdbc.core.JdbcTemplate;
public class EmpDao {
    private JdbcTemplate jt;
    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }
    public List getEmpDetails(String condition){
        List<Map<String,Object>> list=jt.queryForList("select empno,ename,job,sal from emp
        where job in '"+condition, new Object[]{});
        return list;
    }
}

```

EmpService.java

```

package com.nt.service;
import java.util.List;
import java.util.Map;
import com.nt.dao.EmpDao;
public class EmpService {
    private EmpDao dao;

    public void setDao(EmpDao dao) {

```

```

MainServlet.java
package com.nt.controller;
import java.io.IOException;
import java.util.List;
import java.util.Map;
import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.nt.service.EmpService;
public class MainServlet extends HttpServlet {
    ApplicationContext cTransaction management ;
    EmpService service;
    public void init(){
        //create IOC container
        cTransaction management =new
        ClassPathXmlApplicationContext("com/nt/configures/applicationContext.xml");
        // get Service class bean
        service=(EmpService)cTransaction management .getBean("service");
    }

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)

```

```
        throws ServletException, IOException {
            //read form data
            String jobs[]=req.getParameterValues("job");
            //use Service class
            List<Map<String,Object>> list=service.search(jobs);
            // keep results in request attribute
            req.setAttribute("result",list);
            //forward the request to result page
            RequestDispatcher rd=req.getRequestDispatcher("/Result.jsp");
            rd.forward(req,resp);
        } //doGet(-,-)
        @Override
        protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
            doGet(req,res);
        } //doPost(-,-)
    } //class
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.0.xsd">

    <bean id="jofb" class="org.springframework.jndi.JndiObjectFactoryBean">
        <property name="jndiName" value="java:/comp/env/mypool"/>
    </bean>
    <bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
        <property name="dataSource" ref="jofb"/>
    </bean>
    <bean id="dao" class="com.nt.dao.EmpDao">
        <property name="jt" ref="template"/>
    </bean>
    <bean id="service" class="com.nt.service.EmpService">
        <property name="dao" ref="dao"/>
    </bean>
</beans>
```

Mian.jsp

```
<frameset rows="40%,60%">
    <frame name="f1" src="Search.jsp"/>
    <frame name="f2"/>
</frameset>
```

Result.jsp

```
<%-- <@page import="java.util.*" %> --%>
<%-- <@read request attribute
List<Map<String,Object>> list=(List<Map<String,Object>>)request.getAttribute("result");
```



```

    if(list!=null){ %> --%>
    <!--    <table> -->
    <!--    <tr> -->
    <!--        <th>NO</th><th>NAME</th><th>JOB</th><th>Salary</th> -->
    <!--    </tr> -->
    <%--    <% for(Map map:list){ %> --%>
    <!--    <tr> -->
    <%--        <td><%=map.get("EMPNO") %></td> --%>
    <%--        <td><%=map.get("ENAME") %></td> --%>
    <%--        <td><%=map.get("JOB") %></td> --%>
    <%--        <td><%=map.get("SAL") %></td> --%>
    <!--    </tr> -->
    <%--    <%    } //for --%>
    // } //if
    <%--    %> --%>
    <!--    </table> -->
    <%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
    <c:choose>
    <c:when test="\${not empty result}">
        <table>
            <tr>
                <th>NO</th><th>NAME</th><th>JOB</th><th>Salary</th>
            </tr>
            <c:forEach var="map" items="\${result}">
                <tr>
                    <td><c:out value="\${map.EMPNO}"/></td>
                    <td><c:out value="\${map.ENAME}"/></td>
                    <td><c:out value="\${map.JOB}"/></td>
                    <td><c:out value="\${map.SAL}"/></td>
                </tr>
            </c:forEach>
        </table>
    </c:when>
    <c:otherwise>
        No Employee Found
    </c:otherwise>
    </c:choose>

```

Search.jsp

```

<h1><center>Search EmpDetails </center></h1>
<form action="controller" target="f2">
    Select job(s) <select name="job" multiple>
        <option value="CLERK">CLERKS</option>
        <option value="MANAGER">MANAGERS</option>
        <option value="SALESMAN">SALESMEN</option>
        <option value="ANALYST">ANALYSTS</option>
        <option value="PRESIDENT">PRESIDENTS</option>
    </select>

```

```
<input type="submit" value="search"/>
</form>
web.xml
<web-app>
  <servlet>
    <servlet-name>Main</servlet-name>
    <servlet-class>com.nt.controller.MainServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>Main</servlet-name>
    <url-pattern>/controller</url-pattern>
  </servlet-mapping>
</web-app>
```

Jar files: spJDBCLib jarfile+Ojdbc14.jar+JSTL.jar+Servlet-api.jar+standard.jar----->collect from tomcat installation (perform Deployment assembly)

While developing eclipse based web application the jar files placed in build path will not be move to WEB-INF/lib folder automatically. For that we need to perform deploymentAssemble separately. Right click on project----->buildpath--->configure buildpath---->deployment assembly--->Add---->build path entries--->select jar files----->apply--->ok

To deploy Eclipse web project in Ntsp27 Domain of weblogic:

Start N+Sp27 domain server

<weblogic_name>\user-projects\domains\N+Sp27domain\startweblogic.cmd

Export war file to N+Sp27 Doamin server

Right click project-->export-->warfile

Destination: D:\.... AutoDeploy/DAOEmpSearchApp2.war

Test the application.

<http://localhost:7080/DAOEmpSearchApp2/main.html>

Procedure to create user defined domain in Glasspringframeworkish server:

<Galsspringframeworkish_home>\bin>

Asadmin create-domain --adminport=4343 --user=testuser Ntsp27

Enter password: testuser

Re-Enter password: testuser

.....

.....

Domain created

To install glasspringframeworkish 4.x extract the zip file

Glasspringframeworkish uses separate port no for http operations and separate portno for admin operations.

Procedure to create jdbc DataSource pointing to jdbc con pool for oracle in GNNtsp27 Domain server of Glasspringframeworkish:

Place Ojdbc14/6.jar file in <Glasspringframeworkish_home>/domains/GFNTSP27Domain/lib0/ext folder

Start the domain server

<Galsspringframeworkish_home>\bin>asadmin start-domain NtSp27

Open admin console of domain server

<http://localhost:46460>

username: testuser

password: testuser

login

For Stop domain

<Glassspringframeworkish_home>\bin>asadmin Stop_domain Ntsp27

Create jdbc conn pool for Oracle

Admin console---->resource---->jdbc---->hdbcConnection pools---->new---->pool name: pool1

Resource type: javax.sql.DataSource

DB Drivervendor: Oracle----->Next

Initial and min pool size: 3

Max pool size:100

.....

.....

Additional properties

User: scott

DBName: Xp

Password: tiger

Servername: localhost

Url:jdbc:oracle:thin@localhost:1521:xe

Port no:1521

---->finish

Launch pool1--->ping

Create jdbc DataSource pointing to the above jdbc connection pool:

Adminconsole---->resource---->jdbc-->jdbcources---->new---->jNDi name:DBjndi

Pool name: pool1 ---->ok

To deploy application in Glassspringframeworkish Ntsp27Domain server of glassspringframeworkish place directory or war File in

<Glassspringframeworkish_home>\domain\Ntsp27\AutoDeployFolder

Note: While working with spring jdbc in servlet then application context constructor should be ClassPathXmlApplicationContext.

If it will FileSystemXmlAppContext. Then it will not work

Remember here src Folder should not mension in the file path of applicationContext.xml

Make that xsd file to beans-2.0.xsd

Creating jdbc connection pool for oracle in tomcat server:

Step1: make sure that tomcat-jdbc.jar file is available in <tomcat_home>/lib folder

Note: This jar file is builtin file in tomcat but must be arranged separately for other versions of tomcat by extracting apache-tomcat-jdbc-1.1.0.1_bin.zip file.

Step2: place ojdbc 14/6.jar file in <tomcat_home>/confg/context.xml

Step 3: place the following <Resource>tag in <tomcat_home>confg/context.xml

File as the subtag of <context> tag

<Resource name="myPool" type="javax.sql.DataSource"

Factory="org.apache.tomcat.jdbc.pool.DataSourceFactory"

driverClassName="Oracle.jdbc.driver.OracleDriver" url="jdbc:oracle:thin:@localhost:1521:xe"

Username="Scott" password="tiger" maxActive="20" maxIdle="10" validationQuery="SELECT SYSDATE FROM DUAL" />

Step 4: restart the server

Note: To use the above jdbc connection pool in web application we need to place jndi name as shown below

Java:/comp (prefix)/env/myPool (jndiname)

Note: If you deploy any web application in tomcat server through eclipse ide by configuration tomcat server with eclipse Ide. It is recommended to place "context.xml" file in servers folder of package explorer in order to use tomcat server managed jdbc connection pool in that webapplication.

Callback Interfaces:

The method that is called by underlying jvm/container/server/etc.... automatically and dynamically is callback method.

The interface that contains such methods is called callback interfaces.

Spring provides lots of callback interfaces along with lots of templates classes. We can use these callback interfaces to write customized logics to process data and to get data in customized manner.

For example the queryForMap(-) gives single record in the form of map object having column name as key and column values as the values in map dataStructure element. But industry standard is getting that record as Bo class object. For this spring gives RowMapper call back interface.

Various callback interfaces of spring jdbc are

StatementCallback--->Allows to work with statement object

PreparedStatementCallback

CallableStatementCallback

PreparedStatementCreator-->provides con object to create preparedStatement

PreparedStatementSetter-->provides preparedStatement object to set value to queryParams.

RowMapper---->To process 1 record of ResultSet at a time. *alteration for map*

ResultSetExtractor---->To process multiple records of ResultSet at a time. *alteration for List*

PreparedStatementBatchProcess → for batch processing

Note: The implementation class methods of callback interfaces will be called by JdbcTemplate internally.

EX: Application to get single employee record as EmployeeBO class objects and multiple Employee records as multiple EmployeeBO class objects stored in List Collection.

DAOProj3 (CallbakInterfaces)

- src
 - com.nt.bo
 - EmployeeBO.java
 - com.nt.cfigs
 - applicationContext.xml persistence-beans.xml
 - com.nt.dao
 - ListEmpDao.java
 - ListEmpDaoImpl.java
 - com.nt.dto
 - EmployeeDTO.java
 - com.nt.service
 - ListEmpService.java
 - com.nt.test
 - ClientApp.java
- JRE System Library [JavaSE-1.7]
- DAOlib
 - ojdbc14.jar - D:\oracle\product\10.2.0\db_
 - spring-beans-4.1.6.RELEASE.jar - C:\Users\
 - spring-context-4.1.6.RELEASE.jar - C:\Users\
 - spring-context-support-4.1.6.RELEASE.jar -
 - spring-core-4.1.6.RELEASE.jar - C:\Users\di
 - spring-expression-4.1.6.RELEASE.jar - C:\U
 - spring-jdbc-4.1.6.RELEASE.jar - C:\Users\di
 - spring-tx-4.1.6.RELEASE.jar - C:\Users\del\
 - commons-logging.jar - C:\Users\del\Desktop
 - c3p0-0.9.1.jar - C:\Users\del\Desktop
 - commons-collections-3.1.jar - C:\Users\de
 - commons-dbcp-1.2.1.jar - C:\Users\del\De
 - commons-pool-1.2.jar - C:\Users\del\Des

EmployeeBO.java

package com.nt.bo;

public class EmployeeBO {

private int no;

private String ename,job;

private int sal;

public EmployeeBO() {

}

public EmployeeBO(int no, String ename, String job, int sal) {

this.no = no;

this.ename = ename;

this.job = job;

this.sal=sal;

}

public int getNo() {

return no;

}

public void setNo(int no) {

this.no = no;

}

```

public String getENAME() {
    return ename;
}
public void setENAME(String ename) {
    this.ename = ename;
}
public String getJob() {
    return job;
}
public void setJob(String job) {
    this.job = job;
}
public int getSal() {
    return sal;
}
public void setSal(int sal) {
    this.sal = sal;
}
}

```

2. query for object () me takes the given
 Remapper obj (~~the~~ Emp Remapper)
 and calls mapRow(-) having ResultSet obj, this
 mapRow(-) method puts the record of result-set
 obj into po class obj and return to
 queryForObject(-) method,

ListEmpDAO.java

```

package com.nt.dao;
import java.util.List;
import com.nt.bo.EmployeeBO;
public interface ListEmpDao {
    public EmployeeBO getEmpDetailsByNo(int eno);
    public List<EmployeeBO> getEmpDetailsByDesg(String desg);
}

```

ListEmpDAOImpl.java

```

package com.nt.dao;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;
import org.springframework.dao.DataAccessException;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.ResultSetExtractor;
import org.springframework.jdbc.core.RowMapper;
import com.nt.bo.EmployeeBO;
public class ListEmpDaoImpl implements ListEmpDao {
    private static final String GET_EMP_DETAILS_BY_NO="select empno,ename,job,sal from
    emp where empno=?";
    private static final String GET_EMP_DETAILS_BY_DESG="select empno,ename,job,sal from
    emp where job=?";
    private JdbcTemplate jt;
    public void setJt(JdbcTemplate jt) {

```

```

    this.jt = jt; *1 → method flow
    }
    @Override
    public EmployeeBO getEmpDetailsByNo(int eno) {
        EmployeeBO empbo = jt.queryForObject(GET_EMP_DETAILS_BY_NO,
        *1 new Object[]{eno},
        new EmpRowMapper());

        return empbo;
    }
    private class EmpRowMapper implements RowMapper<EmployeeBO>{

```

```

        @Override
        public EmployeeBO mapRow(ResultSet rs, int pos) throws SQLException {
            System.out.println("mapRow(-,-)");
            // Keep the record of ResultSet into BO class obj
            EmployeeBO ebo = new EmployeeBO(rs.getInt(1),
            rs.getString(2),
            rs.getString(3),
            rs.getInt(4));

            return ebo;
        } //mapRow
    } //inner class
    @Override
    public List<EmployeeBO> getEmpDetailsByDesg(String desg) {

```

```

        List<EmployeeBO> list = jt.query(GET_EMP_DETAILS_BY_DESG,
        *1 new Object[]{desg},
        *2 new ResultSetExtractor<List<EmployeeBO>>(){

```

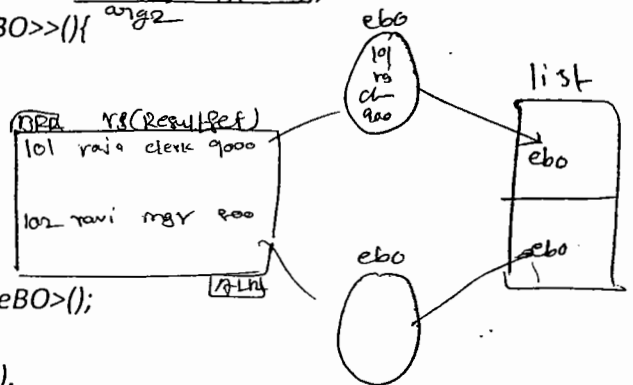
```

            @Override
            public List<EmployeeBO> extractData(ResultSet rs)
            throws SQLException, DataAccessException {
                System.out
                .println("extract Data");
                List<EmployeeBO> list = new ArrayList<EmployeeBO>();
                while(rs.next()){
                    EmployeeBO ebo = new EmployeeBO(rs.getInt(1),
                    rs.getString(2),
                    rs.getString(3),
                    rs.getInt(4));

                    list.add(ebo);
                } //while

                return list;
            } //extractData(-)
        } //anonymous inner class

```



*3 → obj of Anonymous inner class that implements
 RowMapper<EmployeeBO> having the
 definition of extractData method

```
    );//method call  
    return list;  
  }//method  
}//class
```

ListEmpService.java

```
package com.nt.service;  
import java.util.ArrayList;  
import java.util.List;  
import com.nt.bo.EmployeeBO;  
import com.nt.dao.ListEmpDao;  
import com.nt.dto.EmployeeDTO;  
public class ListEmpService {  
    private ListEmpDao dao;  
    public void setDao(ListEmpDao dao) {  
        this.dao = dao;  
    }  
  
    public EmployeeDTO searchEmpByNo(int no){  
        //use DAO class  
        EmployeeBO ebo=dao.getEmpDetailsByNo(no);  
        // Transpringframeworker BO data to DTO data  
        EmployeeDTO dto=new EmployeeDTO(ebo.getNo(),  
                                           ebo.getEname(),  
                                           ebo.getJob(),  
                                           ebo.getSal());  
  
        if(dto!=null)  
            return dto;  
        else  
            return null;  
    }  
  
    public List<EmployeeDTO> searchEmpsByDesg(String desg){  
        //use DAO class  
        List<EmployeeBO> listbo=dao.getEmpDetailsByDesg(desg);  
        // Copy List<BO> data to List<DTO>  
        List<EmployeeDTO> listdto=new ArrayList<EmployeeDTO>();  
        for(EmployeeBO bo:listbo){  
            EmployeeDTO dto=new EmployeeDTO(bo.getNo(),  
                                              bo.getEname(),  
                                              bo.getJob(),  
                                              bo.getSal());  
  
            listdto.add(dto);  
        }//for  
        if(listdto!=null)  
            return listdto;  
        else  
            return null;  
    }  
    }//method  
}//class
```



```
EmployeeDTO.java
package com.nt.dto;
import java.io.Serializable;
public class EmployeeDTO implements Serializable {
    private int no;
    private String name,job;
    private int sal;
    public EmployeeDTO() {
        // TODO Auto-generated constructor stub
    }
    public int getNo() {
        return no;
    }
    public void setNo(int no) {
        this.no = no;
    }

    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getJob() {
        return job;
    }
    public void setJob(String job) {
        this.job = job;
    }
    public int getSal() {
        return sal;
    }
    public void setSal(int sal) {
        this.sal = sal;
    }
    public EmployeeDTO(int no, String name, String job, int sal) {
        super();
        this.no = no;
        this.name = name;
        this.job = job;
        this.sal = sal;
    }
    @Override
    public String toString() {
        return "EmployeeDTO [no=" + no + ", name=" + name + ", job=" + job
            + ", sal=" + sal + "]\n";
    }
}
```

applicationContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.0.xsd">
  <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
    <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
    <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
    <property name="username" value="scott"/>
    <property name="password" value="tiger"/>
  </bean>
  <bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
    <property name="dataSource" ref="dataSource"/>
  </bean>
  <bean id="empDao" class="com.nt.dao.ListEmpDaoImpl">
    <property name="jt" ref="template"/>
  </bean>
  <bean id="empService" class="com.nt.service.ListEmpService">
    <property name="dao" ref="empDao"/>
  </bean>
</beans>

```

Client.java

```

package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.ListEmpService;
public class ClientApp {
  public static void main(String[] args) {
    //create IOC Container
    ApplicationContext cTransaction management =new
    FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
    //get Bean
    ListEmpService service=(ListEmpService)cTransaction management
    .getBean("empService");
    //call b.methods
    System.out.println("7499 Emp Details "+service.searchEmpByNo(7499));
    System.out.println("All CLERK Detials "+service.searchEmpsByDesg("CLERK"));
  } //main
} //class

```

Jarfiles:

|----->spjdbc lib jarfile+

Two methods of jdbcTemplate to work with RowMapper, ResultSetExtractor callback interface for the above requirement.

```

public Object queryForObject (String sql, Object args[] RowMapper<T> rowmapper)
(useful to get Bo class object having single record)
public List<T> query(String sql, Object args[], ResultSetExtractor<T> extractor);

```

In real time the enduser supplied data will be received by service class in the form of Vo or DTO class object data.

After processing request Bo class object will be prepared to pass to DAO. DAO uses Bo class object data for persistence operations and gives another BO class object back to service class. This service class converts Bo class object data to DTO class object data to send to client.

VO, BO class object are not serializable to send over the network where as DTO class objects are serializable objects so we can send to network in D.

Batch processing/Updating: 16/12/15

Instead of performing multiple insert, update, delete operations on DB table by sending same query to database software for multiple times it is recommended to take the support of batch processing. Which sends query to database software for 1 time and sets batch of values to query for one time to reduce network roundtrip between java application and database software.

While working with JdbcTemplate we will use batch update method for this.

batchUpdate(String sql, List<Object []>args)

batchUpdate(String sql, ^{batch}PreparedStatementBatchSetter psbs) → callback interface

batchUpdate(String sql, List<object []> args, int [] args types) and etc.....

PreparedStatementBatchSetter is a callback interface that can be used to setValue to prepared statement provided by JdbcTemplate for each of a number of updates in a batch using same sql. It allows programmer to set batch values of query parameters in customized form by gathering values from java objects like Bo objects not from object[].

This callback interface gives 2 methods

setValues(PreparedStatement ps, int index)

Allows to set values to query params by specifying its index and by specifying query index to execute.

getBatchSize()

Returns numeric value to specify this number of times that setValues(-, -) methods should execute.

Ex: It getBatchSize() returns 5 then setValues(-) method executes for 5 times to assign 5 steps of values to batch. Later the query executes multiple times(5) in DB software having those batch of

* batch processing is only used in offline app not online apps.

* batch process never give guarantee of Tx mgmt.

* Adv of batch processing is reduce n/w round trips b/w db stuff jover app

* while working with simple stmt, we can place multiple queries in a batch

of batch processing

* while working with prepared stmt, we can't place multiple queries in the batch of batch processing but we can keep multiple sets of

query param values to the batch.

* batch processing is not required in online apps, required only in offline

apps like marketing census info.

* batchpreparedstatementsetter callback interface internally provides prepared stmt obj to add multiple sets of query param values to the batch.

DAOProj4(Batch Processing) 16/12/15

- src
 - com.nt.bo
 - StudentBO.java
 - com.nt.cfgs
 - applicationContext.xml
 - com.nt.dao
 - RegisterStudentsDao.java
 - com.nt.dto
 - StudentDTO.java
 - com.nt.service
 - RegisterStudentsService.java
 - com.nt.test
 - ClientApp.java
- JRE System Library [JavaSE-1.7]
 - DAOlib
 - ojdbc14.jar - D:\oracle\product\10.2.0\db_2\
 - spring-beans-4.1.6.RELEASE.jar - C:\Users\del
 - spring-context-4.1.6.RELEASE.jar - C:\Users\del
 - spring-context-support-4.1.6.RELEASE.jar - C:
 - spring-core-4.1.6.RELEASE.jar - C:\Users\del
 - spring-expression-4.1.6.RELEASE.jar - C:\Users
 - spring-jdbc-4.1.6.RELEASE.jar - C:\Users\del
 - spring-tx-4.1.6.RELEASE.jar - C:\Users\del\De
 - commons-logging.jar - C:\Users\del\Desktop
 - c3p0-0.9.1.jar - C:\Users\del\Desktop
 - commons-collections-3.1.jar - C:\Users\del\

StudentBO.java

```

package com.nt.bo;
public class StudentBO {
    private int sno;
    private String sname,sadd;
    public StudentBO() {
    }
    public StudentBO(int sno, String sname, String sadd) {
        this.sno = sno;
        this.sname = sname;
        this.sadd = sadd;
    }

    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {

```

```

    this.sname = sname;
}
public String getSadd() {
    return sadd;
}
public void setSadd(String sadd) {
    this.sadd = sadd;
}
}

```

RegisterStudentsDao.java

```

package com.nt.dao;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.List;
import org.springframework.jdbc.core.BatchPreparedStatementSetter;
import org.springframework.jdbc.core.JdbcTemplate;
import com.nt.bo.StudentBO;
public class RegisterStudentsDao {
    private static final String STUD_INSERT_QRY="insert into student values(?,?,?)";
    private JdbcTemplate jt;
    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }
    public boolean insert(5final List<StudentBO> listbo){
        6int result[]=jt.batchUpdate(STUD_INSERT_QRY, new BatchInserion(listbo));
        if(result!=null) 6
            7return true;
        else
            8return false;
    }

    private static final class BatchInserion implements BatchPreparedStatementSetter{
        private List<StudentBO> listbo;
        public BatchInserion(List<StudentBO> listbo){
            this.listbo=listbo;
        }
        @Override 7
        public int getBatchSize() {
            System.out.println("getBatchSize()");
            return listbo.size();
        }
    }
    @Override 9
    public void setValues(PreparedStatement ps, int index) 10
        throws SQLException {
        11System.out.println("setValues(-,-)");
        ps.setInt(1,listbo.get(index).getSno());
        ps.setString(2,listbo.get(index).getSname());
        ps.setString(3,listbo.get(index).getSadd());
    }
}

```

⁵ This method executes for multiple times to set values to query batch parameters in a batch. The no. of times that this method executes will be decided based on the return value of getBatchSize() method

```
    } //setValues(-,-)
  } //class
}
```

RegisterStudentsService.java

```
package com.nt.service;
import java.util.ArrayList;
import java.util.List;
import com.nt.bo.StudentBO;
import com.nt.dao.RegisterStudentsDao;
import com.nt.dto.StudentDTO;

public class RegisterStudentsService {
    private RegisterStudentsDao dao;
    public void setDao(RegisterStudentsDao dao) {
        this.dao = dao;
    }
    public String registerStudents(List<StudentDTO> listdto){
        //convert ListDTO to ListBO obj
        List<StudentBO> listbo=new ArrayList<StudentBO>();
        for(StudentDTO dto:listdto){
            StudentBO bo=new StudentBO(dto.getSno(),
                                           dto.getSname(),
                                           dto.getSadd());
            listbo.add(bo);
        }
        // Use DAO class
        boolean status=dao.insert(listbo);
        if(status){
            return "Batch insertion success";
        }
        else{
            return "Batch insertion failed";
        }
    } //method
}
```

StudentDTO.java

```
package com.nt.dto;
import java.io.Serializable;
public class StudentDTO implements Serializable {
    private int sno;
    private String sname,sadd;

    public StudentDTO() {
    }
    public StudentDTO(int sno, String sname, String sadd) {
        this.sno = sno;
    }
}
```

```
        this.sname = sname;
        this.sadd = sadd;
    }

    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.0.xsd">
    <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
        <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
        <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
        <property name="username" value="scott"/>
        <property name="password" value="tiger"/>
    </bean>
    <bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
        <property name="dataSource" ref="dataSource"/>
    </bean>
    <bean id="stDao" class="com.nt.dao.RegisterStudentsDao">
        <property name="jt" ref="template"/>
    </bean>
    <bean id="stService" class="com.nt.service.RegisterStudentsService">
        <property name="dao" ref="stDao"/>
    </bean>
</beans>
```

Client.java

```
package test;
import java.util.ArrayList;
```

```

import java.util.List;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.dto.StudentDTO;
import com.nt.service.RegisterStudentsService;

public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext cTransaction-management ctx = new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext
.xml");
        // get Bean
        1 RegisterStudentsService service = cTransaction-management ctx
ctx.2getBean("stService", RegisterStudentsService.class);
        // Prepare Batch of Studnets
        StudentDTO st1 = new StudentDTO(101, "raja", "hyd");
        StudentDTO st2 = new StudentDTO(102, "ravi", "hyd");
        List<StudentDTO> students = new ArrayList<StudentDTO>();
        students.add(st1); students.add(st2);
        //call B, method
        String result = service 3registerStudents(students);
        System.out.println(result);
    } //main
} //class

```

Student table

```

|---->Sno (n) (pk)
|---->Sname (VC2)
|---->Sadd (VC2)

```

In order to use local variables or parameters of the method definition inside the inner class we must take them as final variables.

NamedParameterJdbcTemplate :- 17-12-15

Spring jdbc supports 2 types of params in the SQL Query

- 1) Positional parameter (?)
- 2) Named Parameter (:<name>)

JdbcTemplate supports only positional parameters(?). we need to use var-args values or Object[] values to assign values to positional parameters based on their indexes. In this process if we confuse towards identifying the indexes there is a possibility of getting problems.

To overcome above problem the spring jdbc has given NamedParameterJdbcTemplate as wrapper around JdbcTemplate supporting Named parameters (:<varname>) in the query. This provides all the functionalities of JdbcTemplate but allows us to work with named parameters (:<varname>)

Eg: select * from emp where empno>=:min and empno<=:max;

The underlying jdbc of spring jdbc converts each name parameter into positional parameter before executing the query.

There are 2 ways to set values to NamedParameters

As Map<String, Object> obj

Specify the names of named parameters as keys and the objects as values in elements.

Eg: Map<String, Object> map=new HashMap<String, Object>();
 Map.put("min", 10);
 Map.put("Max", 100);

ii) By using the implementation class objects of SqlParameterSource (i)

i) MapSqlParameterSource

Allows to specify the name, values of named parameters as the args of add(-,-).

SqlParameterSource params=new MapSqlParameterSource();

params.addValue("min", 10);

params.addValue("max", 100);

BeanPropertySqlParameterSource

SqlParameterSource map = new BeanPropertySqlParameterSource();

Note: Allows to specify bean object values the named parameters values and the names of named parameters and BeanProperty names(member variable) must match

* DB sql doesn't support named parameters supports only positional params(?).

DAOProj5(NamedParameterJdbcTemplate) 17/12/15

- src
 - com.nt.bo
 - EmployeeBO.java
 - com.nt.cfgs
 - com.nt.dao
 - EmployeeDao.java
 - com.nt.dto
 - EmployeeDTO.java
 - com.nt.service
 - EmployeeService.java
 - com.nt.test
 - ClientApp.java
- JRE System Library [JavaSE-1.7]
- DAOlib

EmployeeBO.java

package com.nt.bo;

public class EmployeeBO {

private int eno,salary;

private String ename,job;

public EmployeeBO() {

}

public EmployeeBO(int eno, int salary, String ename, String job) {

this.eno = eno;

this.salary = salary;

this.ename = ename;

this.job = job;

}

public int getEno() {

return eno;

}

public void setEno(int eno) {

this.eno = eno;

}

```
public int getSalary() {
    return salary;
}
public void setSalary(int salary) {
    this.salary = salary;
}
public String getEname() {
    return ename;
}
public void setEname(String ename) {
    this.ename = ename;
}
public String getJob() {
    return job;
}
public void setJob(String job) {
    this.job = job;
}
}
```

EmployeeDAO.java

```
package com.nt.dao;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.HashMap;
import java.util.Map;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.jdbc.core.namedparam.BeanPropertySqlParameterSource;
import org.springframework.jdbc.core.namedparam.MapSqlParameterSource;
import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;
import com.nt.bo.EmployeeBO;
public class EmployeeDao {
    private static final String GET_EMP_NAME_QRY="select ename from emp where
    empno=:eno";
    private static final String GET_EMP_DETAILS_BY_NO_QRY="select empno,ename,job,sal
    from emp where empno=:eno";
    private static final String EMP_INSERT_QRY="insert into emp(empno,ename,job,sal)
    values(:eno,:ename,:job,:salary)";
    private NamedParameterJdbcTemplate njt;
    public void setNjt(NamedParameterJdbcTemplate njt) {
        this.njt = njt;
    }
    public String findName(int no){
        //prepare param values
        Map <String,Object> map=new HashMap<String,Object>();
        map.put("eno",no);
    }
}
```

```

String name=njt.queryForObject(GET_EMP_NAME_QRY,
                                map,
                                String.class);

return name;
}
public EmployeeBO findEmpDetails(int no){
    //prepare param values
    MapSqlParameterSource params=new MapSqlParameterSource();
    params.addValue("eno",no);
    //execute Query
    EmployeeBO ebo=njt.queryForObject(GET_EMP_DETAILS_BY_NO_QRY,
                                      params,
                                      new EmpRowMapper());

    return ebo;
}
private static final class EmpRowMapper implements RowMapper<EmployeeBO>{
    @Override
    public EmployeeBO mapRow(ResultSet rs, int pos) throws SQLException {
        // Store ResultSet record into BO obj
        EmployeeBO ebo=new EmployeeBO(rs.getInt(1),
                                      rs.getInt(4),
                                      rs.getString(2),
                                      rs.getString(3));

        return ebo;
    }
}
}
public int insertRecord(EmployeeBO ebo){
    //prepare param values
    BeanPropertySqlParameterSource params=
        new BeanPropertySqlParameterSource(ebo);
    int result=njt.update(EMP_INSERT_QRY, params);
    return result;
}
}

```

*here Bean Property ramer(ebo) and insert query named
sql params must match*

EmployeeService.java

```

package com.nt.service;
import com.nt.bo.EmployeeBO;
import com.nt.dao.EmployeeDao;
import com.nt.dto.EmployeeDTO;
public class EmployeeService {
    private EmployeeDao dao;
    public void setDao(EmployeeDao dao) {
        this.dao = dao;
    }
    public String searchNameByNo(int no){

```

```
//use DAO
String name=dao.findName(no);
return name;
}
public EmployeeDTO searchEmpByNo(int no){
    //use DAO
    EmployeeBO bo=dao.findEmpDetails(no);
    // convert BO into DTO
    EmployeeDTO dto=new EmployeeDTO(bo.getEno(),
                                     bo.getSalary(),
                                     bo.getEname(),
                                     bo.getJob());

    return dto;
}
public String registerEmp(int no,String name,String job,int sal){
    //create DTO
    EmployeeDTO dto=new EmployeeDTO(no,sal,name,job);
    // convert DTO to BO
    EmployeeBO bo=new EmployeeBO(dto.getEno(),
                                  dto.getSalary(),
                                  dto.getEname(),
                                  dto.getJob());

    //use DAO
    int result=dao.insertRecord(bo);
    if(result==1)
        return "Registration Success";
    else
        return "Registraion failed";
}
}
```

EmployeeDTO.java

```
package com.nt.dto;
import java.io.Serializable;
public class EmployeeDTO implements Serializable {
    private int eno,salary;
    private String ename,job;
    public EmployeeDTO() {
    }
    public EmployeeDTO(int eno, int salary, String ename, String job) {
        this.eno = eno;
        this.salary = salary;
        this.ename = ename;
        this.job = job;
    }
    public int getEno() {
        return eno;
    }
}
```

```
public void setEno(int eno) {
    this.eno = eno;
}
public int getSalary() {
    return salary;
}
public void setSalary(int salary) {
    this.salary = salary;
}
public String getEname() {
    return ename;
}

public void setEname(String ename) {
    this.ename = ename;
}
public String getJob() {
    return job;
}
public void setJob(String job) {
    this.job = job;
}
@Override
public String toString() {
    return "EmployeeDTO [eno=" + eno + ", salary=" + salary + ", ename="
        + ename + ", job=" + job + "];"
}
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.0.xsd">
    <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
        <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
        <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
        <property name="username" value="scott"/>
        <property name="password" value="tiger"/>
    </bean>
    <bean id="ntemplate"
        class="org.springframework.jdbc.core.namedparam.NamedParameterJdbc
        Template">
        <constructor-arg ref="dataSource"/>
    </bean>
    <bean id="empDao" class="com.nt.dao.EmployeeDao">
        <property name="njt" ref="ntemplate"/>
    </bean>
</beans>
```

```
<bean id="empService" class="com.nt.service.EmployeeService">
  <property name="dao" ref="empDao"/>
</bean>
</beans>
```

Client.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.EmployeeService;

public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext cTransaction management = new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //get Bean
        EmployeeService service = cTransaction management
        ctx.getBean("empService", EmployeeService.class);
        //call b.methods
        System.out.println("7499 emp name"+service.searchNameByNo(7499));
        System.out.println("7499 emp details"+service.searchEmpByNo(7499));
        System.out.println("Emp
        Registered?" + service.registerEmp(9999, "raja", "clerk", 8999));
    }
}
```

12/12/15

Working with SimpleJdbcInsert, SimpleJdbcCall:

Uses metadata concepts as simplifies environment to call PL/SQL procedures and to insert records.

SimpleJdbcInsert:

It uses Database table metadata concepts are used to insert records dynamically without insert SQL query. It just take table name, column name. column values from the programmers and uses them to generate dynamic SQL insert query to insert records. We can give these column names and values either as Map object or as SqlParameterSource implementation class objects. The map object contains (0) names as keys and column values as values.

To create SimpleJdbcInsert object DataSource object is dependent object and we can setTableName(-) to specify dbtable name and execute(-) to specify (0) names, (0) values and also to execute the dynamically generated Insert SQL query.

If our application deals with only registrations (insert operation) then we need to work with SimpleJdbcInsert class instead of JdbcTemplate.

SimpleJdbcTemplate: The Jdbc Template class of Spring 2.5 doesn't support JDK 1.5 features like varargs, autoboxing, Generics and --etc. Having support to these features ~~for~~ SimpleJdbcTemplate is given in Spring 2.5. The more commonly used functionalities of Jdbc Template class in Spring 3.x JdbcTemplate class is designed supporting JDK 1.5 features like varargs, autoboxing, Generics --. Due to this SimpleJdbcTemplate has been

18/12/15

DAOProj6(SimpleJdbcInsert)

- src
 - com.nt.bo
 - StudentBO.java
 - com.nt.cfgs
 - applicationContext.xml
 - com.nt.dao
 - StudentDao.java
 - com.nt.service
 - StudentService.java
 - com.nt.test
 - ClientApp.java
 - com.nt.vo
 - StudentVO.java
- JRE System Library [JavaSE-1.7]
- DAOlib

StudentBO.java

package com.nt.bo;

```

public class StudentBO {
    private int sno;
    private String sname;
    private String sadd;
    public StudentBO() {
    }
    public StudentBO(int sno, String sname, String sadd) {
        this.sno = sno;
        this.sname = sname;
        this.sadd = sadd;
    }
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
}

```

*JdbcTemplate, NamedParameterJdbc, SimpleJdbc

are thread-safe & all methods are Synchronised, so it degrades the performance

*SimpleJdbcInsert is non-thread safe and all methods are non-synchronised so performance is increased (it allows multiple threads at a time).

*In your app, you can take both SimpleJdbcInsert & JdbcTemplate

*SimpleJdbcInsert is multithread env. ...

StudentDAO.java

```
package com.nt.dao;
import java.util.HashMap;
import java.util.Map;
import org.springframework.jdbc.core.simple.SimpleJdbcInsert;
import com.nt.bo.StudentBO;
public class StudentDao {
    private SimpleJdbcInsert sji;
    public void setSji(SimpleJdbcInsert sji) {
        this.sji = sji;
    }
    public int insert(StudentBO bo){
        //prepare params for execute(-)
        Map<String,Object> map=new HashMap<String,Object>();
        map.put("sno",bo.getSno());
        map.put("sname",bo.getSname());
        map.put("sadd",bo.getSadd());
        //specify table name
        sji.setTableName("student");
        //execute the logic
        int result=sji.execute(map); //it generates dynamically sql query
        return result;
    } //insert(-)
} //class
```

StudentService.java

```
package com.nt.service;
import com.nt.bo.StudentBO;
import com.nt.dao.StudentDao;
import com.nt.vo.StudentVO;
public class StudentService {
    private StudentDao dao;
    public void setDao(StudentDao dao) {
        this.dao = dao;
    }
    public String registerStudent(String sno,String sname,String add){
        //prepare VO
        StudentVO vo=new StudentVO(sno,sname,add);
        //Convert VO obj to BO obj
        StudentBO bo=new StudentBO();
        bo.setSno(Integer.parseInt(vo.getNo()));
        bo.setSname(vo.getSname());
        bo.setSadd(vo.getSadd());
        //use DAO
        int result=dao.insert(bo);
        if(result==0)
            return "Registration failed";
        else
    }
```



```
        return "Registration success";
```

```
    } //method
```

```
 } //class
```

StudentVO.java

```
package com.nt.vo;
```

```
public class StudentVO {
```

```
    private String no,sname,sadd;
```

```
    public StudentVO(String no, String sname, String sadd) {
```

```
        super();
```

```
        this.no = no;
```

```
        this.sname = sname;
```

```
        this.sadd = sadd;
```

```
    }
```

```
    public StudentVO() {
```

```
    }
```

```
    public String getNo() {
```

```
        return no;
```

```
    }
```

```
    public void setNo(String no) {
```

```
        this.no = no;
```

```
    }
```

```
    public String getSname() {
```

```
        return sname;
```

```
    }
```

```
    public void setSname(String sname) {
```

```
        this.sname = sname;
```

```
    }
```

```
    public String getSadd() {
```

```
        return sadd;
```

```
    }
```

```
    public void setSadd(String sadd) {
```

```
        this.sadd = sadd;
```

```
    }
```

```
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```
http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
    <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
```

```
        <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
```

```
        <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
```

```
        <property name="username" value="scott"/>
```

```
        <property name="password" value="tiger"/>
```

```
    </bean>
```

```
    <bean id="simpleJdbcInsert" class="org.springframework.jdbc.core.simple.SimpleJdbcInsert">
```

```

<constructor-arg ref="dataSource"/>
</bean>
<bean id="stDao" class="com.nt.dao.StudentDao">
  <property name="sjl" ref="simpleJdbcInsert"/>
</bean>
<bean id="stService" class="com.nt.service.StudentService">
  <property name="dao" ref="stDao"/>
</bean>
</beans>

```

Client.java

```

package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.StudentService;
public class ClientApp {
    public static void main(String[] args) {
        //Create IOC container
        ApplicationContext cTransaction management = new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //get ServiceBean
        StudentService service=cTransaction management
        .getBean("stService", StudentService.class);
        //call B.method
        System.out.println("Result="+service.registerStudent("1010", "Ramesh", "hyd"));
    }
}

```

18/12/15

✓ VO class is a java bean that doesn't implements Serializable (i) and it's object represents input values coming to the controller Application.

DTO class is java bean that implements Serializable (i) and it's object represents input values or output values to from 1 resource to another resource over the network.

Generally the VO class objects contains String properties holding all the zip values given by enduser like form data, DTO class contains all types of Properties and Bo class contains all types of properties (But these properties types and DB table column types generally should match) SimpleJdbcInsert internally uses JdbcTemplate while completing the persistence operation. JdbcTemplate, SimpleJdbcTemplate, NamedParameterJdbcTemplate class objects are thread safe once configured i.e they allow one thread at a time to perform persistence operations which degrades the performance.

To overcome this problem we can use SimpleJdbcInsert or SimpleJdbcCall whose objects are multithreaded allowing multiple thread symultaneously.

✓ Usecase:

When we ask visitors to register in timeframe (like online web counsiling) we need to allow symultaneous record insertion for better performance so prefer using SimpleJdbcInsert in that situation.

* In one DAO class, we can place both place JdbcTemplate and SimpleJdbcTemplate. we use JdbcTemplate for non-insert operations. and we use SimpleJdbcInsert for simultaneous record insertion.

SimpleJdbcCall: 19/12/15

It is a multithreaded, reusable object representing call to PL/SQL procedure or function. It runs on metadata concepts and simplifies the code that is required to call procedure or function. We just need to provide procedure or function name with map of in parameters to call procedure or function and this returns a Map having out parameter named and values.

Instead of writing common persistence in every module as sqlqueries, it is recommended to write only for one time in db software as PL/SQL procedure or function.

Ex: Instead of writing authentication logic in every module separately, write it only once as PL/SQL procedure or function and use it in multiple modules of a project.

PL/SQL procedures:

- ed > Create or replace procedure getDetails (no in number, name out varchar2, address out varchar2) as
 Begin
 Select sname, sad into name, address from Student where sno=no;
 end;

DAOProj7(SimpleJdbcCall) 19/12/15

- src
 - com.nt.bo
 - StudentBO.java
 - com.nt.cfgs
 - applicationContext.xml
 - com.nt.dao
 - StudentDao.java
 - com.nt.service
 - StudentService.java
 - com.nt.test
 - ClientApp.java
 - com.nt.vo
 - StudentVO.java
- JRE System Library [JavaSE-1.7]
- DAOLib

Assignment

Use SimpleJdbcCall support to call PL/SQL procedure or function that performs authentication.

StudentBO.java

```
package com.nt.bo;

public class StudentBO {
    private int sno;
    private String sname;
    private String sadd;
    public StudentBO() {
    }
    public StudentBO(int sno, String sname, String sadd) {
        this.sno = sno;
        this.sname = sname;
        this.sadd = sadd;
    }
    public int getSno() {
        return sno;
    }
}
```

```

public void setSno(int sno) {
    this.sno = sno;
}
public String getSname() {
    return sname;
}
public void setSname(String sname) {
    this.sname = sname;
}
public String getSadd() {
    return sadd;
}
public void setSadd(String sadd) {
    this.sadd = sadd;
}
}

```

StudentDAO.java

```

package com.nt.dao;
import java.util.HashMap;
import java.util.Map;
import org.springframework.jdbc.core.simple.SimpleJdbcCall;
import org.springframework.jdbc.core.simple.SimpleJdbcInsert;
import com.nt.bo.StudentBO;
public class StudentDao {
    private SimpleJdbcCall sjc;
    public void setSjc(SimpleJdbcCall sjc) {
        this.sjc = sjc;
    }
    public StudentBO getStudentDetails(int id) {
        // set procedure name
        sjc.setProcedureName("getDetails");
        //Prepare IN params
        Map <String, Object> inParams=new HashMap<String, Object>();
        inParams.put("no", id);
        // call PL/sql procedue
        Map<String, Object>outParams=sjc.execute(inParams);
        // Place OutParam values in BO obj
        StudentBO bo=new StudentBO(id,
        (String)outParams.get("NAME"),
        (String)outParams.get("ADDRESS"));
        return bo;
    }
    }
}

```

This method generates query that calls PL/SQL procedure + sets given in param values and returns map obj having out-parameter names, values by executing procedure.

StudentService.java

```
package com.nt.service;
import com.nt.bo.StudentBO;
import com.nt.dao.StudentDao;
import com.nt.dto.StudentDTO;
public class StudentService {
    private StudentDao dao;
    public void setDao(StudentDao dao) {
        this.dao = dao;
    }
    public StudentDTO seachStudentByNo(int no){
        //use DAO
        StudentBO bo=dao.getStudentDetails(no);
        //Convert BO obj to DTO obj
        StudentDTO dto=new StudentDTO(bo.getSno(),
            bo.getSname(),
            bo.getSadd());
        return dto;
    }
}
} //class
```

StudentDTO.java

```
package com.nt.dto;
public class StudentDTO {
    private int no;
    private String sname,sadd;
    public StudentDTO(int no, String sname, String sadd) {
        super();
        this.no = no;
        this.sname = sname;
        this.sadd = sadd;
    }
    public StudentDTO() {
    }
    public int getNo() {
        return no;
    }
    public void setNo(int no) {
        this.no = no;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
```

```

        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
    @Override
    public String toString() {
        return "StudentDTO [no=" + no + ", sname=" + sname + ", sadd=" + sadd
            + "]\n";
    }
}

```

applicationContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
        <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
        <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
        <property name="username" value="scott"/>
        <property name="password" value="tiger"/>
    </bean>
    <bean id="simpleJdbcCall" class="org.springframework.jdbc.core.simple.SimpleJdbcCall">
        <constructor-arg ref="dataSource"/>
    </bean>
    <bean id="stDao" class="com.nt.dao.StudentDao">
        <property name="sjc" ref="simpleJdbcCall"/>
    </bean>
    <bean id="stService" class="com.nt.service.StudentService">
        <property name="dao" ref="stDao"/>
    </bean>
</beans>

```

Client.java

```

package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.StudentService;
public class ClientApp {
    public static void main(String[] args) {
        //Create IOC container
        ApplicationContext ctx = new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //get ServiceBean
    }
}

```

```

StudentService service=Transaction management
ctx.getBean("stService",StudentService.class);
//call B.method
System.out.println("1001 student details"+service.seachStudentByNo(1001));
} //main
} //class

```

Mapping SQL operations as Sub Classes: 20/12/15

This is call technique in spring jdbc to perform persistence operations. This technique make programmer to develop sub classes where we set DataSource, declare parameters and compile query only for 1 time but we execute query multiple times by reusing the object of subclass for multiple times. This concept is similar to JDO (Java Data Objects) programming.

Spring jdbc supplies SqlQuery, SqlUpdate classes to develop these sub classes. Use SqlQuery for select operations, use SqlUpdate for non-select operations.

While working with JdbcTemplate, NamedParameterJdbcTemplate, SimpleJdbcTemplate classes if same query is executed for multiple time then it creates multiple jdbc preparedStatements internally on 1 preparedStatement object for each execution basis. This is bad practice and also degrades the performance. To overcome this problem we can use "Mapping SQL operations as sub classes" Technique.

SQLQuery: 20/12/15

We develop separate subclass extending from SQLQuery for each select query inside DAO class.

This subclass supplies datasource, query to superclass to executes query for multiple times by compiling the query only for 1 time. SqlQuery supplies executes methods to execute query that gives list objects (multiple records) and also give finder(-) methods to execute query that gives only one object (one record). This class also gives mapRow(-, -) method which can be used to process each record of ResultSet object. To executes execute() method internally it calls mapRow(-, -) method. Since SqlQuery in abstract class having more abstract methods we prefer using "MappingSqlQuery" which subclass in the inheritance hierarchy of SqlQuery.

When developing subclass for SqlQuery/MappingSqlQuery classes we use following methods in the subclasses.

```

declareParameter (-)
compile()
executeXxx(-)
findXxx(-) methods and etc.....

```

This technique gives every thing to super class from subclass for query execution and reuse subclass object.

* while working with mapping SQL operations as sub classes technique for every SQL query we need to take 1 inner class in DAO class as the sub class of SqlQuery/mapping SqlQuery (for select operation) or as the subclass of SqlUpdate (for non-select operation).

DAOProj8(MappingSQLOperatios as Sub classes, 20/12/15

- src
 - com.nt.bo
 - StudentBO.java
 - com.nt.cfgs
 - applicationContext.xml
 - com.nt.dao
 - StudentDao.java
 - com.nt.dto
 - StudentDTO.java
 - com.nt.service
 - StudentService.java
 - com.nt.test
 - ClientApp.java
- JRE System Library [JavaSE-1.7]
- DAOlib

StudentBO.java

```
package com.nt.bo;

public class StudentBO {
    private int sno;
    private String sname;
    private String sadd;
    public StudentBO() {
    }

    public StudentBO(int sno, String sname, String sadd) {
        this.sno = sno;
        this.sname = sname;
        this.sadd = sadd;
    }

    public int getSno() {
        return sno;
    }

    public void setSno(int sno) {
        this.sno = sno;
    }

    public String getSname() {
        return sname;
    }

    public void setSname(String sname) {
        this.sname = sname;
    }

    public String getSadd() {
        return sadd;
    }

    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
}}
```


20/12

StudentDAO.java

```

package com.nt.dao;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Types;
import java.util.List;
import javax.sql.DataSource;
import org.springframework.jdbc.core.SqlParameter;
import org.springframework.jdbc.object.MappingSqlQuery;
import org.springframework.jdbc.object.SqlUpdate;
import com.nt.bo.StudentBO;

public class StudentDao {
    private static final String GET_STUDETNS_BY_ADDRESS="select * from student where
sadd=?";

    private DataSource ds;
    private StudentSelector selector;
    private StudentUpdater updater;

    public StudentDao(DataSource ds) {
        this.ds=ds;
        selector=new StudentSelector(ds,GET_STUDETNS_BY_ADDRESS);
    }

    //DAO class method
    public List<StudentBO> findStudentsByAddress(String addr){
        List<StudentBO> listbo=selector.findByAddress(addr);
        return listbo;
    }

    private static final class StudentSelector extends MappingSqlQuery<StudentBO>{
        public StudentSelector(DataSource ds,String qry){
            //supply data source, query details to super class
            super(ds,qry);
            super.declareParameter(new SqlParameter(Types.VARCHAR));
            super.compile();
        }

        @Override
        protected StudentBO mapRow(ResultSet rs, int index) throws SQLException {
            System.out.println("mapRow(-)");
            return new StudentBO(rs.getInt(1),
rs.getString(2),
rs.getString(3));
        }

        public List<StudentBO> findByAddress(String addr){
            List<StudentBO> listbo=super.execute(addr);
            return listbo;
        }
    }
}
//inner class/sub class
//class

```

later that inner class obj will be utilized for multiple times for query execution. so it is far better than the direct methods or jdbc template class which compiles the same sql query for multiple times

when ioc container creates dao class, the inner class obj is created in this process the data source, query will be supplied to super class to compile the query in db only for 1 time

where data source must be injected through constructor injection to pass to the inner class

inner class and subclass taken for each sql query

for each

//supply data source, query details to super class

super(ds,qry);

super.declareParameter(new SqlParameter(Types.VARCHAR));

super.compile(); makes super class to compile sql query by sending the

to db slw

@Override

This method will be protected StudentBO mapRow(ResultSet rs, int index) throws SQLException {
called by execute() method internally and automatically while processing each record of result set

System.out.println("mapRow(-)");

return new StudentBO(rs.getInt(1),

rs.getString(2),

rs.getString(3));

public List<StudentBO> findByAddress(String addr){

List<StudentBO> listbo=super.execute(addr);

return listbo;

}

//inner class/sub class

//class

StudentService.java

```
package com.nt.service;
import java.util.ArrayList;
import java.util.List;
import com.nt.bo.StudentBO;
import com.nt.dao.StudentDao;
import com.nt.dto.StudentDTO;
public class StudentService {
    private StudentDao dao;
    public void setDao(StudentDao dao) {
        this.dao = dao;
    }
    public List<StudentDTO> searchStudentsByAddress(String addr){
        //use DAO (2)
        List<StudentBO> listbo=dao.findStudentsByAdderess(addr);
        //convert listBO obj to listDTO obj (3)
        List<StudentDTO> listdto=new ArrayList<StudentDTO>();
        for(StudentBO bo:listbo){
            StudentDTO dto=new StudentDTO(bo.getSno(),
                                                bo.getSname(),
                                                bo.getSadd());
            listdto.add(dto);
        }
        return listdto;
    } //method (4)
} //class
```

StudentDTO.java

```
package com.nt.dto;
public class StudentDTO {
    private int no;
    private String sname,sadd;
    public StudentDTO(int no, String sname, String sadd) {
        super();
        this.no = no;
        this.sname = sname;
        this.sadd = sadd;
    }
    public StudentDTO() {
    }
    public int getNo() {
        return no;
    }
    public void setNo(int no) {
        this.no = no;
    }
    public String getSname() {
```

20/12

```

        return sname;
    }
    public void setName(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
    @Override
    public String toString() {
        return "StudentDTO [no=" + no + ", sname=" + sname + ", sadd=" + sadd
            + "]";
    }
}

```

Client.java

```

package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.StudentService;
public class ClientApp {
    public static void main(String[] args) {
        //Create IOC container
        ApplicationContext cTransactionManagement = new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //get ServiceBean
        StudentService service = cTransactionManagement
        cfm.getBean("stService", StudentService.class);
        //call B.method
        System.out.println("Hyd student details" + service.searchStudentsByAddress("hyd"));
    }
}

```

applicationContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
        <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
        <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
        <property name="username" value="scott"/>
        <property name="password" value="tiger"/>
    </bean>
    <bean id="stDao" class="com.nt.dao.StudentDao">

```

```
<constructor-arg ref="dataSource"/>
</bean>
<bean id="stService" class="com.nt.service.StudentService">
  <property name="dao" ref="stDao"/>
</bean>
</beans>
```

SqlUpdate: 21/12/15

It is similar to SqlQuery class but given for non-select queries (update queries) execute. We can prepare subclass to SqlUpdate class inside the DAO class where we supply dataSource, query to SqlUpdate class only once. But we use these subclass objects for multiple times to executes same nonselect query with the support of update(-) methods.

SqlUpdate is a concrete class and supplies lots of updates(-) methods to execute non-select query with same or different parameters for multiple times.

For every non-select sqlquery we need to take separate sub class for SqlUpdate inside the DAO class.

Why should we learn SimpleJdbcInsert, SimpleJdbcCall and MappingSQL operations as subclasses techniques even though JdbcTemplate class is already available?

Ans: JdbcTemplate class object is single threaded object/ threadsafe object i.e it allows one thread at a time to perform the persistence operations. To overcome this problem use SimpleJdbcInsert, SimpleJdbcCall methods.

When we use JdbcTemplate class object to execute same query for multiple times it internally creates multiple PreparedStatement object on one object for each execution basis which is a bad practice.

To overcome this problem use MappingSql Operations as Subclasses technique.

21/12/15
DAOPorj9 (SqlUpdate operation us subclass)

- src
 - com.nt.bo
 - StudentBO.java
 - com.nt.cfgs
 - applicationContext.xml
 - com.nt.dao
 - StudentDao.java
 - com.nt.dto
 - StudentDTO.java
 - com.nt.service
 - StudentService.java
 - com.nt.test
 - ClientApp.java
- JRE System Library [JavaSE-1.7]
- DAOLib

StudentBO.java

```
package com.nt.bo;
public class StudentBO {
    private int sno;
    private String sname;
    private String sadd;
    public StudentBO() {
    }
    public StudentBO(int sno, String sname, String sadd) {
        this.sno = sno;
        this.sname = sname;
        this.sadd = sadd;
    }
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
}
```

StudentDAO.java

```
package com.nt.dao;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Types;
import java.util.List;
import javax.sql.DataSource;
import org.springframework.jdbc.core.SqlParameter;
import org.springframework.jdbc.object.MappingSqlQuery;
import org.springframework.jdbc.object.SqlUpdate;
import com.nt.bo.StudentBO;
public class StudentDao {
    private static final String UPDATE_STUDENT_ADDRS_BY_NO="update student set sadd=?
where sno=?";
```

```
private DataSource ds;
StudentUpdator updator;
public StudentDao(DataSource ds) {
    this.ds=ds;
    updator=new StudentUpdator(ds,UPDATE_STUDENT_ADDR_BY_NO);
}

public int updateStudentDetails(int no,String newAddrs){
    int cnt=updator.updateDetails(no, newAddrs);
    return cnt;
}
```

inner class

```
private static final class StudentUpdator extends SqlUpdate{
```

this constructor gives DataSource, query to open class and also makes it to compile the query on once

```
    public StudentUpdator(DataSource ds,String qry) {
        //provide DataSource, Query to super class
        super(ds,qry);
        //Declare Parameters and given to super class
        super.declareParameter(new SqlParameter(Types.VARCHAR));
        super.declareParameter(new SqlParameter(Types.INTEGER));
        //compile the Query
        super.compile();
    }
```

```
    public int updateDetails(int no,String newAddrs){
```

```
        int cnt=super.update(newAddrs,no);
```

```
        return cnt; ↳ method of sqlupdate class which actually executes
```

the already compiled sql query by setting given argument value as the query param value

```
    }
}
//class
```

StudentService.java

```
package com.nt.service;
import java.util.ArrayList;
import java.util.List;
import com.nt.bo.StudentBO;
import com.nt.dao.StudentDao;
import com.nt.dto.StudentDTO;
public class StudentService {
    private StudentDao dao;
    public void setDao(StudentDao dao) {
        this.dao = dao;
    }
    public String modifyStudentDetails(int no,String newAddrs){
        int cnt=dao.updateStudentDetails(no, newAddrs);
        if(cnt==0)
            return no+" Student details are not updated";
        else
            return no+" student details are updated";
    }
}
//method
//class
```

StudentDTO.java

```
package com.nt.dto;
public class StudentDTO {
    private int no;
    private String sname,sadd;
    public StudentDTO(int no, String sname, String sadd) {
        super();
        this.no = no;
        this.sname = sname;
        this.sadd = sadd;
    }
    public StudentDTO() {
    }
    public int getNo() {
        return no;
    }
    public void setNo(int no) {
        this.no = no;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
    @Override
    public String toString() {
        return "StudentDTO [no=" + no + ", sname=" + sname + ", sadd=" + sadd
            + "]\n";
    }
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
        <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
        <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
        <property name="username" value="scott"/>
    </bean>
</beans>
```

```
<property name="password" value="tiger"/>
</bean>
<bean id="stDao" class="com.nt.dao.StudentDao">
  <constructor-arg ref="dataSource"/>
</bean>
<bean id="stService" class="com.nt.service.StudentService">
  <property name="dao" ref="stDao"/>
</bean>
</beans>
```

Client.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.StudentService;
public class ClientApp {
    public static void main(String[] args) {
        //Create IOC container
        ApplicationContext cTransactionManagement = new FileSystemXmlApplicationContext
        ("src/com/nt/configures/applicationContext.xml");
        //get ServiceBean
        StudentService service = cTransactionManagement.getBean("stService", StudentService.class);
        //call B.method
        System.out.println(service.modifyStudentDetails(1001, "hyd11"));
    } //main
} //class
```


21/12/15 Spring AOP

AOP is the methodology of programming that makes the programmer primary logics of application from the secondary logics or helper logics. AOP is not replacement for OOP more ever it complements OOP. In OOP we develop business methods in classes by mixing up both primary, secondary logics. Due to this business methods becomes heavy and the reusability of secondary logics will be killed and we cannot enable or disable secondary logics without touching source code of business method. To overcome above method use AOP that makes the programmer to write primary logics in separate class and secondary logics in separate class and allows to mix and bind both logics dynamically at runtime by generally new classes.

OOP example:

```
class Bank{
public boolean withdraw (int acctno, int amt)
{
//security logic
//logging logic
//Transaction logic
Bal=bal-amt; //primary logic
}
public boolean withdraw deposit (int acctno, int amt)
{
//security logic
//logging logic
//Transaction logic
Bal=bal+amt; //primary logic
}
}
```

AOP example

```
class Bank
```

```
{
withdraw (acctno, amt) primary
{
bal=bal-amt; // primary logic
}
deposit (actno, amt)
{
bal=bal+amt; // primary logic
}
}
```

```
class Security
```

```
{
.....
.....//secondary
.....
}
```

AOP Implementing
software
(or)
AOP enabled
SLW

overcreates
In-Memory
Proxy Class
having both
primary &
secondary logics

Proxy
class

Proxy class

```
public class Bank$Proxy extends Bank {
public boolean withdraw (int acctno, float amt) {
// security logic
// logging logic
bal = bal - amt; // primary logic
}
```

```
public boolean deposit (-, -) {
// security
// logging logic
bal = bal + amt;
}
```

```
class Logging
```

```
{
.....
.....
}
```

22/12/15

The logic that is minimum to create the task is called the primary logic. Without primary logic task will not be completed.

The logic that supports primary logic and makes primary logic as better logic but optional to have in the application is called secondary logic.

While performing withdraw amount operation deducting amount from balance is called primary logic.

Checking the identity of user, logging, auditing and etc are called secondary logics.

Since we need secondary logic in multiple component classes of application or project they can be also called as cross cutting concerns because they need to be applied across the multiple classes.

For example scenario that speaks about need of AOP with bank account use case.

AOP is not replacement for OOP, moreover it complements OOP.

The key component in OOP is class. The key component in AOP Aspect (cross cutting that represents primary logics).

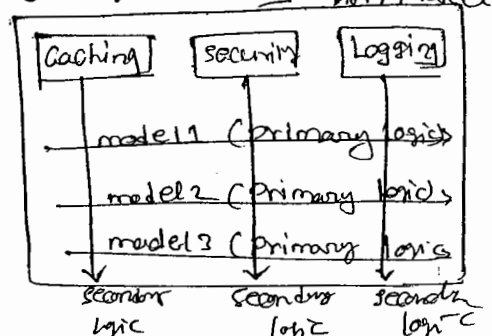
The language that is designed based on OOP principle like abstraction, encapsulation, inheritance and etc... is called OOP enable languages.

Similarly OOP principle we also have AOP principles. The software that supports and implements OOP principles is called AOP enable software. AOP is not just specific to spring. There are multiple AOP enabled softwares like spring AOP, AspectAOP, JBOSS AOP, JAC (java Aspect component), myzilla AOP and etc...

22/12/15

AOP principles/Terminologies *fig: cross cutting*

1. Aspect
2. JoinPoint
3. Advice
4. Pointcut
5. Target class
6. Weaving
7. Proxy class



Aspect means services 1, 2, 3, 4 ⇒ Joinpoints

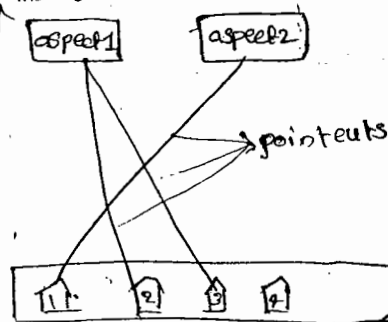


Fig: Target class execution

Aspect:

The logic that has to be applied across the multiple classes of application is called Aspect. This logics is also called as crosscutting concern, secondary logic, middleware service logic.

Ex: Security, logging, transaction and etc...

JoinPoint:

The possible points in classes on which aspect logics can be applied is called joinpoints. They are like fields, constructors, methods and etc....

Spring supports only methods as joinPoint.

Advice:

It specifies or indicates an aspect should be applied on which JoinPoint like applying aspect before executing method or applying at the end of method and etc.....

There are 4 types of advices like

BeforeAdvice, AfterAdvice, ThrowsAdvice, AroundAdvice.

PointCut:

It is a collection of JoinPoint on which aspects are advised (i.e aspects are configured).

Target Class:

The class on which we want to advice the aspects. It is pre-AOP java class having primary logics in methods. If we call methods of target class only primary logic will be execute.

Weaving:

The process of advising aspects on the target class to build proxy class is called weaving. It is all about combining multiple things to build one 1 proxy class.

Proxyclass:

The outcome class of weaving process where primary logics and secondary logics aspects will be mixed up dynamically at runtime. It is post-AOP class. If we call methods on proxy class object the primary logic will be executed along with secondary logic

Aspect weaver is an unit of AOP forward like spring AOP, AspectAOP and etc.. It uses some runtime libraries like jdk libraries or cglib libraries to generate proxy classes through weaving.

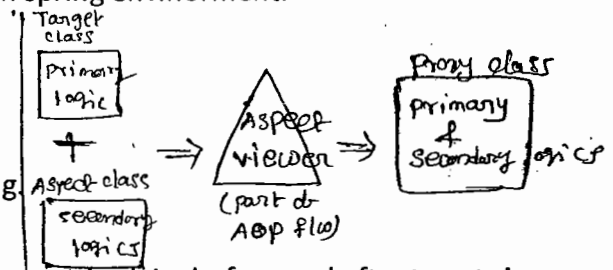
This proxy classes are dynamically generated inmemory classes.

Spring1.x gives spring AOP to work with AOP. From spring 2.x AspectAOP has been integrated with spring. So spring 2.x we can use spring AOP and AspectJAOP to work with AOP cohcepts.

There are 4 approaches to implement AOP concepts in spring envinorment.

1. Spring AOP Declaraive (xml)
2. Spring AOP programmatic (javacode)
3. Spring integrated AspectJ AOP declarative (xml)
4. Spring integrated AspectJ AOP Annotatins (@)

There are 4 types of advices in AOP style programming



AroundAdvice:

Here advice logic logic executes Around the target class method, i.e before and after target class method (Target method)

BeforeAdvice:

Here Advice logic executes before executing the target class method. So once target method is executed control does not come back to advice logic/method.

AfterAdvice:

Here advice logic executes after executing target method and before it returns the control back to caller.

ThrowsAdvice:

Here the Advice logic executes when exception is raised in target method.

Spring AOP declarative Approach:

Here we use xml file to configure Aspect, target classes and to configure the classes that perform weaving.

Around Advice:

Advice indicates the action taken by Aspect on JoinPoint. So Around advice makes its aspects to execute around the JoinPoint i.e Aspect/Advice logic executes before entering to target method and after executing the target method.

This Around advice is useful to implement "perfomance" monitoring, caching, logging and etc usecase:

To develop this Advice/Aspect we need to take a class that implements MethodInterceptor(i) should provide invoke (methodInvocation invocat) as shown below.

Sample Code:

```
public class myAdvice implements MethodInterceptor{
    public Object invoke (MethodInvocation invocation)
    {
        .....
        ..... //logic that executes before entering into target method.
        Object retval=invocation.proceed(); //calls target method
        .....
        .....
        ..... //logic that executes after finishing the execution of target method.
        .....
    }
}
```

Using MethodInvocation obj we can call getName(), getArguments() method names and its arguments. We can call proceed() method to call target method from advice class.

3 important points/control points of around advice

Can access and modify target method arguments.

Can control target method execution i.e we can skip or proceed target method execution being from advice class.

Can access and modify the target method return value.

Jar file: IOCLib jar file+ spring-aop-<ver>.Release.jar+spring-aspect-<ver>.Release.jar+com.springsource.org/aopalliance-1.0.0.jar

Procedure to spring AOP Application in Declarative Approach:

- a) Develop target class having method with primary logic.
 - b) Develop Aspect/Advice class having secondary logic.
 - c) Perform configurations in spring bean configuration file for weaving
 - i) Configure target class
 - ii) Configure Aspect/Advice class
 - iii) Configure proxyFactoryBean class supplying target, Aspect classes to generate proxy class through weaving
 - d) Develop clientApp for testing
 - e) Create IOC container
 - i) Get proxy class object by supplying the bean id of "proxyFactoryBean"
 - ii) Call target class methods on proxy object.
- In spring JoinPoints are methods of classes where Aspects can be advised.
 Pointcut specifies which aspect or aspects should be applied on which JoinPoint or JoinPoints.
 Flow of execution with respect to AOP proj1 of booklet 2 by invoking the generated proxy class:

refer last pages of booklet

- AOPProj1 (Spring AOP-Decl-AroundAdvice) 23/12/15
 src
 com.nt.aspect
 CacheAdvice.java
 LogAroundAdvice.java
 PerformanceMonitoringAdvice.java
 com.nt.cfgs
 applicationContext.xml
 com.nt.service
 IntrAmtCalculator.java
 com.nt.test
 ClientApp.java
 JRE System Library [JavaSE-1.7]
 AOPlib
 spring-aop-4.1.6.RELEASE.jar - C:\Users\de
 spring-aspects-4.1.6.RELEASE.jar - C:\Users
 spring-beans-4.1.6.RELEASE.jar - C:\Users\de
 spring-context-4.1.6.RELEASE.jar - C:\Users
 spring-context-support-4.1.6.RELEASE.jar -
 spring-core-4.1.6.RELEASE.jar - C:\Users\de
 spring-expression-4.1.6.RELEASE.jar - C:\U
 commons-logging.jar - C:\Users\de\Desk
 com.SpringSource.org\aspectalliance-1.0.0.jar

CacheAdvice.java

```

package com.nt.aspect;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;
import org.aopalliance.intercept.MethodInterceptor;
import org.aopalliance.intercept.MethodInvocation;
public class CacheAdvice implements MethodInterceptor {
    Map <String, Object> cache=new HashMap<String, Object>();
    @Override
    public Object invoke(a2MethodInvocation invocation) throws Throwable {
        String key=invocation.getMethod().getName()+Arrays.toString(invocation.getArguments());

        Object retValue=null;
        if(!cache.containsKey(key)){ a3? b3?
            retValue=invocation.proceed(); a4
            cache.put(key, retValue);
a5 {
                System.out.println("From target method");
            }
            return retValue;
        }
        retValue=cache.get(key); b4
        System.out.println("From cache");
        return retValue; b5
    }
}
  
```

cache (map)

calcIntrAmt	52F00
key	value

12/11/15
LogAroundAdvice.java
package com.nt.aspect;
import java.util.Arrays;
import org.aopalliance.intercept.MethodInterceptor;
import org.aopalliance.intercept.MethodInvocation;
public class LogAroundAdvice implements MethodInterceptor{

@Override

public Object invoke(MethodInvocation invocation) throws Throwable {
 System.out.println("Entering into "+invocation.getMethod().getName()+" with
args"+Arrays.toString(invocation.getArguments()));

// Modify argument values

Object args[]=invocation.getArguments();
if(((Float)args[0])<50000){
 args[1]=1.0f;
}

Object retValue=null; controlling target method execution

if(((Float)args[0])==0.0f || ((Float)args[1])==0.0f || ((Float)args[2])==0.0f){
 return 0.0f;
}

else{

retValue=invocation.proceed();

}

System.out.println("Leaving from "+invocation.getMethod().getName()+" with
args"+Arrays.toString(invocation.getArguments()));

if(((Float)retValue)<=25000)
 return retValue;

else

return ((Float)retValue)+(((Float)retValue)*0.1f);

}

}

23/12/15
PerformanceMonitoringAdvice.java

package com.nt.aspect;

import org.aopalliance.intercept.MethodInterceptor;

import org.aopalliance.intercept.MethodInvocation;

public class PerformanceMonitoringAdvice implements MethodInterceptor{

long start,end;

@Override

public Object invoke(MethodInvocation invocation) throws Throwable {

start=System.currentTimeMillis();

Object retValue=invocation.proceed();

end=System.currentTimeMillis();

System.out.println(invocation.getMethod().getName()+"has taken "+(end-start)+"ms
to complete execution");

return retValue;

}

}

24/12/14 IntrAmtCalculator.java

```
package com.nt.service;
public class IntrAmtCalculator {
    public float calcIntrAmt(float principle, float rate, float time) {
        System.out.println("Target class : Method");
        return (principle * rate * time) / 100.0f;
    }
}
```

24/12/14 applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.0.xsd">
    <bean id="target" class="com.nt.service.IntrAmtCalculator"/>
    <bean id="logAdvice" class="com.nt.aspect.LogAroundAdvice"/>
    <bean id="pmAdvice" class="com.nt.aspect.PerformanceMonitoringAdvice"/>
    <bean id="cacheAdvice" class="com.nt.aspect.CacheAdvice"/>
    <bean id="pfb" class="org.springframework.aop.framework.ProxyFactoryBean">
        <property name="target" ref="target"/>
        <property name="interceptorNames">
            <list>
                ③ <value>pmAdvice</value>
                ② <value>logAdvice</value>
                ① <value>cacheAdvice</value>
            </list>
        </property>
    </bean></beans>
```

⇒ cache advice will always encounter before if the data is present at cache memory there is no need of calculating performance & logging of the application

⇒ cache is the temp memory that stores the data temporarily.

24/12/14 Client App.java

```
package test;
import java.util.Arrays;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.IntrAmtCalculator;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext cTransactionManagement = new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //get Bean
        IntrAmtCalculator proxy = cTransactionManagement
        at.getBean("pfb", IntrAmtCalculator.class);
        //Call B.method
        float intrAmt = proxy.calcIntrAmt(80000, 2, 20); ①
        System.out.println("IntrAmt = " + intrAmt);
        System.out.println("-----");
        float intrAmt1 = proxy.calcIntrAmt(80000, 2, 20); ②
    }
}
```

```

    }//main
} //class

```

method	method, object args[], object target)
1 etc	1 represents
target method	target method
	target class
	object

The important /control point of beforeAdvice:

- ① → Can access n modify target method arguments
- ② → Can not access modify target method return value because the control doesn't come back to before advice once the target method execution is completed.
- ③ → Can not control target method execution being from advice but can stop/abort target method execution by throwing exception in the before (-,-,-) method.
 - Logging keep tracks of various components that are involved in the flow of execution of App.
 - Auditing keeps track of various activities that are taken place in the execution of application. Generally the audit info will be written to the audit log file or audit table, it's like username, Ordertime, ordered and. etc

25/12/15

AOPProj3(Before Advice-Auditing)

src

com.nt.aspect

AuditingAdvice.java

com.nt.cfgs

applicationContext.xml

com.nt.service

OrderApprover.java

com.nt.test

ClientApp.java

JRE System Library [JavaSE-1.7]

AOPlib + com.springSource.org.eopalliance-1.0.0.jar

AuditingAdvice.java

package com.nt.aspect;

import java.io.FileWriter;

import java.lang.reflect.Method;

import java.util.Arrays;

import java.util.Date;

import org.springframework.aop.MethodBeforeAdvice;

public class AuditingAdvice implements MethodBeforeAdvice {

@Override

public void before(Method method, Object[] args, Object target)

throws Throwable {

System.out.println("Target class name"+target.getClass());

System.out.println("Target method name"+method.getName());

System.out.println("Target method args"+Arrays.toString(args));

if((Integer)args[0]==0)

throw new IllegalArgumentException("Wrong Order id ");

if((Integer) args[0]<0)

args[0]=((Integer)args[0])*-1;

//Auditing

FileWriter fw=new FileWriter("E:\\Spring\\AuditLog.Transaction management",true);

fw.write(args[0]+" order has come for approval at"+new Date()+"\n");

fw.flush();

fw.close(); }

OrderApprover.java

```
package com.nt.service;
import java.util.Calendar;
public class OrderApprover {
    public String approveOrder(int orderId){
        // get system date
        Calendar cl=Calendar.getInstance();
        // get date current month
        int month=cl.get(Calendar.MONTH);
        if(month>=7 && month<=10)
            return orderId+" order is not approved";
        else
            return orderId+" order is Approved";
    }
}

```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.0.xsd">
    <bean id="targetBean" class="com.nt.service.OrderApprover"/>
    <bean id="auditingAdvice" class="com.nt.aspect.AuditingAdvice"/>
    <bean id="pfb" class="org.springframework.aop.framework.ProxyFactoryBean">
        <property name="target" ref="targetBean"/>
        <property name="interceptorNames">
            <list>
                <value>auditingAdvice</value>
            </list>
        </property>
    </bean>
</beans>

```

ClientApp.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.OrderApprover;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext cexTransactionManagement = new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //get Bean
        OrderApprover proxy=cexTransactionManagement.getBean("pfb",OrderApprover.class);
        //Call B.methods
        System.out.println("Order Approved?" + proxy.approveOrder(1234));
    }
}

```

```

System.out.println("-----");
System.out.println("Order Approved?" + proxy.approveOrder(-4567));
System.out.println("-----");
System.out.println("Order Approved?" + proxy.approveOrder(0));
System.out.println("-----");
} // main
} // class

```

Security means authentication and authorization. Checking identity of a user is called authentication. Checking the access permission is called authorization. Security is mainly authentication.

We need to authentication the user before entering into target method/business method. So it is recommended to develop security logic as before advice logic.

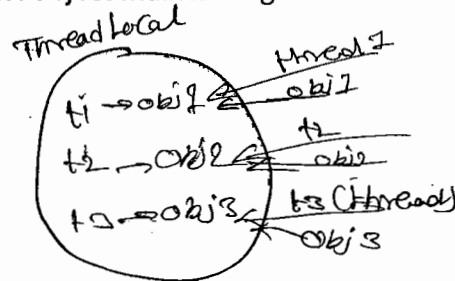
Since target method call cannot supply username, password everytime. We need to make client performing logging operation, submitting username, password before calling target method. These username, password must be saved temporarily to use then when Real B.Method/ target method is called for this we can use ThreadLocal object.

If we want to make data specific to each thread having global visibility without using local variable and synchronization concept then we can keep that data in ThreadLocal object that is taken as instance variable.

This ThreadLocal object allows each thread to keep one object and doesn't allow other threads to access and use that object. So we can say ThreadLocal object maintaining data in Thread scope.

In java we can keep data in 4 scopes.

- 1) Local
- 2) Instance
- 3) Class
- 4) ThreadLocal obj



Ex: userlist (db table in Oracle)

Username(VC2)

Raja
King

password(VC2)

Rani
Kingdom

→ without using synchronization concept, if we want to maintain data having global visibility use ThreadLocal obj. The data placed by one thread in ThreadLocal obj only by that thread. ThreadLocal obj is internally a map collection having Thread name as KEY and obj as the value.

→ while while operating ATM, we give card no, PIN but they ~~will not~~ will be used for Authentication immediately. They will be actually used for authentication before serious business operations like withdraw & deposit etc.

- AOPProj4(SpringAOP-Decl BeforeAdvice-Security)
 - src
 - com.nt.aspect
 - AuthenticationManager.java (helper class before advice)
 - SecurityAdvice.java (before advice)
 - UserDetails.java (variable to maintain username, pwd)
 - com.nt.cfgs
 - applicationContext.xml
 - com.nt.dao
 - AuthenticateDAO.java
 - com.nt.service
 - IntrAmtCalculator.java
 - com.nt.test
 - ClientApp.java
 - JRE System Library [JavaSE-1.7]
 - AOPIib
 - Referenced Libraries
 - spring-jdbc-4.1.6.RELEASE.jar - C:\Users\ade
 - spring-tx-4.1.6.RELEASE.jar - C:\Users\ade
 - ojdbc14.jar - D:\oracle\product\10.2.0\db_1

AuthenticateDAO.java

```
package com.nt.dao;
import org.springframework.jdbc.core.JdbcTemplate;
import com.nt.aspect.UserDetails;
public class AuthenticateDAO {
    private static final String AUTH_QRY="select count(*) from userlist where uname=? and
pwd=?";
    private JdbcTemplate jt;
    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }
    public boolean authenticate(h UserDetails ud){
        int cnt=jt.queryForInt(AUTH_QRY, ud.getUser(),ud.getPwd());
        if(cnt==0)
            return false;
        else
            return true;i
    }
}
```

AuthenticationManager.java

```
package com.nt.aspect;
import com.nt.dao.AuthenticateDAO;
public class AuthenticationManager {
    private ThreadLocal<UserDetails> tl=new ThreadLocal<UserDetails>();
    private AuthenticateDAO dao;
    public void setDao(AuthenticateDAO dao) {
        this.dao = dao;
    }
}
```

```

    }
    public void signIn(6String user,String pwd){
        UserDetails ud=new UserDetails(user,pwd);
        7tl.set(ud); → keeps username, pwd in ThreadLocal obj as the obj of
    } userDetails class
    public void signOut(8){
        9tl.remove(); → removes username & pwd from ThreadLocal obj
    }
    public boolean isAuthenticated(10){
        //get UserDetails obj
        UserDetails ud=tl.get();
        boolean flag=dao.authenticate(9ud);
        return flag;11
    }
}

```

SecurityAdvice.java

```

package com.nt.aspect;
import java.lang.reflect.Method;
import org.springframework.aop.MethodBeforeAdvice;
public class SecurityAdvice implements MethodBeforeAdvice{
    private AuthenticationManager am;
    public void setAm(AuthenticationManager am) {
        this.am = am;
    }
    @Override
    public void before(Method method, Object[] args, Object target)
        throws Throwable {
        //perform Authentication
        boolean flag=am.isAuthenticated();
        if(flag==false)
            throw new IllegalArgumentException("In Valid Credentials");
    }
}

```

UserDetails.java

```

package com.nt.aspect;
public class UserDetails {
    private String user,pwd;
    public String getUser() {
        return user;
    }
    public void setUser(String user) {
        this.user = user;
    }
    public String getPwd() {
        return pwd;
    }
}

```

```
public void setPwd(String pwd) {
    this.pwd = pwd;
}
public UserDetails() {
}
public UserDetails(String user, String pwd) {
    this.user = user;
    this.pwd = pwd;
}
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.0.xsd">
    <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
        <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
        <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
        <property name="username" value="scott"/>
        <property name="password" value="tiger"/>
    </bean>
    <bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
        <property name="dataSource" ref="dataSource"/>
    </bean>
    <bean id="authDAO" class="com.nt.dao.AuthenticateDAO">
        <property name="jt" ref="template"/>
    </bean>
    <bean id="authManager" class="com.nt.aspect.AuthenticationManager">
        <property name="dao" ref="authDAO"/>
    </bean>
    <bean id="targetBean" class="com.nt.service.IntrAmtCalculator"/>
    <bean id="securityAdvice" class="com.nt.aspect.SecurityAdvice">
        <property name="am" ref="authManager"/>
    </bean>
    <bean id="pfb" class="org.springframework.aop.framework.ProxyFactoryBean">
        <property name="target" ref="targetBean"/>
        <property name="interceptorNames">
            <list>
                <value>securityAdvice</value>
            </list>
        </property>
    </bean>
</beans>
```

IntrAmtCalculator.java

```
package com.nt.service;
public class IntrAmtCalculator {
    public float calcIntrAmt(float p, float t, float r){
        return p*t*r/100.0f;
    }
}
```

ClientApp.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.aspect.AuthenticationManager;
import com.nt.service.IntrAmtCalculator;
public class ClientApp {
    public static void main(String[] args) throws Exception{
        //Create IOC Container
        ApplicationContext cTransactionManagement = new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        // get AuthenticationManager obj
        AuthenticationManager authManager = cTransactionManagement
        .getBean("authManager", AuthenticationManager.class);
        //signIn
        authManager.signIn("raja", "rani");
        //call b.method
        IntrAmtCalculator proxy = cTransactionManagement
        .getBean("pfb", IntrAmtCalculator.class);
        float amt = proxy.calcIntrAmt(20000, 10, 2);
        System.out.println("Intr Amt="+amt);
        //signOUT
        authManager.signOut();
    } //main
} //class
```

Jar: AOP+DAO+IOC

AfterAdvice/AfterReturningAdvice:

This advice method/Logic executes once target method finishes the execution and before it returns the control to control method.

This advice can catch the return value but cannot modify the return value. To develop this advice we need to take a class implementing AfterReturningAdvice(i) and should provide implementation for afterReturning(-,-,-) method.

Use case:

Generating discount voucher for next purchase based on current purchase bill amount.
ATM pin generation.

Important/Control Points:

We can access target method arguments but there is no use of modifying them because control comes to target method only after finishing execution of target method.

Cannot control target method execution because control comes to advice method only after executing target method. But by throwing exception we can stop target method sending return value to caller method.

Can access the return value but cannot modify return value if modify that will not reflect.

AOP is designed based on decorator, proxy design patterns.

▲ AOPProj5(SpringAOP-Decl-AfterAdvice-Discoun

▲ src

- ▲ com.nt.aspect
 - ▷ DiscountCuponAdvice.java
- ▲ com.nt.cfgs
 - ▷ applicationContext.xml
- ▲ com.nt.service
 - ▷ ShoppingStore.java
- ▲ com.nt.test
 - ▷ ClientApp.java

▷ JRE System Library [JavaSE-1.7]

▲ AOPlib

- ▷ spring-aop-4.1.6.RELEASE.jar - C:\Users\de
- ▷ spring-aspects-4.1.6.RELEASE.jar - C:\Users\de
- ▷ spring-beans-4.1.6.RELEASE.jar - C:\Users\de
- ▷ spring-context-4.1.6.RELEASE.jar - C:\Users\de
- ▷ spring-context-support-4.1.6.RELEASE.jar - C:\Users\de
- ▷ spring-core-4.1.6.RELEASE.jar - C:\Users\de
- ▷ spring-expression-4.1.6.RELEASE.jar - C:\Users\de
- ▷ commons-logging.jar - C:\Users\de\Desl

Example Application on AfterAdvice to generate discount coupon based on the bill amount

DiscountCuponAdvice.java

```
package com.nt.aspect;
import java.io.FileWriter;
import java.lang.reflect.Method;
import org.springframework.aop.AfterReturningAdvice;
public class DiscountCuponAdvice implements AfterReturningAdvice {
    @Override
    public void afterReturning(Object returnValue, Method method, Object[] args,
        Object target) throws Throwable {
        float billAmt=(Float)returnValue;
        //we can not modify return Value.
        billAmt=billAmt+100;
        returnValue=billAmt;
        String msg=null;
        if(billAmt>=50000)
            msg="Avail 30% Discount on next purchase";
        else if(billAmt>=30000)
            msg="Avail 20% Discount on next purchase";
        else
            msg="Avail 10% Discount on next purchase";
        //store discount cupon in a file
        FileWriter fw=new FileWriter("D:\\cupon.Transaction management t");
```



```
        fw.write(msg);
        fw.flush();
        fw.close();
    } //afterReturning
} //class
```

ShoppingStore.java

```
package com.nt.service;
public class ShoppingStore {
    public float calcBillAmt(String item, float price, int qty){
        return price*qty;
    }
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="targetBean" class="com.nt.service.ShoppingStore"/>
    <bean id="discountAdvice" class="com.nt.aspect.DiscountCouponAdvice"/>
    <bean id="pfb" class="org.springframework.aop.framework.ProxyFactoryBean">
        <property name="target" ref="targetBean"/>
        <property name="interceptorNames">
            <list>
                <value>discountAdvice</value>
            </list>
        </property> </bean>
</beans>
```

ClientApp.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.ShoppingStore;
public class ClientApp {
    public static void main(String[] args) {
        // create IOC Container
        ApplicationContext cTransaction management =new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //get Proxy obj
        ShoppingStore proxy=cTransaction management
        .getBean("pfb",ShoppingStore.class);
        //call target method
        float billAmt=proxy.calcBillAmt("table", 3000,20);
        System.out.println("Bill Amount="+billAmt);
    } //main & Class
```

We can use after advice to check whether the return value generated by target method is valid value or not. If not valid we can throw exception in Afteradvice and we can stop that return value being return to ClientApp (caller).

Usecase:

If target method returns ATM pin we can check whether that generated pin is weak pin or strong pin in Afteradvice. If it is a weak pin then we

- ▲ AOPProj6(SpringAOP-Decl AfterAdvice-PinVerifier)
 - ▲ src
 - ▲ com.nt.aspect
 - CheckPinAdvice.java
 - ▲ com.nt.cfigs
 - applicationContext.xml
 - ▲ com.nt.service
 - PinGenerator.java
 - ▲ com.nt.test
 - ClientApp.java
 - JRE System Library [JavaSE-1.7]
 - AOPlib
 - Referenced Libraries

CheckPinAdvice.java

```
package com.nt.aspect;
import java.lang.reflect.Method;
import java.util.Arrays;
import org.springframework.aop.AfterReturningAdvice;
public class CheckPinAdvice implements AfterReturningAdvice {
    @Override
    public void afterReturning(Object retValue, Method method, Object[] args,
        Object target) throws Throwable {
        System.out.println("Generated pin"+retValue);
        System.out.println("target class"+target.getClass());
        System.out.println("target method"+method.getName());
        System.out.println("target method args"+Arrays.toString(args));
        int cardNo=(Integer)args[0]+100;
        //get return value
        int pin=(Integer)retValue;
        //check the pin
        if(pin<=999)
            throw new IllegalArgumentException(" Weak Pin generated");
    }
}

```

PinGenerator.java

```
package com.nt.service;
import java.util.Random;
public class PinGenerator {
    public int generatePin(int cardNo){
        //generate ATM Pin
    }
}

```

```

        Random rad=new Random();
        int pin =rad.nextInt(9999);
        return pin;
    }
}

```

applicationContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="targetBean" class="com.nt.service.PinGenerator"/>
<bean id="pinAdvice" class="com.nt.aspect.CheckPinAdvice"/>
<bean id="pfb" class="org.springframework.aop.framework.ProxyFactoryBean">
<property name="target" ref="targetBean"/>
<property name="interceptorNames">
<list>
<value>pinAdvice</value>
</list>
</property>
</bean>
</beans>

```

ClientApp.java

```

package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.PinGenerator;
public class ClientApp {
    public static void main(String[] args) {
        // create IOC Container
        ApplicationContext cTransaction management =new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //get Proxy obj
        PinGenerator proxy=cTransaction management .getBeah("pfb",PinGenerator.class);
        //call target method
        int pin=proxy.generatePin(88888);
        System.out.println("Generated Pin="+pin);
    }
}

```

ThrowsAdvice

This advice will execute when exception is raised in target method. Using this advice we cannot catch and handle the exception, but we can see the details of exception that is raised.

We can use this advice to develop common/central; Exception logger that logs all the exception related messages that are raised in various target methods of target class.

Note: ThrowsAdvice logic not only execute when Exception is raised in Target method, it also respond for the Exception raised in All Advices configured on the Target method.

We can catch and handle the exception raised by target method in client App where target method will be called.

To develop this advice we need to take class that implements "ThrowsAdvice (i)" (marker interface-->Empty interface) and we need to place one or more afterThrowing methods having the following signatures.

public void afterThrowing (<sub class of Throwable>)

Or recommended

public void afterThrowing (Method method, Object args[], Object target <subclass of throwable>)

throwable class
(or)

Control points/important points:

- We can access Target Method params, but there is no use of modifying them.
- We can not control target method execution because control comes to throwsAdvice after executing target methods that to for exception raising.
- We cannot catch the return value of target method.

29/12/15
AOPProj7(Spring AOP Decl-ThrowAdvice-Exception)

- src
 - com.nt.aspect
 - ExceptionLoggerAdvice.java
 - com.nt.cfgs
 - applicationContext.xml
 - com.nt.service
 - PriceCalculator.java
 - com.nt.test
 - ClientApp.java
- JRE System Library [JavaSE-1.7]
- AOPlib

~~THE~~ ThrowsAdvice is used to
converted the technology based exception
to application based exception

ExceptionLoggerAdvice.java

```
package com.nt.aspect;
import java.lang.reflect.Method;
import java.util.Arrays;
import org.springframework.aop.ThrowsAdvice;
public class ExceptionLoggerAdvice implements ThrowsAdvice {
    public void afterThrowing(IllegalArgumentException ie){
        throw new NumberFormatException("Changed exception");
    }
    public void afterThrowing(Method method, Object args[], Object target, Throwable ie){
        System.out.println(ie.getClass()+" Exception is raised in "+method.getName()+" with args
        "+Arrays.toString(args)+" in "+target.getClass());
    }
}
```

PriceCalculator.java

```
package com.nt.service;
public class PriceCalculator {
    public float calcPrice(float price, float qty){
        if(price==0.0f || qty==0.0f)
            throw new IllegalArgumentException(" price and qty must not be zero..");
        return price*qty; } }
```

29/12/15
applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="targetBean" class="com.nt.service.PriceCalculator"/>
    <bean id="throwsAdvice" class="com.nt.aspect.ExceptionLoggerAdvice"/>
    <bean id="pfb" class="org.springframework.aop.framework.ProxyFactoryBean">
        <property name="target" ref="targetBean"/>
        <property name="interceptorNames">
            <list>
                <value>throwsAdvice</value>
            </list>
        </property>
    </bean>
</beans>
```

ClientApp.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.PriceCalculator;
public class ClientApp {
    public static void main(String[] args) {
        // create IOC Container
        ApplicationContext cTransaction management = new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //get Proxy obj
        PriceCalculator proxy=cTransaction management
        .getBean("pfb",PriceCalculator.class);
        //call target method
        try{
            float price=proxy.calcPrice(45.65f,0);
            System.out.println("Price="+price);
        }
        catch(Exception ie){
            System.out.println("Internal Problem"+ie.getClass()+" "+ie.getMessage());
        }
        //main
        //class
        Jars: same
```

Note: Throws advice logic is not only executes when exception is raised in the target method. It is also responds for the exception raised in other advices configured on the target method

If exception is raised in target method, control comes to throws advice and looks for afterThrowing method in the following order.

Exact Exception type

Super class raised Exception type.

Note: If multiple afterThrowing method contains exact match the more params method executes.

2) Note: If multiple afterThrowing method contains super class of raised exception then more params method executes.

3) Note: If 1 afterThrowing() contains exact matching and another afterThrowing() contains super type then the exact match afterThrowing will executed. If both different then nearest superclass param contains method will execute.

→ We can also use ThrowsAdvice to wrap/cascade/Translate one form of Exception to another form of Exception (app specific). Like ^{graph} translating the ^{graph} bitier comp generated technology specific exception to app specific Exception.

4. Advice Type	When advice logic executes	Access to target method params	Access to target method return value	Control to target method execution
1. Around Advice	Around target method execution	Can see ^{access} and modify b. method args	Can see ^{access} and modify <u>use case:</u> Catching Around Logging performing monitoring	NA
2. Before Advice	Before target method Execution	Can see ^{access} and modify b. method args	NA (Not Application) <u>use cases:</u> Auditing Security check	NA (can stop by throwing Exception)
3. After Advice	After target method execution ^{executing} ^{before client app} before returning value	Can see ^{access} but not modify	Can see ^{access} but not modify <u>use case:</u> → Gift voucher Generator → ATM Pin Generator	NA
Throws Advice	When exception is raised in target method	Can see but not modify	NA <u>use case:</u> 1. common Exception Logger 2. ExceptionTranslator.	Na

29/12/15
30/12/15

Programatic Spring AOP:

Here no xml file support or configuration support will be taken to generate proxy class by supplying Target class, advice class.

All intraction will be given directly in client application through java statement.

This is not recommended approach.

Here we use proxyFactory class directly in client app to generate the proxy class.

ProxyFactory factory = new ProxyFactory();

factory.setTarget ();

```
factory.addAdvice();
```

```
proxy=factory.getProxy();
```

→ generates temporary proxy class dynamically applying advice logics on the methods of target class

```
proxy.bm1();
```

```
proxy.bm2(); //call B.methods
```

Note: Programmatic Aop not recommended to use because

it makes programmer to modify java code to enable or disable aspects and doesn't allow programmer to take the benefit of IOC container.

So far the aspects that they are developing will be applied on all the methods of target class. In some situations we don't want to apply aspects on certain joinpoints or methods of target class. For this we can write additional conditional logic in advice class to execute advice logic only for specific joinPoint methods (Target methods)

In the above approach the generated proxy class overrides all the methods of target class having logic to apply aspects on target method. But advice logic will be executed only for ^{specific method} ~~add~~ because of the condition that is specified. This indicates proxy class is not generated in optimized manner because it is also overriding those target methods on which we don't want to apply aspects moreover those methods of target class should be executed directly without touching proxy class

When they are called from clients.

To overcome the above problem use pointcuts. Pointcut is a collection of join points on whom you want to advice the aspects. It specifies which aspects has to be applied on which joinPoints.

There are 2 types of pointcuts.

1) Static Pointcut:

Allows to specify target class name, method names while preparing the Pointcut. But that can't specify target methods argument values. This indicates static Pointcuts will be processed during weaving process that generates proxy class.

2) Dynamic Pointcut:

This allows to specify target class, target method names and their arguments values. That means this pointcut will be processed after target method call on proxy object dynamically at runtime.

Ex: Apply aspect on "add" method is static pointcut.

Apply aspect on "add" method when argument values are <100 is dynamic pointcut.

While working with pointcut the generated proxy class override only those methods that are specified in pointcut to apply aspects. Due to this proxy class code will be optimized to improve performance.

To develop userdefined pointcut we need to take a class implementing pointcut Interface. Spring api has supplied some predefined classes implementing this interface.

They are

StaticMethodMatcherPointcut	→ static pointcut class
NameMatchMethodPointcut	
jdkRegexpMethodPointcut	

Dynamic pointcut :-

DynamicMethodMatcherPointcut

→ To develop custom pointcut classes extend from above classes instead of implementing Pointcut Interface.

→ Advisor is the combination of Pointcut+Aspect/advice.

In order to apply aspects specifying pointcuts we take the support of advisor. Every advisor class is the implementation class of Advisor (i)

The spring API supplied predefined Advisor are

DefaultPointAdvisor
NameMatchMethodPointcutAdvisor and etc..

While working with static pointcut classes we need to override matches method that gives access to target class and target method. If this method returns true for certain target method aspect will be applied for that method, otherwise it will not be applied.

In order to develop dynamic pointcut we need to take a class extending from `DynamicMethodMatcherPointcut` class and implements matches (Method method, Object [] args, class<?> target) method.

If this method returns true the certain target method then aspect will be applied on that method otherwise aspect will not be applied.

Instead of developing separate userdefined class by extending from abstract class `StaticMethodMatcherPointcut` or `DynamicMethodMatcherPointcut` we can use spring API supplied concrete static pointcut class `NameMatchMethodPointcut` as shown below

```
NameMatchMethodPointcut nmp=new NameMatchMethodPointcut();
nmp.setMappedNamed("Sum", "null");
DefaultPointcutAdvisor advisor=new DefaultPointcutAdvisor (nmp, new
PerformanceMonitoringAdvice());
nmp.addAdvisor(advisor);
```

Working with Pointcut in declarative spring AOP:

In this approach we can configure pointcut classes, ^{se}advisor classes, ^{as}Advisor classes ~~se~~ beans in configuration file and we can specify advisor directly as the values of interceptor name properly while configuring `ProxyFactoryBean`.

Instead of working with "NameMatchedMethodPointcut" "DefaultPointcutAdvisor" classes separately we can use "NameMatchedMethodPointcutAdvisor" class directly to bring the same effect.

for APP, on spring AOP declarative with pointcuts refer AopProj10 d-softcopy.

EX: 30/12/15

- ▲ AOPProj9(Spring AOP Decl-Pointcuts)
 - ▲ src
 - ▲ com.nt.aspect
 - CacheAdvice.java
 - LogAroundAdvice.java
 - PerformanceMonitoringAdvice.java
 - ▲ com.nt.cfigs
 - applicationContext.xml
 - ▲ com.nt.service
 - ArithmeticService.java
 - ▲ com.nt.test
 - ClientApp.java
 - JRE System Library [JavaSE-1.7]
 - AOPlib

CacheAdvice.java

```
package com.nt.aspect;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;
import org.aopalliance.intercept.MethodInterceptor;
import org.aopalliance.intercept.MethodInvocation;
public class CacheAdvice implements MethodInterceptor {
    Map <String,Object> cache=new HashMap<String,Object>();
    @Override
```



```
public Object invoke(MethodInvocation invocation) throws Throwable {
    String key=invocation.getMethod().getName()+Arrays.toString(invocation.getArguments());
    Object retValue=null;
    if(!cache.containsKey(key)){
        retValue=invocation.proceed();
        cache.put(key, retValue);
        System.out.println("From target method");
        return retValue;
    }
    retValue=cache.get(key);
    System.out.println("From cache");
    return retValue;
}
```

PerformanceMonitoringAdvice.java

```
package com.nt.aspect;
import java.util.Arrays;
import org.aopalliance.intercept.MethodInterceptor;
import org.aopalliance.intercept.MethodInvocation;
public class PerformanceMonitoringAdvice implements MethodInterceptor {
    long start,end;
    @Override
    public Object invoke(MethodInvocation invocation) throws Throwable {
        start=System.currentTimeMillis();
        Object retVal=invocation.proceed();
        end=System.currentTimeMillis();
        System.out.println(invocation.getMethod().getName()+" with
args"+Arrays.toString(invocation.getArguments())+" has taken "+(end-start)+"ms");
        return retVal;
    }
}
```

CommonDynaPointcut.java

```
package com.nt.poincut;
import java.lang.reflect.Method;
import org.springframework.aop.support.DynamicMethodMatcherPointcut;
import com.nt.service.ArithmeticService;
public class CommonDynaPointcut extends DynamicMethodMatcherPointcut {
    @Override
    public boolean matches(Method method, Class<?> target, Object[] args) {
        if(target==ArithmeticService.class && method.getName().equals("mul") &&
(Integer)args[0]>=1000 && (Integer)args[1]>=1000)
            return true;
        else
            return false;
    }
}
```

PerformanceMonitoringPointcut.java *(Static pointcut)*

```
package com.nt.poincut;
import java.lang.reflect.Method;
import org.springframework.aop.support.StaticMethodMatcherPointcut;
import com.nt.service.ArithmeticService;
public class PerformanceMonitoringPointcut extends StaticMethodMatcherPointcut {
    @Override
    public boolean matches(Method method, Class<?> target) {
        if(target==ArithmeticService.class && method.getName().equals("add"))
            return true;
        else
            return false;
    }
}
```

ArithmeticService.java *(Target Class)*

```
package com.nt.service;
public class ArithmeticService {
    public int add(int x,int y){
        return x+y;
    }
    public int sub(int x,int y){
        return x-y;
    }
    public int mul(int x,int y){
        return x*y;
    }
}
```

ClientApp.java

```
package test;
import org.springframework.aop.framework.ProxyFactory;
import org.springframework.aop.support.DefaultPointcutAdvisor;
import org.springframework.aop.support.NameMatchMethodPointcut;
import com.nt.aspect.CacheAdvice;
import com.nt.service.ArithmeticService;
public class ClientApp {
    public static void main(String[] args) {
        //Get ProxyFactory class obj
        ProxyFactory pf=new ProxyFactory();
        //set Target class obj
        pf.setTarget(new ArithmeticService());
        /* //set Advice class obj
        pf.addAdvice(new PerformanceMonitoringAdvice()); */
        /* //prepre advisor (poincut+ advice/aspect)
        DefaultPointcutAdvisor advisor=new DefaultPointcutAdvisor(new
        PerformanceMonitoringPointcut());
```

```

        new PerformanceMonitoringAdvice());
        pf.addAdvisor(advisor); */
    /*      DefaultPointcutAdvisor PMAvisor=new DefaultPointcutAdvisor(new
CommonDynaPointcut()
PerformanceMonitoringAdvice());
        DefaultPointcutAdvisor Cacheadvisor=new
DefaultPointcutAdvisor(new CommonDynaPointcut(),
        new CacheAdvice());
        pf.addAdvisor(0,Cacheadvisor);
        pf.addAdvisor(1, PMAvisor); */
    /*      DefaultPointcutAdvisor PMAvisor=new DefaultPointcutAdvisor(new
PerformanceMonitoringPointcut(),
        new PerformanceMonitoringAdvice());
        DefaultPointcutAdvisor Cacheadvisor=new DefaultPointcutAdvisor(new
CommonDynaPointcut(),
        new CacheAdvice());
        pf.addAdvisor(0,PMAvisor);
        pf.addAdvisor(1,Cacheadvisor); */
        NameMatchMethodPointcut nmp=new
NameMatchMethodPointcut();
        nmp.setMappedNames("add","mul");
        DefaultPointcutAdvisor advisor=new DefaultPointcutAdvisor(nmp,new
CacheAdvice());
        pf.addAdvisor(advisor);
        //get Proxy obj
        ArithmeticService proxy=(ArithmeticService)pf.getProxy();
        // Call target method
        System.out.println("Sum="+proxy.add(1000,2000));
        System.out.println("-----");
        System.out.println("Sub="+proxy.sub(100000,20000));
        System.out.println("-----");
        System.out.println("Mul="+proxy.mul(10000,20000));
        System.out.println("-----");
        System.out.println("Mul="+proxy.mul(10000,20000));
        System.out.println("-----");
        System.out.println("Mul="+proxy.mul(100,200));
        System.out.println("-----");
    } //main
} //class

```

Wrt to Spring Aop Declarative pointcut

We can develop static Pointcut either as usedefined class extending from StaticMethodMatcherPointcut or we can use spring specified direct concrete classes like NameMatchMethodPointcut.

Inorder to work with DynamicMethodPointcut we must take user defined class extending from DynamicMethodMatcherPointcut and there is no direct concrete classes supplied by spring API to work with dynamic pointcut.

Aspect J AOP: 1/1/16

A third party AOP framework where we can develop aspect classes as pojo classes without having tight coupling with spring API. i.e aspect class need not implement spring api interface and need not to extend spring API class.

But we need to use aspectJ api inside the aspects classes. Different between spring AOP and AspectJ AOP. The limitation with spring AOP is the advice classes are invasive (Should implement or extend from Spring AOP Interface or class)

Spring AOP	Plain Aspectj AOP
→ Perform runtime weaving	→ Perform compile time weaving
→ Generate proxy class at runtime	→ Generate proxy class at compile time. Use AspectJ compiler.
→ Supports only methods as ^{Joinpoint} Pointcut	→ Supports fields, constructors methods as joinpoints.
→ Support both static and dynamic pointcut	→ Supports only static Pointcut
→ Doesn't allow annotations	→ Allows annotation.
→ Can be used in programmatic and	→ can be used in declarative
→ Decalarative(xml) approach	(xml) through annotations.

→ Advice classes are non-pojo class
From spring 2.0 the AspectJ AOP is integrated with spring AOP due to this limitations are applied on

Aspect AOP

- Supports only method level joinpoints.
- Performs runtime weaving i.e proxy class will be generated at runtime.
- Still supports only static pointcuts.
- No AspectJ compiler is required.
- We can use spring integrated AspectJ AOP in 2 ways
- Declarative AspectJ AOP (using xml)
- Annotations AspectJ AOP

Note: Aspects AOP also supports 4 types of Advices

Declarative Aspects AOP:

Here we develop aspect classes as pojo classes by using aspects api internally and we configure these xml file by using tags as aspect classes.

We need to import AOP namespace for these configuration.

The important tags are

<aop: config>---->to enable AspectJ AOP from spring container.

<aop: aspect>---->to cfg spring bean as Aspect class.

<aop: around>----->to configure aspect class method as Around advice.

<aop: before>----->to configure aspect class method as after advice

<aop: after-returning>----> to configure aspect class method as after advice.

<aop: after-throwing>----->to configure aspect class method s throws advice

<aop: pointcut>---->to configure pointcut

We use OGNL for pointcut expression

Syntax:

execution (<return type><package name>.<class name>.<method name> (...))

Around Advice:

This advice executes around target methods.

The use cases are caching, logging, performanceMonitoring and etc....

To develop this advice we can use any arbitrary class having any arbitrary method with following signature.

`public Object <method>(ProceedingJoinPoint pjp) throws Throwable`

ProceedingJoinPoint pjp can be used to get target class, method info and to control target method invocation.

Ex:

- AspectJ AOP Proj (AroundAdvice) 2/2/16
- src
 - com.nt.aspect
 - CacheAdvice.java
 - LogAroundAdvice.java
 - PerformanceMonitoringAdvice.java
 - com.nt.cfgs
 - applicationContext.xml
 - com.nt.service
 - ArithmeticService.java
 - com.nt.test
 - ClientApp.java
 - JRE System Library [JavaSE-1.7]
 - AOPlib + aspectjrt-1.5.3.jar
aspectjweaver.jar

NOTE: while working with AspectJ AOP, specifying pointcut expression is mandatory even though if we want to apply all advices on the target methods of target class. Because if target class (or) if the implementation proxy interface their in ~~com~~ LOGICAL expression of pointcut will not be ~~there~~ there. when `ctx.getBean()` method is called.

CacheAdvice.java

```
package com.nt.aspect;
import java.util.Arrays;
import java.util.HashMap;
import org.aspectj.lang.ProceedingJoinPoint;
public class CacheAdvice {
    HashMap<String, Object> cache = new HashMap<String, Object>();
    public Object caching(ProceedingJoinPoint pjp) throws Throwable {
        String key = pjp.getSignature() + Arrays.toString(pjp.getArgs());
        if (!cache.containsKey(key)) {
            Object retVal = pjp.proceed();
            cache.put(key, retVal);
            System.out.println("From target method");
            return retVal;
        } else {
            System.out.println("From cache");
            return cache.get(key);
        }
    }
}
//caching
//class
```

LogAroundAdvice.java

```
package com.nt.aspect;
import java.util.Arrays;
import org.aspectj.lang.ProceedingJoinPoint;
public class LogAroundAdvice {
```

```
public Object logging(ProceedingJoinPoint pjp)throws Throwable{
    System.out.println("Entering into "+pjp.getSignature()+" with
args"+Arrays.toString(pjp.getArgs()));
    //Get args and modify args
    Object args[]=pjp.getArgs();
    if((Float)args[0]<=50000){
        args[2]=(Float)args[2]-1;
    }
    Object retVal=null;
    if((Float)args[0]==0 || (Float)args[1]==0 || (Float)args[2]==0 )
        throw new IllegalArgumentException("Provide +ve values as the p,t,r
values");
    else
        retVal=pjp.proceed(args);
    // add Service tax on intr amt
    retVal=(Float)retVal+(Float)retVal*0.05f;
    System.out.println("Exiting from "+pjp.getSignature()+" with
args"+Arrays.toString(pjp.getArgs()));
    return retVal;
} //method
} //class
```

PerformanceMonitoringAdvice.java

```
package com.nt.aspect;
import java.util.Arrays;
import org.aspectj.lang.ProceedingJoinPoint;
public class PerformanceMonitoringAdvice {
    long start,end;
    public Object pMonitor(ProceedingJoinPoint pjp)throws Throwable{
        start=System.currentTimeMillis();
        Object retVal=pjp.proceed();
        end=System.currentTimeMillis();
        System.out.println(pjp.getSignature()+" with" +Arrays.toString(pjp.getArgs())+" has
taken "+(end-start)+"ms");
        return retVal;
    }
}
```

IntrAmtCalculator.java

```
package com.nt.service;
public class IntrAmtCalculator {
    public float calcSimpleIntrAmt(float p,float t,float r){
        return (p*t*r)/100.0f;
    }
    public float calcCompoundIntrAmt(float p,float t,float r){
        return (float) (p*Math.pow((1+r/100.0f),t)-p);
    }
}
```

applicationContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:aop="http://www.springframework.org/schema/aop"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd">
<bean id="target" class="com.nt.service.IntrAmtCalculator"/>
<bean id="logAdvice" class="com.nt.aspect.LogAroundAdvice"/>
<bean id="cacheAdvice" class="com.nt.aspect.CacheAdvice"/>
<bean id="PMAAdvice" class="com.nt.aspect.PerformanceMonitoringAdvice"/>
<aop:config>
<aop:pointcut id="ptc1" expression="execution(* com.nt.service.*(..))"/>
<aop:pointcut id="ptc2" expression="execution(
*com.nt.service.IntrAmtCalculatorInterface.calcCompoundIntrAmt(..))"/>
<aop:aspect id="logAspect" order="3" ref="logAdvice">
<aop:around pointcut-ref="ptc1" method="logging"/>
</aop:aspect>
<aop:aspect id="cacheAspect" order="1" ref="cacheAdvice">
<aop:around pointcut-ref="ptc2" method="caching"/>
</aop:aspect>
<aop:aspect id="PMAAspect" order="2" ref="PMAAdvice">
<aop:around pointcut-ref="ptc1" method="pMonitor"/>
</aop:aspect>
</aop:config>
</beans>

```

Aspect classes

→ aopml exp printing
to all the methods of
all the classes
available in
com.nt.service pkg

any order
args

ClientApp.java

```

package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.IntrAmtCalculator;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext cTransaction management = new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //GEt Proxy obj
        IntrAmtCalculator proxy=cTransaction management
        .getBean("target",IntrAmtCalculator.class);
        System.out.println(proxy.getClass());
        //Call target method
        System.out.println("Simple Intr Amt"+proxy.calcSimpleIntrAmt(10000,20,2));
        System.out.println("-----");
        System.out.println("Compound Intr Amt"+proxy.calcCompoundIntrAmt(60000, 20,
        2));
    }
}

```

```

        System.out.println("-----");
        System.out.println("Compound Intr Amt"+proxy.calcCompoundIntrAmt(60000, 20,
2));
    }
}
Jar AOPLib+aspectjrt1.5.3.jar+aspectweaver-1.5.3.jar

```

Note: While working with AspectJ AOP AroundAdvice the modification done in target method args in Around advice will be reflected only when we call `pjp.proceed(args)` having that `obj[]` as arguments. Here `Object[]` holds the modified argument values.

When we run the ClientApp of spring intrigated AspectJAOP the internally created IOC container maintains inmemory metadata along with AOP configurations based on `<aop:config>` and its sub tags.

When `cTransactionmanagement.getBean(-)` method is called with target class bean id, the IOC container doesn't create target class object because of `<aop:config>` tag based configuration. It generates proxy class extending from target class and returns that proxy class object to the clientApplication.

Like spring AOP if we start working with proxyInterface (The interface that is implemented by target class) in AspectJAOP then the generated proxy class will come because of jdk libraries. For this we need to specify proxy interface name in OGNL expression of pointcut. In this AspectJAOP if proxy interfaces are not specified the `CGlib` libraries will be utilized to generate the proxy class.

BE : `<aop:pointcut id="ptC1" expression="execution(* com.nt.service.*(..))"/>` *proxy Interface*

Before Advice:

This advice executes before executing target method.

Using this advice we can access target method parameters but we can't control target method executed and we can't catch the return value.

Using this advice we can stop target method execution by throwing exception.

To develop this advice we can take any class having any method with following signature.

```
public void <Method> (joinPoint jp)
```

or

```
public void <method> (JoinPoint jp, param1, param2,.....)
```

To hold the target method argument values

JoinPoint object just gives target method method details where as ProceedingJoinPoint object gives target method details, control on target method invocation.

beforeAdvice use cases are security check, auditing and etc

AspectJAOP3(Before Advice- Auditing)

src

- com.nt.aspect
 - AuditingAdvice.java
- com.nt.cogs
 - applicationContext.xml
- com.nt.service
 - OrderApprover.java
- com.nt.test
 - ClientApp.java
- JRE System Library [JavaSE-1.7]
- AOPLib

AuditingAdvice.java

```
package com.nt.aspect;
import java.io.FileWriter;
import java.util.Arrays;
import java.util.Date;
import org.aspectj.lang.JoinPoint;
public class AuditingAdvice {
    public void audit(JoinPoint jp, int oid)throws Throwable{
        System.out.println("target method:"+jp.getSignature());
        System.out.println("target class "+jp.getTarget().getClass());
        System.out.println("target method args"+Arrays.toString(jp.getArgs()));
        System.out.println("Order Id is"+oid);
        Object args[]=jp.getArgs();
        args[0]=(Integer)args[0]+1;
        jp.getArgs()[0]=args[0];
        //perform auditing
        FileWriter fw=new FileWriter("e:\\audit.log",true);
        fw.write("order with order id "+oid+" sumited for approval on Date"+new Date());
        fw.flush();
        fw.close();
    }
}
}
```

OrderApprover.java

```
package com.nt.service;
import java.util.Calendar;
public class OrderApprover {
    public String approve(int orderid){
        // get Current month
        Calendar cl=Calendar.getInstance();
        int h=cl.get(Calendar.MONTH);
        if(h>=7 && h<=10)
            return "We do not accept this "+orderid +"order in rainy seasno";
        else
            return orderid +" order is approved";
    }
}
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:aop="http://www.springframework.org/schema/aop"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd">
    <bean id="orderApprover" class="com.nt.service.OrderApprover"/>
</beans>
```

```

<bean id="auditAdvice" class="com.nt.aspect.AuditingAdvice"/>
<aop:config>
  <aop:pointcut id="ptc1" expression="execution(* com.nt.service.*(..) and args(oid))"/>
    <aop:aspect id="auditAspect" ref="auditAdvice">
      <aop:before method="audit" pointcut-ref="ptc1"/>
    </aop:aspect>
  </aop:config>
</beans>

```

indicates the argument or value of approveOrder(-) will be stored in the "oid" parameter of advice(-)
oid

ClientApp.java

```

package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.OrderApprover;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext cTransactionManagement = new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //get Proxy
        OrderApprover proxy = cTransactionManagement
        .getBean("orderApprover", OrderApprover.class);
        System.out.println("status:" + proxy.approve(1001));
    }
}

```

Object graph navigation language (OGNL)

In AspectJ AOP the beforeAdvice can access TargetMethod arguments but cannot modify their values. It is not similar to spring AOP before Advice.

While working with beforeAdvice we can even get Targetmethod arguments without making the support of JoinPoint object. We need to take additional parameters support in advice method to store target method arguments and we must specify those parameters in pointcut expression.

applicationContext.xml:

```

<aop: pointcut id="ptc1" expression="execution(* com.nt.service.*(..) and args (oid))"/>

```

While working with AspectJ AOP if we call cTransactionManagement.getBean(-) method by using the bean id whose class name is specified in pointcut OGNL expression then we get proxy class object. Otherwise we will get the given bean id related object.

AfterAdvice/After Returning Advice:

This advice method executes when target method complete execution and before it returns the control/value to caller.

To develop this advice we can take any class having the following signature method.

```

public void <method>(JoinPoint JP, <param to hold return value>)

```

Usecase:

Generating discountCoupon for next purchase.

ATM pin verifier.

This advice allows to access target method args values.

This advice allows to access return value but doesn't allow to modify.

By throwing exception in this advice we can stop return value being return to caller method.
The parameter taken in advice method to hold return value must be configured in <aop:after-returning> by using "returning" attribute.

<aop: after-returning method="coupon" pointcut-ref="ptc" returning="billAmt"/>

↳ parameter name of advice

method to hold target method
return value.

- AspectJAOP5(AfterAdvice- DiscountCoupon)
 - src
 - com.nt.aspect
 - DiscountCouponAdvice.java
 - com.nt.cfgs
 - applicationContext.xml
 - com.nt.service
 - ShoppingStore.java
 - com.nt.test
 - ClientApp.java
 - JRE System Library [JavaSE-1.7]
 - AOPlib

DiscountCouponAdvice.java

```
package com.nt.aspect;
import java.io.FileWriter;
import org.aspectj.lang.JoinPoint;
public class DiscountCouponAdvice {
    public void generateCoupon(JoinPoint jp, float billAmt) throws Throwable{
        String couponMsg=null;
        if(billAmt>=50000)
            couponMsg="Avail 30% Discount on next purchase";
        else if(billAmt>=30000)
            couponMsg="Avail 20% Discount on next purchase";
        else
            couponMsg="Avail 10% Discount on next purchase";
        // Save Coupon in a file
        FileWriter fw=new FileWriter("e:\\cupont.Transaction management t");
        fw.write(couponMsg);
        fw.flush();
        fw.close();
    } //generateCoupon
} //class
```

ShoppingStore.java

```
package com.nt.service;
public class ShoppingStore {
    public float generateBillAmt(String item, float price, float qty ){
        return price*qty;
    }
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:aop="http://www.springframework.org/schema/aop"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd">
  <bean id="shopping" class="com.nt.service.ShoppingStore"/>
  <bean id="discountAdvice" class="com.nt.aspect.DiscountCouponAdvice"/>
  <aop:config>
    <aop:pointcut id="ptc1" expression="execution(* com.nt.service.*(..))"/>
    <aop:aspect id="discountAspect" ref="discountAdvice">
      <aop:after-returning method="generateCoupon" pointcut-ref="ptc1" returning="billAmt"/>
    </aop:aspect>
  </aop:config>
</beans>
```

Advice pattern name to hold
return value of target method

ClientApp.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.ShoppingStore;
public class ClientApp {
  public static void main(String[] args) {
    //create IOC container
    ApplicationContext cTransaction management =new
    FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
    //Get Proxy obj
    ShoppingStore proxy=cTransaction management
    .getBean("shopping",ShoppingStore.class);
    //call B.method
    System.out.println("Bill Amt"+proxy.generateBillAmt("shirt",900,2));
  }
}
```

Throws Advice also responds for the exception that are raised in other
advice that are applied on the target method.

Throws Advice:

This advice method executes when exception is raised in target method. This advice is useful to develop central, common exception logger.

This advice doesn't catch and handle the exception, but allows to see the details of exception and translate one form of exception to another form of exception.

To develop this advice we can take a class having method with following signature.

public void <method name>(JoinPoint JP, <param of type subclass of Throwable>);

The additional param of this advice method, that is capable of referring raised exception must be specified in <aop:after-throwing> tag by using "throwing" attribute.

Ex: <aop:after-throwing method="....." pointcut-ref="....." throwing="....."/>

→ This allows to keep presentation time compo untouchable even though service, DAO class

AspectJAOP (ThrowsAdvice-CommonExceptionHandler)

4/1/16

ThrowsAdvice even useful for exception graphing, which is nothing but transpringframeworkorm of one form of exception to another form.

4/1/16

AspectJAOP6(ThrowsAdvice-CommonExceptionHandler)

- src
 - com.nt.aspect
 - CommonExceptionHandler.java
 - com.nt.cfgs
 - applicationContext.xml
 - com.nt.service
 - Math.java
 - com.nt.test
 - ClientApp.java
- JRE System Library [JavaSE-1.7]
- AOPlib

This throwing advice is more useful to graph (or) translate the technology specific exception of Service, Dao classes into App. specific exception. This allows to keep presentation tier camps untouched even though Service, Dao technologies are changed.

CommonExceptionHandler.java

```
package com.nt.aspect;
import java.util.Arrays;
import org.aspectj.lang.JoinPoint;
public class CommonExceptionHandler {
    public void exceptionLogging(JoinPoint jp, ArithmeticException ae) {
        System.out.println(ae.getClass()+"Exception raised With Msg "+ae.getMessage()+" in "+jp.getSignature()+" with args"+Arrays.toString(jp.getArgs()));
        throw new IllegalArgumentException("Provide valid inputs");
    }
}
```

TOP
While writing AspectJAOP, specifying pointcut expression is mandatory even though if you want to apply all the advices on all the target methods of target class, because if target class name or if the implemented any interface is not there in the pointcut expression, the proxy class will not be when the target method is called.

If throws advice class contain multiple overloaded methods to respond for exception that is raised in target. The method with least no. of parameters

Math.java

```
package com.nt.service;
public class Math {
    public float div(int x, int y) {
        if (y == 0)
            throw new ArithmeticException("Divider can not Zero");
        else
            return (float)x/y;
    }
}
```

gets priority to execute (AspectJAOP). If multiple overloaded methods having same no. of parameters, error will be raised. It is different from Spring AOP.

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:aop="http://www.springframework.org/schema/aop"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd">
```

```

<bean id="math" class="com.nt.service.Math"/>
<bean id="exLogAdvice" class="com.nt.aspect.CommonExceptionHandler"/>
<aop:config>
  <aop:pointcut id="ptc1" expression="execution(* com.nt.service.*(..))"/>
  <aop:aspect id="exLogAspect" ref="exLogAdvice">
    <aop:after-throwing method="exceptionLogging"
      pointcut-ref="ptc1" throwing="ae"/>
  </aop:aspect>
</aop:config>
</beans>

```

*parameter name of Advice method
to catch the raised Exception*

ClientApp.java

```

package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.Math;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext cTransaction management = new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //GET Proxy obj
        Math proxy=cTransaction management .getBean("math",Math.class);

        //call B.method
        try{
            System.out.println("Result="+proxy.div(10,20));
            System.out.println("-----");
            System.out.println("Result="+proxy.div(10,0));
        }
        catch(ArithmeticException ae){
            System.out.println("Internal Problem");
        }
    }
}

```

Annotations driven AspectJ AOP:

Here we don't use xml file to configure advice classes and advice methods rather we use list of annotations given by aspectjAOP.

- ✓ @Aspect-----> to configure class as AspectClass. *spring bean*
- ✓ @Around-----> to configure method as around advice method.
- ✓ @AfterReturning-----> to configure method as AfterAdvice
- ✓ @AfterThrowing-----> to configure method as throw advice
- ✓ @Pointcut -----> to specify pointcut expression with reusability
- ✓ @Order-----> to specify order

To make IOC container recognizing all these annotations we need to place <aop:aspectj-autoproxy> tag in xml file. This is equal to <aop:config> tag

While working with AspectJ AOP we can take Advice classes as pojo classes and we can place multiple advice logics in a single class.

@Around, @AfterReturning, @Before, @AfterThrowing annotations allow to place pointcut expressions directly.

To place pointcut expression with reusability we can use @Pointcut.

4/1/16
5/1/16

AspectJ AOP7 (Anno-AroundAdvice) ⇒

src

com.nt.aspect

CacheAdvice.java

LogAroundAdvice.java

PerformanceMonitoringAdvice.java

com.nt.cfgs

applicationContext.xml

com.nt.service

IntrAmtCalculator.java

com.nt.test

ClientApp.java

JRE System Library [JavaSE-1.7]

AOPLib

...

com.nt.aspect:aspectJ-autoproxy />

makes IOC container to modify In memory metadata

by recognising target, advice classes that are configured through Annotations.

⇒ @component, @service makes Java class as

Spring bean where as @Aspect makes

Spring bean as aspect class.

CacheAdvice.java

```
package com.nt.aspect;
```

```
import java.util.Arrays;
```

```
import java.util.HashMap;
```

```
import org.aspectj.lang.ProceedingJoinPoint;
```

```
import org.aspectj.lang.annotation.Around;
```

```
import org.aspectj.lang.annotation.Aspect;
```

```
import org.springframework.core.annotation.Order;
```

```
import org.springframework.stereotype.Component;
```

```
@Aspect
```

```
@Order(1)
```

```
@Component
```

```
public class CacheAdvice {
```

```
    HashMap<String, Object> cache = new HashMap<String, Object>();
```

```
    @Around("execution(* com.nt.service.*(..))")
```

```
    public Object caching(ProceedingJoinPoint pjp) throws Throwable {
```

```
        String key = pjp.getSignature() + Arrays.toString(pjp.getArgs());
```

```
        if (!cache.containsKey(key)) {
```

```
            Object retVal = pjp.proceed();
```

```
            cache.put(key, retVal);
```

```
            System.out.println("From target method");
```

```
            return retVal;
```

```
        }
```

```
        else {
```

```
            System.out.println("From cache");
```

```
            return cache.get(key);
```

```
}  
    }  
} // caching  
} // class
```

LogAroundAdvice.java

```
package com.nt.aspect;  
import java.util.Arrays;  
import org.aspectj.lang.ProceedingJoinPoint;  
import org.aspectj.lang.annotation.Around;  
import org.aspectj.lang.annotation.Aspect;  
import org.springframework.core.annotation.Order;  
import org.springframework.stereotype.Component;  
@Aspect  
@Order(2)  
@Component  
public class LogAroundAdvice {  
    @Around("execution(* com.nt.service.*(..))")  
    public Object logging(ProceedingJoinPoint pjp) throws Throwable {  
        System.out.println("Entering into "+pjp.getSignature()+" with  
args"+Arrays.toString(pjp.getArgs()));  
        //Get args and modify args  
        Object args[]=pjp.getArgs();  
        if((Float)args[0]<=50000){  
            args[2]=(Float)args[2]-1;  
        }  
        Object retVal=null;  
        if((Float)args[0]==0 || (Float)args[1]==0 || (Float)args[2]==0 )  
            throw new IllegalArgumentException("Provide +ve values as the p,t,r  
values");  
        else  
            retVal=pjp.proceed(args);  
        // add Service tax on intr amt  
        retVal=(Float)retVal+(Float)retVal*0.05f;  
        System.out.println("Exiting from "+pjp.getSignature()+" with  
args"+Arrays.toString(pjp.getArgs()));  
        return retVal;  
    } //method  
} //class
```

PerformanceMonitoringAdvice.java

```
package com.nt.aspect;  
import java.util.Arrays;  
import org.aspectj.lang.ProceedingJoinPoint;  
import org.aspectj.lang.annotation.Around;  
import org.aspectj.lang.annotation.Aspect;  
import org.springframework.core.annotation.Order;  
import org.springframework.stereotype.Component;
```



```

@Aspect
@Order(3)
@Component
public class PerformanceMonitoringAdvice {
    long start,end;
    @Around("execution(* com.nt.service.*.*(..))")
    public Object pMonitor(ProceedingJoinPoint pjp)throws Throwable{
        start=System.currentTimeMillis();
        Object retVal=pjp.proceed();
        end=System.currentTimeMillis();
        System.out.println(pjp.getSignature()+" with" +Arrays.toString(pjp.getArgs())+" has
taken "+(end-start)+"ms");
        return retVal;
    }
}

```

IntrAmtCalculator.java

```

package com.nt.service;
import org.springframework.stereotype.Component;
@Component("targetBean")
public class IntrAmtCalculator {
    public float calcSimpleIntrAmt(float p,float t,float r){
        return (p*t*r)/100.0f;
    }
    public float calcCompoundIntrAmt(float p,float t,float r){
        return (float) (p*Math.pow((1+r/100.0f),t)-p);
    }
}

```

applicationContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xmlns:aop="http://www.springframework.org/schema/aop"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
<!-- <bean id="targetBean" class="com.nt.service.IntrAmtCalculator"/>
<bean id="pMAdvice" class="com.nt.aspect.PerformanceMonitoringAdvice"/>
<bean id="logAdvice" class="com.nt.aspect.LogAroundAdvice"/>
<bean id="cacheAdvice" class="com.nt.aspect.CacheAdvice"/> -->
<aop:aspectj-autoproxy/>
<context:component-scan base-package="com.nt.aspect,com.nt.service"/>
</beans>

```

ClientApp.java

```

package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.IntrAmtCalculator;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext cTransactionbemanagement = new
        FileSystemXmlApplicationContext("src/com/nt/beconfigures/applicationContext.xml");
        //Get Proxy obj
        IntrAmtCalculator proxy = cTransactionbemanagement
        .getBean("targetBean", IntrAmtCalculator.class);
        //Call target method
        System.out.println("Simple Intr Amt"+proxy.calcSimpleIntrAmt(10000,20,2));
        System.out.println("-----");
        System.out.println("Compound Intr Amt"+proxy.calcCompoundIntrAmt(60000, 20,
2));
        System.out.println("-----");
        System.out.println("Compound Intr Amt"+proxy.calcCompoundIntrAmt(60000, 20,
2));
    }
}

```

By working with AspectJ AOP with annotations the order they follow to apply advices on target method will be decided based on the order that is used to configure advice classes as spring beans in applicationContext.xml

If advice/aspect classes are not configured in xml file and they are configured through annotations like

@Component then we can define the order by using

@Order(-) annotations

@Component or @Service can be used to configure ordinary java class as spring bean.

@Aspect can't be used to configure ordinary java class as aspect class. But it can be used to configure spring bean as aspect class.

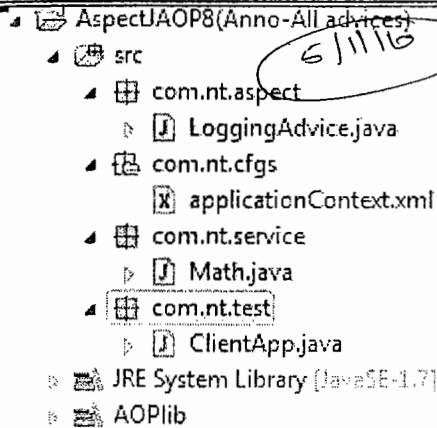
While working with annotation-based AspectJ AOP

<aop:aspect-autoproxy> is alternate to <aop:config> tag to enable aspectjAOP based on the annotations.

ThrowsAdvice not only executes for the exception that is raised in target method, it also executes for the exception that is raised in advices that are related to target method.

When we define multiple advice methods in ^{simple} aspect class having same pointcut expression then instead of defining that pointcut expression separately for every method it is recommended to take dummy method in same Aspect class having pointcut expression defined through

@Pointcut and specify that dummy method name in every advice method as pointcut expression.



Ex:

LoggingAdvice.java

```
package com.nt.aspect;
import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.AfterReturning;
import org.aspectj.lang.annotation.AfterThrowing;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;
import org.springframework.stereotype.Component;

@Component
@Aspect
public class LoggingAdvice {
    @Pointcut("execution(* com.nt.service.Math.*(..))")
    public void myPointcut(){
    }

    @Around("myPointcut()")
    public Object log(ProceedingJoinPoint pjp)throws Throwable{
        System.out.println("Entering into "+pjp.getSignature());
        Object retVal=pjp.proceed();
        System.out.println("Leaving from "+pjp.getSignature());
        return retVal;
    }

    @Before("myPointcut()")
    public void checkInputs(JoinPoint jp){
        Object args[]=jp.getArgs();
        if((Integer)args[0]<=0 || (Integer)args[1]<=0){
            throw new IllegalArgumentException("Provide proper inputs");
        }
    }

    @AfterReturning(value="myPointcut()",returning="result")
    public void checkResult(JoinPoint jp,int result){
        if(result<=0)
```

```
throw new IllegalArgumentException("Invalid result");
```

```
}  
@AfterThrowing(value="myPointcut()",throwing="e")  
public void exceptionLogger(JoinPoint jp,Exception e){  
    System.out.println(jp.getSignature()+" has raised"+e.getClass()+" Exception");  
}  
}
```

Math.java

```
package com.nt.service;  
import org.springframework.stereotype.Component;  
@Component("math")  
public class Math {  
    public int sum(int x,int y){  
        return x+y;  
    }  
    public int sub(int x,int y){  
        return x-y;  
    }  
    public int mul(int x,int y){  
        return x*y;  
    }  
    public int div(int x,int y){  
        return (int)x/y;  
    }  
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"  
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
xmlns:context="http://www.springframework.org/schema/context"  
xmlns:aop="http://www.springframework.org/schema/aop"  
xsi:schemaLocation="http://www.springframework.org/schema/beans  
http://www.springframework.org/schema/beans/spring-beans.xsd  
http://www.springframework.org/schema/aop  
http://www.springframework.org/schema/aop/spring-aop.xsd  
http://www.springframework.org/schema/context  
http://www.springframework.org/schema/context/spring-context.xsd">  
    <aop:aspectj-autoproxy/>  
    <context:component-scan base-package="com.nt.aspect,com.nt.service"/>  
</beans>
```

ClientApp.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.Math;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext cTransaction management =new
        FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
        //GEt Proxy obj
        Math proxy=cTransaction management.getBean("math",Math.class);
        //Call target method
        System.out.println(proxy.sum(10,20));
        System.out.println("-----");
        System.out.println(proxy.sub(10,30));
        System.out.println("-----");
        System.out.println(proxy.mul(20,30));
        System.out.println("-----");
        System.out.println(proxy.div(20,10));
    }
}
```

Transaction Management

The process of combining related operations into single unit and executing then by applying do everything or nothing principle called "**Transaction Management**"

The withdraw amount from source account, deposit amount into dest amount must happen in Transaction environment.

In employee registration employee details insertion into DB and insertion into payroll DB must happen in Transactional environment.

Transaction management allows to implement ACID properties.

A-->Atomicity

C--->Consistency

I---->Isolation

D---->Durability

The process of combining indivisible operations into single unit is called Atomicity. Atomicity word is derived from the term call Atom.

Even though rules kept on DB software (like balance must not be negative) are violated during the course Transaction management, but if there is a guarrenty that no rule will be violated at the end of Transaction management, then we can say DB is consistent.

Applying locks on DB software and allowing one user per one application at a time to manipulate db data is called isolation.

Brining data back to normal state by log file, backup file through it is crashed indicates durability of DB

Every Transaction contains 3 operations

Begin Transaction

.....
..... //continue Transaction (Execute logics)
.....

Commit/Rollback Transaction

Transaction management resource must allow data commit or rollback operations.

File can't be taken as Transaction resource. DB software, message Queues can be taken as Transaction resource.

Based on the Transaction management applications instruction the Transaction manager connects to resource through resource manager to commit or rollback Transaction.

Based on number of resources that are involved in a transaction there are 2 types of transaction.

1) Local Transaction

Single resource will participate in transaction boundary

Ex: Transferring money operation between 2 account of

2) Global Transaction / Distributed Transaction / XA Transaction

Multiple resources will participate in takes place on multiple resources in a single shot.

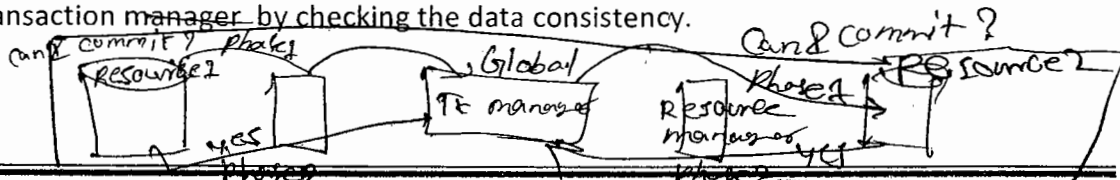
Ex: Transferring money operation between 2 accounts of 2 different bank

Global transaction runs based on 2 pc protocol (2 phase commit)

Phase-1:

Transaction manager talks with all the resources through resource manager seeking permission to commit the Transaction manager by checking the data consistency.

Phase-2:



If all the resource managers respond and says "OK" then the data of multiple resources will be committed. If any resource manager "no" then data of multiple database will be rollback. Jdbc, hibernate supports only local Transaction management . Spring, EJB, supports both local and global Transaction management

Transaction Models: [1. Flat Tx model, 2. Nested Tx model]

When the tour plan is given to flat Transaction management model application, if one journey tkt are not available it will cancel the remaining journey tkt. Because all booking will be taken as direct operations of main transaction.

If above tour plan is given to nested Transaction management . Model application it will confirm remaining journey tickets even though one or another journey takes are not available . Because here each journey take booking will be taken as sub Transaction management of main Transaction management so the success or failure of one transaction doesn't affect other sub Transaction management

Flat Transaction

Nested Transaction

Main Transaction {
journey1 tkt booking
Journey2 tkt booking
.....

Journeryn tkt booking
}



Nested Tx model

Main Tx {

sub Tx1 {

--- Journey1 ticket booking

}

sub Tx2 {

--- Journey2 ticket booking

}

Jdbc, Hibernate, EJB supports only flat transactions spring supports both flat Transaction management and Nested Transaction management .

Generally we use JDBC code for local Transaction Management as shown below

Local Transaction management sample code using jdbc:

```

try {
con.setAutoCommit(false); // Transaction management began
.....
..... //Transaction management .logic
.....
con.commit ();
} catch (Exception e){
con.rollback();
}

```

To perform distributed Transaction management we need either application server (JEE server) or Third party api like atomokis that gives userTransaction Object as transaction manager. We also need XA-jdbc drivers, XA Datasource manager to get Transaction manager required for distributed Transaction management .

Here are never directly call con.commit() or con.rollback. we ask TransactionManager to perform these operations by getting connection through Resource Manager (Datasource). The server like weblogic supply "userTransaction" object in its jndi registry having fixed jndi name like

"javax.transaction.userTransaction"

We use JTA support to perform global Transaction management in programmatic approach. We use EJB to perform Global Transaction management in declarative approach (xml configurations/annotation configurations).

- In programmatic approach we write Transaction management code inside the B.methods. So disabling Transaction management service is very complex (we need to modify java code)
- In declarative approach b.method contains only b.logic but the Tx Transaction configurations will be placed in xml file. The container/server/framework enable Transaction management service on b.methods based on xml file.
- Jdbc, JTA supports only programmatic Transaction management.
- EJB supports both programmatic & declarative management.
- Global Transaction management in the component that is deployable in the server using JTA (programmatic approach) :-

```
InitialContext IC=new InitialContext();
UserTransaction ut=(UserTransaction)IC.lookup("javad/transaction/userTransaction");
try{
    Ut.begin(); //begin Transaction management
    .....
    ..... Transaction management .logic
    .....
    ut.commit();
} catch (Exception e){
    ut.rollback();
}
}
```

Declarative Transaction management Management Using EJB:

EJB component

```
public class Mycomp implements SessionBean
{
    public void bml()
    {
        .....
        ..... // only B.logic or no Transaction management logic
        .....
    }
}
```

Ejb-jar.xml

class configures

→ Transaction management configuration :-

Based on xml configures the EJB container applies Transaction management services on to b.methods of EJB component.

→ The EJB's declarative Transaction management service is really nice But it is heavy weight.

Spring Transaction:

In the above discussion we need to use different java, JEE apis or 3rd party apis like JPA, hibernate and eTransaction management To perform different modes of Transaction management.

- Due to this switching from local Transaction management to distributed or switching from one Transaction management implementation to another Transaction management implementation become very complex for the programmer.
- Spring provides unified environment to work with any Transaction management because it is a framework providing abstraction layer on multiple technologies.
- Due to this we can make spring to use internally any implementation of Transaction management.
- But we can write the unified code for Transaction management.
- Due to this switching to distributed Transaction management from local Transaction management and switching from one Transaction management implementation to another Transaction management implementation doesn't make to change spring code much. 1,2,3 → priority

Spring supports 4 ways of transaction management

- 1) Programatic Transaction management *(Long back outdated)*
- 2) Declarative Transaction management using spring AOP *(2,3) → only in maintenance mod*
- * 3) Declarative Transaction management using AspectJAOP *(1) (Gmt)*
- 4) Annotations based Transaction management using AspectJAOP *(2)*

- In spring Transaction service is secondary logic/cross cutting concern. It is given as a pre-defined Aspect logic as the combination of Around advice and throwsAdvice type.
- Transaction management will be begin while entering into method and will be commit when method returns a value or will be rollback when methods throws exceptions. So the Transaction management service is combination of AroundAdvice and throwsAdvice.
- In order to perform Transaction management we need to choose the appropriate Transaction manager/ Transaction management coordinator.
- DataSourceTransactionManager → For underlying jdbc *code* (Local Transaction management)
- HibernateTransactionManager → for hibernate *code* (Local Transaction management)
- JDOTransactionManager → for the *code* (Transaction management) → *JDO*
- JtaTransactionManager → for Distributed Transaction management and etc.....

File 16

Declarative Transaction management management using spring AOP:

TransactionProxyFactoryBean → Generates proxy class applying Transaction service on Target method

- NameMatchTransactionAttributeSource → Allows to specify Transaction management attribute on target methods it is like pointcut

Note: To create DataSourceTransactionManager the "dataSource" object is dependent.

- In programmatic Transaction management we write Transaction management code inside the b.method which is against of AOP, so don't use programmatic Transaction management management.
- In declarative Transaction management we write Transaction management related configure in xml file so target method just contain business logic. But Transaction management service will be applied on business method because of proxy class *Hand*

TxMgmtProj1(Decl Txmgmt Spring AOP Local, Flat Tx)

8/1/16

- src
 - com.nt.cfgs
 - applicationContext.xml
 - com.nt.dao
 - BankDAO.java
 - BankDAOImpl.java
 - com.nt.service
 - BankService.java
 - com.nt.test
 - ClientApp.java

SQL> desc tx_account;

name	null?	type
ACNO	NOT NULL	number(5)
HOLDER		varchar2(20)
BALANCE		number(20)

Referenced Libraries

- TXlib
 - spring-jdbc-4.1.6.RELEASE.jar - C:\Users\deli\
 - spring-tx-4.1.6.RELEASE.jar - C:\Users\deli\De
 - ojdbc14.jar - D:\oracle\product\10.2.0\db_23\
 - spring-aspects-4.1.6.RELEASE.jar - C:\Users\deli\
 - spring-beans-4.1.6.RELEASE.jar - C:\Users\deli\
 - spring-context-4.1.6.RELEASE.jar - C:\Users\deli\
 - spring-context-support-4.1.6.RELEASE.jar - C\
 - spring-core-4.1.6.RELEASE.jar - C:\Users\deli\
 - spring-expression-4.1.6.RELEASE.jar - C:\Users\
- JRE System Library [jre7]

BankDAO.java

```
package com.nt.dao;

public interface BankDAO {

    public int withdraw(int acno,int amt);

    public int deposit(int acno,int amt);

}
```

BankDAOImpl.java

```
package com.nt.dao;

import org.springframework.jdbc.core.JdbcTemplate;

public class BankDAOImpl implements BankDAO{

    private JdbcTemplate jt;

    private static final String WITHDRAW_QRY="update tx_account set balance=balance-?
where accno=?";

    private static final String DEPOSITE_QRY="update tx_account set balance=balance+? where
accno=?";

    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }

    @Override
    public int withdraw(int acno, int amt) {
        int cnt=jt.update(WITHDRAW_QRY,amt,acno);
        return cnt;
    }

    @Override
```

```

    public int deposit(int acno, int amt) {
        int cnt=jt.update(DEPOSITE_QRY,amt,acno);
        return cnt;
    }
}

```

BankService.java

```

package com.nt.service;
import com.nt.dao.BankDAO;
public class BankService {
    private BankDAO dao;
    public void setDao(BankDAO dao) {
        this.dao = dao;
    }
    public boolean transfertranspringframeworkerMoney(int srcNo,int destNo,int amt){
        int result1=dao.withdraw(srcNo, amt);
        int result2=dao.deposit(destNo, amt);

        if(result1==0 || result2==0){
            throw new RuntimeException("Money not Transpringframeworker(Tx
            rolled back)");
            ↳ must be unchecked Exception to make Transaction management
            to rollback the transaction
        }
        else{
            System.out.println("Money Transpringframeworker (Tx committed)");
            return true;
        }
    }
}

```

applicationContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.0.xsd">
    <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
        <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
        <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
        <property name="username" value="scott"/>
        <property name="password" value="tiger"/>
    </bean>
    <bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
        <property name="dataSource" ref="dataSource"/>
    </bean>
    <bean id="bankDAO" class="com.nt.dao.BankDAOImpl">
        <property name="jt" ref="template"/>
    </bean>

```

```

<bean id="txMgr"
class="org.springframework.jdbc.datasource.DataSourceTransactionManager"> <property
name="dataSource" ref="dataSource"/>
</bean>
<bean id="bankService" class="com.nt.service.BankService">
  <property name="dao" ref="bankDAO"/>
</bean>
<bean id="nmtas"
class="org.springframework.transaction.interceptor.NameMatchTransactionAttributeSource">
  <property name="properties">
    <props>
      <prop key="transpringframeworkerMoney">PROPAGATION_REQUIRED</prop>
    </props>
  </property>
</bean>
<bean id="tpfb"
class="org.springframework.transaction.interceptor.TransactionProxyFactoryBean">
  <property name="transactionManager" ref="txMgr"/>
  <property name="target" ref="bankService"/>
  <property name="transactionAttributeSource" ref="nmtas"/>
</bean>
</beans>

```

↳ To enable Transaction manager on Spring jdbc persistence logic

↳ It is like pointcut to apply transaction attributes on a target class method

transfer

generates proxy class by applying transaction service

ClientApp.java

```

package com.nt.test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.BankService;
public class ClientApp {
public static void main(String[] args) {
  //create IOC container
  ApplicationContext ctx=new
  FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
  //get Proxy obj
  BankService proxy=ctx.getBean("tpfb",BankService.class);
  //call b.method
  System.out.println("Money transfer? "
  Transpringframeworker?"+proxy.transpringframeworkerMoney(1002, 1001,500));
}
}
Jar: AOPLib+springjdbcLib.

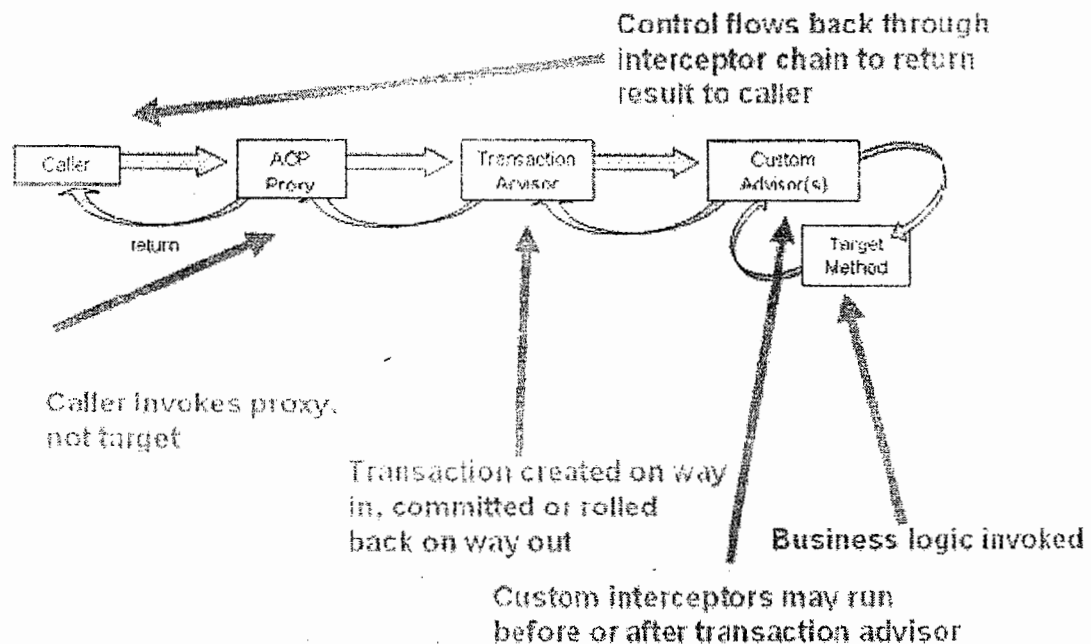
```

ctx

transfer

Transpringframeworker?

Conceptually, calling a method on a transactional proxy looks like this...



In declarative Transaction management the Transaction management manager rollback the Transaction management automatically if unchecked exception is raised in the target method (RunTimeException and its subclass) otherwise the Transaction management will be committed.

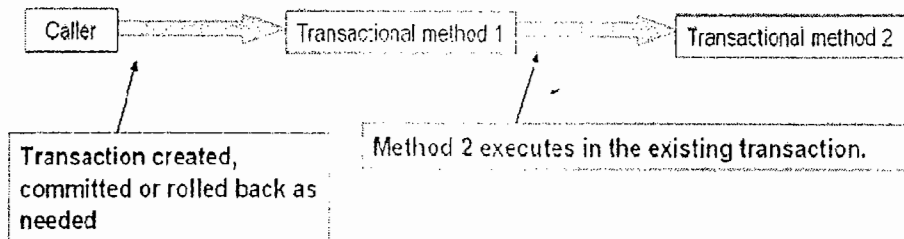
In declarative Transaction management we can specify Transaction management attribute on the target method to make target method running with no transaction or new transaction or client supplied Transaction.

<Tx:method> attributes:

Attribute	Required?	Default	Description
name	Yes		Method name(s) with which the transaction attributes are to be associated. The wildcard (*) character can be used to associate the same transaction attribute settings with a number of methods; for example, get*, handle*, on*Event, and so forth.
propagation	No	REQUIRED	Transaction propagation behavior.
isolation	No	DEFAULT	Transaction isolation level.
Timeout	No	-1	Transaction timeout value (in seconds).
read-only	No	False	Is this transaction read-only?
rollback-for	No		Exception(s) that trigger rollback; comma-delimited. For example, com.foo.MyBusinessException, ServletException.
no-rollback-for	No		Exception(s) that do not trigger rollback; comma-delimited. For example, com.foo.MyBusinessException, ServletException.

Transaction Attribute:

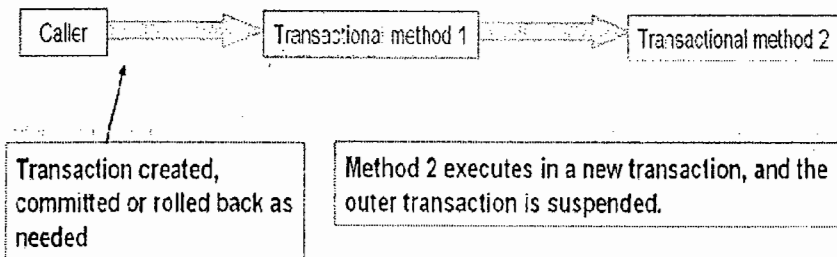
Required	(PROPAGATION_REQUIRED) — most regular/2 used
Requirednew	(PROPAGATION_REQUIRES_NEW)
Supports	(PROPAGATION_SUPPORTS) — default
Never	(PROPAGATION_NEVER)
notSupported	(PROPAGATION_NOT_SUPPORTED)
mandatory	(PROPAGATION_MANDATORY)

Required:**REQUIRED****Transaction**

If clientApp calls the B.method in a Transaction management then b.method runs ⁱⁿ that Transaction management.

If clientApp calls the B.method without Transaction management then b.method runs ⁱⁿ a new Transaction management.

Note: It is most popularly used Transaction management attribute.

RequiresNew:**REQUIRES_NEW****Transaction 1****Transaction 2**

B.method/target method always runs in a new Transaction management irrespective of ^{whether} clientApp calls b.method in a Transaction management or not.

Supports:

B.method runs ^{with} in client Application Transaction management, if client application calls B.method in a Transaction management.

B.method runs without Transaction management, if client application calls the b.method without Transaction management.

It is default attribute.

Never:

ClientApp must not call b.method in a Transaction management, if called Exception will be raised.

Not supported: If b.method/target method always runs in a ^{No Transaction} irrespective of whether client app ~~calls~~ calls b.method ^{with or without} Transaction.

Manadatory:

ClientApp must call `B.method()` in `Transaction` otherwise `Exception` will be raised

Transaction management attribute	client Transaction	Bean's Transaction
Required	none	T2 (new)
	T1	T1
RequiresNew	None	T2
	T1	T2
Supports	None	None
	T1	T1
Manadatory	None	Error
	T1	T1
Not Supported	None	None
	T1	None
Never	None	None
	T1	error

Declarative Transaction management Management Using AspectJ AOP:

It is most regularly used Transaction management management mechanism. To work with this we need to import AOP; ^{Tx namespace} Transaction management namespaces. In order to activate Transaction management service as Advice ^{having} bean id to specify Transaction management attribute on target methods we can use `<tx:advice>`, `<tx:attributes>`, `<tx:method>` tags. The Transaction management advice performs commit or rollback operation through Transaction management manager so we need to specify Transaction management manager while configuring Transaction management Advice.

In order to apply Transaction management advice on the specific methods of Target/service class use `<aop:config>`, `<aop:advisor>`, `<aop:pointcut>` tag as shown below.

▲ TxMgmtProj2(Decl Txmgmnt AspectJ AOP Local, Flat

▲ src

▲ com.nt.cfgs

applicationContext.xml

▲ com.nt.dao

BankDAO.java

BankDAOImpl.java

▲ com.nt.service

BankService.java

▲ com.nt.test

ClientApp.java

▶ Referenced Libraries

▶ TXlib

▶ JRE System Library [jre7]

Add ~~app~~ aspectJ 2 jar files

BankDAO.java

```
package com.nt.dao;
```

```
public interface BankDAO {
```

```
    public int withdraw(int acno, int amt);
```

```
    public int deposit(int acno, int amt);
```

```
}
```

BankDAOImpl.java

```
package com.nt.dao;
import org.springframework.jdbc.core.JdbcTemplate;
public class BankDAOImpl implements BankDAO{
    private JdbcTemplate jt;
    private static final String WITHDRAW_QRY="update tx_account set balance=balance-?
where accno=?";
    private static final String DEPOSITE_QRY="update tx_account set balance=balance+? where
accno=?";
    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }
    @Override
    public int withdraw(int acno, int amt) {
        int cnt=jt.update(WITHDRAW_QRY,amt,acno);
        return cnt;
    }
    @Override
    public int deposit(int acno, int amt) {
        int cnt=jt.update(DEPOSITE_QRY,amt,acno);
        return cnt;
    }
}
```

BankService.java

```
package com.nt.service;
import com.nt.dao.BankDAO;
public class BankService {
    private BankDAO dao;
    public void setDao(BankDAO dao) {
        this.dao = dao;
    }
    public boolean transpringframeworkerMoney(int srcNo,int destNo,int amt){
        int result1=dao.withdraw(srcNo, amt);
        int result2=dao.deposit(destNo, amt);

        if(result1==0 || result2==0){
            throw new RuntimeException("Money not Transpringframeworkered(Tx
rolled back)");
        }
        else{
            System.out.println("Money Transpringframeworkered (Tx committed)");
            return true; } } }
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:aop="http://www.springframework.org/schema/aop"
xmlns:tx="http://www.springframework.org/schema/tx"
xsi:schemaLocation="
```



```

http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/tx
http://www.springframework.org/schema/tx/spring-tx.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd">
  <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
    <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
    <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
    <property name="username" value="scott"/>
    <property name="password" value="tiger"/>
  </bean>
  <bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
    <property name="dataSource" ref="dataSource"/>
  </bean>
  <bean id="bankDAO" class="com.nt.dao.BankDAOImpl">
    <property name="jt" ref="template"/>
  </bean>
  <bean id="txMgr"
class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
    <property name="dataSource" ref="dataSource"/>
  </bean>
  <bean id="bankService" class="com.nt.service.BankService">
    <property name="dao" ref="bankDAO"/>
  </bean>
  <tx:advice id="txAdvice" transaction-manager="txMgr">
    <tx:attributes>
      <tx:method name="transpringframeworkerMoney" propagation="REQUIRED"
read-only="false"/>
    </tx:attributes>
  </tx:advice>
  <aop:config>
    <aop:pointcut id="ptc1"
      expression="execution(*
com.nt.service.BankService.transpringframeworkerMoney(..))"/>
    <aop:advisor advice-ref="txAdvice" pointcut-ref="ptc1"/>
  </aop:config> </beans>

```

Transaction Attribute
optional indicating non select persistence operations are happening

ClientApp.java

```

package com.nt.test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.BankService;
public class ClientApp {
  public static void main(String[] args) {
    //create IOC container
    ApplicationContext ctx=new
    FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");

```

```
//get Proxy obj
BankService proxy=ctx.getBean("bankService",BankService.class);
try{
//call b.method
System.out.println("Money
Transpringframeworkfered?" + proxy.transferMoney(1006, 1001,500));
}
catch(Exception e){
e.printStackTrace();
}
}
```

We can repeat <tx:method> tag for multiple times to apply different Transaction management attributes on different target method. If we specify * then it represents all the methods of target class.

```
<tx:method name="*" propagation="REQUIRED"/>
```

In declarative, annotation based Transaction management management the Transaction manager will rollback the transction only when target method throws unchecked exception (RuntimeException and it's subclass) doesn't rollback for checked exception.

We Generally design DAO class one one per table statergy. In stead of applying Transaction management service on DAO class methods directly use DAO classes in service class method and apply Transaction management service on that methods of service class.

Annotations driven AspectJAOP Transaction management management:

For this Transaction the Transaction management configurations will not be done in xml file.They will be configured directly on target class methods by using @Transactional.

To make IOC container detecting and using these annotations we can use <tx:annotation-driven transaction-manager="...."/>

▲ TxMgmtProj3(Anno Tx mgmt Local Tx,Flat Tx)

- ▲ src
 - ▲ com.nt.cfgs
 - applicationContext.xml
 - ▲ com.nt.dao
 - BankDAO.java
 - BankDAOImpl.java
 - ▲ com.nt.service
 - BankService.java
 - ▲ com.nt.test
 - ClientApp.java
- ▶ Referenced Libraries
- ▶ TXlib
- ▶ JRE System Library [jre7]

BankDAO.java

```
package com.nt.dao;
public interface BankDAO {
    public int withdraw(int acno,int amt);
    public int deposit(int acno,int amt);
}
```

BankDAOImpl.java

```
package com.nt.dao;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;
@Repository("bankDAO")
public class BankDAOImpl implements BankDAO{
    private JdbcTemplate jt;
    private static final String WITHDRAW_QRY="update tx_account set balance=balance-?
where accno=?";
    private static final String DEPOSITE_QRY="update tx_account set balance=balance+? where
accno=?";
    @Autowired
    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }
    @Override
    public int withdraw(int acno, int amt) {
        int cnt=jt.update(WITHDRAW_QRY,amt,acno);
        return cnt;
    }
    @Override
    public int deposit(int acno, int amt) {
        int cnt=jt.update(DEPOSITE_QRY,amt,acno);
        return cnt;
    }
}
```

BankService.java

```
package com.nt.service;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;
import com.nt.dao.BankDAO;
@Component("bankService")
public class BankService {
    private BankDAO dao;
    @Autowired
    public void setDao(BankDAO dao) {
        this.dao = dao;
    }
    @Transactional(propagation=Propagation.REQUIRED)
    public boolean transferMoney(int srcNo,int destNo,int amt){
        int result1=dao.withdraw(srcNo, amt);
        int result2=dao.deposit(destNo, amt);
        if(result1==0 || result2==0){
            throw new RuntimeException("Money not Transferred(Tx
rolled back)");
        }
    }
}
```

```
    }  
    else{  
        System.out.println("Money Transpringframeworkered (Tx committed)");  
        return true;  
    }  
}  
}
```

ApplicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"  
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
xmlns:aop="http://www.springframework.org/schema/aop"  
xmlns:tx="http://www.springframework.org/schema/tx"  
xmlns:context="http://www.springframework.org/schema/context"  
xsi:schemaLocation="  
http://www.springframework.org/schema/beans  
http://www.springframework.org/schema/beans/spring-beans.xsd  
http://www.springframework.org/schema/tx  
http://www.springframework.org/schema/tx/spring-tx.xsd  
http://www.springframework.org/schema/aop  
http://www.springframework.org/schema/aop/spring-aop.xsd  
http://www.springframework.org/schema/context  
http://www.springframework.org/schema/context/spring-context.xsd">  
    <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">  
        <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>  
        <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>  
        <property name="username" value="scott"/>  
        <property name="password" value="tiger"/>  
    </bean>  
    <bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">  
        <property name="dataSource" ref="dataSource"/>  
    </bean>  
    <bean id="txMgr" class="org.springframework.jdbc.datasource.DataSourceTransactionManager">  
        <property name="dataSource" ref="dataSource"/>  
    </bean>  
    <context:component-scan base-package="com.nt.service,com.nt.dao"/>  
    <tx:annotation-driven transaction-manager="txMgr"/>  
</beans>
```

ClientApp.java

```
package com.nt.test;  
import org.springframework.context.ApplicationContext;  
import org.springframework.context.support.FileSystemXmlApplicationContext;  
import com.nt.service.BankService;  
public class ClientApp {  
    public static void main(String[] args) {  
        //create IOC container
```

```

ApplicationContext ctx=new FileSystemXmlApplicationContext ("src/com/ nt/configures/
                                                                    applicationContext.xml");

//get Proxy obj
BankService proxy=ctx.getBean("bankService",BankService.class);
try{
//call b.method
System.out.println("Money
Transferred?"+proxy.transferMoney(1001, 1008,500));
}
catch(Exception e){
    e.printStackTrace();
}
}
}

```

@Transactional properties

Property	Type	Description
<u>value</u>	String	Optional qualifier specifying the transaction manager to be used.
<u>propagation</u>	enum: Propagation	Optional propagation setting.
<u>isolation</u>	enum: Isolation	Optional isolation level.
<u>readOnly</u>	Boolean	Read/write vs. read-only transaction
<u>Timeout</u>	int (in seconds granularity)	Transaction timeout.
<u>rollbackFor</u>	Array of Class objects, which must be derived from Throwable.	Optional array of exception classes that must cause rollback.
<u>rollbackForClassname</u>	Array of class names. Classes must be derived from Throwable.	Optional array of names of exception classes that must cause rollback.
<u>noRollbackFor</u>	Array of Class objects, which must be derived from Throwable.	Optional array of exception classes that must not cause rollback.
<u>noRollbackForClassname</u>	Array of String class names, which must be derived from Throwable.	Optional array of names of exception classes that must not cause rollback.

While working with @Transactional we can specify read only param and propagation param for transaction attributes.

In order to apply same Transaction management attribute on all methods of target class we can place @Transactional on top of class.

Other use cases of transactionmanagement:

In employee region its details should be saved in hrDB and in finance DB.

In student register its details should be save in library db and admission DB.

In online shopping goods delivery and payment must be executed in a transaction.

```

<bean id="atomikostx" class="com.atomikos.icatch.jta.userTransaction"/>
<bean id="atomikosUT" class="com.atomikos.location.jta.UserTransactionManager">
<property name="tx" ref="atomikostx"/>
<property name="userTransaction" ref="atomikosUT"/>
</bean>

```

EX: DISTRIBUTED Transfer Money operation b/w two accounts of two different banks (Two

1. TxMgmtProj1(Anno-Distributed Tx) different Databases(oracle, mysql) Transaction Management:-

2. src

3. com.nt.cfgs

4. applicationContext.xml

5. com.nt.dao

6. DepositDAO.java

7. DepositDAOImpl.java

8. WithdrawDAO.java

9. WithdrawDAOImpl.java

10. com.nt.service

11. AccountService.java

12. com.nt.test

13. ClientApp.java

14. Referenced Libraries

15. TXlib

16. JRE System Library [jre7]

→ It uses 2pc principle by involving multiple resources in a single transaction boundary. For this we need to use

XA JDBC driver, XA Databases & DataSources.

→ oracle 11g versions, MYSQL 5.5+ are XA Databases

→ To perform Distributed Tx management, we use

JTA Transaction manager by supplying Distributed

Transaction manager object and Transaction Service obj.

We can get these 2 objects either from JNDI registry,

or app server or by using 3rd party API like Atomikos

Sample code for Distributed Tx mgmt :-

DepositDAO.java

package com.nt.dao;

public interface DepositDAO {

public boolean deposit(int accNo, int amt);

}

DepositDAOImpl.java

package com.nt.dao;

import org.springframework.jdbc.core.JdbcTemplate;

public class DepositDAOImpl implements DepositDAO {

private static final String DEPOSITE_QRY="update tx_account set balance=balance+? where accno=?";

private JdbcTemplate jt;

public void setJt(JdbcTemplate jt){

this.jt=jt;

}

@Override

public boolean deposit(int accNo, int amt) {

int cnt=jt.update(DEPOSITE_QRY,amt,accNo);

if(cnt==0)

return false;

else

return true;

}

}

WithdrawDAO.java

package com.nt.dao;

public interface WithdrawDAO {

public boolean withdraw(int accNo, int amt);

}

WithdrawDAOImpl.java

```
package com.nt.dao;
import org.springframework.jdbc.core.JdbcTemplate;
public class WithdrawDAOImpl implements WithdrawDAO {
    private static final String WITHDRAW_QRY="update tx_account set balance=balance-?
where accno=?";
    private JdbcTemplate jt;
    public void setJt(JdbcTemplate jt){
        this.jt=jt;
    }
```

@Override

```
public boolean withdraw(int accNo, int amt) {
    int cnt=jt.update(WITHDRAW_QRY,amt,accNo);
    if(cnt<=0)
        return false;
    else
        return true;
}
```

It represents
"Distributed Transaction
manager".
It represents
"Distributed Transaction
Service"

AccountService.java

```
package com.nt.service;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;
import com.nt.dao.DepositeDAO;
import com.nt.dao.WithdrawDAO;
public class AccountService {
    private DepositeDAO depositeDAO;
    private WithdrawDAO withdrawDAO;
    public void setDepositeDAO(DepositeDAO depositeDAO) {
        this.depositeDAO = depositeDAO;
    }
    public void setWithdrawDAO(WithdrawDAO withdrawDAO) {
        this.withdrawDAO = withdrawDAO;
    }
    @Transactional(propagation=Propagation.REQUIRED)
    public boolean transpringframeworkerMoney(int srcAcno,int destAcno,int amt){
        boolean result1=withdrawDAO.withdraw(srcAcno, amt);
        boolean result2=depositeDAO.deposite(destAcno, amt);
        if(result1==false || result2==false){
            throw new RuntimeException(" Money not transpringframeworkered");
        }
        return true;
    }
    //transpringframeworkerMoney(-,-)
    //class transferMoney (-,-)
```

appliionContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:aop="http://www.springframework.org/schema/aop"
xmlns:tx="http://www.springframework.org/schema/tx"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/tx
http://www.springframework.org/schema/tx/spring-tx.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
<bean id="orads" class="com.atomikos.jdbc.AtomikosDataSourceBean" init-method="init" destroy-
method="close">
  <property name="uniqueResourceName" value="XAOrs"/>
  <property name="xaDataSourceClassName" value="oracle.jdbc.xa.client.OracleXADataSource"/>
  <property name="xaProperties">
    gives atomikos AP2 supplied DataSource class
    that gives XADataSource object for oracle.
    <props>
      <prop key="databaseName">xe</prop>
      <prop key="user">scott</prop>
      <prop key="password">tiger</prop>
      <prop key="URL">jdbc:oracle:thin:@localhost:1521:xe</prop>
    </props>
  </property>
  <property name="poolSize" value="10"/>
</bean>
<bean id="mysqls" class="com.atomikos.jdbc.AtomikosDataSourceBean" init-method="init"
destroy-method="close">
  <property name="uniqueResourceName" value="XAMysql"/>
  <property name="xaDataSourceClassName"
value="com.mysql.jdbc.jdbc2.optional.MysqlXADataSource"/>
  <property name="xaProperties">
    Atomikos AP2 supplied DataSource class
    that gives XADataSource obj for MYSQL
    <props>
      <prop key="databaseName">ntsp7</prop>
      <prop key="user">root</prop>
      <prop key="password">root</prop>
      <prop key="URL">jdbc:mysql:///ntsp7</prop>
    </props>
  </property>
  <property name="poolSize" value="10"/>
</bean>
<bean id="template1" class="org.springframework.jdbc.core.JdbcTemplate">
  <property name="dataSource" ref="orads"/>
</bean>
<bean id="template2" class="org.springframework.jdbc.core.JdbcTemplate">

```



```

<property name="dataSource" ref="mysqlds"/>
</bean>
<bean id="wDao" class="com.nt.dao.WithdrawDAOImpl">
  <property name="jt" ref="template1"/>
</bean>
<bean id="dDao" class="com.nt.dao.DepositeDAOImpl">
  <property name="jt" ref="template2"/>
</bean>
<bean id="service" class="com.nt.service.AccountService">
  <property name="depositDAO" ref="dDao"/>
  <property name="withdrawDAO" ref="wDao"/>
</bean>
<!-- Construct Atomikos UserTransactionManager, needed to configure Spring --> atomikos AP2
<bean id="atomikosTxMgr" class="com.atomikos.icatch.jta.UserTransactionManager" supplied
  init-method="init" destroy-method="close"> Tx manger
<!-- when close is called, should we force transactions to terminate or not? -->
  <property name="forceShutdown" value="true"/>
</bean>
<!-- Also use Atomikos UserTransactionImp, needed to configure Spring --> atomikos AP2 supplied
<bean id="atomikosUT" class="com.atomikos.icatch.jta.UserTransactionImp"> UserTransaction Imp
  <property name="transactionTimeout" value="300"/> 3rd party transaction service
</bean>
<!-- Configure the Spring framework to use JTA transactions from Atomikos -->
<bean id="dtxmgr" class="org.springframework.transaction.jta.JtaTransactionManager">
  <property name="transactionManager" ref="atomikosTxMgr"/> springs transaction manager
  <property name="userTransaction" ref="atomikosUT"/> that uses atomikos AP2
</bean>
<tx:annotation-driven transaction-manager="dtxmgr"/> supplied distributed Tx manager,
</beans> service

```

ClientApp.java

```

package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.AccountService;
public class ClientApp {
  public static void main(String[] args) {
    //create IOC container
    ApplicationContext ctx=new
    FileSystemXmlApplicationContext("src/com/nt/configures/applicationContext.xml");
    //get Proxy obj
    AccountService proxy=ctx.getBean("service",AccountService.class);
    try{
      //call b.method
      System.out.println("Money
      Transpringframeworkfered?" + proxy.transpringframeworkerMoney(1201, 1002,500));
    }
    catch(Exception e){
      e.printStackTrace();
    }
  }
}

```

Jars: Spring jdbc-lib, AOP library, Aspectjrt.jar, AspectJweaver.jar

Atomikos-transaction-api.jar, atomikos-transactions-jta.jar, atomikos-util-3.7.0.jar, com.mysql.jdbc-5.1.5.jar, jta.jar ojdbc.jar, transactions-3.7.0.jar, Transactions-jdbc-3.7.0.jar.

DB table in Oracle:

Transaction-management_account

Accno	holder	balance
101	Raja	9000

DB table in mysql 5.x (global db:ntsp 27)

Transaction-management_account

Accno	holder	balance
102	Ravi	8000

While working with distributed Transaction management the UserTransaction object represent Transaction service and TransactionManager object represents the Transaction management manager that performs commit or rollback by using Transaction service.

JtaTransactionManager can use either server supplied or 3rd party API supplied Tx manager, UserTransaction objects to perform Tx management.

Q. While developing MVC2 Arch based web app can we change the roles of various technologies that are used?

Ans: Yes we can change. but it is not recommended

Servlet as view: presentation logic changes at regular intervals. view modification in jsp is easy compared to view modifications in ~~Servlet~~ ^{Servlet} so it is recommended to use jsp as view

JSP as Controller: writing java code in jsp is a bad practice.

Use js.

When we use jsp as controller: we should place java code in jsp to interact with model layer ~~give response~~ ^{business components}.

It is recommended to use Servlet/ServletFilter ~~place java code~~ as controller. there place java code not a problem.

→ java class, EJB components, Spring beans as view and controller components.

This thought is wrong ~~best~~ we can't take request from client and

we can't send response to browser.

Spring MVC

Different Web App Development

Model1

- Here either jsp or Servlets will be taken as main web resource programs having multiple logics.
- Since every servlet/jsp contains Multiple logics, there is no Separation between logics. So this Architecture is not suitable for Large scale web application.
- MVC1 architecture we take Single resource (server/jsp) Having view, controller separate logics and we take resources for model layer logics.

Model2

In model 2 we take multiple technologies in multiple layers to develop various logic. This gives clean separation Between logics.

→ In MVC2 architecture we take separate resources for view, separate resources for model and separate resources for controller.

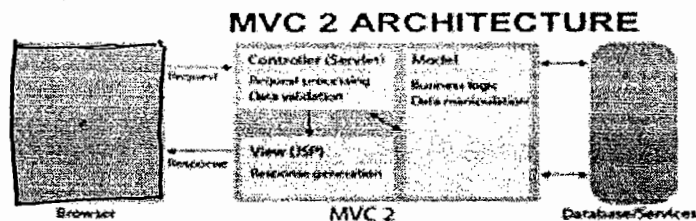
M---->Model---->Data+B.logic+persistence logic

V---->View----->persistence logic

C----->controller--->integration logic/controller logic.

⇒ Controller maintains logic all the activities of web application like takes the request, passes the request to appropriate model component, gathers requests from model and passes the results to view.

MVC2 Architecture:-



* MVC 2 isolates the presentation logic from business logic.

* In MVC 2, controller is servlet and model is java class.

* Supports re usability of application components.

Diagram:

Multiple backend software together is called EIS (Enterprise Information S/m).
The project that is developed by using old technology or old versions of existing technologies is called Legacy project.

Eg: COBOL project, Java 1.0 project

Advantages of MVC2:

- Because of multiple layers we can get clean separations between logics.
- The modification done in layer logics, does not affect other layer logics.
- Maintenance and enhancement, so gives good productivity.
- While using EJB, spring in the model layer we can take advantage of built-in middleware services.

Disadvantages:

- Knowledge on more technologies are required.
- For parallel development, more programmers are required.
- Every web application contains some common logic to handle along with application logics.

1. Request graphing/wrapping :-

Reading data from the form comps of form pages and placing that data in a java class object.

2. Request Submission/form submission:

We get request from different places of web pages like through hyperlinks, through submit buttons, through other components.

These request may come with or without data, so we should handle all the request properly.

3. Form Validation/Form Handling :-

- Verifying the pattern, format of form data and displaying same form page with errors having old values in form component.

→ If we develop web application by using JEE technologies directly we need to write the above 3 common logics in multiple servlet programs, which leads boiler plate code problem.

→ If we develop MVC2 architecture web applications by using JEE technologies directly (Servlet, JSP) we need to develop all the logics of all the layers manually, moreover we need to write both common and application specific logics.

MVC2 Design pattern/Architecture rules:

- Every layer is designed to have specific logics, so place only those logics, don't place any additional logics.
- All the operations of application must be take place under the control or monitoring of controller servlet.
- There can be multiple resources in view layer, multiple resources in model layer but there must be only 1 servlet acting as controller. This must be taken as front controller.
- The view and model layer comps must not interact with each other directly. They must interact through controller.

→ If we develop mvc2 architecture based web application manually, All the logics of all the layers (including common, application specific logic) must be taken care by programmer.

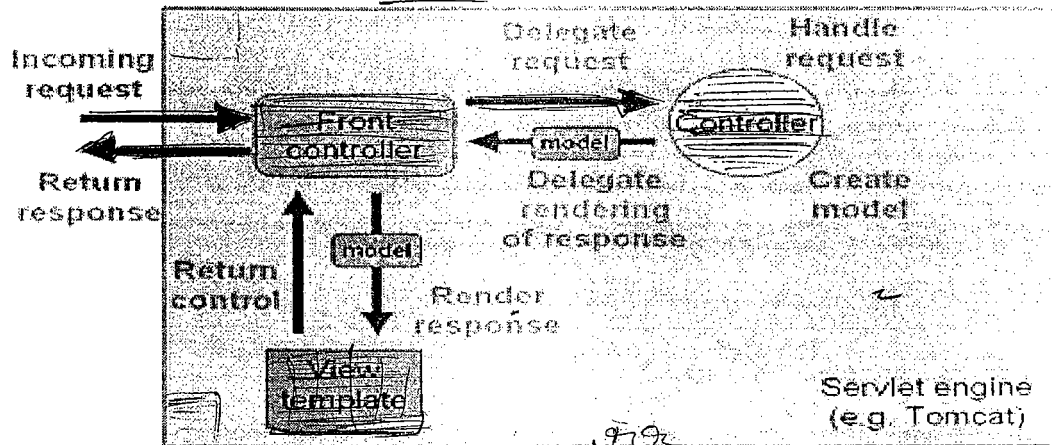
→ If we develop mvc2 architecture based web application by using web framework then the common logics will be taken care by the framework by giving one built-in servlet as frontController, so the programmer just needs to develop application specific logic.

The common logics are like request graphing, form/request, form having security, logging etc

The web frameworks are

- *Struts -----> apache
- JSPRINGFRAMEWORK (JSF) -----> from SunMS (Oracle) (2)
- Webwork -----> from opson symphony
- ADF -----> from Oracle corp (3) → commercial
- MVC -----> from interface21 (1)

Front Controller Diagram: 20/11/16



→ java classes of web app can't take req directly from browser. so we make those reqs getting trapped by Front Controller to delegate the req to appropriate java classes.

→ The special web component of web application that acts as entry point for all the request and applies common system services for all the request like (request graphing, request submission, form handling is called "front controller". security, logging -etc)

→ We can take either Servlet/JSP ^{as} for front controller. To make front controller trapping and taking multiple request coming from clients we must configure then either with directory match or extension match url pattern. ^{it (Front Controller)}

eg *do,*.tm,*.http etc are extension match url pattern

eg /x/y/* and etc are directory match url pattern.

→ In struts ActionServlet is builtin front controller servlet.

→ In spring MVC DispatcherServlet is builtin front controller servlet.

Spring MVC Web application: Flow: refer last pages ^{as} for diagram

1) Browser gives request to web application.

2) As frontcontroller Dispatcherservlet traps and takes the request and applies the common system - services like security and etc....

3) DispatchServlet uses HandlerMapping component to decide the Handler class to utilize based on the incoming request uri.

4. Dispatcherservlet passes the control to Handler class by calling method.

5. Handler class internally writes the received form data to command class object.

6. Handler class process the request and generates the output, if needed it also uses command object work with form data.

7. Handler class returns logical view name back to DispatchServlet.

8. DispatchServlet uses view Resolver to get view object having view layer technology and resource name.

9. DispatchServlet uses the view object to pass the control to view resource.

10. The view resource format the resources and sends the output to browser (response to browser).

Spring MVC features:

1) Allows to use various technologies in the view layer like html, jsp, velocity, freemaker, xslt, etc...

2) Allows to design command class (whose object holds form data) as pojo class (Need not to extend or implement spring api class/interfaces)

3) Allow to take any type properties in command class to hold the form data.

4) Supports internationalization (i18n) ^{as} ^{→ E18n}
(making our app working for different Locales)

5) Allows to configure themes for rendering better presentation.

6) Allows to develop b.logics, form validation logics and other logics ^{as} ^{→ reusable} logics.

7) Provides clean separation between logics/comps by involving different handler mapping, views, viewresolver, command classes, handler/controller classes and etc...

1/1/16 → Allows to integrate with other modules of spring framework and etc..

→ Spring MVC is designed around DispatcherServlet because various operations on the request will be done by DispatcherServlet by using different components to process the Request and to deliver the response.

→ Even though it is predefined servlet its configuration in web.xml is mandatory. We configure this servlet with either extension match or directory match url pattern to make that servlet as front controller to take multiple requests.

→ This servlet internally activates IOC container of type WebApplicationContext by taking <DispatcherServlet logical name in web.xml>-servlet.xml or specified filename (as namespace init param value) as spring bean configure file.

Spring bean configure file contains Handler mapping, Controller class, View Resolver Configurations,

web.xml

<servlet>

<servlet-name>dispatcher</servlet-name>

<servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>

<init-param>

<param-name>namespace</param-name>

<param-value>applicationContext</param-value>

</init-param>

<load-on-startup>2</load-on-startup>

</servlet>

<servlet-mapping>

<servlet-name>dispatcher</servlet-name>

<url-pattern>*.htm</url-pattern>

</servlet-mapping>

name as the spring bean cfg file. Due to this, all the beans defined in spring bean cfg file will be pre-instantiated and also completes dependency injection.

⇒ when spring mvc app is deployed, the container creates obj for dispatcherServlet either during server startup or during

the deployment of web app because of load-on-startup. In this

process, the init method of dispatcher Servlet

executes and that logic creates IOC container of type web application context by taking default name or specified

1/1/16 → If this "namespace" init parameter is not configured the IOC container created by DispatcherServlet (ds) creates dispatcher-servlet.xml as spring bean configure file. If it is configured the IOC container takes applicationContext.xml as spring bean configuration file.

⇒ It is recommended to place jsp files inside WEB-INF folder or in its sub folders. The reasons are

1) We can hide the jsp technology that is used in the webapplication development from enduser.

2) If jsp page is displaying request attributes given by servlet the direct request to jsp will display ugly values. We can avoid this ugly messages by not allowing direct request to jsp, for that place jsp inside the WEB-INF folder. Eg: book my show.com

3) If session is not logged out properly, hackers can access previous session by appending sessionid to the request url of jsp collected from browser history. This can be avoided by keeping jsps inside the WEB-INF folder.

→ In order to develop our controller/handler class, we must take a class implementing org.springframework.web.servlet.mvc.Controller

Interface and should provide implementation for handleRequest (-,-) method having logic to process the request and to return ModelAndView object having logical view name.

→ Ex: public class MyController implements Controller {
public ModelAndView handleRequest (HttpServletRequest req, HttpServletResponse res)
{
 return home method

return ModelAndView ("home");

```
}}
```

→ Every controller/handler class must be configured in spring configuration file having bean id and that bean id must be mapped with incoming request url by using one or other HandlerMapping.

→ HandlerMapping is responsible to map incoming request urls to handler classes (controller classes).

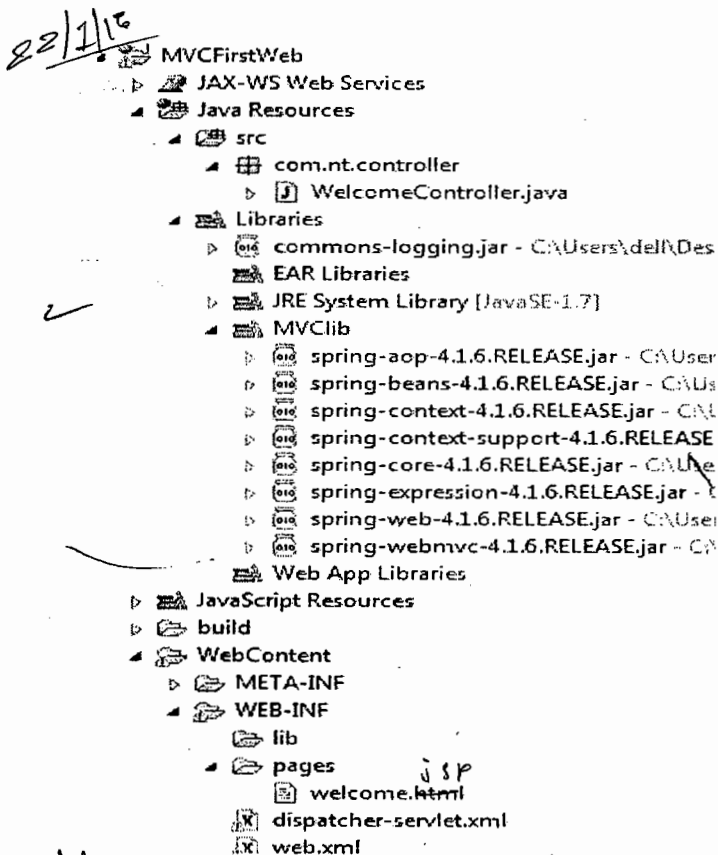
Every HandlerMapping is a class that implements org.springframework.web.servlet.HandlerMapping interface the most regularly used handlermapping class is "SimpleUrlHandlerMapping".

Sample Code:

```
<!--controller class.....>
<bean id="wc" class="com.nt.controller.WelcomeController"/>
<!--HandlerMapping>
<bean id="surl" class="org.springframework.web.servlet.SimpleUrlHandlerMapping">
  <property name="mappings">
    <props>
      <prop key="home.html">wc</prop>
    </props>
  </property>
</bean>
```

Handwritten notes: "match" with an arrow pointing from "wc" to "mappings", and "m" under "mappings".

1st spring mvc web application that displays "welcome.jsp" of WEB-INF/pages folder upon receiving the request



url Request http://localhost:3030/MVCFirstWeb/home.jsp

Web.xml

```
<web-app>
  <servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
```

Handwritten notes: "html" above "home.jsp", and a bracket around the servlet configuration.

```

<load-on-startup>2</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>dispatcher</servlet-name>
  <url-pattern>*.htm</url-pattern>
</servlet-mapping>
</web-app>

```

dispatcher-servlet.xml *(Spring bean config file)*

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
  <bean id="wc" class="com.nt.controller.WelcomeController"/>
  <bean id="surl" class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
    <property name="mappings">
      <props>
        <prop key="home.htm">wc</prop>
      </props>
    </property>
  </bean>

```

```

  <bean id="vr" class="org.springframework.web.servlet.view.InternalResourceViewResolver">

```

```

    <property name="prefix" value="/WEB-INF/pages/" />
    <property name="suffix" value=".jsp" />
  </bean>
</beans>

```

by appending these given prefix and suffixes to logical view name the physical view name will be decided as /WEB-INF/pages/welcome.jsp

Welcome.jsp

```

<h1><center> HOME PAGE</center></h1>

```

```

${requestScope.msg}

```

WelcomeController.java

```

package com.nt.controller;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.Controller;
public class WelcomeController implements Controller {
  @Override
  public ModelAndView handleRequest(HttpServletRequest req,
    HttpServletResponse res) throws Exception {
    return new ModelAndView("welcome", "msg", "Welcome To Spring MVC");
  }
}

```

Viewresolver:

This view Resolver takes the logical ^{view name} given by controller class and resolves physical view name by adding prefixes, suffixes and returns View interface implementation class object back to dispatcher servlet having physical view name.

- DispatcherServlet calls render() method on the given View implementation class object and this render() method passes the control to physical view resource.
- Every ViewResolver is a class that implements org.springframework.web.servlet.ViewResolver interface class. Spring has given multiple built-in viewResolver class in that InternalResourceViewResolver is useful to return view implementation class object having servlet, jsp programs placed in the WEB-INF folder of the web application (internal resource).
- Spring MVC web app2 that launches homepage and process the request based on the hyperlink clicked in homepage. For diagram, refer last pages, for code, refer page no: 252, 253

23/1/16 **Note:**

In spring mvc application only welcome pages will be placed outside the WEB-INF Folder and remaining jsps are recommended to place inside WEB-INF folder for security reason. Welcome page executes automatically for every request given to web application and gives implicit request by using <jsp:forward> tag or response.sendRedirect() method to launch other pages that are there inside WEB-INF folder through controller/handler classes.

- It is always recommended to develop our controller/handler class by extending from predefined XxxController classes. That means it is not recommended to develop same class by implementing Controller interface directly.

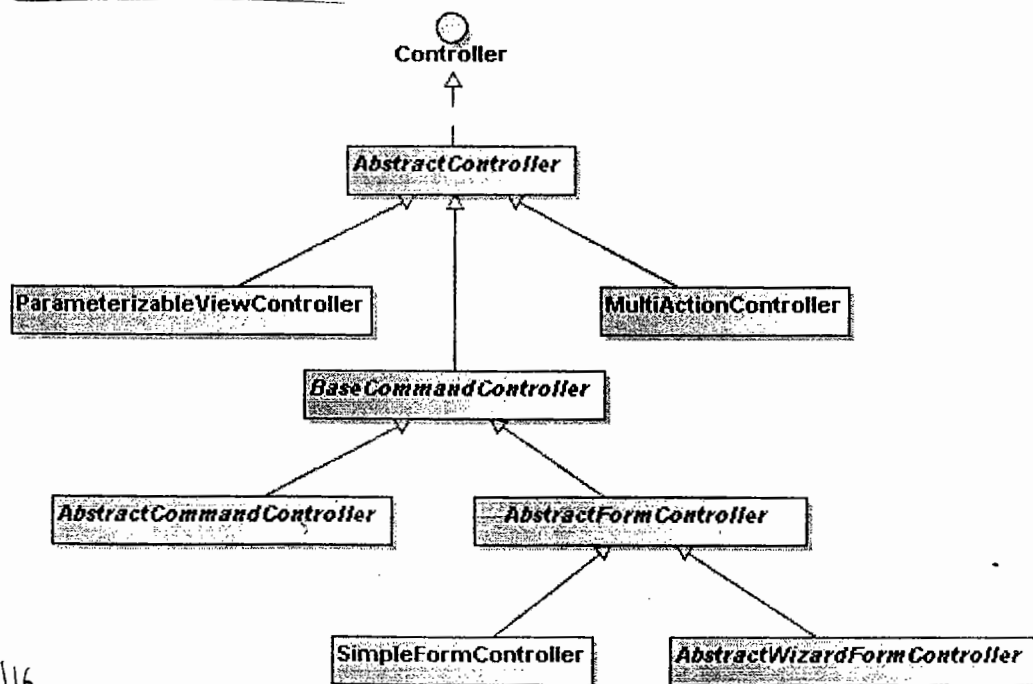
Controller/Handler:

The java class that implements org.springframework.servlet.mvc.Controller is called controller/Handler. It is mainly useful to render logical viewname and data that is required to display in view layer resource.

Placing b.logic/request processing logic in controller class is not recommended process. It is recommended to place such logics in service classes and communicate with them from controller class.

Spring API^{has} supplied lots of built-in controller classes implementing "Controller" interface. These classes are given to handle the request coming from pages, hyperlinks and from other form comps.

23/1/16 **The Controller Classes are:** *Controller Hierarchy*



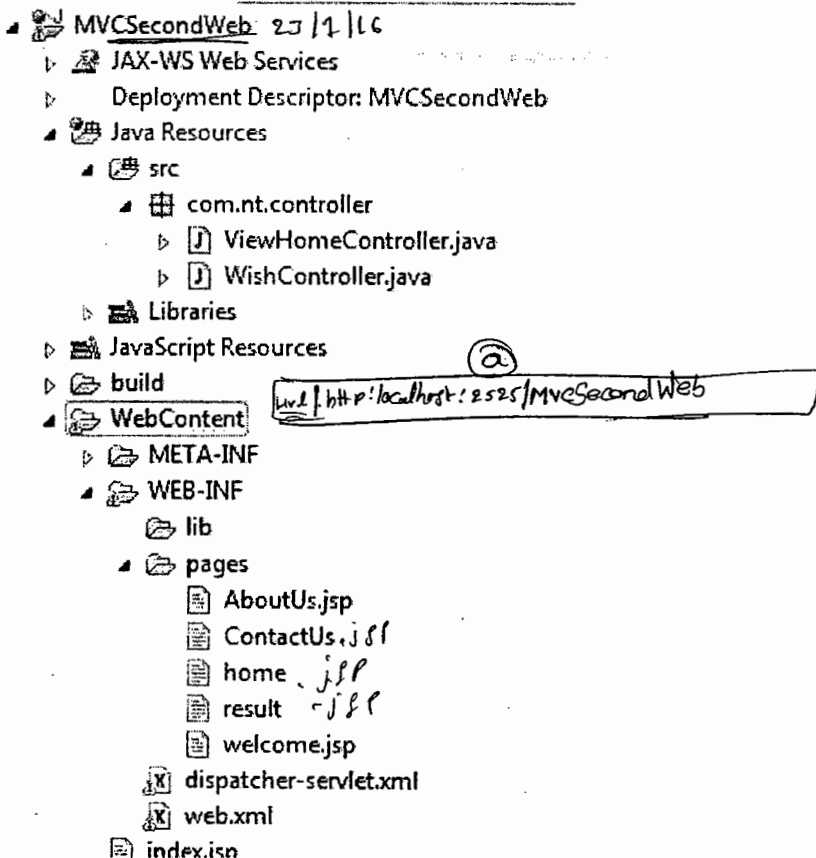
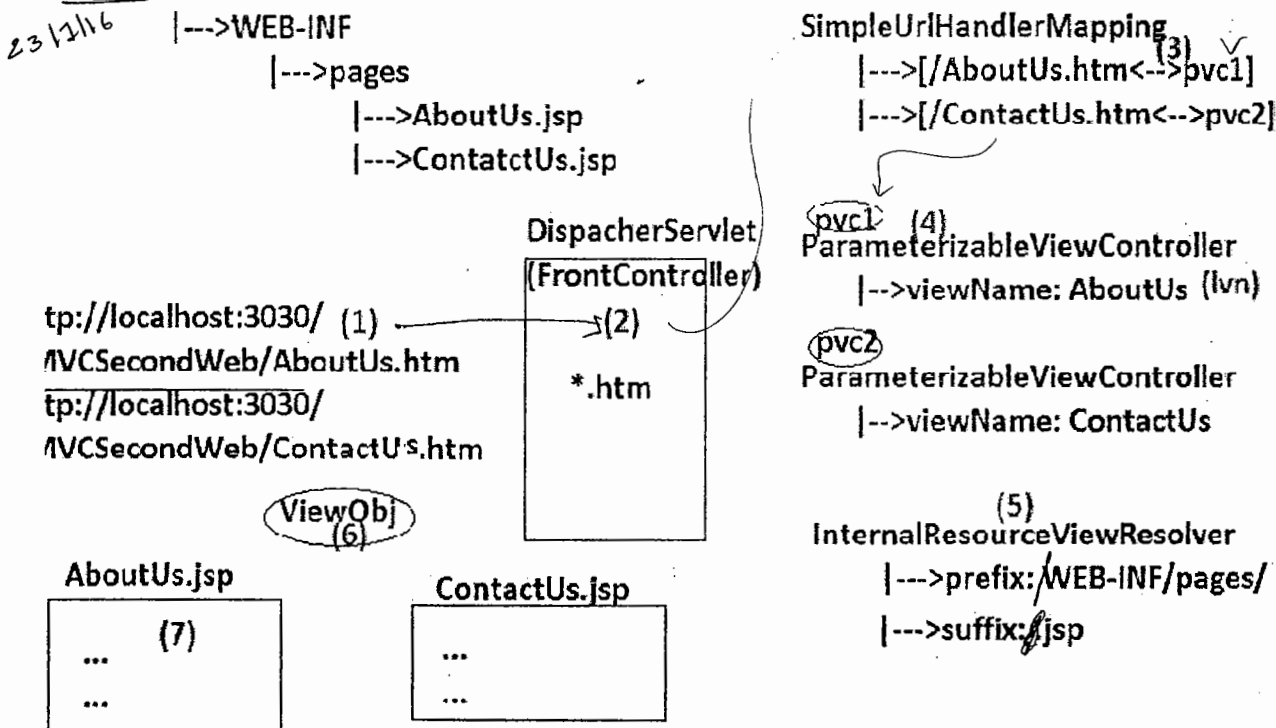
23/2/16

ParameterizableViewController:

→ In order to display webpage without processing request, then instead of taking separate user defined controller class we can configure this class by specifying the destination view name as "viewName" property value. This is useful to display webpage for hyperlink generated request without having any request processing logics

→ Eg: clicking "AboutUs", "ContactUs" hyperlinks. For multiple request of this kind we need to configure this class for multiple times.

Diagram:



```

SimpleUrlHandlerMapping
    |--->[/Aboutus.htm<->pvc1]
    |--->[/contact.htm<->pvc2]
PVC1
ParameterizableViewController
|----->ViewName: AboutUs (logical viewname) No use
PVC2
ParameterizableViewController
    |----->ViewName: ContactUs
InternalResourceViewResolver
    |----->Prefix: /WEB-INF/Pages/
    |----->Softfix: .jsp

```

23/1

ViewHomeController.java

```

package com.nt.controller;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.AbstractController;
public class ViewHomeController extends AbstractController {
    @Override
    protected ModelAndView handleRequestInternal(HttpServletRequest req,
        HttpServletResponse res) throws Exception {
        return new ModelAndView("home");
    }
}

```

23/1

WishController.java

```

package com.nt.controller;
import java.util.Calendar;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.AbstractController;
public class WishController extends AbstractController {
    @Override
    protected ModelAndView handleRequestInternal(HttpServletRequest req,
        HttpServletResponse res) throws Exception {
        // get System date and time
        Calendar cl=Calendar.getInstance();
        //get current hour of the day
        int hour=cl.get(Calendar.HOUR_OF_DAY);
        String msg=null;
        if(hour<12)
            msg="Good morning";
        else if(hour<16)
            msg="Good afternoon";
        else if(hour<20)
            msg="Good evening";
    }
}

```

```

else
    msg="Good Night";
    return new ModelAndView("result", "wmsg", msg);
}

```

23/12 ^(b) Index.jsp

```
<jsp:forward page="home.htm"/>
```

web.xml

```
<web-app>
```

```
<servlet>
```

```
<servlet-name>dispatcher</servlet-name>
```

```
<servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
```

```
<load-on-startup>2</load-on-startup>
```

```
</servlet>
```

```
<servlet-mapping>
```

```
<servlet-name>dispatcher</servlet-name>
```

```
<url-pattern>*.htm</url-pattern>
```

```
</servlet-mapping>
```

```
</web-app>
```

dispatcher-servlet.xml (k)

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
```

```
<bean id="surl" class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
```

```
<property name="mappings">
```

```
<props>
```

```
<prop key="home.htm">vhc</prop> (d)
```

```
<prop key="wish.htm">wc</prop>
```

```
<prop key="AboutUs.htm">ufv</prop>
```

```
<prop key="ContactUs.htm">ufv</prop> (l)
```

```
</props>
```

```
</property>
```

```
</bean> (e)
```

23/11/16 <bean id="vhc" class="com.nt.controller.ViewHomeController"/>

```
<bean id="wc" class="com.nt.controller.WishController"/>
```

```
(n) <bean id="ufv" class="org.springframework.web.servlet.mvc.UrlFilenameViewController"/>
```

```
<!-- <bean id="pvc1" class="
org.springframework.web.servlet.mvc.ParameterizableViewController">
```

```
<property name="viewName" value="AboutUs"/>
```

```
</bean>
```

```
<bean id="pvc2" class="
```

```
org.springframework.web.servlet.mvc.ParameterizableViewController">
```

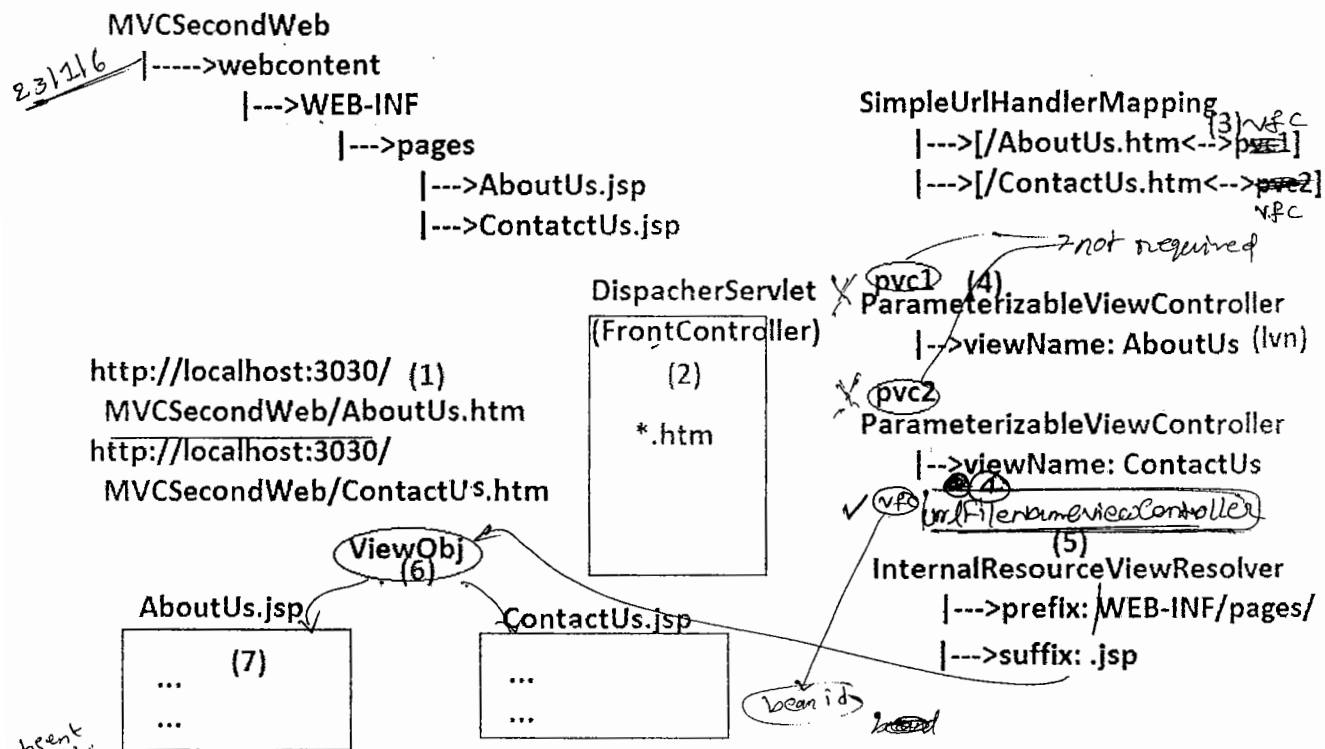
```
<property name="viewName" value="ContactUs"/>
```

```
</bean> -->
```

```
<bean id="vr" class="org.springframework.web.servlet.view.InternalResourceViewResolver">
```

```
(h) <property name="prefix" value="/WEB-INF/pages/">
```

```
(n) <property name="suffix" value=".jsp"/> (n)
```

Note: If request is there without form data (from hyperlinks) and no request processing is required but displaying views is required, then go for ParameterizableViewController or UriFileNameViewController (like working with AboutUs, ContactUs hyperlink).

→ In real-time every spring application will be designed to have 2 IOC containers as parent, child containers because it is recommended to maintain presentation tier components like HandlerMapping, ViewResolver, Controller class in one container (WebApplicationContextContainer) given by DispatcherServlet and business tier component like (service class, DAO class, AOP, Aspect class) in another container (ApplicationContext) IOC container given by **ContextLoaderListner**.
ContextListner is a spring supplied predefined ServletContextListner which activates ApplicationContext container when servletContext object is created (either during server startup or during the deployment of web application) by taking **contextConfigLocation** as name and contextParamValue as the springbean configuration file.

Code in web.xml:

```

<web-app>
<servlet>
<servlet-name>dispatcher</servlet-name>
<servlet-class>org.springframework.web.servlet.DispatcherServlet </servlet-class>
<load-on-startup>2</load-on-startup>
</servlet>
<servlet-mapping>
<servlet-name>dispatcher</servlet-name>
<url-pattern>*.htm</url-pattern>
</servlet-mapping>
<context-param>
<param-name>contextConfigLocation</param-name>
<param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>

```

This dispatcherServlet gives web app context container as child container to manage presentation tier compo.

```

<listener>
<listener-class>org.springframework.web.context.ContextLoaderListener
</listener-class> This contextLoaderListener activated container and becomes
</listener> parent parent container to manage business tier comp's.
</web-app>

```

If namespaceInitParameter is not configured for Dispatcher servlet then the activated webapp context container takes <servlet-name>-servlet.xml as spring bean configuration file. (dispatcher-servlet.xml)

Generally ServletListener object will be created first compare to the servlet object on which load-on-startup is enabled, but some servers create servlet object first before Listener object is load-on-startup priority value is 1.

We want to see Dispatcher^{servlets} webApplicationContext contains^{ex} as child container to contextLoaderListener's ApplicationContext container in all situations and servers for this it is recommended to enable load-on-startup on dispatcher servlet with priority value other than 1. Since controller classes are managed by beans in child container so they can be injected with service class object/beans managed by parent container.

AbstractController:

Take this class to develop controller class when there is no form submission in request but request processing required to display dynamic webpage as output.

Generally for hyperlink generated request to get dynamic content having request processing we should go for AbstractController class.

Use case:

- Getting current user profile in facebook when "viewprofile" link is clicked.
- Getting all the jobs that are posted when "current" trend link is clicked.
- Getting all employee details when "List of Employees" hyperlink is clicked.

Here no form submission, so there will not be any request graphing form handling.

Since there is no form submission the handleRequestInternal(-, -) does not take any command object as argument.

handleRequestInternal(-, -) returns ModelAndView object having logical view name and model Data that has to be displayed as dynamic content.

Note:

The process of storing form data into an object is called request graphing (request wrapping). Validating form data and displaying same form page with old when form validation errors are there is called Form handling.

Note:

Writing business logic in controller class is not a good practice, always practice logic in service class and communication from container class.

Understanding 2 containers work to previous page (54) web.xml: -

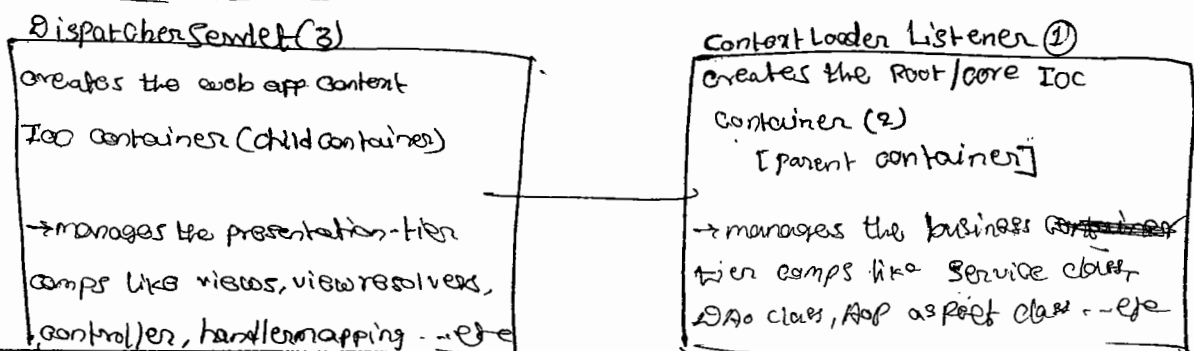
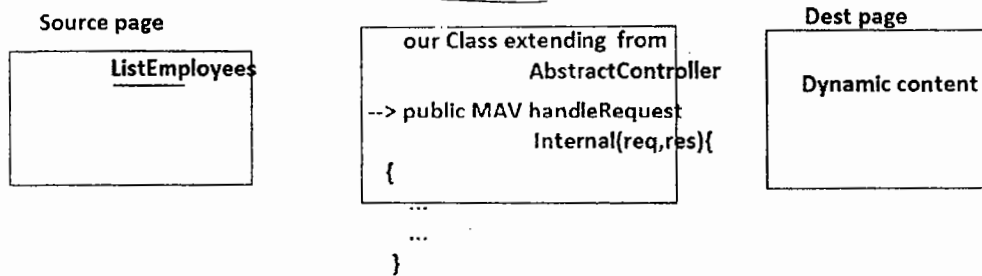


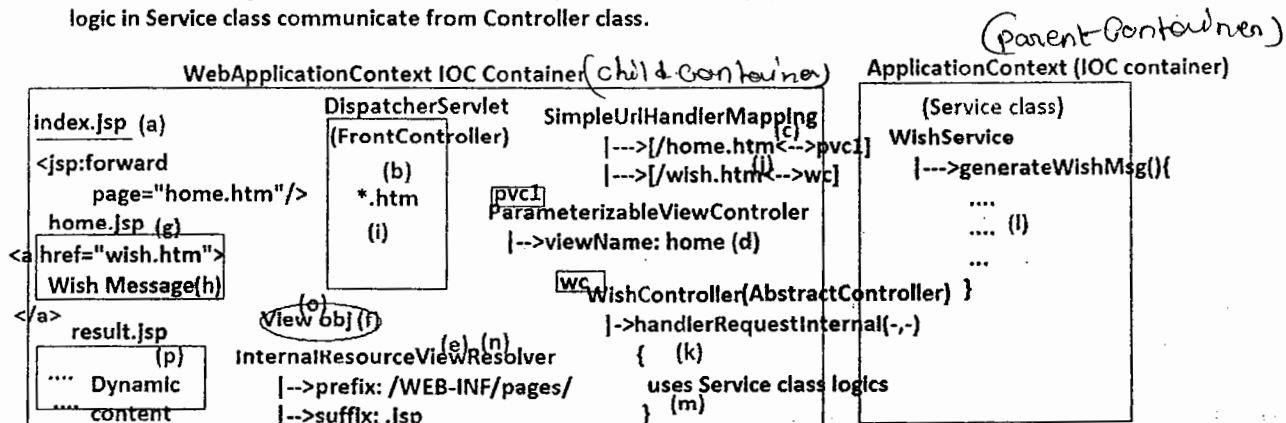
Diagram: Flow of AbstractController

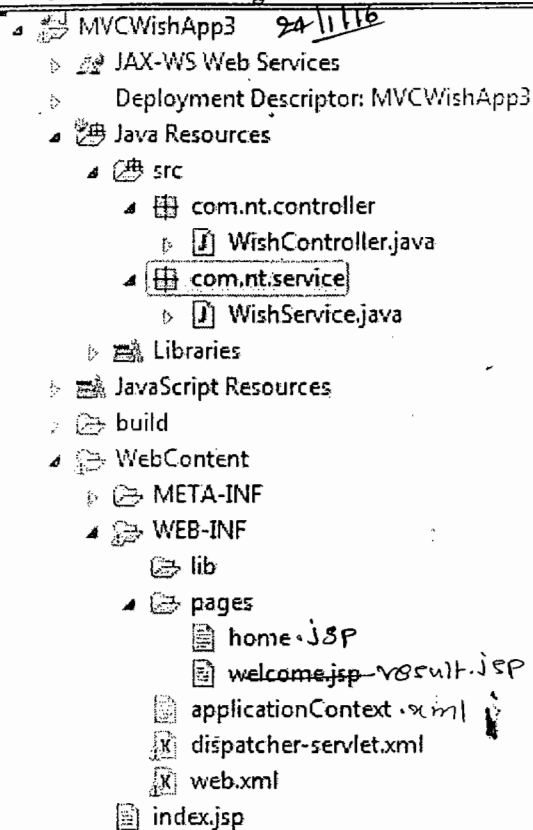


- a) Here no form submission, So there will not be any request grapping, form handling.
 b) Since there is no form submission the handleRequestInternal(-,-) does not take any Command obj as argument.
 c) handleRequestInternal(-,-) returns ModelAndView obj having logical view name and Model Data that has to be displayed as dynamic content.

note: -> The process of storing form data into an obj is called request grapping.
 -> Validating form data and displaying same form page with old when form validation errors are there is called Form handling.

note: Writing b.logic in controller class is not a good practice, Always place this logic in Service class communicate from Controller class.



**WishController.java**

```

package com.nt.controller;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.AbstractController;
import com.nt.service.WishService;

public class WishController extends AbstractController {
    private WishService service;

    public void setService(WishService service) {
        this.service = service;
    }

    @Override
    protected ModelAndView handleRequestInternal(HttpServletRequest req,
        HttpServletResponse res) throws Exception {
        //Use Service class b.logic
        String wmsg=service.generateWishMsg();
        //create and return ModelAndView obj
        return new ModelAndView("result","wresult",wmsg);
    }
}

```

WishService.java

```

package com.nt.service;
import java.util.Calendar;

public class WishService {
    public String generateWishMsg() {

```

```
//get System date and time
Calendar cl=Calendar.getInstance();
//get current hour of the day
int h=cl.get(Calendar.HOUR_OF_DAY);
String msg=null;
if(h<=12)
    msg="Good morning";
else if(h<=16)
    msg="GoodAfter noon";
else if(h<=20)
    msg="Good Evening";
else
    msg="Good Night";
return msg;
}
}
```

Index.jsp

```
<jsp:forward page="home.htm"/>
```

web.xml

```
<web-app>
<servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>2</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>*.htm</url-pattern>
</servlet-mapping>
<listener>
    <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
<context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>
</web-app>
```

dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
    <bean id="surl"
        class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
        <property name="mappings">
            <props>
```

```

    <prop key="/home.htm">pvc1</prop>
    <prop key="/wish.htm">wc</prop>
  </props>
</property>
</bean>
  <bean id="wc" class="com.nt.controller.WishController">
    <property name="service" ><ref parent="wService"/></property>
  </bean>
  <bean id="pvc1" class="
org.springframework.web.servlet.mvc.ParameterizableViewController">
    <property name="viewName" value="home"/>
  </bean>
  <bean id="vr" class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="prefix" value="/WEB-INF/pages/">
    <property name="suffix" value=".jsp"/>
  </bean>
</beans>

```

applicationContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="wService" class="com.nt.service.WishService"/>
</beans>

```

Home.jsp result.jsp

```

<h1><center>Result Page</center></h1>
<br><br>
<font color="red"> ${wresult}</font>
<br>
<a href="home.htm">home</a>

```

Welcome.jsp home.jsp

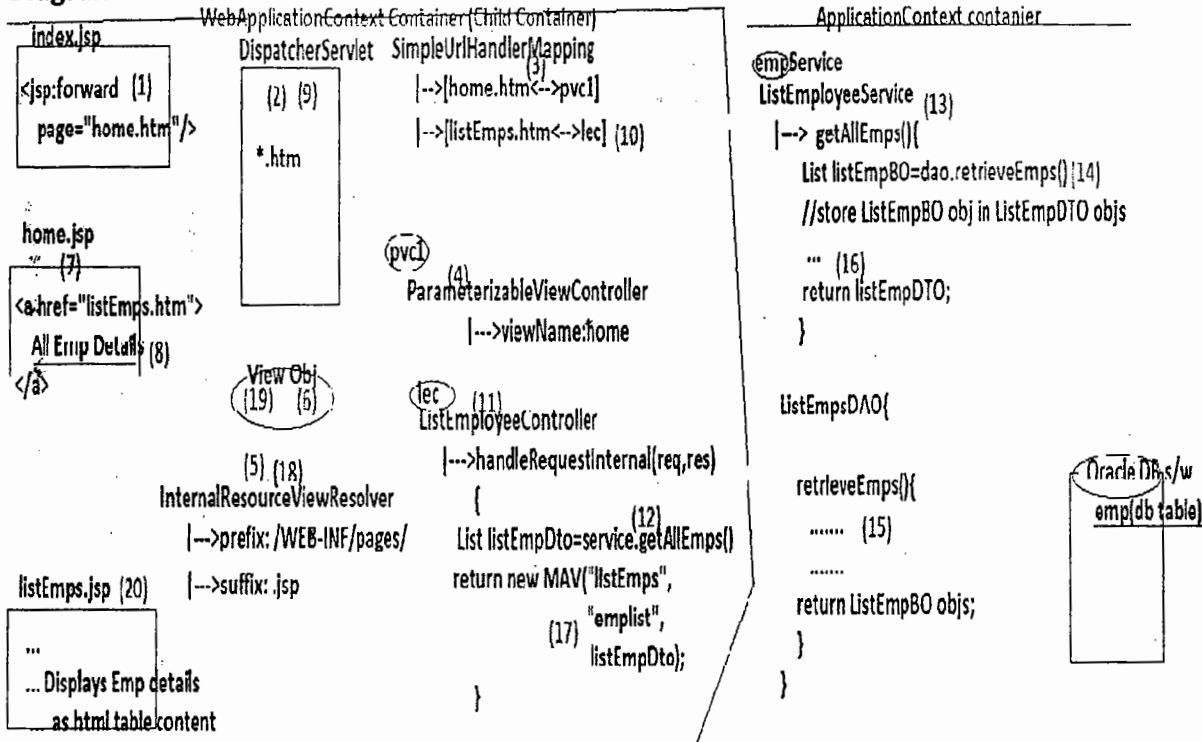
```

<H1><center> HOME PAGE</center></H1>
<br>
<br>
<a href="wish.htm">Wish Message</a>

```

In real time beans of service layer will be configured with separate xml(service-beans.xml) file, similarly the beans of persistence layer will be configured in separate xml file and all this xml will be imported in main applicationContext.xml

Diagram:



List of jars

MVClib + DAOlib + jstl.jar + standard.jar

MVCEmpApp4AC

- ▶ JAX-WS Web Services
- ▶ Deployment Descriptor: MVCEmpApp4AC
- ▶ Java Resources
 - src
 - com.nt.bo
 - EmpBO.java
 - com.nt.controller
 - ListEmpController.java
 - com.nt.dao
 - ListEmpDAO.java
 - com.nt.dto
 - EmpDTO.java
 - com.nt.service
 - ListEmpService.java
 - Libraries
 - JavaScript Resources
 - build
 - WebContent
 - META-INF
 - WEB-INF
 - lib
 - pages
 - home.jsp
 - listEmps.jsp
 - applicationContext.xml
 - dispatcher-servlet.xml
 - persistence-beans.xml
 - service-beans.xml
 - web.xml
 - index.jsp

EmpBO.java

```
package com.nt.bo;
public class EmpBO {
    private int no;
    private String name, job;
    private int salary;
    public EmpBO() {
        System.out.println("EmpBO:0-param constructor");
    }
    public EmpBO(int no, String name, String job, int salary) {
        System.out.println("EmpBO:4-param constructor");
        this.no = no;
        this.name = name;
        this.job = job;
        this.salary = salary;
    }
    public int getNo() {
        return no;
    }
    public void setNo(int no) {
        this.no = no;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getJob() {
        return job;
    }
    public void setJob(String job) {
        this.job = job;
    }
    public int getSalary() {
        return salary;
    }
    public void setSalary(int salary) {
        this.salary = salary;
    }
}
```

ListEmpDAO.java

```
package com.nt.dao;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.List;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.RowMapper;
import com.nt.bo.EmpBO;
public class ListEmpDAO {
    private static final String GET_ALL_EMP_DETAILS="SELECT EMPNO,ENAME,JOB,SAL FROM
    EMP";
    private JdbcTemplate jt;
    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }
    public List<EmpBO> retrieveEmps(){
        List<EmpBO> listEmpBO=(List<EmpBO>)jt.query(GET_ALL_EMP_DETAILS,
                                                    new EmpRowMapper());
        return listEmpBO;
    }
    private class EmpRowMapper implements RowMapper<EmpBO>{

        @Override
        public EmpBO mapRow(ResultSet rs, int pos) throws SQLException {
            //Place ResultSet obj record into BO class obj
            EmpBO empbo=new EmpBO(rs.getInt(1),
                                   rs.getString(2),
                                   rs.getString(3),
                                   rs.getInt(4));

            return empbo;
        }
    }
}
//inner class
//clas
```

EmpDTO.java

```
package com.nt.dto;
import java.io.Serializable;
public class EmpDTO implements Serializable {
    private int no;
    private String name,job;
    private int salary;

    public EmpDTO() {
        System.out.println("EmpDTO:0-param constructor");
    }
    public EmpDTO(int no, String name, String job, int salary) {
        System.out.println("EmpDTO:4-param constructor");
        this.no = no;
    }
}
```

```
        this.name = name;
        this.job = job;
        this.salary = salary;
    }

    public int getNo() {
        return no;
    }
    public void setNo(int no) {
        this.no = no;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getJob() {
        return job;
    }
    public void setJob(String job) {
        this.job = job;
    }
    public int getSalary() {
        return salary;
    }
    public void setSalary(int salary) {
        this.salary = salary;
    }
}
```

ListEmpService.java

```
package com.nt.service;
import java.util.ArrayList;
import java.util.List;
import com.nt.bo.EmpBO;
import com.nt.dao.ListEmpDAO;
import com.nt.dto.EmpDTO;
public class ListEmpService {
    private ListEmpDAO dao;
    public void setDao(ListEmpDAO dao) {
        this.dao = dao;
    }
    public List<EmpDTO> getAllEmps(){
        //use DAO class
        List <EmpBO> listEmpBO=dao.retrieveEmps();
        // Copy List of BO objs to List of DTO objs
        List <EmpDTO> listEmpDTO=new ArrayList<EmpDTO>();
```

```
for(EmpBO bo:listEmpBO){  
    EmpDTO empdto=new EmpDTO(bo.getNo(),  
    bo.getName(),bo.getJob(),bo.getSalary());  
    listEmpDTO.add(empdto);  
} //for  
return listEmpDTO; } //method } //class
```

ListEmpController.java

```
package com.nt.controller;  
import java.util.List;  
import javax.servlet.http.HttpServletRequest;  
import javax.servlet.http.HttpServletResponse;  
import org.springframework.web.servlet.ModelAndView;  
import org.springframework.web.servlet.mvc.AbstractController;  
import com.nt.dto.EmpDTO;  
import com.nt.service.ListEmpService;  
public class ListEmpController extends AbstractController {  
    private ListEmpService service;  
  
    public void setService(ListEmpService service) {  
        this.service = service;  
    }  
  
    @Override  
    protected ModelAndView handleRequestInternal(HttpServletRequest req,  
        HttpServletResponse res) throws Exception {  
        //use service class logics  
        List <EmpDTO> listEmpDTO=service.getAllEmps();  
        return new ModelAndView("listEmps","empList",listEmpDTO);  
    }  
}
```

Index.jsp

```
<jsp:forward page="home.htm"/>
```

web.xml

```
<web-app>  
<servlet>  
    <servlet-name>dispatcher</servlet-name>  
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>  
    <load-on-startup>2</load-on-startup>  
</servlet>  
<servlet-mapping>  
    <servlet-name>dispatcher</servlet-name>  
    <url-pattern>*.htm</url-pattern>  
</servlet-mapping>  
<listener>  
    <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>  
</listener>
```



```
<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>
</web-app>
```

service-beans.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="empService" class="com.nt.service.ListEmpService">
    <property name="dao" ref="empDao"/>
  </bean>
</beans>
```

p - small p Persistence-beans.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
    <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
    <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
    <property name="username" value="scott"/>
    <property name="password" value="tiger"/>
  </bean>
  <bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
    <property name="dataSource" ref="dataSource"/>
  </bean>
  <bean id="empDao" class="com.nt.dao.ListEmpDAO">
    <property name="jt" ref="template"/>
  </bean>
</beans>
```

Dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
  <bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
    <property name="mappings">
      <props>
        <prop key="/home.htm">pvc1</prop>
        <prop key="/listEmps.htm">lec</prop>
      </props>
    </property>
  </bean>
```

```

</property>
</bean>
<bean id="lec" class="com.nt.controller.ListEmpController">
  <property name="service" ref="empService"/>
</bean>
<bean id="pvc1" class="
org.springframework.web.servlet.mvc.ParameterizableViewController">
  <property name="viewName" value="home"/>
</bean>
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="prefix" value="/WEB-INF/pages/" />
  <property name="suffix" value=".jsp" />
</bean>
</beans>

```

applicationContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<import resource="persistence-beans.xml"/>
<import resource="service-beans.xml"/>
</beans>

```

home.jsp

```

<h1><Center>HOME PAGE</Center></h1>
<br>
<a href="listEmps.htm"> All Employees</a>

```

listEmps.jsp

```

<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<table border="1" bgcolor='red'>
  <tr bgcolor="blue"><th>eno</th><th>ename</th><th>JOB</th><th>Salary</th></tr>
  <c:forEach var="emp" items="${empList}">
    <tr bgcolor="cyan">
      <td><c:out value="${emp.no}" /></td>
      <td><c:out value="${emp.name}" /></td>
      <td><c:out value="${emp.job}" /></td>
      <td><c:out value="${emp.salary}" /></td>
    </tr>
  </c:forEach>
</table>

```

Command Class: 25/11/16

The java bean class that would be instantiated by spring mvc to store form data of form page is called command class. The command class properties names and the form page components must match. This avoids programmers burden of calling request.getParameter(-) and writing form data to java bean object.

A command class properties can be of any type hold different types of values given by form page.

25/11/16 AbstractCommandController:

This controller is capable of reading form data of form page and writes that to command class, i.e it takes care of request graphing/wrapping.

→ Use this controller when source page having form submission with no serverside form validation, client side form validation optional, and no form handling (no need of displaying same form page with old value if error occurs) and request wrapping is required, destination page should be static or dynamic page that should come after request processing. (Request processing is there)

⇒ When we develop controller class extending from AbstractCommandController we must implement handle(-,-) method *→ 4 params*

public ModelAndView handle(HttpServletRequest req, HttpServletResponse res, Object cmd, BindException be)

Form submission,

No serverside form validation, optional client side form validations.

No form handling.

Source-page:

Form with submit button or javascript based form submission having optional client side form validation

Since form is there request graphing/wrapping is required.

No form handling

Controller:

Receives form data and stores in command class object (request graphing/wrapping)

Process the request by taking support of service, DAO classes.

Returns ModelAndView object logical view name and model data (result)

Dest Page:

Static or dynamic content that comes after request processing.

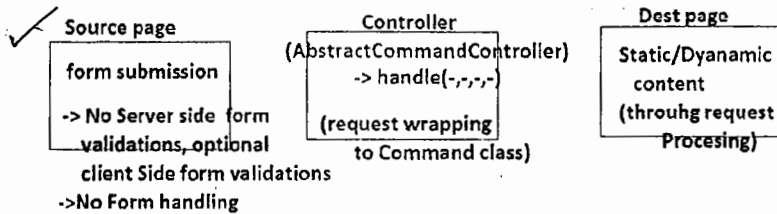
Usecase:

Searching employee details based on given ^{id} details or 1/2/3.....details in a form.

While configuring controller class whose type is AbstractCommandController we must specify property "commandClass" having fully qualified command class name as the value.

25/12/16

Diagram: Abstract Command Controller



Source page:

- > Form with submit button or java script based form submission having optional client side form validations
- > Since form is there request wrapping/graphing is required
- > No Form handling

Controller

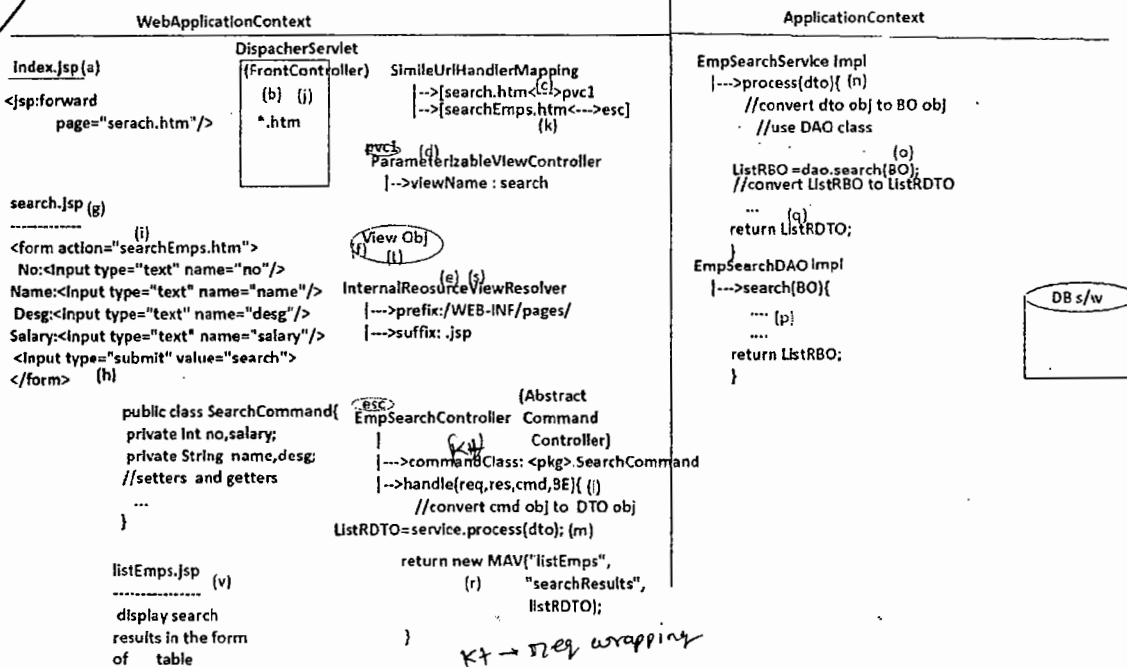
- => Recives form data and stores in command class obj (request wrapping/graphing)
- => Process the request by taking support of Service,DAO classes
- => Returns ModelAndView obj having logical view name and Model data (results)

Dest page

- => Static /dynamic content that comes after request processing.

Usease: Searching employee details based given no details or 1/2/3.. details in a form .

=> While cfg Controller class whose type is AbstractCommandController we must specify property "commandClass" having fully qualified Command class name as the value.



- ▲  MVCEmpSearchApp5ACC
 - ▷  JAX-WS Web Services
 - ▷ Deployment Descriptor: MVCEmpSearchApp
 - ▲  Java Resources
 - ▲  src
 - ▲  com.nt.bo
 - ▷  SearchBO.java
 - ▷  SearchResultBO.java
 - ▲  com.nt.command
 - ▷  SearchCommand.java
 - ▲  com.nt.controller
 - ▷  EmpSearchController.java
 - ▲  com.nt.dao
 - ▷  EmpSearchDAO.java
 - ▷  EmpSearchDAOImpl.java
 - ▲  com.nt.dto
 - ▷  SearchDTO.java
 - ▷  SearchResultDTO.java
 - ▲  com.nt.service
 - ▷  EmpSearchService.java
 - ▷  EmpSearchServiceImpl.java
 - ▷  Libraries
 - ▷  JavaScript Resources
 - ▷  build
 - ▲  WebContent
 - ▷  META-INF
 - ▲  WEB-INF
 - ▷  lib
 - ▷  pages
 - ▷  aop-beans.xml

SearchBO.java

```
package com.nt.bo;
```

```
public class SearchBO {
```

```
    private int no;
```

```
    private String name,desg;
```

```
    private int salary;
```

```
    public SearchBO() {
```

```
    }
```

```
    public SearchBO(int no, String name, String desg, int salary) {
```

```
        this.no = no;
```

```
        this.name = name;
```

```
        this.desg = desg;
```

```
        this.salary = salary;
```

```
    }
```

```
    public int getNo() {
```

```
        return no;
```

```
    }
```

```
    public void setNo(int no) {
```

```
        this.no = no;
```

```
    }
```

```
public String getName() {  
    return name;  
}  
public void setName(String name) {  
    this.name = name;  
}  
public String getDesg() {  
    return desg;  
}  
public void setDesg(String desg) {  
    this.desg = desg;  
}  
public int getSalary() {  
    return salary;  
}  
public void setSalary(int salary) {  
    this.salary = salary;  
}  
}
```

SearchResultBO.java

```
package com.nt.bo;  
public class SearchResultBO {  
    private int no;  
    private String name, desg;  
    private int salary, deptno, mgrno;  
    public SearchResultBO() {  
    }  
    public SearchResultBO(int no, String name, String desg, int salary,  
        int deptno, int mgrno) {  
        this.no = no;  
        this.name = name;  
        this.desg = desg;  
        this.salary = salary;  
        this.deptno = deptno;  
        this.mgrno = mgrno;  
    }  
    public int getNo() {  
        return no;  
    }  
    public void setNo(int no) {  
        this.no = no;  
    }  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

```
}  
    public String getDesg() {  
        return desg;  
    }  
    public void setDesg(String desg) {  
        this.desg = desg;  
    }  
    public int getSalary() {  
        return salary;  
    }  
    public void setSalary(int salary) {  
        this.salary = salary;  
    }  
    public int getDeptno() {  
        return deptno;  
    }  
    public void setDeptno(int deptno) {  
        this.deptno = deptno;  
    }  
  
    public int getMgrno() {  
        return mgrno;  
    }  
    public void setMgrno(int mgrno) {  
        this.mgrno = mgrno;  
    }  
}
```

EmpSearchDAO.java

```
package com.nt.dao;  
import java.util.List;  
import com.nt.bo.SearchBO;  
import com.nt.bo.SearchResultBO;  
public interface EmpSearchDAO {  
    public List<SearchResultBO> search(SearchBO sbo);  
}
```

EmpSearchDAOImpl.java

```
package com.nt.dao;  
import java.sql.ResultSet;  
import java.sql.SQLException;  
import java.util.List;  
import org.springframework.jdbc.core.JdbcTemplate;  
import org.springframework.jdbc.core.RowMapper;  
import com.nt.bo.SearchBO;  
import com.nt.bo.SearchResultBO;  
public class EmpSearchDAOImpl implements EmpSearchDAO {  
    private static final String EMP_SEARCH_QUERY="select empno,ename,job,sal,mgr,deptno  
from emp "
```

+ " where (empno is not null and empno=?

)"

+ "or (ename is not null and ename like ?)"

+ "or (job is not null and job like ?)"

+ "or (sal is not null and sal=?)"

```
private JdbcTemplate jt;
public void setJt(JdbcTemplate jt) {
    this.jt = jt;
}
@Override
public List<SearchResultBO> search(SearchBO sbo) {
    List <SearchResultBO> listRBO=jt.query(EMP_SEARCH_QUERY, new
EmpRowMapper(),sbo.getNo(),
sbo.getName(),
sbo.getDesg(),
sbo.getSalary());
    return listRBO;
}
private static final class EmpRowMapper implements RowMapper<SearchResultBO>{
    @Override
    public SearchResultBO mapRow(ResultSet rs, int pos) throws SQLException {
        //copy ResultSet record into SearchResultBO obj
        SearchResultBO rbo=new SearchResultBO();
        rbo.setNo(rs.getInt(1));
        rbo.setName(rs.getString(2));
        rbo.setDesg(rs.getString(3));
        rbo.setSalary(rs.getInt(4));
        rbo.setMgrno(rs.getInt(5));
        rbo.setDeptno(rs.getInt(6));
        return rbo;
    }
}
```

SearchDTO.java

```
package com.nt.dto;
import java.io.Serializable;
public class SearchDTO implements Serializable {
    private int no;
    private String name,desg;
    private int salary;
    public SearchDTO() {
    }
    public SearchDTO(int no, String name, String desg, int salary) {
        this.no = no;
        this.name = name;
        this.desg = desg;
    }
}
```



```
        this.salary = salary;
    }
    public int getNo() {
        return no;
    }
    public void setNo(int no) {
        this.no = no;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getDesg() {
        return desg;
    }
    public void setDesg(String desg) {
        this.desg = desg;
    }
    public int getSalary() {
        return salary;
    }
    public void setSalary(int salary) {
        this.salary = salary;
    }
}
```

SearchResultDTO.java

```
package com.nt.dto;
import java.io.Serializable;
public class SearchResultDTO implements Serializable {
    private int no;
    private String name,desg;
    private int salary,mgrno,deptno;
    public SearchResultDTO() {
    }
    public SearchResultDTO(int no, String name, String desg, int salary,
        int mgrno, int deptno) {
        this.no = no;
        this.name = name;
        this.desg = desg;
        this.salary = salary;
        this.mgrno = mgrno;
        this.deptno = deptno;
    }
    public int getNo() {
        return no;
    }
}
```

```
}  
public void setNo(int no) {  
    this.no = no;  
}  
public String getName() {  
    return name;  
}  
public void setName(String name) {  
    this.name = name;  
}  
public String getDesg() {  
    return desg;  
}  
public void setDesg(String desg) {  
    this.desg = desg;  
}  
public int getSalary() {  
    return salary;  
}  
public void setSalary(int salary) {  
    this.salary = salary;  
}  
public int getMgrno() {  
    return mgrno;  
}  
public void setMgrno(int mgrno) {  
    this.mgrno = mgrno;  
}  
public int getDeptno() {  
    return deptno;  
}  
public void setDeptno(int deptno) {  
    this.deptno = deptno;  
}  
}
```

EmpSearchService.java

```
package com.nt.service;  
import java.util.List;  
import com.nt.dto.SearchDTO;  
import com.nt.dto.SearchResultDTO;  
public interface EmpSearchService {  
    public List<SearchResultDTO> process(SearchDTO sdto);  
}
```

EmpSearchServiceImpl.java

```
package com.nt.service;  
import java.util.ArrayList;  
import java.util.List;
```

```
import com.nt.bo.SearchBO;
import com.nt.bo.SearchResultBO;
import com.nt.dao.EmpSearchDAO;
import com.nt.dto.SearchDTO;
import com.nt.dto.SearchResultDTO;
public class EmpSearchServiceImpl implements EmpSearchService {
    private EmpSearchDAO dao;
    public void setDao(EmpSearchDAO dao) {
        this.dao = dao;
    }
    @Override
    public List<SearchResultDTO> process(SearchDTO sdto) {
        //Convert SearchDTO obj to SearchBO obj
        SearchBO sbo=new SearchBO();
        sbo.setNo(sdto.getNo());
        sbo.setName(sdto.getName());
        sbo.setDesg(sdto.getDesg());
        sbo.setSalary(sdto.getSalary());
        //Use DAO
        List<SearchResultBO> listRBO=dao.search(sbo);
        //Convert ListRBO to ListRDTO obj
        List<SearchResultDTO> listRDTO=new ArrayList<SearchResultDTO>();
        for(SearchResultBO rbo:listRBO){
            SearchResultDTO rdto=new SearchResultDTO();
            rdto.setNo(rbo.getNo());
            rdto.setName(rbo.getName());
            rdto.setDesg(rbo.getDesg());
            rdto.setSalary(rbo.getSalary());
            rdto.setMgrno(rbo.getMgrno());
            rdto.setDeptno(rbo.getDeptno());
            listRDTO.add(rdto);
        }
        return listRDTO;
    }
}
//process
}
//class
```

SearchCommand.java

```
package com.nt.command;
public class SearchCommand {
    private int no;
    private String name,desg;
    private int salary;
    public SearchCommand() {
        System.out.println("SearchCommand:0-param constructor");
    }

    public int getNo() {
        return no;
    }
}
```

```
}  
public void setNo(int no) {  
    this.no = no;  
}  
public String getName() {  
    return name;  
}  
public void setName(String name) {  
    this.name = name;  
}  
public String getDesg() {  
    return desg;  
}  
public void setDesg(String desg) {  
    this.desg = desg;  
}  
public int getSalary() {  
    return salary;  
}  
public void setSalary(int salary) {  
    this.salary = salary;  
}  
}
```

EmpSearchController.java

```
package com.nt.controller;  
import java.util.List;  
import javax.servlet.http.HttpServletRequest;  
import javax.servlet.http.HttpServletResponse;  
import org.springframework.validation.BindException;  
import org.springframework.web.servlet.ModelAndView;  
import org.springframework.web.servlet.mvc.AbstractCommandController;  
import com.nt.command.SearchCommand;  
import com.nt.dto.SearchDTO;  
import com.nt.dto.SearchResultDTO;  
import com.nt.service.EmpSearchService;  
public class EmpSearchController extends AbstractCommandController {  
    private EmpSearchService service;  
    public void setService(EmpSearchService service) {  
        this.service = service;  
    }  
    @Override  
    protected ModelAndView handle(HttpServletRequest req,  
        HttpServletResponse res, Object cmd, BindException be)  
        throws Exception {  
        //get Command obj  
        SearchCommand command=(SearchCommand)cmd;  
        //convert Command obj into DTO obj
```

```
SearchDTO dto=new SearchDTO();
dto.setNo(command.getNo());
dto.setName(command.getName());
dto.setDesg(command.getDesg());
dto.setSalary(command.getSalary());
//Use Service class
List <SearchResultDTO> listRDTO=service.process(dto);
return new ModelAndView("listEmps","searchResults",listRDTO);
}
}
```

Index.jsp

```
<jsp:forward page="search.htm"/>
```

web.xml

```
<web-app>
<servlet>
  <servlet-name>dispatcher</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  <load-on-startup>2</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>dispatcher</servlet-name>
  <url-pattern>*.htm</url-pattern>
</servlet-mapping>
<listener>
  <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>
</web-app>
```

Service-beans.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="esService" class="com.nt.service.EmpSearchServiceImpl">
    <property name="dao" ref="esDao"/>
  </bean>
</beans>
```

persistence-beans.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
  <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
  <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
  <property name="username" value="scott"/>
  <property name="password" value="tiger"/>
</bean>
<bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
  <property name="dataSource" ref="dataSource"/>
</bean>
<bean id="esDao" class="com.nt.dao.EmpSearchDAOImpl">
  <property name="jt" ref="template"/>
</bean>
</beans>
```

dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
<bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
  <property name="mappings">
    <props>
      <prop key="/search.htm">pvc1</prop>
      <prop key="/searchEmps.htm">esc</prop>
    </props>
  </property>
</bean>
<bean id="esc" class="com.nt.controller.EmpSearchController">
  <property name="commandClass" value="com.nt.command.SearchCommand"/>
  <property name="service" ref="esService"/>
</bean>
<bean id="pvc1" class="
org.springframework.web.servlet.mvc.ParameterizableViewController">
  <property name="viewName" value="search"/>
</bean>
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="prefix" value="/WEB-INF/pages/">
  <property name="suffix" value=".jsp"/>
</bean>
</beans>
```

applicationContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <import resource="persistence-beans.xml"/>
  <import resource="service-beans.xml"/>
  <import resource="aop-beans.xml"/>
</beans>

```

aop-beans.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:aop="http://www.springframework.org/schema/aop"
xmlns:tx="http://www.springframework.org/schema/tx"
xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/tx
http://www.springframework.org/schema/tx/spring-tx.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd">
  <bean id="txMgr"
class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
    <property name="dataSource" ref="dataSource"/>
  </bean>
  <tx:advice id="txAdvice" transaction-manager="txMgr">
    <tx:attributes>
      <tx:method name="process" propagation="REQUIRED"
read-only="true"/>
    </tx:attributes>
  </tx:advice>
  <aop:config>
    <aop:pointcut id="ptc1"
expression="execution(* com.nt.service.EmpSearchService.process(..))"/>
    <aop:advisor advice-ref="txAdvice" pointcut-ref="ptc1"/>
  </aop:config>
</beans>

```

Search.jsp

```

<center><H1>Search Employees</H1> </center>
<form action="searchEmps.htm" method="POST">
  Emp no: <input type="text" name="no"><br>
  Emp Name: <input type="text" name="name"><br>
  Emp Desg: <input type="text" name="desg"><br>
  Emp Salary: <input type="text" name="salary"><br>
  <input type="submit" value="Search"> </form>

```

listEmps.jsp

```

<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<center><h1>Search Results</h1></center>
<c:if test="${not empty searchResults}">
<table>
<tr>
<th>Emp No</th>
<th>Emp Name</th>
<th>Salary</th>
<th>Desg</th>
<th>Dept no</th>
<th>Manger No</th>
</tr>
<c:forEach items="${searchResults}" var="result">
<tr>
<td><c:out value="${result.no}"/></td>
<td><c:out value="${result.name}"/></td>
<td><c:out value="${result.salary}"/></td>
<td><c:out value="${result.desg}"/></td>
<td><c:out value="${result.deptno}"/></td>
<td><c:out value="${result.mgrno}"/></td>
</tr>
</c:forEach>
</table>
</c:if>

```

For understanding Abstract Command Controller
go to Last pages

28/11/16

⇒ The moment we enable the Transaction management the underlying database software will be applied with read committed isolated level. i.e means it allows to read only committed data from database.

For every request given our AbstractCommand controller class, one object of command class will be created for request graphing.

DispatcherServlet always gets our controller class objects from webApplicationContext container by cTransaction management .getBean (-) and calls handleRequest(req, res) on that object. If not available then the super class handleRequest(-, -) executes.

DispatcherServlet doesn't create command class object. The handleRequest(-, -) of superclass to our controller class like AbstractCommandController/SimpleFormController Will create this command class object.

* SimpleFormController: 28/11/16

It is a multipurpose controller which can be used for form submission, form validations, request graphing/wrapping, solving double posting and etc....

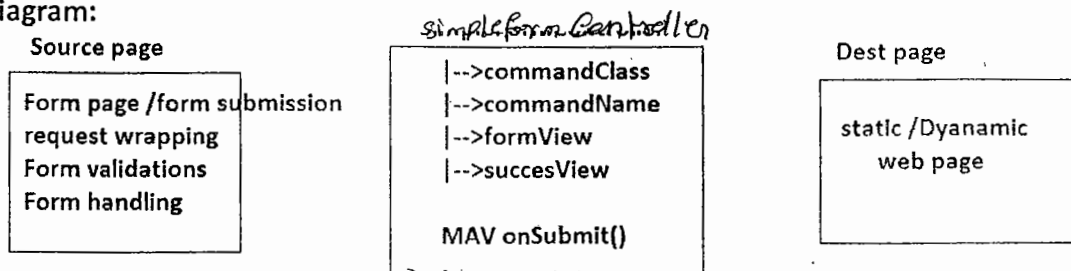
If our source page having ^{with} form page in form validation, form handling and looking to get Dest page either as static/dynamic page having request processing then use SimpleFormController.

→ If you ~~use~~ use this simpleformController, no need to use

parameterisable view Controller for building form page.

28/2/16

Diagram:



→ The SimpleFormController displays form page based on "formView" value if the request is initialphase request (request method: GET) i.e there is no need taking separate Parameterizable ViewController for launching form page.

The same controller becomes ready to process the request if the request isPostBack request (request method: POST)

- 1) DispatcherServlet calls handleRequest(req,res) on our SimpleFormController class obj
- 2) Since not available, super class (SimpleFormController) handleRequest(-,-) method executes
- 3) The handleRequest(-,-) checks whether request is initialphase request or post back request

Initial Phase Request (if req.getMethod.equals("GET")){

- a) Creates an empty Command class obj
- b) Keeps Command class obj either in Session/request scope
- c) gather formView property value and returns it as logical View name to DispatcherServlet
- d) ViewResolver takes lvn (logical view name) and gives View obj to render form page to browser. This form page can display the initial values by collecting from Command obj

Post back Request (if req.getMethod.equals("POST")){

- a) creates /Locates Command class obj and writes form data to it (request wrapping)
- b) creates an empty BindException obj
- c) calls onSubmit() having req,res,command,BinException objs (4-params)
- d) onSubmit() our Controller class will process the request and returns MAV obj having logical view name
- e) ViewResolver takes lvn and gives view obj to render result page.

SimpleFormController

```

|----->CommandClass
|----->CommandName
|----->formView: form page name
|----->successView: Result page name
|----->sessionForm and etc.....
|----->true---->keep cmd obj in session obj.
|----->false---->keep cmd obj in requestScope.
  
```

Methods

```

|----->public MAV onSubmit (Object cmd)
|----->public MAV onSubmit (@Object cmd, BindException be)
|----->public MAV onSubmit (req,res, Object cmd, BindException be)
  
```

z/1/1/ The SimpleFormController displays form page based on "formView" value, if the request is initial phase request (request method: GET) i.e. there is no need to take parameterizableViewController for launching form page.

The same page controller becomes ready to process the request if the request is postback request. (request method: POST)

Points:

- DispatcherServlet calls `handleRequest(req, res)` on our SimpleFormController (Subclass of SimpleformController) `handleRequest(-,-)` checks whether request is initial phase request or postback request.
- `InitialPhaseRequest` (if `req.getMethod().equals("GET")`)
- Creates an empty command class object.
- Keeps command class object either in session/request scope
- Gather `formView` property value and return if a logical view name to ViewResolver. ViewResolver takes logical view name (env) and gives View object to render form page to browser, this form page can display the initial value by collecting from command object. (Springframeworkcontroller difference by checking its methodology.)

Postback Request (if `req.getMethod().equals("POST")`):

Locate the command class object from session scope

- Creates an empty bind exception object.
- Calls `onSubmit(-)` having req, res, command, BindException object. In `onSubmit(-)` our controller will process the request and returns MAV object having logical view name. ViewResolver takes logical View name(lvn) and gives view object to render formpage to browser.

Note:

- While working with SimpleformController our formpage request method must be "POST".
- In form handling we need to display the same page with old values when form validation error are there. In this process we need to write lots of additional code for this if we work with traditional HTML pages.
- If we use spring supplies tag library tags then automatically it take care of this process which is nothing but reading old values from command object and displaying in the form page.
- Spring gives 2 jsp tag libraries to develop that jsps of web application

Generic jsp tag library

```
|----->general tags
|----->tld File=spring.tld
|----->taglib uri=http://www.springframework.org/tags
```

Form jsp tag library

```
|-----> form tags
|----->tld File: Spring-form.tld
|----->taglib uri=http://www.springframewrok.org/tags/form
```

Diagram:

Insert Usecase of SimpleFormController

Index.jsp
(GET)
<jsp:forward
page="student.htm"/>
(h)
student.jsp
<%@taglib uri="..." prefix="form"%>
<form:form method="POST"
commandName="stCmd">
No: <form:input path="sno">

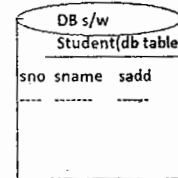
Name: <form:input path="sname">

Sadd: <form:input path="sadd">

<input type="submit" value="register">
</form:form>
StudentCmd{
private int sno; (i)
private String sname, sadd;
//setters and getters
....
Result.jsp
....
}

DispatcherServlet
(b)
*.htm
(j)
SimpleUrlHandlerMapping
|--->{/student.htm<-->sic
(c) (k)
StudentInsertController (SimpleFormController)
|--->formView:student (e)
|--->successView:Result (f)
|--->commandClass: <pkg>.StudentCmd (l)
|--->commandName: stCmd (d)
|--->public MAV onSubmit(req,res, cmd, BE){
//Convert Cmd obj to DTO obj
....
//use service class (m)
(t) String result=service.register(dto);
return new MAV["Result", "resMsg", result];
(u)
InternalResourceViewResolver
|--->prefix: /WEB-INF/pages/
|--->suffix: .jsp
(f) (v)

Insert
public StudentServiceImpl{
(n)
public String register(dto){
//Convert DTO Into BO
(o)
(c) int cnt=dao.Insert{bo};
if(cnt==0)
return "Registration failed"; (s)
else
return "Registration succeeded";
}
}
Insert
StudentDAOImpl{
(p)
public int Insert(BO){
....
} return 0/1 (q)
}



(WebApplicationContext container)

a-->h: InitialPhase request
i-->x: Post back request

(ApplicationContext container)

- 29/12/16
completed app
- MVCStudentInsertApp6SFC
 - JAX-WS Web Services
 - Deployment Descriptor: MVCStudentInsertAp
 - Java Resources
 - src
 - com.nt.bo
 - StudentBO.java
 - com.nt.command
 - StudentCommand.java
 - com.nt.controller
 - StudentInsertController.java
 - com.nt.dao
 - StudentInsertDAO.java
 - StudentInsertDAOImpl.java
 - com.nt.dto
 - StudentDTO.java
 - com.nt.service
 - StudentInsertService.java
 - StudentInsertServiceImpl.java
 - Libraries
 - JavaScript Resources
 - build
 - WebContent
 - META-INF
 - WEB-INF
 - lib
 - pages
 - applicationContext.xml
 - dispatcher-servlet.xml
 - persistence-beans.xml

StudentBO.java

```
package com.nt.bo;
public class StudentBO {
    private int sno;
    private String sname,sadd;
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
}
```

StudentInsertDAO.java

```
package com.nt.dao;
import com.nt.bo.StudentBO;
public interface StudentInsertDAO {
    public int insert(StudentBO sbo);
}
```

StudentInsertDAOImpl.java

```
package com.nt.dao;
import org.springframework.jdbc.core.JdbcTemplate;
import com.nt.bo.StudentBO;
public class StudentInsertDAOImpl implements StudentInsertDAO {
    private static final String INSERT_STUDENT_QRY="INSERT INTO STUDENT values(?,?,?)";
    private JdbcTemplate jt;
    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }
    @Override
    public int insert(StudentBO sbo) {
        int cnt=jt.updqt(INSERT_STUDENT_QRY,sbo.getSno(),
        sbo.getSname(),sbo.getSadd());
        return cnt;
    }
}
```

Handwritten notes:
- A box is drawn around the `setJt` method.
- An arrow points from the text "update method" to the `updqt` method call in the `insert` method.
- The `sbo.getSname()` is crossed out and replaced with `sbo.getSname()`.

29/11/16 **StudentDTO.java**

```
package com.nt.dto;
import java.io.Serializable;
public class StudentDTO implements Serializable {
    private int sno;
    private String sname,sadd;
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
}
```

StudentInsertService.java

```
package com.nt.service;
import com.nt.dto.StudentDTO;
public interface StudentInsertService {
    public String register(StudentDTO dto);
} StudentInsertServiceImpl :-
package com.nt.service;
import com.nt.bo.StudentBO;
import com.nt.dao.StudentInsertDAO;
import com.nt.dto.StudentDTO;
public class StudentInsertServiceImpl implements StudentInsertService{
    private StudentInsertDAO dao;
    public void setDao(StudentInsertDAO dao) {
        this.dao = dao;
    }
    @Override
    public String register(StudentDTO dto) {
        // Convert DTO obj into BO obj
        StudentBO sbo=new StudentBO();
        sbo.setSno(dto.getSno());
        sbo.setSname(dto.getSname());
        sbo.setSadd(dto.getSadd());
    }
}
```

Convert DTO obj into BO obj

29/11/16

```
//Use DAO
int cnt=dao.insert(sbo);
if(cnt==0)
    return " Student Registration Failed";
else
    return " Student Registration Succeeded with no"+sbo.getSno();
}
}
```

StudentInsertController.java

```
package com.nt.controller;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.validation.BindException;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.SimpleFormController;
import com.nt.command.StudentCommand;
import com.nt.dto.StudentDTO;
import com.nt.service.StudentInsertService;
public class StudentInsertController extends SimpleFormController {
    private StudentInsertService service;
    public void setService(StudentInsertService service) {
        this.service = service;
    }
    @Override
    protected ModelAndView onSubmit(HttpServletRequest request,
        HttpServletResponse response, Object command, BindException errors)
        throws Exception {
        //type cast cmd obj
        StudentCommand cmd=(StudentCommand)command;
        //Convert Command obj to DTO obj
        StudentDTO dto=new StudentDTO();
        dto.setSno(cmd.getSno());
        dto.setSname(cmd.getSname());
        dto.setSadd(cmd.getSadd());
        //Use Service class
        String result=service.register(dto);
        //return ModelAndView obj
        return new ModelAndView(this.getSuccessView(),"resMsg",result);
    }
}
```

StudentCommand.java

```
package com.nt.command;
public class StudentCommand {
    private int sno=101;
    private String sname="raja",sadd="hyd";
    public int getSno() {
```

```
29/16
    return sno;
}
public void setSno(int sno) {
    this.sno = sno;
}
public String getSname() {
    return sname;
}
public void setSname(String sname) {
    this.sname = sname;
}
public String getSadd() {
    return sadd;
}
public void setSadd(String sadd) {
    this.sadd = sadd;
}
}
```

Index.jsp

```
<jsp:forward page="student.htm"/>
```

web.xml

```
<web-app>
<servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>2</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>*.htm</url-pattern>
</servlet-mapping>

<listener>
    <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
<context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>
</web-app>
```

dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
    <bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
```

29/2/16

```
<property name="mappings">
```

```
<props>
```

```
<prop key="/student.htm">sic</prop>
```

```
</props>
```

```
</property>
```

```
</bean>
```

```
<bean id="sic" class="com.nt.controller.StudentInsertController">
```

```
<property name="formView" value="student"/>
```

```
<property name="successView" value="Result"/>
```

```
<property name="commandClass" value="com.nt.command.StudentCommand"/>
```

```
<property name="commandName" value="stCmd"/>
```

```
<property name="sessionForm" value="true"/>
```

```
<property name="service" ref="stService"/>
```

```
</bean>
```

```
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
```

```
<property name="prefix" value="/WEB-INF/pages/">
```

```
<property name="suffix" value=".jsp"/>
```

```
</bean>
```

```
</beans>
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xmlns:context="http://www.springframework.org/schema/context"
```

```
xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```
http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<import resource="persistence-beans.xml"/>
```

```
<import resource="service-beans.xml"/>
```

```
</beans>
```

service-beans.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xmlns:context="http://www.springframework.org/schema/context"
```

```
xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```
http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<bean id="stService" class="com.nt.service.StudentInsertServiceImpl">
```

```
<property name="dao" ref="stDao"/>
```

```
</bean>
```

```
</beans>
```

persistence-beans.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xmlns:context="http://www.springframework.org/schema/context"
```

```
xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```
http://www.springframework.org/schema/beans/spring-beans.xsd">
```

here you have to parameter view

Result page that has to generate controller for home hyper link

displayed after proceeding from in Result page,

gathered req

form page name

It is optional

command class name

Command class obj attribute name, which will be used form page.

Inherited property

new Resolver


```

<bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
  <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
  <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
  <property name="username" value="scott"/>
  <property name="password" value="tiger"/>
</bean>
<bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
  <property name="dataSource" ref="dataSource"/>
</bean>
<bean id="stDao" class="com.nt.dao.StudentInsertDAOImpl">
  <property name="jt" ref="template"/>
</bean>
</beans>

```

student.jsp

tag <%@taglib uri="http://www.springframework.org/tags/form" prefix="form" %>

<h1><center> Register Student Details</center></h1>

form <form:form method="POST" commandName="stCmd">

Number: <form:input path="sno"/>

Name : <form:input path="sname"/>

Address : <form:input path="sadd"/>

<input type="submit" value="register"/>

</form:form>

Result.jsp

<h1><center> Result Page</center></h1>

{resMsg}

Do observe that there is no action attribute but still form page submitted req goes to our simple form controller class (StudentInsertController) becaz this form page is launched through same controller using same url. So the same url will be utilized when the form is submitted

Form validation in SimpleformController:

Verifying the format and pattern of form data is called form validation. Spring MVC allows to write form validation logic in separate class so that it can be linked with multiple controller classes having reusability. For server side validation, spring mvc gives good support. For client side validation,

Sample Code: we need to write plain java script code

① Develop a validator class implementing Validation(i) and implements supports(-) and validate (-,) methods

② Develop properties file having validation Error messages.

③ Configure the properties file in "dispatcher-servlet.xml".

④ Configure validator class in dispatcher-Servlet.xml and link that class with controller class using the property "validator".

⑤ Place <form:errors path="*" /> tag in student.jsp to display the form validation error messages.

The SimpleController can handle 3 types of validation.

① Form Validations (like number, name can't be blank)

② typeMismatch errors (like storing string value to Sno properties)

③ Application logic errors (like not allowing "delhi" as address for student). Instead of hardcoding error msgs directly in validator class and controller class it is recommended to place in properties file having our choice keys and values, but typeMismatch errors the key should be "typeMismatch.<propertyname>"

④ What is the use of implementing supports (Class class) method in validator class of simpleformcontroller?

This support ~~(-)~~ method holds the configuration "CommandClass" in class parameter and we can check whether correct command class is configured or not by using class. isAssignable ~~form~~ (-) method, if configured correctly then it returns true, so validate (-, -) method executes, if not configured correctly then it returns false, so validate (-, -) will not execute. Due to this we can avoid mis configuration in spring bean configure file.

Org.springframework.validation.Errors(i)

Extends ~~implements~~ extends

Org.springframework.validation.BindingResult(i)

Extends ~~implements~~ implements

Org.springframework.validation.BindException(~~@C~~)

30/11/16
MVCStudentInsertApp6SFCValidations

- ▶ JAX-WS Web Services
- ▶ Deployment Descriptor: MVCStudentInsertApp6S
- ▶ Java Resources
 - ▶ src
 - ▶ com.nt.bo
 - ▶ com.nt.command
 - ▶ com.nt.controller
 - ▶ com.nt.dao
 - ▶ com.nt.dto
 - ▶ com.nt.service
 - ▶ com.nt.validations
 - ▶ StudentInsertValidator.java
 - ▶ StCmdValidationErrors.properties
 - ▶ Libraries
 - ▶ JavaScript Resources
 - ▶ build
 - ▶ WebContent
 - ▶ META-INF
 - ▶ WEB-INF
 - ▶ lib
 - ▶ pages
 - ▶ applicationContext.xml
 - ▶ dispatcher-servlet.xml
 - ▶ persistence-beans.xml
 - ▶ service-beans.xml
 - ▶ web.xml
 - ▶ index.jsp

StudentBO.java

```
package com.nt.bo;
public class StudentBO {
    private int sno;
    private String sname,sadd;
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
}
```

```
public String getSname() {  
    return sname;  
}  
public void setName(String sname) {  
    this.sname = sname;  
}  
public String getSadd() {  
    return sadd;  
}  
public void setSadd(String sadd) {  
    this.sadd = sadd;  
}  
}
```

StudentInsertDAO.java

```
package com.nt.dao;  
import com.nt.bo.StudentBO;  
public interface StudentInsertDAO {  
    public int insert(StudentBO sbo);  
}
```

StudentInsertDAOImpl.java

```
package com.nt.dao;  
import org.springframework.jdbc.core.JdbcTemplate;  
import com.nt.bo.StudentBO;  
public class StudentInsertDAOImpl implements StudentInsertDAO {  
    private static final String INSERT_STUDENT_QRY="INSERT INTO STUDENT values(?,?,?)";  
    private JdbcTemplate jt;  
    public void setJt(JdbcTemplate jt) {  
        this.jt = jt;  
    }  
    @Override  
    public int insert(StudentBO sbo) {  
        int cnt=jt.update(INSERT_STUDENT_QRY,sbo.getSno(),  
                                                                    sbo.getSname(),  
                                                                    sbo.getSadd());  
        return cnt;  
    }  
}
```

StudentInsertService.java

```
package com.nt.service;  
import com.nt.dto.StudentDTO;  
public interface StudentInsertService {  
    public String register(StudentDTO dto);  
}
```

StudentDTO.java

package com.nt.dto;
import java.io.Serializable;
public class StudentDTO implements Serializable {
 private int sno;
 private String sname,sadd;
 public int getSno() {
 return sno;
 }
 public void setSno(int sno) {
 this.sno = sno;
 }
 public String getSname() {
 return sname;
 }
 public void setSname(String sname) {
 this.sname = sname;
 }
 public String getSadd() {
 return sadd;
 }
 public void setSadd(String sadd) {
 this.sadd = sadd;
 }
}

StudentInsertServiceImpl.java

package com.nt.service;
import com.nt.bo.StudentBO;
import com.nt.dao.StudentInsertDAO;
import com.nt.dto.StudentDTO;
public class StudentInsertServiceImpl implements StudentInsertService{
 private StudentInsertDAO dao;
 public void setDao(StudentInsertDAO dao) {
 this.dao = dao;
 }
 @Override
 public String register(StudentDTO dto) {
 // Convert DTO obj BO obj
 StudentBO sbo=new StudentBO();
 sbo.setSno(dto.getSno());
 sbo.setSname(dto.getSname());
 sbo.setSadd(dto.getSadd());
 //Use DAO
 int cnt=dao.insert(sbo);
 if(cnt==0)
 return "Student Registration Failed";
 else

```

    } } 30111 return "Student Registration Succeeded with no"+sbo.getSno();

```

StudentCommand.java

```

package com.nt.command;
public class StudentCommand {
    private int sno=101;
    private String sname="raja",sadd="hyd";
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
}

```

StCmdValidationErrors.java *properties*

~~typeMismatch~~.sno=student no can not empty or can not be String
 sno.invalid=Student no must be positive value
 sname.required=Student name is mandatory
 sadd.required=Student address is mandatory
 sadd.minlength= Addr's can not be less than 3 chars
 sadd.city=Address can be not "delhi".

Note:

StudentInsertValidator.java

```

package com.nt.validator;
import org.springframework.validation.Errors;
import org.springframework.validation.Validator;
import com.nt.command.StudentCommand;
public class StudentInsertValidator implements Validator {

    @Override
    public boolean supports(Class<?> clazz) {

        return clazz.isAssignableFrom(StudentCommand.class);
    }
}

```

30/1/16

```
@Override
public void validate(Object command, Errors errors) {
    //type casting
    StudentCommand cmd=(StudentCommand)command;
    //form validations
    if(cmd.getSno()<=0)
        errors.rejectValue("sno", "sno.invalid");
    if(cmd.getSname().equals("") || cmd.getSname()==null)
        errors.rejectValue("sname", "sname.required");
    if(cmd.getSadd().equals("") || cmd.getSadd()==null)
        errors.rejectValue("sadd", "sadd.required");
    else{
        if(cmd.getSadd().length()<3){
            errors.rejectValue("sadd", "sadd.minlength");
        }
    }
}
```

StudentInsertController.java

```
package com.nt.controller;
import javax.servlet.RequestDispatcher;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.validation.BindException;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.SimpleFormController;
import com.nt.command.StudentCommand;
import com.nt.dto.StudentDTO;
import com.nt.service.StudentInsertService;
public class StudentInsertController extends SimpleFormController {
    private StudentInsertService service;
    public void setService(StudentInsertService service) {
        this.service = service;
    }
}
```

@Override

```
protected ModelAndView handleInvalidSubmit(HttpServletRequest req,
    HttpServletResponse res) throws Exception {
    return new ModelAndView("dblpst");
}
```

@Override

```
protected ModelAndView onSubmit(HttpServletRequest request,
    HttpServletResponse response, Object command, BindException errors)
    throws Exception {
    //type cast cmd obj
    StudentCommand cmd=(StudentCommand)command;
    // B.logic related validation
```

30/11/16

```
if(cmd.getSadd().equals("delhi")){
    errors.rejectValue("sadd", "sadd.city");
    return showForm(request, response, errors);
}
```

//Convert Command obj to DTO obj

```
StudentDTO dto=new StudentDTO();
dto.setSno(cmd.getSno());
dto.setSname(cmd.getSname());
dto.setSadd(cmd.getSadd());
//Use Service class
String result=service.register(dto);
//return ModelAndView obj
return new ModelAndView(this.getSuccessView(), "resMsg", result);
```

}

@Override

```
protected Object formBackingObject(HttpServletRequest request)
    throws Exception {
```

```
    int no=Integer.parseInt(request.getParameter("id"));
    StudentCommand cmd=new StudentCommand();
    cmd.setSno(no);
    cmd.setSname("raja111");
    cmd.setSadd("hyd111");
    return cmd;
```

}

}

StudentCommand.java

```
package com.nt.command;
public class StudentCommand {
    private int sno=101;
    private String sname="raja",sadd="hyd";
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
```

30/11/16
}

```
public void setSadd(String sadd) {  
    this.sadd = sadd;  
}
```

Index.jsp

```
<jsp:forward page="student.htm"/>
```

web.xml

```
<web-app>  
<servlet>  
    <servlet-name>dispatcher</servlet-name>  
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>  
    <load-on-startup>2</load-on-startup>  
</servlet>  
<servlet-mapping>  
    <servlet-name>dispatcher</servlet-name>  
    <url-pattern>*.htm</url-pattern>  
</servlet-mapping>  
<listener>  
    <listener-class>org.springframework.web.context.ContextLoaderListener  
</listener-class>  
</listener>  
<context-param>  
    <param-name>contextConfigLocation</param-name>  
    <param-value>/WEB-INF/applicationContext.xml</param-value>  
</context-param>  
</web-app>
```

dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"  
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
xsi:schemaLocation="http://www.springframework.org/schema/beans  
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">  
    <bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">  
        <property name="mappings">  
            <props>  
                <prop key="/student.htm">sic</prop>  
            </props>  
        </property>  
    </bean>  
    <bean id="messageSource"  
        class="org.springframework.context.support.ResourceBundleMessageSource">  
        <property name="basename" value="com/nt/validator/StCmdValidationErrors"/>  
    </bean>  
    <bean id="stValidator" class="com.nt.validator.StudentInsertValidator"/>  
    <bean id="sic" class="com.nt.controller.StudentInsertController">  
        <property name="formView" value="student"/>  
        <property name="validator" ref="stValidator"/>  
    </bean>
```


30/1/16

```
<property name="successView" value="Result"/>
<property name="commandClass" value="com.nt.command.StudentCommand"/>
<property name="commandName" value="stCmd"/>
<property name="sessionForm" value="true"/>
<property name="service" ref="stService"/>
</bean>
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="prefix" value="/WEB-INF/pages/" />
  <property name="suffix" value=".jsp" />
</bean>
</beans>
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <import resource="persistence-beans.xml"/>
  <import resource="service-beans.xml"/>
</beans>
```

service-beans.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="stService" class="com.nt.service.StudentInsertServiceImpl">
    <property name="dao" ref="stDao"/>
  </bean>
</beans>
```

persistence-beans.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
    <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
    <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
    <property name="username" value="scott"/>
    <property name="password" value="tiger"/>
  </bean>
  <bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
    <property name="dataSource" ref="dataSource"/>
  </bean>
```

30/06
`<bean id="stDao" class="com.nt.dao.StudentInsertDAOImpl">
 <property name="jt" ref="template"/>
</bean>
</beans>`

student.jsp

```
<%@taglib uri="http://www.springframework.org/tags/form" prefix="form" %>
  <h1><center>Register Student Details</center></h1>
  <form:form method="POST" commandName="stCmd">
    <!-- <font color='red'> -->
    <%-- <form:errors path="*" /> --%>
    <!-- </font> -->
    <!-- <br> -->
    Number: <form:input path="sno" /> <form:errors path="sno" /> <br>
    Name : <form:input path="sname" /> <form:errors path="sname" /> <br>
    Address : <form:input path="sadd" /> <form:errors path="sadd" /> <br>
    <input type="submit" value="register" />
  </form:form>
```

Result.jsp

```
<h1><center>Result Page</center></h1>
${resMsg}
```

dbpost.jsp

```
<h1>Warning!!!!</h1>
```

01/11/16 <p style="color:red"> double posting is not allowed </p>

Internal flow of form validation:

Form page Submit the form (post back request)----->Dispatcher servlet traps request and uses handler mapping to map the request to our SimpleFormController and calls handleRequest (-,-)----->this method checks whether request is "initialphase" or "postback" request. Since the request is post back request.

1) Creates or locates command class object and writes form data to it.
(request wrapping).

2) Creates an empty BindException object.

3) Checks "validation" is configured or not and it notices configured in controller class.

4) get validator object and calls support (-) with class object that points to command class if this method returns true then validate (-,-) will be called having ErrorsObject (BindException), command object otherwise validate (-,-) will not be called.

5) validate (-,-) perform form validations and stores validationErrors in ErrorsObject.

6) handleRequest (-,-) checks the size of ErrorsObject to know whether validation Errors are there or not

7) If Errors obj size=0 (Then no validation errors)
Then {

1) Calls onSubmit (-,-,-) having req,res,command and BindException objs.

2) onSubmit (-,-,-) will process the request
}

→ If (Error obj size>0) [form validation errors]

{
1) keep Errorsobjs in request scope

II (9) Returns MAV obj having formview property value as lvn, commandClassName, CommandClass obj as model attribute.

III) Dispatcher servlet keeps the received command class obj in req scope and use ViewResolver to get view obj having form page as physical view Name.

IV) Form page will be displayed. In that <form:input> tags collect command object data to show old values in comps and <form:errors> uses "ErrorObj" data to show validation error message.

31/11/16 What is double posting problem and duplicate form submission problem and how can we solve that problem? 31/11/16 absent

Pressing refresh button on the result page of form submission submits the form one more time, clicking on submit button for multiple times continuously. This leads duplicate form submission and gives problem

like duplicate registrations, deducting amount from credit card, debit card accounts for multiple times and etc..... This is also called as "doublePosting" problem.

We can overcome this problem with tokens support, when form page is submitted one token will be created and will be maintained as session attribute. Once onSubmit(-,-) method process the request it removes the token from session. If any request comes without token because of double posting problem error will be raised.

In spring MVC all these activities are handle internally if any request comes without token, the handleInvalidSubmit (-,-) method will be called. So we can place logic to render error page in that method. refer 298 → db.post.jsp and 294 (Handle Invalid submit as the ex: to consider)

Importance of formBackingObject () 31/11/16 absent

Generally when handleRequest method is processing initial phase request it internally calls formBackingObject() to create command class object. The predefined method always creates empty command object. If you want to create command object with dynamic data according to application requirement then we need to override this method in our controller class that extends from SimpleFormController.

Use case:

When we clicked on student no hyperlink from list of student details, if we want to display edit.jsp having that student details initial values. For example app, on formBackingObject and various

31/11/16 **referenceData (HttpServletRequest)** other options or simple form controller, app given 307-320

While displaying form page as the response of initial phase request or as the response of post back request when form validation errors are there, if you want to display additional data apart from the data that is there in command object, we can override referenceData(-) in our SimpleFormController class to construct method that data dynamically. This method returns map object having data in the form of values <List> with keys.

→ DispatcherServlet keeps this map obj in request scope. In order to display this map object data in certain component we need key of map obj in the "items" attribute of <form:options>, <form:checkboxes>

3/2/16
14/2/16

MVCStudentApp7SPRINGFRAMEWORKC (reference-data)

- ▶ JAX-WS Web Services
- ▶ Deployment Descriptor: MVCStudentApp7SPRING
- ▶ Java Resources
 - ▶ src
 - ▶ com.nt.command
 - ▶ RegistrationCommand.java
 - ▶ com.nt.controller
 - ▶ StudentRegistrationController.java
 - ▶ com.nt.validations
 - ▶ StudentInsertValidator.java
 - ▶ StCmdValidationErrors.properties
 - ▶ Libraries
 - ▶ JavaScript Resources
 - ▶ build
 - ▶ WebContent
 - ▶ META-INF
 - ▶ WEB-INF
 - ▶ lib
 - ▶ pages
 - ▶ applicationContext.xml
 - ▶ dispatcher-servlet.xml
 - ▶ persistence-beans.xml
 - ▶ service-beans.xml
 - ▶ web.xml
 - ▶ index.jsp

add spring 3.2.x jars only

<form:checkboxes> and etc.... tags

StudentRegistrationController.java

```
package com.nt.controller;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.validation.BindException;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.SimpleFormController;
public class StudentRegistrationController extends SimpleFormController {
    @Override
    protected Map referenceData(HttpServletRequest request) throws Exception {
        System.out.println("SRC: referenceData(-)");
        Map<String, List> map=new HashMap<String, List>();
        List<String> courses=new ArrayList();
        courses.add("spring"); courses.add("hibernate");
        courses.add("adv.java"); courses.add("webservices");
        map.put("courseList", courses);
        return map;
    }
    @Override
```

```
protected ModelAndView onSubmit(HttpServletRequest request,
                                HttpServletResponse response, Object command, BindException errors)
    throws Exception {
    return new ModelAndView("register_result");
}
```

RegistrationCommand.java

```
package com.nt.command;
public class RegistrationCommand {
    private int sno;
    private String sname;
    private String course;
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getCourse() {
        return course;
    }
    public void setCourse(String course) {
        this.course = course;
    }
}
```

StCmdValidationErrors.java

typeMismatch.sno=student no can not empty or can not be String
sno.invalid=Student no must be positive value
sname.required=Student name is mandatory

StudentInsertValidator.java

```
package com.nt.validator;
import org.springframework.validation.Errors;
import org.springframework.validation.Validator;
import com.nt.command.RegistrationCommand;
public class StudentInsertValidator implements Validator {
    @Override
    public boolean supports(Class<?> clazz) {
        return clazz.isAssignableFrom(RegistrationCommand.class);
    }
}
```

@Override *let us assume validation errors are there*

```
public void validate(Object command, Errors errors) {  
    //type casting  
    RegistrationCommand cmd=(RegistrationCommand)command;  
    //form validations  
    if(cmd.getSno()<=0)  
        errors.rejectValue("sno", "sno.invalid");  
    if(cmd.getSname().equals("") || cmd.getSname()==null)  
        errors.rejectValue("sname", "sname.required");  
}
```

Index.jsp

```
<jsp:forward page="register.htm"/>
```

web.xml

```
<web-app>  
<servlet>  
    <servlet-name>dispatcher</servlet-name>  
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>  
    <load-on-startup>2</load-on-startup>  
</servlet>  
<servlet-mapping>  
    <servlet-name>dispatcher</servlet-name>  
    <url-pattern>*.htm</url-pattern>  
</servlet-mapping>  
<listener>  
    <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>  
</listener>  
<context-param>  
    <param-name>contextConfigLocation</param-name>  
    <param-value>/WEB-INF/applicationContext.xml</param-value>  
</context-param>  
</web-app>
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xmlns:context="http://www.springframework.org/schema/context"  
    xsi:schemaLocation="http://www.springframework.org/schema/beans  
        http://www.springframework.org/schema/beans/spring-beans.xsd">  
</beans>
```

Dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="http://www.springframework.org/schema/beans  
        http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">  
    <bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">  
        <property name="mappings">
```

```

<props>
  <prop key="/register.htm">src</prop>
</props>
</property>
</bean>
<bean id="messageSource"
  class="org.springframework.context.support.ResourceBundleMessageSource">
  <property name="basename" value="com/nt/validator/StCmdValidationErrors"/>
</bean>
<bean id="stValidator" class="com.nt.validator.StudentInsertValidator"/>
<bean id="src" class="com.nt.controller.StudentRegistrationController">
  <property name="formView" value="register"/>
  <property name="validator" ref="stValidator"/>
  <property name="successView" value="register_result"/>
  <property name="commandClass" value="com.nt.command.RegistrationCommand"/>
  <property name="commandName" value="regCmd"/>
  <property name="sessionForm" value="true"/>
</bean>
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="prefix" value="/WEB-INF/pages/">
  <property name="suffix" value=".jsp"/>
</bean>
</beans>

```

Register
Result.jsp

```

<%@taglib uri="http://www.springframework.org/tags/form" prefix="form" %>
<form:form method="POST" commandName="regCmd">
<form:errors path="*" />
<br>

```

No : <form:input path="sno"/>

Name : <form:input path="sname"/>

Courses :

```

  <form:select path="course">
    <form:options items="${courseList}" /> <br>
  </form:select>
  <input type="submit" value="register" />
</form:form>

```

→ displays map obj data given by reference data method

Register result.jsp

<center> Registration Success</center>

Use case: Displaying roles in form page (Select box)

For initial phase request and it in post back request when it generates form validation errors.

12/16 MVCDataBinderSFCApp10

- ▶ JAX-WS Web Services
- ▶ Deployment Descriptor: MVCDataBinderSFCApp1
- ▲ Java Resources
 - ▲ src
 - ▲ com.nt.command
 - ▶ StudentCommand.java
 - ▲ com.nt.controller
 - ▶ DataBindController.java
 - ▶ Libraries
 - ▶ JavaScript Resources
 - ▶ build
 - ▲ WebContent
 - ▶ META-INF
 - ▲ WEB-INF
 - ▶ lib
 - ▲ pages
 - ▶ student_register.jsp
 - ▶ student_result.jsp
 - ▶ applicationContext.xml
 - ▶ dispatcher-servlet.xml
 - ▶ web.xml
- ▶ index.jsp add 3.2.x spring jars

StudentCommand.java

```
package com.nt.command;
import java.util.Date;
public class StudentCommand {
    private String name;
    private Date dob, doj, dom;
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public Date getDob() {
        return dob;
    }
    public void setDob(Date dob) {
        this.dob = dob;
    }
    public Date getDoj() {
        return doj;
    }
    public void setDoj(Date doj) {
        this.doj = doj;
    }
    public Date getDom() {
        return dom;
    }
}
```



```
public void setDom(Date dom) {
    this.dom = dom;
}
@Override
public String toString() {
    return "StudentCommand [name=" + name + ", dob=" + dob + ", doj=" + doj
        + ", dom=" + dom + "]";
}
}
```

DataBindController.java

```
package com.nt.controller;
import java.text.SimpleDateFormat;
import java.util.Date;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.beans.propertyeditors.CustomDateEditor;
import org.springframework.validation.BindException;
import org.springframework.web.bind.ServletRequestDataBinder;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.SimpleFormController;
import com.nt.command.StudentCommand;
public class DataBindController extends SimpleFormController {
    @Override
    protected void initBinder(HttpServletRequest request,
        ServletRequestDataBinder binder) throws Exception {
        SimpleDateFormat sdf=new SimpleDateFormat("dd-MM-yyyy");
        binder.registerCustomEditor(Date.class,
            new CustomDateEditor(sdf,true));
        ↳ predefined classes
    }
    @Override
    protected ModelAndView onSubmit(HttpServletRequest request,
        HttpServletResponse response, Object command, BindException errors)
        throws Exception {
        StudentCommand cmd=(StudentCommand)command;
        return new ModelAndView("student_result","cmdData",cmd);
    }
}
```

Index.jsp

```
<jsp:forward page="student.htm"/>
```

web.xml

```
<web-app>
<servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>2</load-on-startup>
</servlet>
```

```
<servlet-mapping>
  <servlet-name>dispatcher</servlet-name>
  <url-pattern>*.htm</url-pattern>
</servlet-mapping>
<listener>
  <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>
</web-app>
```

Dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
  <bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
    <property name="mappings">
      <props>
        <prop key="/student.htm">dbc</prop>
      </props>
    </property>
  </bean>
  <bean id="dbc" class="com.nt.controller.DataBindController">
    <property name="formView" value="student_register"/>
    <property name="successView" value="student_result"/>
    <property name="commandClass" value="com.nt.command.StudentCommand"/>
    <property name="commandName" value="regCmd"/>
    <property name="sessionForm" value="true"/>
  </bean>
  <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="prefix" value="/WEB-INF/pages/">
    <property name="suffix" value=".jsp"/>
  </bean>
</beans>
```

appliactionContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
</beans>
```

Student_register.jsp

```

<%@taglib uri="http://www.springframework.org/tags/form" prefix="form"%>
<form:form method="POST" commandName="regCmd">
  <form:errors path="*" />
  Name : <form:input path="name" /> <br>
  DOB : <form:input path="dob" /> <br>
  DOJ : <form:input path="doj" /> <br>
  DOM : <form:input path="dom" /> <br>
  <input type="submit" value="register" />
</form:form>

```

Student_result.jspInitBinder method() :-

The given details are :

`{cmdData} InitBinder()` :-

property editors of spring are very useful to convert given values to required type properties of Bean class or command class.

Spring gives built-in property editors and allows us to develop custom property editors.

→ Custom DateEditor is convert given spring data values into java.util.Date class object by specifying date format.

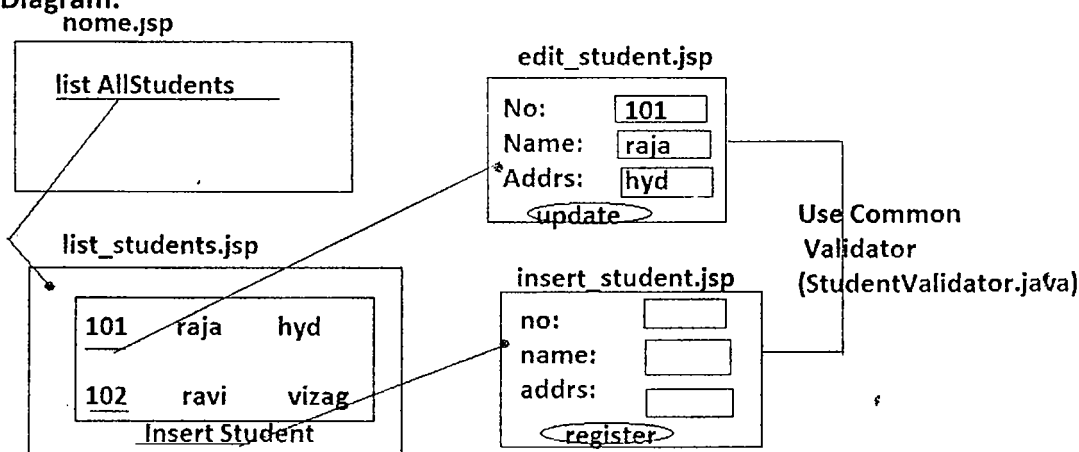
While working with SimpleFormController we can override initBinder() method to register property editors that are required to use while binding given form data (request param values) to command class object properties.

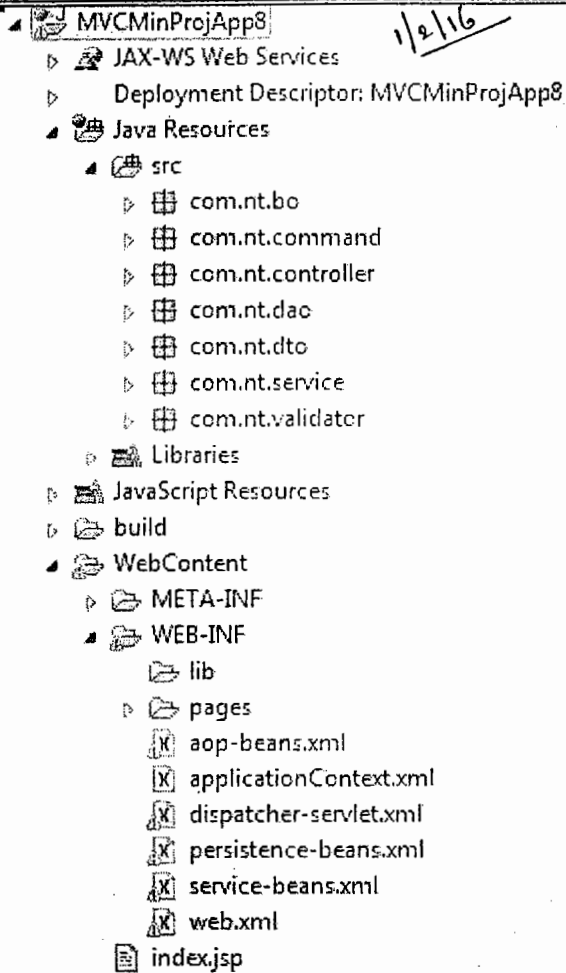
→ This method gives request, ServletRequest DataBinder object. We can register property editors with ServletRequestDataBinder obj because it is responsible to bind the form submitted request parameter values to command object properties. for example app or InitBinder, refer 304 app

Usecase:

Storing string value of for component to command class properties as java.util.Date class obj.

Internal flow of execution in handleRequest method of

SimpleFormController:**Diagram:**

**StudentBO.java**

```
package com.nt.bo;

public class StudentBO {
    private int sno;
    private String sname;
    private String sadd;
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd; } }
}
```

StudentDAO.java

```
package com.nt.dao;
import java.util.List;
import com.nt.bo.StudentBO;
public interface StudentDAO {
    public List<StudentBO> getAllStudents();
    public int insert(StudentBO bo);
    public StudentBO getStudent(int no);
    public int update(StudentBO bo);
}
```

StudentDAOImpl.java

```
package com.nt.dao;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.List;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.RowMapper;
import com.nt.bo.StudentBO;
public class StudentDAOImpl implements StudentDAO {
    private static final String GET_ALL_STUDENTS_QRY="SELECT SNO,SNAME,SADD FROM STUDENT ";
    private static final String INSERT_STUDENT_QRY="INSERT INTO STUDENT VALUES(?,?,?)";
    private static final String GET_STUDENT_BY_SNO="SELECT SNO,SNAME,SADD FROM STUDENT WHERE SNO=?";
    private static final String UPDATE_STUDENT_BY_NO="UPDATE STUDENT SET SNAME=?, SADD=? WHERE SNO=?";

    private JdbcTemplate jt;
    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }
    @Override
    public List<StudentBO> getAllStudents() {
        List<StudentBO> listBO=jt.query(GET_ALL_STUDENTS_QRY, new StudentRowMapper());
        return listBO;
    }
    private class StudentRowMapper implements RowMapper<StudentBO>{
        @Override
        public StudentBO mapRow(ResultSet rs, int pos) throws SQLException {
            StudentBO bo=new StudentBO();
            bo.setSno(rs.getInt(1));
            bo.setSname(rs.getString(2));
            bo.setSadd(rs.getString(3));
            return bo;
        }
    }
    } //inner class
```

```
@Override
public int insert(StudentBO bo) {
    int cnt=jt.update(INSERT_STUDENT_QRY, bo.getSno(),
bo.getSname(),
bo.getSadd());
    return cnt;
}
@Override
public StudentBO getStudent(int no) {
    StudentBO bo=jt.queryForObject(GET_STUDENT_BY_SNO, new
StudentRowMapper(), no);
    return bo;
}
public int update(StudentBO bo){
    int cnt=jt.update(UPDATE_STUDENT_BY_NO,
bo.getSname(),bo.getSadd(),bo.getSno());
    return cnt;
}
}
```

StudentDTO.java

```
package com.nt.dto;
import java.io.Serializable;
public class StudentDTO implements Serializable {
    private int sno;
    private String sname;
    private String sadd;
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
} }
```

StudentService.java

```
package com.nt.service;
import java.util.List;
import com.nt.dto.StudentDTO;
public interface StudentService {
    public List<StudentDTO> listStudents();
    public String registerStudent(StudentDTO dto);
    public StudentDTO searchStudentBySno(int no);
    public String updateStudentBySno(StudentDTO dto);
}
```

StudentServiceImpl.java

```
package com.nt.service;
import java.util.ArrayList;
import java.util.List;
import com.nt.bo.StudentBO;
import com.nt.dao.StudentDAO;
import com.nt.dto.StudentDTO;
public class StudentServiceImpl implements StudentService{
    private StudentDAO dao;
    public void setDao(StudentDAO dao) {
        this.dao = dao;
    }
    @Override
    public List<StudentDTO> listStudents() {
        //use dao
        List<StudentBO> listBO=dao.getAllStudents();
        //Copy ListBO to List DTO
        List<StudentDTO> listDTO=new ArrayList<StudentDTO>();
        if(listBO!=null){
            for(StudentBO bo:listBO){
                StudentDTO dto=new StudentDTO();
                dto.setSno(bo.getSno());
                dto.setSname(bo.getSname());
                dto.setSadd(bo.getSadd());
                listDTO.add(dto);
            }
        }
        return listDTO;
    }
    @Override
    public String registerStudent(StudentDTO dto) {
        //Convert DTO obj BO obj
        StudentBO bo=new StudentBO();
        bo.setSno(dto.getSno());
        bo.setSname(dto.getSname());
        bo.setSadd(dto.getSadd());
        //use dao
    }
}
```



```
public boolean supports(Class<?> clazz) {
    return clazz.isAssignableFrom(StudentCommand.class);
}

@Override
public void validate(Object command, Errors errors) {
    //type casting
    StudentCommand cmd=(StudentCommand)command;
    //form validations
    if(cmd.getSno()==null || cmd.getSno()<=0)
        errors.rejectValue("sno", "sno.invalid");
    if(cmd.getSname().equals("") || cmd.getSname()==null)
        errors.rejectValue("sname", "sname.required");
    if(cmd.getSadd().equals("") || cmd.getSadd()==null)
        errors.rejectValue("sadd", "sadd.required");
    else{
        if(cmd.getSadd().length()<3){
            errors.rejectValue("sadd", "sadd.minlength");
        }
    }
}
}
```

ListStudentsController.java

```
package com.nt.controller;
import java.util.List;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.AbstractController;
import com.nt.dto.StudentDTO;
import com.nt.service.StudentService;
public class ListStudentsController extends AbstractController {
    private StudentService service;
    public void setService(StudentService service) {
        this.service = service;
    }

    @Override
    protected ModelAndView (i)handleRequestInternal(HttpServletRequest req,
        HttpServletResponse res) throws Exception {
        //use Service class
        List<StudentDTO> listDTO=service.listStudents();
        //create MAV obj (j)
        ModelAndView mav=new ModelAndView();
        mav.setViewName("list_students");
        mav.addObject("stList",listDTO);
        return mav; (p)
    }
}
```

StudentInsertController.java

```
package com.nt.controller;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import org.springframework.validation.BindException;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.SimpleFormController;
import com.nt.command.StudentCommand;
import com.nt.dto.StudentDTO;
import com.nt.service.StudentService;
public class StudentInsertController extends SimpleFormController {
    private StudentService service;

    public void setService(StudentService service) {
        this.service = service;
    }

    @Override
    protected ModelAndView onSubmit(HttpServletRequest req,
        HttpServletResponse res, Object command, BindException errors)
        throws Exception {
        // type cast command
        StudentCommand cmd=(StudentCommand)command;
        //convert Command obj to DTO obj
        StudentDTO dto=new StudentDTO();
        dto.setSno(cmd.getSno());
        dto.setSname(cmd.getSname());
        dto.setSadd(cmd.getSadd());
        //use service
        String result=service.regisiterStudent(dto);
        return new ModelAndView("insert_result","resMsg",result);
    }

    @Override
    protected ModelAndView handleInvalidSubmit(HttpServletRequest request,
        HttpServletResponse response) throws Exception {
        return new ModelAndView("dblpst");
    }
}
```

StudentUpdateController.java

```
package com.nt.controller;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.validation.BindException;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.SimpleFormController;
import com.nt.command.StudentCommand;
import com.nt.dto.StudentDTO;
```

```

import com.nt.service.StudentService;
public class StudentUpdateController extends SimpleFormController {
    private StudentService service;

    public void setService(StudentService service) {
        this.service = service;
    }
    @Override
    protected Object formBackingObject(HttpServletRequest request)
        throws Exception {
        //read sno
        int no=Integer.parseInt(request.getParameter("id"));
        //use Service class
        StudentDTO dto=service.searchStudentBySno(no);
        //create Command class obj and place DTO obj
        StudentCommand cmd=new StudentCommand();
        cmd.setSno(dto.getSno());
        cmd.setSname(dto.getSname());
        cmd.setSadd(dto.getSadd());
        return cmd;
    } //method
    @Override
    protected ModelAndView onSubmit(HttpServletRequest request,
        HttpServletResponse response, Object command, BindException errors)
        throws Exception {
        // typecast command
        StudentCommand cmd=(StudentCommand)command;
        //convert Command obj to DTO obj
        StudentDTO dto=new StudentDTO();
        dto.setSno(cmd.getSno());
        dto.setSname(cmd.getSname());
        dto.setSadd(cmd.getSadd());
        //use Service class
        String updMsg=service.updateStudentBySno(dto);
        return new ModelAndView(getSuccessView(),"updResult",updMsg);
    }
}

```

Index.jsp

```
<jsp:forward page="home.htm"/>
```

web.xml

```

<web-app>
  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
  </welcome-file-list>
  <servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>

```

```

<load-on-startup>2</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>dispatcher</servlet-name>
  <url-pattern>*.htm</url-pattern>
</servlet-mapping>
<listener>
  <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>
</web-app>

```

dispatcher-servlet.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
  <bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
    <property name="mappings">
      <props>
        <prop key="/home.htm">pvc1</prop>
        <prop key="/listStudents.htm">stLc</prop>
        <prop key="/insert.htm">sic</prop>
        <prop key="/edit.htm">suc</prop>
      </props>
    </property>
  </bean>
  <bean id="pvc1"
    class="org.springframework.web.servlet.mvc.ParameterizableViewController">
    <property name="viewName" value="home"/>
  </bean>
  <bean id="stLc" class="com.nt.controller.ListStudentsController">
    <property name="service" ref="stService"/>
  </bean>
  <bean id="messageSource"
    class="org.springframework.context.support.ResourceBundleMessageSource">
    <property name="basename" value="com/nt/validator/StCmdValidationErrors"/>
  </bean>
  <bean id="stValidator" class="com.nt.validator.StudentInsertValidator"/>
  <bean id="sic" class="com.nt.controller.StudentInsertController">
    <property name="formView" value="insert_student"/>
    <property name="successView" value="insert_result"/>
    <property name="commandClass" value="com.nt.command.StudentCommand"/>
    <property name="commandName" value="stCmd"/>
    <property name="validator" ref="stValidator"/>
  </bean>

```

```

<property name="sessionForm" value="true"/>
  <property name="service" ref="stService"/>
</bean>
<bean id="suc" class="com.nt.controller.StudentUpdateController">
  <property name="formView" value="edit_student"/>
  <property name="successView" value="edit_result"/>
  <property name="commandClass" value="com.nt.command.StudentCommand"/>
  <property name="commandName" value="stCmd"/>
  <property name="validator" ref="stValidator"/>
  <property name="sessionForm" value="true"/>
  <property name="service" ref="stService"/>
</bean>
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="prefix" value="/WEB-INF/pages/">
  <property name="suffix" value=".jsp"/>
</bean>
</beans>

```

applicationContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <import resource="persistence-beans.xml"/>
  <import resource="service-beans.xml"/>
  <import resource="aop-beans.xml"/>
</beans>

```

persistence-beans.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="jofb" class="org.springframework.jndi.JndiObjectFactoryBean">
  <property name="jndiName" value="DsJndi"/>
</bean>
<bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
  <property name="dataSource" ref="jofb"/>
</bean>
<bean id="stDao" class="com.nt.dao.StudentDAOImpl">
  <property name="jt" ref="template"/>
</bean>
</beans>

```

service-beans.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="stService" class="com.nt.service.StudentServiceImpl">
    <property name="dao" ref="stDao"/>
  </bean>
</beans>
```

aop-beans.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:aop="http://www.springframework.org/schema/aop"
xmlns:tx="http://www.springframework.org/schema/tx"
xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/tx
http://www.springframework.org/schema/tx/spring-tx.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd">
  <bean id="txMgr"
    class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
    <property name="dataSource" ref="jofb"/>
  </bean>
  <tx:advice id="txAdvice" transaction-manager="txMgr">
    <tx:attributes>
      <tx:method name="list*" propagation="REQUIRED"
        read-only="true"/>
      <tx:method name="register*" propagation="REQUIRED"
        read-only="false"/>
      <tx:method name="search*" propagation="REQUIRED"
        read-only="true"/>
      <tx:method name="update*" propagation="REQUIRED"
        read-only="false"/>
    </tx:attributes>
  </tx:advice>
  <aop:config>
    <aop:pointcut id="ptc1"
      expression="execution(* com.nt.service.StudentService.*(..))"/>
    <aop:advisor advice-ref="txAdvice" pointcut-ref="ptc1"/>
  </aop:config>
</beans>
```

Dbpost.jsp

```
<h1 style="color:red"><center> Double Posting is not Alloed </center></h1>
<a href="home.htm">home</a>
```

Edit_result.jsp

```
<h1><center> Result Page</center></h1>
${updResult}
<a href="home.htm">home</a>
```

Edit_student.jsp

```
<%@taglib uri="http://www.springframework.org/tags/form" prefix="form"%>
<h1><center> Update Student Details</center></h1>
<form:form method="post" commandName="stCmd">
  Sno : <form:input path="sno"/><span style="color:red"> <form:errors path="sno"/> </span>
  <br>
  Sname : <form:input path="sname"/><span style="color:red"> <form:errors path="sname"/>
  </span> <br>
  Sadd : <form:input path="sadd"/><span style="color:red"> <form:errors path="sadd"/>
  </span> <br>
  <input type="submit" value="update"/>
</form:form>
```

Home.jsp

```
<h1> Home Page</h1>
<a href="listStudents.htm">List All Students</a>
```

Insert_result.jsp

```
<h1><center> Result page</center></h1>
${resMsg}
<br>
<a href="home.htm">home</a>
```

Insert_student.jsp

```
<%@taglib uri="http://www.springframework.org/tags/form" prefix="form"%>
<h1><center> Register Student Details</center></h1>
<form:form method="post" commandName="stCmd">
  Sno : <form:input path="sno"/><span style="color:red"> <form:errors path="sno"/> </span>
  <br>
  Sname : <form:input path="sname"/><span style="color:red"> <form:errors path="sname"/>
  </span> <br>
  Sadd : <form:input path="sadd"/><span style="color:red"> <form:errors path="sadd"/>
  </span> <br>
  <input type="submit" value="register"/>
</form:form>
```

List_students.jsp

```
<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<h1><center> Student Details</center></h1>
<table bgcolor=red border=1 width="100%" >
  <tr bgcolor="green"><th>Sno </th><th>Sname</th><th>sadd</th></tr>
<c:choose>
  <c:when test="{stList ne null}">
    <c:forEach var="st" items="{stList}">
```

```

<tr>
  <td><a href="edit.htm?id=${st.sno}"><c:out value="${st.sno}" /></a></td>
  <td><c:out value="${st.sname}" /></td>
  <td><c:out value="${st.sadd}" /></td>
</tr>
</c:forEach>
</c:when>
<c:otherwise>
  <span style="color:red"> No Records Found </span>
</c:otherwise>
</c:choose>
</table>
<br><br>
<center>
  <a href="insert.htm"> Insert Student</a>
</center>
<br>
<br>
<a href="home.htm">home</a>

```

MultiActionController:

When formpage contains multiple submit button then instead of taking multiple AbstractCommandController having multiple user-defined methods to process request of multiple submit buttons on one per method basis. *For SimpleForm Controller take single multiAction Controller*

→ The userdefined methods of this class can have following standard signature

```

public (ModelAndView|Map<String|void>) <methodname>
(HttpServletRequest request, HttpServletResponse response, [HttpSession], [anyObject]);

```

→ While configuring our multiActionController class we must specify the MethodResolver obj as dependent obj that contains additional parameter name based on which the method that has to be executed in MultiActionController class will be decided. *name*

Conclusion:-

If sourcepage is form submission having multiple submit buttons with optional form validation and the required dest page is static/dynamic webpage that should come after request processing then we should go after MultiActionController.

Flow for insert submit button:

Insert submit button---->DispatcherServlet---->HandlerMapping---->controller (MAController)-----

>handleRequest(-,-)----->create command obj and performs request wrapping

Reads paramName(opt), defaultMethodName(invalid) from the injected MethodNamesResolver obj.

Reads the values of "opt" req param (insert) *named submit buttons*

Calls insert(req,res,command) method. *it automatically detects cmd obj.*

→ In MAController, we no need to configure command obj, it automatically detects cmd obj.

- MVCMultiActionApp11 5/2/16
 - ▷ JAX-WS Web Services *app completed*
 - ▷ Deployment Descriptor: MVCMultiActionApp11
 - Java Resources *Add spring 4.2 jars at MVC*
 - src
 - com.nt.command
 - ▷ TestCommand.java
 - com.nt.controller
 - ▷ PerformOperationsController.java
 - ▷ Libraries
 - ▷ JavaScript Resources
 - ▷ build
 - WebContent
 - ▷ META-INF
 - WEB-INF
 - ▷ lib
 - ▷ pages
 - applicationContext.xml
 - dispatcher-servlet.xml
 - web.xml
 - ▷ index.jsp

Ex:

TestCommand.java internally creates command class vbi having form data

```

package com.nt.command;
public class TestCommand {
    private int sno;
    private String sname;
    private String sadd;
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
}

```

5/2/16 **PerformOperationsController.java**

```

package com.nt.controller;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.multiaction.MultiActionController;
import com.nt.command.TestCommand;
public class PerformOperationsController extends MultiActionController {

    public ModelAndView insert(HttpServletRequest req, HttpServletResponse res, TestCommand
cmd){
        System.out.println(cmd.getSno());
        return new ModelAndView("user", "operation", "Insert Operation");
    }

    public ModelAndView update(HttpServletRequest req, HttpServletResponse
res, TestCommand cmd){
        return new ModelAndView("user", "operation", "Update Operation");
    }

    public ModelAndView delete(HttpServletRequest req, HttpServletResponse res, TestCommand
cmd){
        return new ModelAndView("user", "operation", "delete Operation");
    }

    public ModelAndView invalid(HttpServletRequest req, HttpServletResponse res, TestCommand
cmd){
        return new ModelAndView("user", "operation", "Invalid Operation");
    }
}

```

Index.jsp

```
<jsp:forward page="form.htm"/>
```

web.xml

```

<web-app>
    <welcome-file-list>
        <welcome-file>index.jsp</welcome-file>
    </welcome-file-list>
    <servlet>
        <servlet-name>dispatcher</servlet-name>
        <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
        <load-on-startup>2</load-on-startup>
    </servlet>
    <servlet-mapping>
        <servlet-name>dispatcher</servlet-name>
        <url-pattern>*.htm</url-pattern>
    </servlet-mapping>
    <listener>
        <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
    </listener>

```

5/2/16

```

<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>
</web-app>

```

dispatcher-servlet.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
  <bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
    <property name="mappings">
      <props>
        <prop key="/form.htm">pvc1</prop>
        <prop key="/perform.htm">poc</prop>
      </props>
    </property>
  </bean>

  <bean id="pvc1"
    class="org.springframework.web.servlet.mvc.ParameterizableViewController">
    <property name="viewName" value="user"/>
  </bean>

  <bean id="mnr"
    class="org.springframework.web.servlet.mvc.multiaction.ParameterMethodName
    Resolver">
    <property name="paramName" value="opt"/>
    <property name="defaultMethodName" value="invalid"/>
  </bean>

  <bean id="poc" class="com.nt.controller.PerformOperationsController">
    <property name="methodNameResolver" ref="mnr"/>
  </bean>

  <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="prefix" value="/WEB-INF/pages/">
    <property name="suffix" value=".jsp"/>
  </bean>
</beans>

```

applicationContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:context="http://www.springframework.org/schema/context"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">
  </beans>

```

User.jsp

```

<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<form action="perform.htm" method="post">
  Sno : <input type="text" name="sno"/> <br>

```

```

Sname :<input type="text" name="sname"/> <br>
Sadd :<input type="text" name="sadd"/> <br>
<input type="submit" name="opt" value="insert"/>
<input type="submit" name="opt" value="update"/>
<input type="submit" name="opt" value="delete"/>
<input type="submit" name="opt1" value="view"/>
</form>
<c:if test="${operation ne null}">
<c:out value="${operation}"/> Successpringframeworkul
</c:if>

```

Note: Only SimpleFormController is removed from spring 4.0

While working with MultiActionController there is no need of configuring command class anywhere here because the superclass `HandlerRequest` internally uses Reflection API to take specified 3rd parameter type or 4th parameter type (if 3rd parameter is `HttpSession`) as command class name.

AbstractWizardFormController: it is equal to Session Tracking

So far we worked with single form page based controller classes like SimpleFormController, MultiActionController and etc... in order to work with multiple form pages that comes in a sequence having wizard style we need to use AbstractWizardFormController by specifying list form pages to display in sequence. In this controller, the data of all the form pages will come and store in single command class object. Based on the name of additional request parameter that comes along with the request and this will display the form page or process the request.

_cancel: leaves the wizard without performing final action (cancel button)

_finish: leaves the wizard by performing final Action (finish button)

_target: displays the nth page from the list of form pages that are configured (for previous, next button)

Welcomepage-->formpage1---->formpage2----->formpage3----->Result

We can override the following methods in our AbstractWizardFormController.

formBackingObject(-): creates and returns command object

processSpringFrameworkinish(-): takes final action of request processing for finish button.

processFinish(-)

If you configure command class in dispatcherServlet-servlet.xml, no need to override formBackingObject, otherwise you have to override

processCancel(-): executes the logic related to "Cancel button"

- ▲ MVCWizardFormApp12 *6/2/16 completed app*
 - ▷ JAX-WS Web Services
 - ▷ Deployment Descriptor: MVCWizardFormApp12
 - ▲ Java Resources
 - ▲ src
 - ▲ com.nt.command
 - ▷ RegisterCommand.java
 - ▲ com.nt.controller
 - ▷ RegisterController.java
 - ▷ Libraries
 - ▷ JavaScript Resources
 - ▷ build
 - ▲ WebContent
 - ▷ META-INF
 - ▲ WEB-INF
 - ▷ lib
 - ▲ pages
 - ▷ Page1Form.jsp
 - ▷ Page2Form.jsp
 - ▷ Page3Form.jsp
 - ▷ result.jsp
 - ▷ welcome.jsp
 - ▷ applicationContext.xml
 - ▷ dispatcher-servlet.xml
 - ▷ web.xml
 - ▷ index.jsp

button"

* add 2.2 jar file

RegisterCommand.java

```
package com.nt.command;

public class RegisterCommand{
    //Login Credentials
    private String userName;
    private String password;
    //Contact Details
    private String emailId;
    private String phone;
    //Address Details
    private String city;
    private String pincode;
    public RegisterCommand(){
        System.out.println("RegisterCommand:0-param constructor");
    }
    public String getUserName() {
        return userName;
    }
    public void setUserName(String userName) {
        this.userName = userName;
    }
    public String getPassword() {
```

6/2/16

```
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
    public String getEmailId() {
        return emailId;
    }
    public void setEmailId(String emailId) {
        this.emailId = emailId;
    }
    public String getPhone() {
        return phone;
    }
    public void setPhone(String phone) {
        this.phone = phone;
    }
    public String getCity() {
        return city;
    }
    public void setCity(String city) {
        this.city = city;
    }
    public String getPincode() {
        return pincode;
    }
    public void setPincode(String pincode) {
        this.pincode = pincode;
    }
    @Override
    public String toString() {
        return "RegisterCommand [userName=" + userName + ", password="
            + password + ", emailId=" + emailId + ", phone=" + phone
            + ", city=" + city + ", pincode=" + pincode + "]\n";
    }
}
```

RegisterController.java

```
package com.nt.controller;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.validation.BindException;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.AbstractWizardFormController;
import com.nt.command.RegisterCommand;
public class RegisterController extends AbstractWizardFormController{
    /*public RegisterController(){
        setCommandName("userForm");
    }
}
```

```

    }*/
    @Override
    protected Object formBackingObject(HttpServletRequest request)
        throws Exception {
        System.out.println("RegisterController.formBackingObject(-)");
        return new RegisterCommand();
    }

    @Override
    protected ModelAndView 33 finish processspringframeworkinish (HttpServletRequest request,
        HttpServletResponse response, Object command, BindException errors)
        throws Exception {
        System.out.println("RegisterController.processspringframeworkinish(-,-,-)");
        //Get the data from command object
        RegisterCommand cmd = (RegisterCommand)command;

        //where is the finish page?
        return new ModelAndView("result", "res", cmd);
    }

    @Override
    protected ModelAndView processCancel(HttpServletRequest request,
        HttpServletResponse response, Object command, BindException errors)
        throws Exception {
        System.out.println("RegisterController.processCancel(-,-,-)");
        //where is the cancel page?
        return new ModelAndView("welcome");
    }
}

```

Index.jsp

```
<jsp:forward page="home.htm"/>
```

web.xml ¹

```
<web-app>
```

```
<servlet>
```

```
<servlet-name>dispatcher</servlet-name>
```

```
<servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
```

```
<load-on-startup>1</load-on-startup>
```

```
</servlet>
```

```
<servlet-mapping>
```

```
<servlet-name>dispatcher</servlet-name>
```

```
<url-pattern>*.htm</url-pattern>
```

```
</servlet-mapping>
```

```
<listener>
```

```
<listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
```

```
</listener>
```

```
<context-param>
```

```
<param-name>contextConfigLocation</param-name>
```

```
<param-value>/WEB-INF/applicationContext.xml</param-value>
```

```
</context-param>
```

```
<welcome-file-list>
```

6/2/16
 <welcome-file>index.jsp</welcome-file>
 </welcome-file-list>
 </web-app>

Dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans-2.5.xsd">
```

```
<bean id="surl" class="org.springframework.web.servlet.handler.SimpleUrlHandler
Mapping">
```

```
<property name="mappings">
```

```
<props>
```

```
<prop key="/home.htm">pvc1</prop>
```

```
<prop key="/register.htm">rc</prop>
```

```
</props>
```

```
</property>
```

```
</bean>
```

```
<bean id="pvc1" class="
```

```
org.springframework.web.servlet.mvc.ParameterizableViewController">
```

```
<property name="viewName" value="welcome"/>
```

```
</bean>
```

```
<bean id="rc" class="com.nt.controller.RegisterController" {initial phase neg}
  <property name="commandName" value="regCmd"/>
  <property name="pages">
    <property name="commandClass" value="com.nt.command.RegisterCmd"/>
    <list>
```

```
<value>Page1Form</value>
```

```
<value>Page2Form</value>
```

```
<value>Page3Form</value>
```

```
</list>
```

```
</property>
```

```
</bean>
```

```
<bean id="viewResolver"
```

```
class="org.springframework.web.servlet.view.InternalResourceViewResolver">
```

```
<property name="prefix" value="/WEB-INF/pages/">
```

```
<property name="suffix" value=".jsp"/>
```

```
</bean>
```

```
</beans>
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:context="http://www.springframework.org/schema/context"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">
  </beans>
```

Page1Form.jsp

```
<%@ taglib prefix="form" uri="http://www.springframework.org/tags/form"%>
```


6/2/16 **Page1Form.jsp** (15)

```

<form:form method="POST" commandName="regCmd">
  Username : <form:input path="userName"/><br>
  Password: <form:password path="password"/><br>
  <input type="submit" value="Next" name="_target1"/>
  <input type="submit" value="Cancel" name="_cancel" />
</form:form>

```

Page2Form.jsp (21)

```

<%@ taglib prefix="form" uri="http://www.springframework.org/tags/form"%>
<form:form method="POST" commandName="regCmd">
  Email :<form:input path="emailId"/><br>
  Phone:<form:input path="phone" /><br>
  <input type="submit" value="Previous" name="_target0" />
  <input type="submit" value="Next" name="_target2"/>
  <input type="submit" value="Cancel" name="_cancel" />
</form:form>

```

Page3Form.jsp (28)

```

<%@ taglib prefix="form" uri="http://www.springframework.org/tags/form"%>
<form:form method="POST" commandName="regCmd">
  City : <form:input path="city" /><br>
  Pincode:<form:input path="pincode" /><br>
  <input type="submit" value="Previous" name="_target1" />
  <input type="submit" value="Finish" name="finish" />
  <input type="submit" value="Cancel" name="_cancel" />
</form:form>

```

Result.jsp (36)

```

<h1><center> Result page</center></h1>
${res}

```

Welcome.jsp (6)

```

<H1><center> HOME PAGE</center></H1>
<br>
<br>
<a href="register.htm"> Register</a>

```

6/2/16 **Diagram: Flow** :

- When we give initial phase request to AbstractWizardFormController it internally calls FormBackingObject() through keep that object in session scope. handleRequest(-) method to create Command class
- When postback request is given having _target<x> request parameter name it displays next or previous page. postback request having the request param name _cancel calls processCancel(-) through handleRequest(-) method
- Similarly calls processSpringFrameworkFinish(-) for postback request that contains _finish request param.

Handler Mapping: 7/2/16

Handler Mapping is a component that maps incoming request to controller/handler class. All handlermapping classes are the implementation classes of "HandlerMapping" (i). The important HandlerMapping classes are

1. BeanNameUrlHanlderMapping (default for xml configuration) → default
2. SimpleUrlHandlerMapping
3. ControllerClassNameHandlerMapping

4. DefaultAnnotationHandlerMapping (default for annotation configuration)

7/12/16 BeanNameUrlHandlerMapping:

This maps incoming request to controller class based on the servlet path or virtual path of incoming request url. i.e the virtual path of incoming request url must be taken as bean name of controller class.

Eg:

```
<bean class="org.springframework.web.servlet.handler.BeanNameUrlHandlerMapping"/>
```

```
<bean name="/home.htm" class="com.nt.controller.WelcomeController"/>
```

If the incoming request url contains "/home.htm" then it is maps the request to WelcomeController class.

→ servlet path or virtual path (or) servlet url

7/12/16 SimpleUrlHandlerMapping:

This is useful to map the incoming requests with controller classes based on virtual paths of request urls and bean ids/names of controller classes.

→ We need to supply the details of values of "mappings" property of type (java.util.properties)

Ex:

```
<bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
```

```
<props>
```

```
<prop key="/home.htm">wc</prop>
```

```
</props>
```

```
</property>
```

```
</bean>
```

```
<bean name="wc" class="com.nt.controller.WelcomeController"/>
```

→ If request url contains "/home.htm" then it passes the request to WelcomeController class whose bean id wc. (or) bean name is "wc".

ControllerClassNameHandlerMapping:

Maps incoming request with controller classes based on virtual paths and controller class names.

It takes virtual path, removes extension, makes firstLetter as uppercase letter and appends

"controller" word to it and looks for controller class whose name is that name.

- 7/12/16 completed
 - ▶ MVCHandlerMappingApp13
 - ▶ JAX-WS Web Services
 - ▶ Deployment Descriptor: MVCHandlerMappingApp13.xml
 - ▶ Java Resources
 - ▶ src
 - ▶ com.nt.controller
 - ▶ WelcomeController.java
 - ▶ Libraries
 - ▶ JavaScript Resources
 - ▶ build
 - ▶ WebContent
 - ▶ META-INF
 - ▶ WEB-INF
 - ▶ lib
 - ▶ pages
 - ▶ dispatcher-servlet.xml
 - ▶ web.xml
 - ▶ index.jsp

SimpleUrlHandlerMapping is mostly used.
not required other handler mapping

WelcomeController.java

```

package com.nt.controller;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.Controller;
public class WelcomeController implements Controller {
    @Override
    public ModelAndView handleRequest(HttpServletRequest req,
        HttpServletResponse res) throws Exception {
        return new ModelAndView("welcome", "msg", "Welcome To Spring MVC");
    }
}

```

Index.jsp

```
<jsp:forward page="welcome.htm"/>
```

web.xml

```

<web-app>
<servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>2</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>*.htm</url-pattern>
</servlet-mapping>
</web-app>

```

dispatcher-servlet.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
    <bean id="wc" class="com.nt.controller.WelcomeController"/>
    <bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
        <property name="mappings">
            <props>
                <prop key="welcome.htm">wc</prop>
            </props>
        </property>
        <property name="order" value="1"/>
    </bean>
    <bean class="org.springframework.web.servlet.mvc.support.ControllerClassNameHandler
Mapping">
        <property name="order" value="2"/>
    </bean>
    <bean id="vr" class="org.springframework.web.servlet.view.InternalResourceView
Resolver">
        <property name="prefix" value="/WEB-INF/pages/">
        <property name="suffix" value=".jsp"/>

```

```

</bean>
</beans>
welcome.jsp
<h1><center> HOME PAGE</center></h1>
${requestScope.msg}

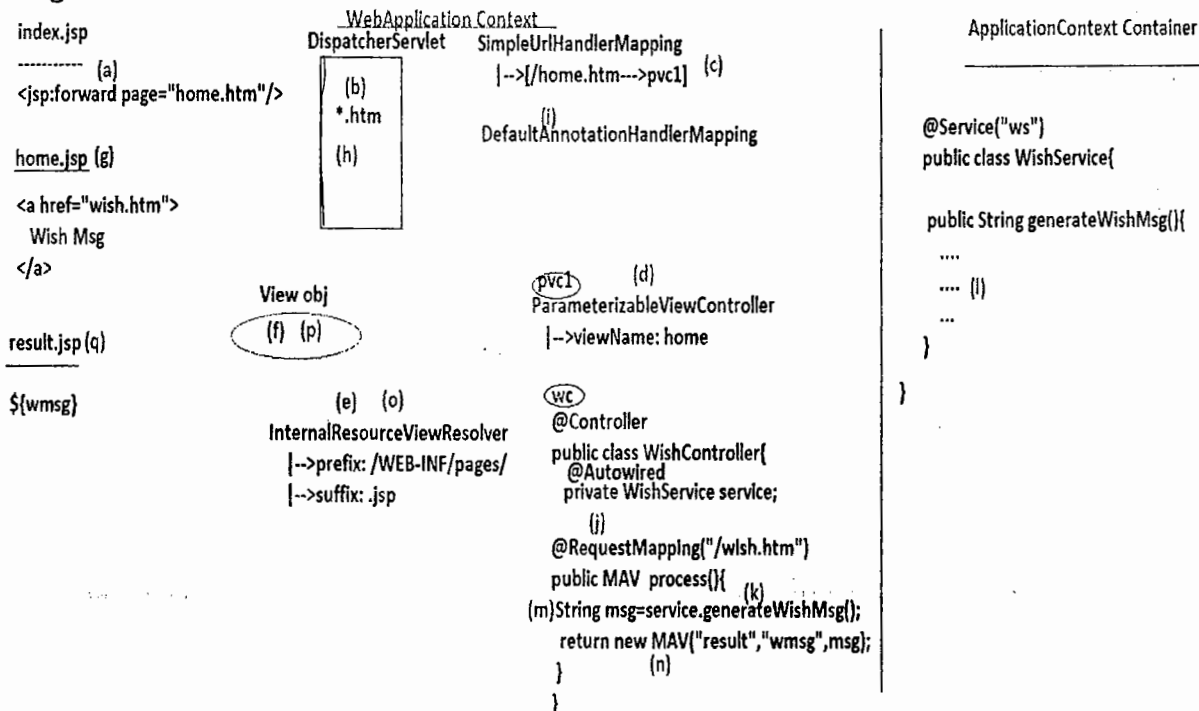
```

⇒ If request url contains "welcome.htm" then it maps the request to a controller class whose name is "WelcomeController"

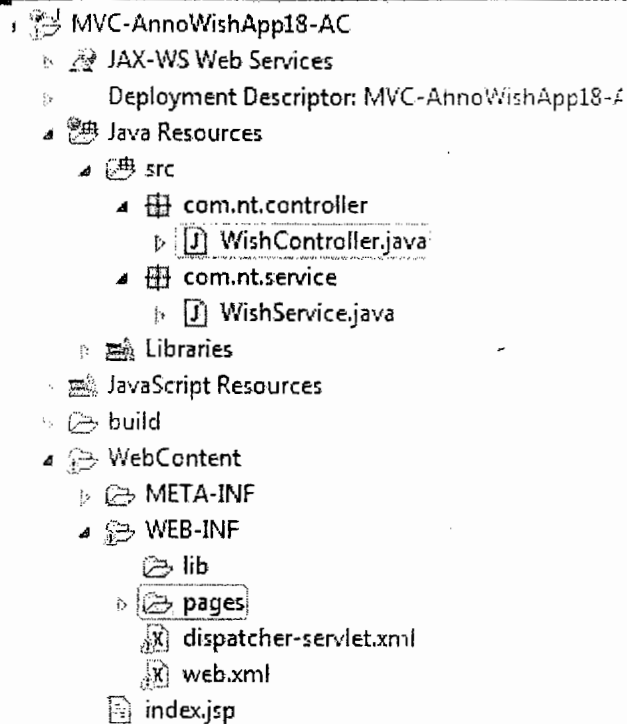
DefaultAnnotationHandlerMapping: ~~4211~~

Maps the incoming request url with controller class methods based on @RequestMapping annotation info.

Diagram:



APR 10 2010 10:10:10

**WishService.java**

```
package com.nt.service;
import java.util.Calendar;
import javax.inject.Named;
@Named("ws")
public class WishService {
    public String generateWishMsg(){
        // generate wish Msg based on current hour the day
        Calendar cl=Calendar.getInstance();
        int hour=cl.get(Calendar.HOUR_OF_DAY);
        if(hour<=12)
            return "Good morning";
        else if(hour<=16)
            return "Good After noon";
        else if (hour<=20)
            return "Good Evening";
        else
            return "Good Night";
    }
}
//method
}
//class
```

WishController.java

```
package com.nt.controller;
import javax.annotation.Resource;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;
import com.nt.service.WishService;
```

```

@Controller
public class WishController {
    @Resource
    private WishService service;
    /* @RequestMapping("/wish.htm")
    public ModelAndView process(){
        //use service class
        String msg=service.generateWishMsg();

        return new ModelAndView("result","wmsg",msg);
    } */
    /* @RequestMapping("/wish.htm")
    public String process(Model model){

        //use service class
        String msg=service.generateWishMsg();

        // add data to mode
        model.addAttribute("wmsg",msg);
        return "result";
    } */
    /* @RequestMapping("/wish.htm")
    public String process(ModelMap map){
        //use service class
        String msg=service.generateWishMsg();
        map.addAttribute("wmsg", msg);
        return "result";
    } */
    /* @RequestMapping("/wish.htm")
    public String process(Map<String,Object> map){
        //use service class
        String msg=service.generateWishMsg();
        map.put("wmsg",msg);
        return "result";
    } */
    @RequestMapping("/wish.htm")
    public void process(Model model){
        //Use service class
        String msg=service.generateWishMsg();
        model.addAttribute("wmsg",msg);
    }
}
}

```

Index.jsp

```
<jsp:forward page="home.htm"/>
```

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<web-app xmlns:web="http://xmlns.jcp.org/xml/ns/javaee">
```

```
<welcome-file-list>
  <welcome-file>index.jsp</welcome-file>
</welcome-file-list>
<servlet>
  <servlet-name>dispatcher</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  <load-on-startup>2</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>dispatcher</servlet-name>
  <url-pattern>*.htm</url-pattern>
</servlet-mapping>
<listener>
  <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>
</web-app>
```

dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
  <bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
    <property name="mappings">
      <props>
        <prop key="/home.htm">pvc1</prop>
      </props>
    </property>
  </bean>
  <bean id="pvc1"
    class="org.springframework.web.servlet.mvc.ParameterizableViewController">
    <property name="viewName" value="home"/>
  </bean>
  <bean class="org.springframework.web.servlet.mvc.annotation.DefaultAnnotation
HandlerMapping"/>
  <!-- <bean class="org.springframework.web.servlet.view.DefaultRequest
ToViewNameTranslator"/> -->
  <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="prefix" value="/WEB-INF/pages/">
    <property name="suffix" value=".jsp"/>
  </bean>
```

```
<context:component-scan base-package="com.nt.controller"/>
</beans>
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
<context:component-scan base-package="com.nt.service"/>
</beans>
```

Home.jsp

```
<h1><center> Welcome to Annotations Based Spring MVC</center></h1>
<a href="wish.htm"> wish Msg</a>
```

Result.jsp

```
<center><h1> Result Page</h1> </center>
<br>
```

```
Wish Message : ${wmsg}
<a href="home.htm">home</a>
```

Wish.jsp

```
<b> wish.jsp</b>
Result is : ${wmsg}
```

✶ If request url contains "/home.htm" then request goes to m1() method

✶ If request url contains "/welcome.htm" then request goes to m2() method.

⇒ When we configure multiple handler mapping in which order they should be executed will be decided based on "order priority" value, high value indicates low priority and low value indicates high priority.

Request: welcome.htm

Eg:

```
<bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
<property name="mappings">
<props>
</property>
<property name="order" value="1"/>
</bean>
<bean
class="org.springframework.web.servlet.mvc.support.ControllerClassNameHandlerMapping">
<property name="order" value="2"/>
</bean>
```

While configuring handler mapping we can also specify the order and handler interceptor.

✶ HandlerInterceptor: (It is equal to filter concepts in J2EE)

It act as hook (additional) for controller/handler class to have pre and post processing logics. It is like ServletFilter of J2EE web application development, But it is different from filter in 1 way. Filter can have only pre request processing and post response processing logics. Where handler

interceptor can ^{have} pre-request processing, before rendering view, after rendering view logics with response to controller classes.

→ To develop interceptor we need to ^{take} back a class that implements "HandlerInterceptor" (i), But there is adaptor class for this whose name is "HandlerInterceptorAdapter", so we can extend from this ^{adaptor} class to override our choice methods.

* preHandler(--) -----> executes before control goes to controller class

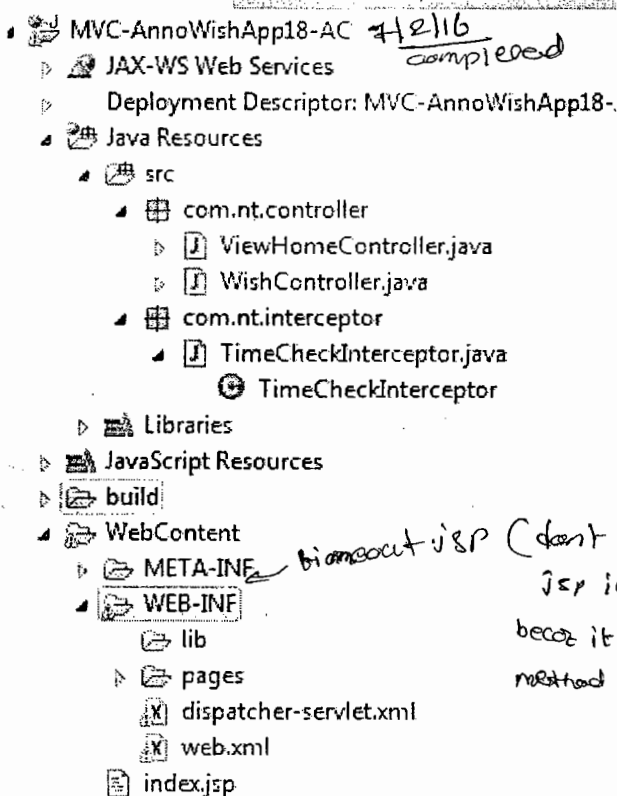
* postHandler(--) -----> executes once controller class finished the execution but before rendering the view.

* afterCompletion() : executes after rendering the view.

Usecases of HandlerInterceptor:

We can develop HandlerInterceptor class for controller only to take the request between 9am to 5pm

→ We can develop Handlerinterceptor class for controller only to allow the request that are coming from certain browser and etc....



TimeCheckInterceptor.java

```
package com.nt.interceptor;
import java.util.Calendar;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.handler.HandlerInterceptorAdapter;
public class TimeCheckInterceptor extends HandlerInterceptorAdapter {
    @Override
    public boolean preHandle(HttpServletRequest request,
                               HttpServletResponse response, Object handler) throws Exception {
        Calendar cl=Calendar.getInstance(); //sys date and time
        int hour=cl.get(Calendar.HOUR_OF_DAY); //current hour in 24hr format
```

7/12/16
if(hour<9 || hour>=17){
 response.sendRedirect("timeout.jsp");
 return false;

}

return true;

}//preHandle

}//class

ViewHomeController.java

package com.nt.controller;

import javax.servlet.http.HttpServletRequest;

import javax.servlet.http.HttpServletResponse;

import org.springframework.web.servlet.ModelAndView;

import org.springframework.web.servlet.mvc.AbstractController;

public class ViewHomeController extends AbstractController {

 @Override

 protected ModelAndView handleRequestInternal(HttpServletRequest req,
 HttpServletResponse res) throws Exception {

 return new ModelAndView("home");

 }

}

WishController.java

package com.nt.controller;

import java.util.Calendar;

import javax.servlet.http.HttpServletRequest;

import javax.servlet.http.HttpServletResponse;

import org.springframework.web.servlet.ModelAndView;

import org.springframework.web.servlet.mvc.AbstractController;

public class WishController extends AbstractController {

 @Override

 protected ModelAndView handleRequestInternal(HttpServletRequest req,
 HttpServletResponse res) throws Exception {

 // get System date and time

 Calendar cl=Calendar.getInstance();

 //get current hour of the day

 int hour=cl.get(Calendar.HOUR_OF_DAY);

 String msg=null;

 if(hour<12)

 msg="Good morning";

 else if(hour<16)

 msg="Good afternoon";

 else if(hour<20)

 msg="Good evening";

 else

 msg="Good Night";

7/2/16

</list>

```

7/12/16 </ref bean="timeCheckInterceptor"/>
</list>
</property>
</bean>
<bean id="timeCheckInterceptor" class="com.nt.interceptor.TimeCheckInterceptor"/>
<bean id="vhc" class="com.nt.controller.ViewHomeController"/>
<bean id="wc" class="com.nt.controller.WishController"/>
<bean id="ufv" class="org.springframework.web.servlet.mvc.UrlFilenameViewController"/>
<!-- <bean id="pvc1" class="
org.springframework.web.servlet.mvc.ParameterizableViewController">
  <property name="viewName" value="AboutUs"/>
</bean>
  <bean id="pvc2" class="
org.springframework.web.servlet.mvc.ParameterizableViewController">
  <property name="viewName" value="ContactUs"/>
</bean> -->
<bean id="vr" class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="prefix" value="/WEB-INF/pages/">
  <property name="suffix" value=".jsp"/>
</bean>
</beans>

```

web.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:web="http://xmlns.jcp.org/xml/ns/javaee"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
    http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">
  <servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>2</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>*.htm</url-pattern>
  </servlet-mapping>
</web-app>

```

ViewResolver:

This component takes logical view name given controller thorough DispatcherServlet and returns "ViewObject" (The implementation class object of View (i)) pointing to physical view. DispatcherServlet calls render (-,-) method on this view object to render the control to view resource.

Like html file, jspfile, bean class that implements "ViewResolver" interface. Spring has supplied few predefined viewResolver classes. They are

1) InternalResourceViewResolver: - (default)

Useful to resolve jsp, servlet programs of WEB-INF folder as views

By default takes "JstView" as view class

UrlBasedViewResolver: —

Super class for (a) and we must specify view class explicitly

c) ^{perce}ResolverBundleViewResolver: —

Allows to configure views in properties file.

d) XmlViewResolver:

Allows to configure views in xml file.

e) BeanNameViewResolver:

Useful to resolve spring beans as views.

^{urlBased}UrlBasedViewResolver: —

→ Takes logical viewname from controller class and searches for view resources whose name is same as logical viewname. We must configure "viewClass" explicitly while configuring this view Resolver class.

→ This viewResolver can be used to resolve any view like html, jsp, beans class and etc... by configuring an appropriate view class (viewclass configuration is mandatory)

→ We configure 'JstlView' as view class to make ViewResolver to resolve Html or Servlets or jsp prgms as views, i.e viewResolver gives JstlView class object and render() will be called on that object to render the view to browser.

Syntax

```
<bean class="org.springframework.web.servlet.view.UrlBasedViewResolver">
  <property name="viewClass" value="org.springframework.web.servlet.view.JstlView"/>
  <property name="suffix" value=".jsp"/> to work with html or servlet or jsp programs as the views
  </bean> <property name="prefix" value="/WEB-INF/pages"/>
```

^{urlBased}InternalResourceViewResolver: → (default view Resolver)

→ It is the subclass of UrlBasedViewResolver, that takes 'JstlView' as default view class having capability to resolve the servlet, jsp prgms of web application that are there under WEB-INF folder.

→ It is most regularly used viewresolver.

Syntax

```
<bean class="org.springframework.web.servlet.InternalResourceViewResolver">
  <property name="prefix" value="/WEB-INF/Pages"/>
  <property name="suffix" value=".jsp"/>
</bean>
```

7/12 ResourceBundleViewResolver:

→ The problem with above 2 view Resolver is logical view name ^{and} jsp file name must match. If jsp filename is changed later then modifying logical viewname by opening the source code of controller class is very difficult.

→ In order to overcome this problem we can use ResourceBundleViewResolver where we can configure views in the properties springframework file.

Views.properties:

Default name also views.properties

(src folder)

logical view name
Welcome(class)=org.springframework.web.servlet.view.JstlView

location
Welcome.url=/WEB-INF/Pages/Welcome.jsp

In dispatcherServlet:

Syntax

```
<bean id="messageSource" vs
  class="org.springframework.web.servlet.view.ResourceBundleViewResolver">
  <property name="basename" value="views"/>
</bean>
```

→ We can use ResourceBundleViewResolver to enable internationalization on our spring MVC applications.

→ We can take multiple jsps for multiple locales and we can configure them in multiple locale specific properties file.

XmlViewResolver:

→ Similar to ResourceBundleViewResolver, but allows to take xml file to specify view configurations. Takes "view.xml" as the default name. we need to specify this file name as the value of "location" property of XmlViewResolver in configuration file.

Ex:

Views.xml (WEB-INF folder)

```
<bean id="welcome" class="org.springframework.web.servlet.view.JstlView">
  <property name="location" value="/WEB-INF/views.xml"/>
</bean>
```

In dispatcher-servlet.xml :-

```
<bean id="vr" class="org.springframework.web.servlet.view.xml.XmlViewResolver"/>
```

2/2/16 BeanNameViewResolver: [If you want to use java bean as view, go for this]

Resolves java class/spring bean as the viewResource, for this the logical view name gathered from controller class must be taken as bean id or name for the class/bean that is acting as view.

dispatcher-servlet.xml

<!-- Java class/Spring class acting as view -->

```
<bean name="welcome" class="com.nt.view.MyReportExcel"/>
```

```
<bean id="ur" class="org.springframework.web.servlet.view.BeanNameViewResolver"/>
```

3/2/16

Working with Views:

When we configure multiple viewResolvers they will execute in a chain.

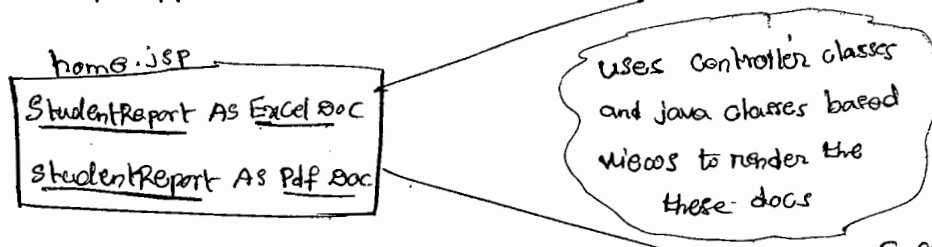
→ InternalResourceViewResolver always gets high priority when compare to other ViewResolvers in chain so it is always recommended to configure InternalResourceViewResolver as the last ViewResolver in the chain.

→ Spring allows to take different technologies in the view layer like jsp,html,servlet,velocity, freemarker and etc... In spring we can take java classes/spring beans as views by extending them from AbstractTransactionManagementView classes (like AbstractPdfView, AbstractExcelView.....) To render response as pdf, excel and etc docs.

→ In spring view is an abstract entity so we can take anything as view.

ViewResolver gives "View obj" (View(i) implementation class object) and DispatcherServlet calls render(-) on the object. This render(-) method passes the control to physical view Files/classes (like the jsp or above discussed classes)

Example app on BeanNameViewResolver and java classes as views:



we can use poi api support to render response as ~~pdf~~ excel doc
 we can use iText api support to render response as ~~Excel~~ pdf doc
 + poi is one 3rd party given by apache to render the response as Excel sheet

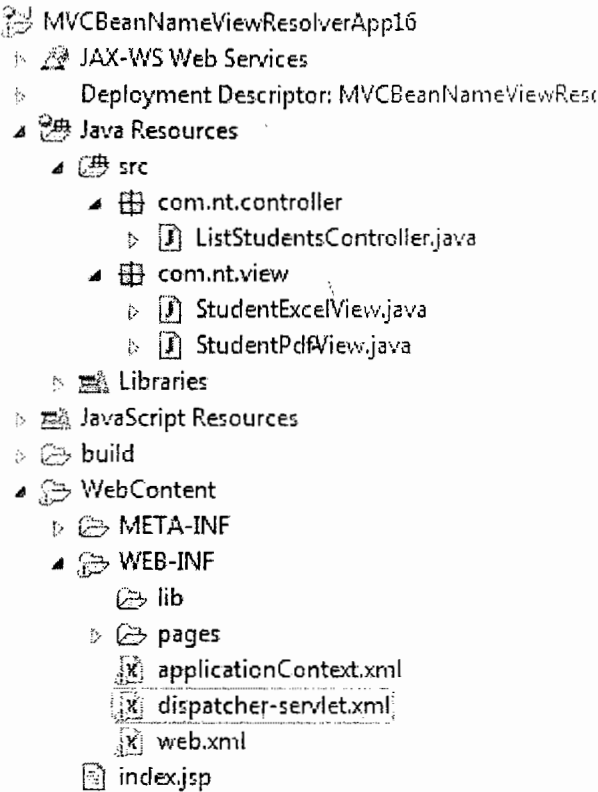
8/2/16
completed

Diagram:

ListStudentsController.java

```

package com.nt.controller;
import java.util.ArrayList;
import java.util.List;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.AbstractController;

public class ListStudentsController extends AbstractController {
    @Override
    protected ModelAndView handleRequestInternal(HttpServletRequest req,
                                                    HttpServletResponse res) throws Exception {
        List<String> list=new ArrayList<String>();
        list.add("raja"); list.add("ravi"); list.add("jani");
        if(req.getParameter("report").equals("pdf"))
        {
            return new ModelAndView("pdfView","stList",list);
        }
        else
        {
            return new ModelAndView("excelView","stList",list);
        }
    }
}
  
```

StudentExcelView.java

```
package com.nt.view;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.apache.poi.hsspringframework.usermodel.HSSFPRINGFRAMEWORKRichTextString;
import org.apache.poi.hsspringframework.usermodel.HSSFPRINGFRAMEWORKSheet;
import org.apache.poi.hsspringframework.usermodel.HSSFPRINGFRAMEWORKWorkbook;
import org.springframework.web.servlet.view.document.AbstractExcelView;
public class StudentExcelView extends AbstractExcelView{
    @Override
    protected void buildExcelDocument(Map<String, Object> map,
        HSSFPRINGFRAMEWORKWorkbook book, HttpServletRequest req,
        HttpServletResponse res)
        throws Exception {
        List list =(ArrayList)map.get("stList");
        HSSFPRINGFRAMEWORKSheet sheet=book.createSheet("Sheet1");
        getCell(sheet,0,0).setCellValue(new HSSFPRINGFRAMEWORKRichTextString("Students
Names"));
        getCell(sheet,1,0).setCellValue(new
HSSFPRINGFRAMEWORKRichTextString(list.get(0)+" ");
        getCell(sheet,2,0).setCellValue(new
HSSFPRINGFRAMEWORKRichTextString(list.get(1)+" ");
        getCell(sheet,3,0).setCellValue(new
HSSFPRINGFRAMEWORKRichTextString(list.get(2)+" ");
    }
}
```

StudentPdfView.java

```
package com.nt.view;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.view.document.AbstractPdfView;
import com.lowagie.text.Document;
import com.lowagie.text.Paragraph;
import com.lowagie.text.Table;
import com.lowagie.text.pdf.PdfWriter;
public class StudentPdfView extends AbstractPdfView {
    @Override
    protected void buildPdfDocument(12Map<String, Object> map, Document doc,
        PdfWriter writer, HttpServletRequest req, HttpServletResponse res)
        throws Exception {
```



```
//read Model obj attribute value
List list =(ArrayList)map.get("stList");
Paragraph p = new Paragraph("Student details");
p.setAlignment("center");
doc.add(p);
Table t = new Table(1);
t.addCell(list.get(0)+" ");
t.addCell(list.get(1)+" ");
t.addCell(list.get(2)+" ");
doc.add(t);
}
```

Index.jsp

```
<jsp:forward page="home.htm"/>
```

web.xml

```
<web-app>
```

```
<servlet>
```

```
<servlet-name>dispatcher</servlet-name>
```

```
<servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
```

```
<load-on-startup>2</load-on-startup>
```

```
</servlet>
```

```
<servlet-mapping>
```

```
<servlet-name>dispatcher</servlet-name>
```

```
<url-pattern>*.htm</url-pattern>
```

```
</servlet-mapping>
```

```
<listener>
```

```
<listener-class>org.springframework.web.context.ContextLoaderListener
```

```
</listener-class>
```

```
</listener>
```

```
<context-param>
```

```
<param-name>contextConfigLocation</param-name>
```

```
<param-value>/WEB-INF/applicationContext.xml</param-value>
```

```
</context-param>
```

```
</web-app>
```

dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
```

```
<bean id="surl"
```

```
class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
```

```
<property name="mappings">
```

```
<props>
```

```
<prop key="home.htm">pvc1</prop>
```

```
<prop key="listStudents.htm">lsc</prop>
```

```
</props>
```

```
</property>
```

```

</bean>
<bean id="lsc" class="com.nt.controller.ListStudentsController"/>
<bean id="pvc1" class="
4 org.springframework.web.servlet.mvc.ParameterizableViewController">
  <property name="viewName" value="home"/>
</bean>
<bean id="excelView" class="com.nt.view.StudentExcelView"/>
<bean id="pdfView" class="com.nt.view.StudentPdfView"/>
<bean id="bnvr" class="org.springframework.web.servlet.view.BeanNameViewResolver"/>
<bean id="vr" class="org.springframework.web.servlet.view.InternalResource
5 ViewResolver">
  <property name="prefix" value="/WEB-INF/pages/">
  <property name="suffix" value=".jsp"/>
</bean>
</beans>
applicationContext.xml

```

```

<beans
xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:util="http://www.springframework.org/schema/util"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
</beans>

```

Home.jsp

```

<b><center> Home page</center></b>
<br>
<a href="listStudents.htm?report=pdf">Student Report as Pdf</a>
<br>
<br>
<a href="listStudents.htm?report=excel">Student Report as Excel</a>

```

Jars:

- ✓ MVC library
- ✓ Standard.jar
- ✓ Jstl.jar
- ✓ iText-2.0.7.jar (---->spring 2.5)---->higher version for spring 4
- ✓ Poi-3.0.1.jar

i18n (Internationalization)

Making our application working for different locales by rendering labels in different languages is called enabling i18n on the application. This improves customer no of customer/visitors to the application. For this we need to take multiple properties files for multiple locales on 1 per basis like Welcome.properties, welcome_for_FR.properties, Welcome_zh_cn.properties and etc... locale basis.

- To enable i18n on spring mvc application we need to configure LocaleResolver, LocalChangeInterceptor classes as beans and we need to pass interceptor reference to link interceptor with controller class.
 - SessionLocaleResolver is implementation class of LocaleResolver interface that allows to configure locale explicitly otherwise fallbacks to default locale cfg file accept-language header value.
- ```

<bean id="localeResolver" class="org.springframework.web.servlet.i18n.SessionLocaleResolver">
 <property name="defaultLocale" value="en"/>

```

```
</bean>
```

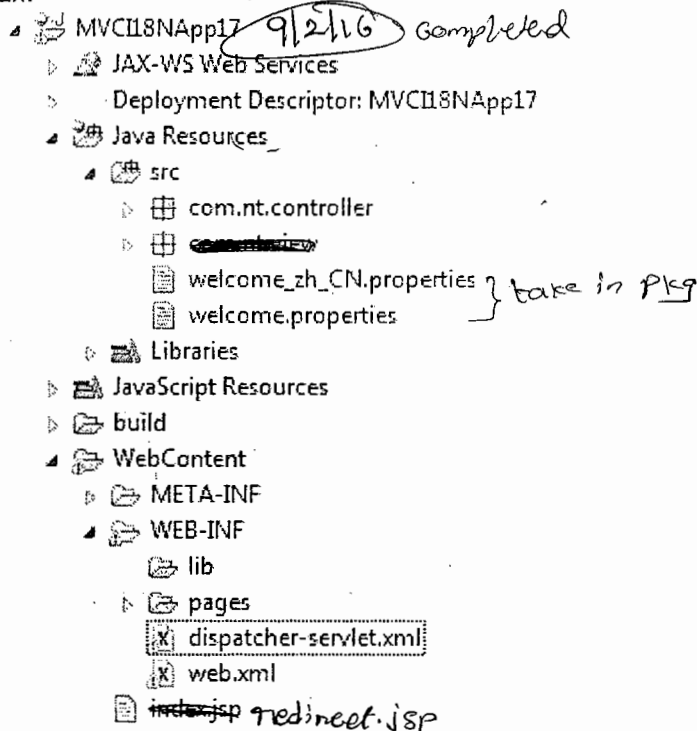
- ✶ LocaleChangeInterceptor allows to change locale for every request with respect to controller based on the additional request param that is configured.

```
<bean id="lci" class="org.springframework.web.servlet.i18n.LocaleChangeInterceptor">
 <property name="paramName" value="language"/>
</bean>
```

Based on the value of language request parameter the locale will be changed for every request by overriding default locale.

- ➔ Based on this locale properties file will be picked up to render the label.
- ➔ While preparing multiple properties files in the application of internationalization, all properties springframework files must have same keys and different labels specific to each locale.

Ex:



#### welcome.properties

my.un= Enter UserName

my.pwd=Enter Password

#### welcome\_zh\_CN.properties

my.un=\u0917\u094a\u092a\u093f\u0905\u0928\u094d

my.pwd=PICHACHU

#### WelcomeController.java

```
package com.nt.controller;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.AbstractController;
public class WelcomeController extends AbstractController{
 @Override
 protected ModelAndView handleRequestInternal(HttpServletRequest request,
 HttpServletResponse response) throws Exception {
```

```
ModelAndView model = new ModelAndView("Register");
```

```
return model;
```

9/12/16  
} }  
}  
  
Redirect.jsp (2)

```
<jsp:forward page="welcome.htm"/>
```

web.xml (3)

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xmlns:web="http://xmlns.jcp.org/xml/ns/javaee"
```

```
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
```

```
http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">
```

```
<servlet>
```

```
<servlet-name>dispatcher</servlet-name>
```

```
<servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
```

```
<load-on-startup>1</load-on-startup>
```

```
</servlet>
```

```
<servlet-mapping>
```

```
<servlet-name>dispatcher</servlet-name>
```

```
<url-pattern>*.htm</url-pattern>
```

```
</servlet-mapping>
```

```
<welcome-file-list>
```

```
<welcome-file>redirect.jsp</welcome-file>
```

```
</welcome-file-list>
```

```
</web-app>
```

Dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```
http://www.springframework.org/schema/beans/spring-beans-2.5.xsd">
```

```
<bean id="localeResolver"
```

```
class="org.springframework.web.servlet.i18n.SessionLocaleResolver">
```

```
<property name="defaultLocale" value="en" />
```

```
</bean>
```

```
<bean id="localeChangeInterceptor"
```

```
class="org.springframework.web.servlet.i18n.LocaleChangeInterceptor">
```

```
<property name="paramName" value="language" />
```

```
</bean>
```

```
<!-- if url=/welcome, it will call WelcomeController -->
```

```
<bean class="org.springframework.web.servlet.mvc.support.Controller
```

```
ClassNameHandlerMapping">
```

```
<property name="interceptors">
```

```
<list>
```

```
<ref bean="localeChangeInterceptor" />
```

```
</list>
```

```
</property>
```

we req having language = en-CN as the additional req

Naresh i Technologies, Opp. Satyam Theatre, Ameerpet, Hyd, Ph: 040-23746666, [www.fb.com/nareshit](http://www.fb.com/nareshit) ::348::  
parameter value, the same becomes the response locale.

u can take simple URL handler mapping

since the req doesn't contain additional req parameter language it will take default locale 'en' as configured in SessionLocaleResolver

```

</bean>
9/2/16 <!-- Register the bean -->
<bean class="com.nat.controller.WelcomeController"/> (14) (17)
<!-- Register the welcome.properties -->
<bean id="messageSource"
class="org.springframework.context.support.ResourceBundleMessageSource">
 <property name="basename" value="welcome" />
</bean>
<bean id="viewResolver"
class="org.springframework.web.servlet.view.InternalResourceViewResolver"> (16) (20)
 <property name="prefix">
 <value>/WEB-INF/pages/</value>
 </property>
 <property name="suffix">
 <value>.jsp</value>
 </property>
</bean></beans>

```

Register.jsp (11) (21)

```

<%@taglib prefix="spring" uri="http://www.springframework.org/tags" %>
<%@page isELIgnored="false" %>
Registration Page

Language : English|

```

```

 Chinese

```

```

<h3>

```

```

 <spring:message code="my.un" text="default text" />
 <input type="text" name="userName">

 <spring:message code="my.pwd" text="default text" />
 <input type="text" name="password">


```

```

</h3>

```

```

Selected locale: ${pageContext.response.locale}

```

```

</body>

```

```

</html>

```

keys in the properties file  
 Since there is no properties file for locale "en", the base property file welcome.properties will be utilized to render the labels.  
 Based on locale zh-CN, the messages will be collected from welcome.zh-CN.properties

### \*Annotation Driven Spring MVC: (9/2/16) Introduction

In xml configuration based programmatic approach of spring MVC, we develop controller class by implementing spring specific interfaces or by extending spring specific classes.

→ This kills non-invasive behavior of spring.

→ It also makes the programmers to implement or override fixed methods with fixed signatures where programmer will be forced to place unnecessarily parameters like req, res, params even though it is not using them. <sup>and</sup> same thing can be said about return type of methods.

⇒ To develop controller classes as pojo classes having flexibility of placing user-defined methods with user-defined signatures we need to go for Annotations driven spring mvc. These annotations are introduced from spring 2.5

→ All new projects and in the migration projects annotations driven spring mvc programming is required.

Two important annotations are

@Controller -----> configure java class/spring mvc controller class and spring bean.

@RequestMapping----> To map incoming request urls with the methods of controller class based the virtual path of request url.

While working with annotations based spring mvc controllers, we can place flexible methods having flexible parameter types or flexible return types.

```
Ex: @Controller
public class MyController
{
 @RequestMapping ("/home.htm")
 public <> <method name>()
 {

 }
}
```

#### Important possible param types:

HttpServletRequest

HttpServletRequest

HttpSession

Command class

@ModelAttribute

BindResults/Errors, Map, Model, ModelAndView and etc.....

(lots of param supporting web services)

#### Important possible return types:

Void, String, ModelAndView, Model,

ModelMap, View and etc.....

(lots of other return types supporting webservice)

#### Note:

In spring mvc only controller classes can be configured with annotations and the remaining compos like HandlerMappings, ViewResolver and etc must be configure through xmlfile. (According to 3.2, 4.1.0).

#### AbstractController (Using annotation) :

Source page: No form submission

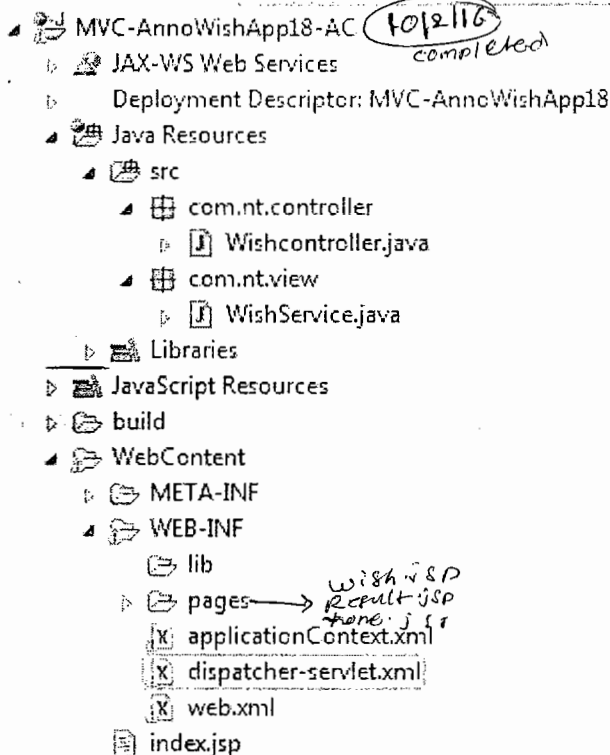
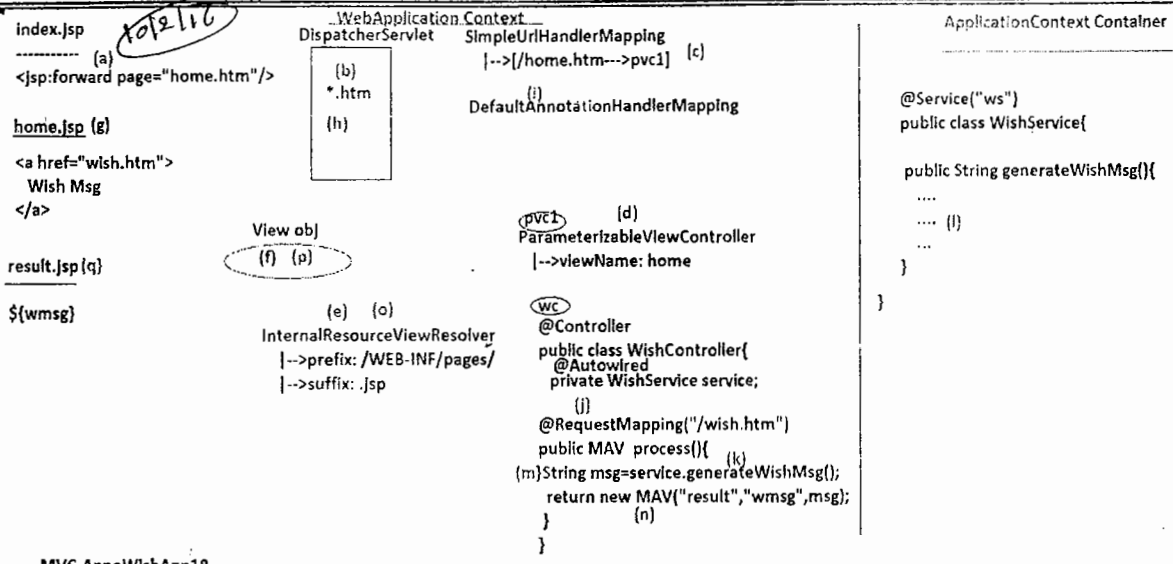
Dest pag: Static/dynamic form page with request processing

Note: we still use built-in forwarding controller like ParameterizableViewController using xml configure"

Diagram: upto spring 2.5, we can work with only xml based sp

⇒ from 2.5, we can develop spring mvc by mixing xml with Annotations (from 2.5, 3.x all versions, 4.x initial versions)

⇒ From spring 4.2, we can develop MVC by elementing all xml files including web.xml (every configuration through Annotation)



Jars: same +AOP+mvc+javainject.jar

### WishService.java

```
package com.nt.service;
import java.util.Calendar;
import javax.inject.Named;
@Named("ws")
public class WishService {
 public String generateWishMsg(){
 // generate wish Msg based on current hour the day
 Calendar cl=Calendar.getInstance();
 int hour=cl.get(Calendar.HOUR_OF_DAY);
 if(hour<=12)
```

10/11/16

```

 return "Good morning";
 else if(hour<=16)
 return "Good After noon";
 else if (hour<=20)
 return "Good Evening";
 else
 return "Good Night";
 } //method
} //class

```

**WishController.java**

```

package com.nt.controller;
import javax.annotation.Resource;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;
import com.nt.service.WishService;
@Controller
public class WishController {

 @Resource It's not spring specific
 private WishService service;

 /* @RequestMapping("/wish.htm")
 public ModelAndView process(){
 //use service class
 String msg=service.generateWishMsg();

 return new ModelAndView("result","wmsg",msg);
 } //process */

 /* @RequestMapping("/wish.htm")
 public String process(Model model){

 //use service class
 String msg=service.generateWishMsg();

 // add data to mode
 model.addAttribute("wmsg",msg); attribute
 return "result"; attribute name
 } */

 /* @RequestMapping("/wish.htm")
 public String process(ModelMap map){ It's good (non-invasive if m)

 //use service class
 String msg=service.generateWishMsg();
 }

```



```

 map.addAttribute("wmsg", msg);
 return "result";
 }*/

 /* @RequestMapping("/wish.htm")
 public String process(Map<String,Object> map){

 //use service class
 String msg=service.generateWishMsg();

 map.put("wmsg",msg);
 return "result";
 }*/

```

```

 @RequestMapping("/wish.htm")
 public void process(Model model){
 //Use service class
 String msg=service.generateWishMsg();

 model.addAttribute("wmsg",msg);
 }
}

```

#### Index.jsp

```
<jsp:forward page="home.htm"/>
```

#### web.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:web="http://xmlns.jcp.org/xml/ns/javaee">
 <welcome-file-list>
 <welcome-file>index.jsp</welcome-file>
 </welcome-file-list>
 <servlet>
 <servlet-name>dispatcher</servlet-name>
 <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
 <load-on-startup>2</load-on-startup>
 </servlet>
 <servlet-mapping>
 <servlet-name>dispatcher</servlet-name>
 <url-pattern>*.htm</url-pattern>
 </servlet-mapping>
 <listener>
 <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
 </listener>
 <context-param>
 <param-name>contextConfigLocation</param-name>
 <param-value>/WEB-INF/applicationContext.xml</param-value>
 </context-param>
</web-app>

```

**Dispatcher-servlet.xml**

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
 <bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
 <property name="mappings">
 <props>
 <prop key="/home.htm">pvc1</prop>
 </props>
 </property>
 </bean>
 <bean id="pvc1"
 class="org.springframework.web.servlet.mvc.ParameterizableViewController">
 <property name="viewName" value="home"/>
 </bean>
 <bean class="org.springframework.web.servlet.mvc.annotation.DefaultAnnotation
HandlerMapping" />
 <!-- <bean class="org.springframework.web.servlet.view.DefaultRequestTo
ViewNameTranslator" /> --> not required in configuration in xml file.
 <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
 <property name="prefix" value="/WEB-INF/pages/" />
 <property name="suffix" value=".jsp" />
 </bean>
 <context:component-scan base-package="com.nt.controller"/>
</beans>

```

**applicationContext.xml**

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
 <context:component-scan base-package="com.nt.service"/>
</beans>

```

**home.jsp**

```

<h1><center> Welcome to Annotations Based Spring MVC</center></h1>
 wish Msg

```

**Result.jsp**

```

<center><h1> Result Page</h1> </center>

Wish Message : ${wmsg}

 home

```

Wish.jsp

<b> wish.jsp</b>

Result is : \${wmsg}

DefaultRequestToViewNameTranslator:-

If controller class handler() (annotated with @RequestMapping) returntype is void (i.e not returning logical view name) then DispatcherServlet takes request uri (virtual path) of current request and translates into view name by trimming slashes and extension based on "org.springframework.servlet.view.RequestToViewNameTranslator"

In controller class:

```
@RequestMapping ("/wish.htm")
public void process (Model model)
{
.....
.....
}
```

→ if the request url is <http://localhost:3030/WishApp/Wish.htm> then "wish" will be taken as logical view name and wish.jsp will be taken as view Resource if the resolver is InternalResourceViewResolver. For this DispatcherServlet internally uses. DefaultRequestToViewNameTranslator. This translator strips/removes leading and trailing slashes and also remove extension from Request uri (makes "/home.htm" as "home") to use the result as logical viewname.

Annotation based AbstractCommandController:

Source Page: 1012/116

Form page with form submission (command class required)

No serverside form validation

No form handling

Static/dynamic page through Request processing.

→ Use @ModelAttribute base param in handler method of controller class to specify the command class and its id to perform request wrapping (writing form data into command class object)

Diagram: meet

Annotation based AbstractCommandController

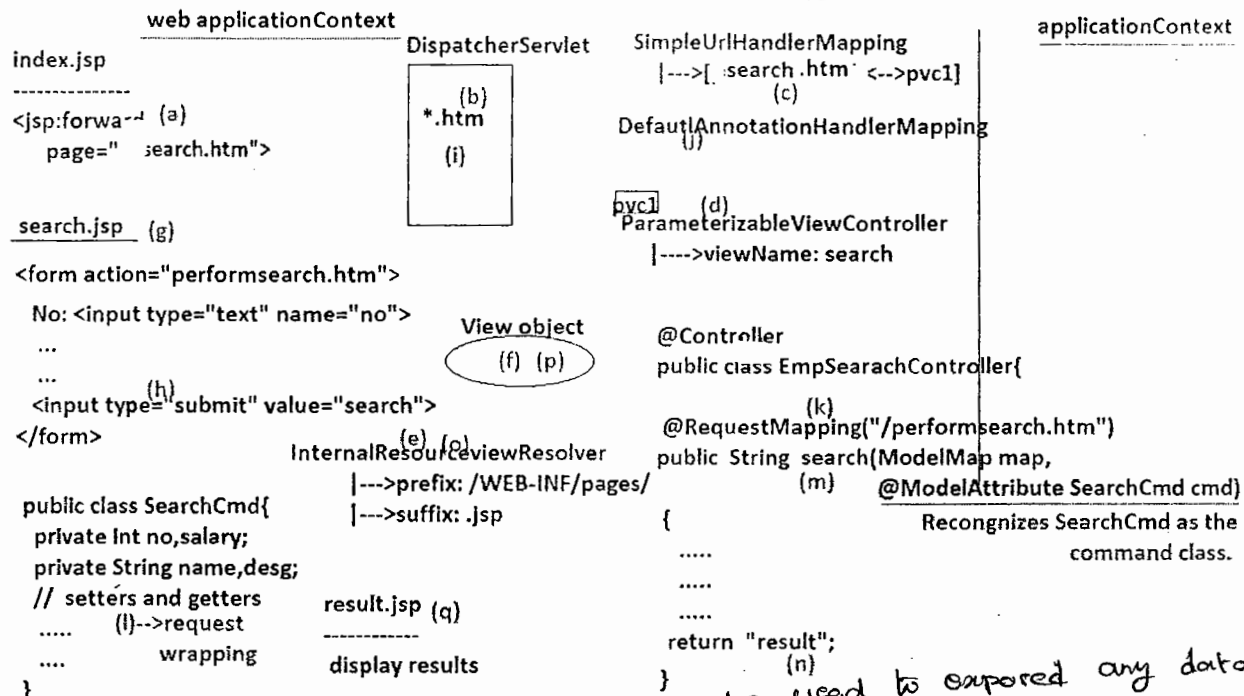
Source Page:

- > Form page with form submission (Command class is required)
- > No Server Side form validations
- > No Form handling

Dest page:

- > Static/Dynamic page through request processing

Use @ModelAttribute based param in handler method of controller class to specify the Command class and its obj to perform request wrapping (Writing form data into Command class obj)



ModelAttribute is multipurpose annotation. It can be used to exposed any data to web view, to make parameter of handler(-) method as command class, to place the return value of handler(-) method as modelAttribute and -- etc

- 11/2/16  
completed
- ▶ MVC-AnnoEmpSearchApp19-ACC
    - ▶ JAX-WS Web Services
    - ▶ Deployment Descriptor: MVC-AnnoEmpSearchApp19-ACC.xml
    - ▶ Java Resources
      - ▶ src
        - ▶ com.nt.command
          - ▶ SearchCmd.java
        - ▶ com.nt.controller
          - ▶ EmpSearchController.java
        - ▶ com.nt.dto
          - ▶ EmployeeDTO.java
      - ▶ Libraries
      - ▶ JavaScript Resources
      - ▶ build
      - ▶ WebContent
        - ▶ META-INF
        - ▶ WEB-INF
          - ▶ lib
          - ▶ pages
          - ▶ applicationContext.xml
          - ▶ dispatcher-servlet.xml
          - ▶ web.xml
        - ▶ index.jsp

### SearchCmd.java

```
package com.nt.command;
public class SearchCmd {
 private int no,salary;
 private String name,desg;
 public int getNo() {
 return no;
 }
 public void setNo(int no) {
 this.no = no;
 }
 public int getSalary() {
 return salary;
 }
 public void setSalary(int salary) {
 this.salary = salary;
 }
 public String getName() {
 return name;
 }
 public void setName(String name) {
 this.name = name;
 }
 public String getDesg() {
 return desg;
 }
 public void setDesg(String desg) {
```

```
 this.desg = desg;
}
```

### EmployeeDTO.java

```
package com.nt.dto;
import java.io.Serializable;
public class EmployeeDTO implements Serializable {
 private int no,salary;
 private String name,desg;
 public EmployeeDTO() {
 // TODO Auto-generated constructor stub
 }
 public EmployeeDTO(int no, int salary, String name, String desg) {
 this.no = no;
 this.salary = salary;
 this.name = name;
 this.desg = desg;
 }
 public int getNo() {
 return no;
 }
 public void setNo(int no) {
 this.no = no;
 }
 public int getSalary() {
 return salary;
 }
 public void setSalary(int salary) {
 this.salary = salary;
 }
 public String getName() {
 return name;
 }
 public void setName(String name) {
 this.name = name;
 }
 public String getDesg() {
 return desg;
 }
 public void setDesg(String desg) {
 this.desg = desg;
 }
}
```

### EmpSearchController.java

```
package com.nt.controller;
import java.util.ArrayList;
```

```

import java.util.List;
import org.springframework.stereotype.Controller;
import org.springframework.ui.ModelMap;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.RequestMapping;
import com.nt.command.SearchCmd;
import com.nt.dto.EmployeeDTO;
@Controller
public class EmpSearchController {
 @RequestMapping("/performsearch.htm")
 public String search(ModelMap map, @ModelAttribute SearchCmd cmd){
 System.out.println(cmd.getNo()+" "+cmd.getName());
 List<EmployeeDTO> emplist=new ArrayList<EmployeeDTO>();
 EmployeeDTO dto1=new EmployeeDTO(101,5000,"raja","clerk");
 EmployeeDTO dto2=new EmployeeDTO(102,6000,"ravi","clerk");
 emplist.add(dto1); emplist.add(dto2);
 // Add result to ModelMap
 map.addAttribute("searchResultsList",emplist);
 return "emplist";
 }
}

```

maxes SearchCmd as command class obj and performs Requestwrapping before the search() method is called

@Resource → for injecting one class in another class

### Index.jsp

```
<jsp:forward page="search.htm"/>
```

### web.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:web="http://xmlns.jcp.org/xml/ns/javaee">
 <welcome-file-list>
 <welcome-file>index.jsp</welcome-file>
 </welcome-file-list>
 <servlet>
 <servlet-name>dispatcher</servlet-name>
 <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
 <load-on-startup>2</load-on-startup>
 </servlet>
 <servlet-mapping>
 <servlet-name>dispatcher</servlet-name>
 <url-pattern>*.htm</url-pattern>
 </servlet-mapping>
 <listener>
 <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
 </listener>
 <context-param>
 <param-name>contextConfigLocation</param-name>
 <param-value>/WEB-INF/applicationContext.xml</param-value>
 </context-param>
</web-app>

```

**Dispatcher-servlet.xml**

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
 <bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
 <property name="mappings">
 <props>
 <prop key="/search.htm">pvc1</prop>
 </props>
 </property>
 </bean>
 <bean id="pvc1" class="org.springframework.web.servlet.mvc.Parameterizable
ViewController">
 <property name="viewName" value="search"/>
 </bean>
 <bean class="org.springframework.web.servlet.mvc.annotation.
DefaultAnnotationHandlerMapping"/>
 <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
 <property name="prefix" value="/WEB-INF/pages/">
 <property name="suffix" value=".jsp"/>
 </bean>
 <context:component-scan base-package="com.nt.controller"/>
</beans>

```

**applicationContext.xml**

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
 </beans>

```

**EmpList.jsp**

```

<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<center><h1>Search Results</h1></center>
<c:if test="${not empty searchResultsList}">
 <table>
 <tr>
 <th>Emp No</th>
 <th>Emp Name</th>
 <th>Salary</th>
 <th>Desg</th>
 </tr>
 <c:forEach items="${searchResultsList}" var="result">

```



```

<tr>
 <td><c:out value="{result.no}"/></td>
 <td><c:out value="{result.name}"/></td>
 <td><c:out value="{result.salary}"/></td>
 <td><c:out value="{result.desg}"/></td>
</tr>
</c:forEach>
</table>
</c:if>
<search.jsp>
 <h1><center> Emp Search Page</center></h1>

 <form action="performsearch.htm" method="post">
 No : <input type="text" name="no">

 Name : <input type="text" name="name">

 Salary : <input type="text" name="salary">

 Desg : <input type="text" name="desg">

 <input type="submit" value="search">
 </form>

```

### Annotation Based SimpleformController: 11/2/16

#### Source Page:

- Form page-->form submission
- Form validation (server side)
- Form handling

#### Destination Page:

- Static/Dynamic page that comes after request process

#### InitialPhase Request (reqmethod=GET) (wrt to aml)

- Create command object
- init Binder ()
- add command obj to model attribute
- referencedata() and etc...

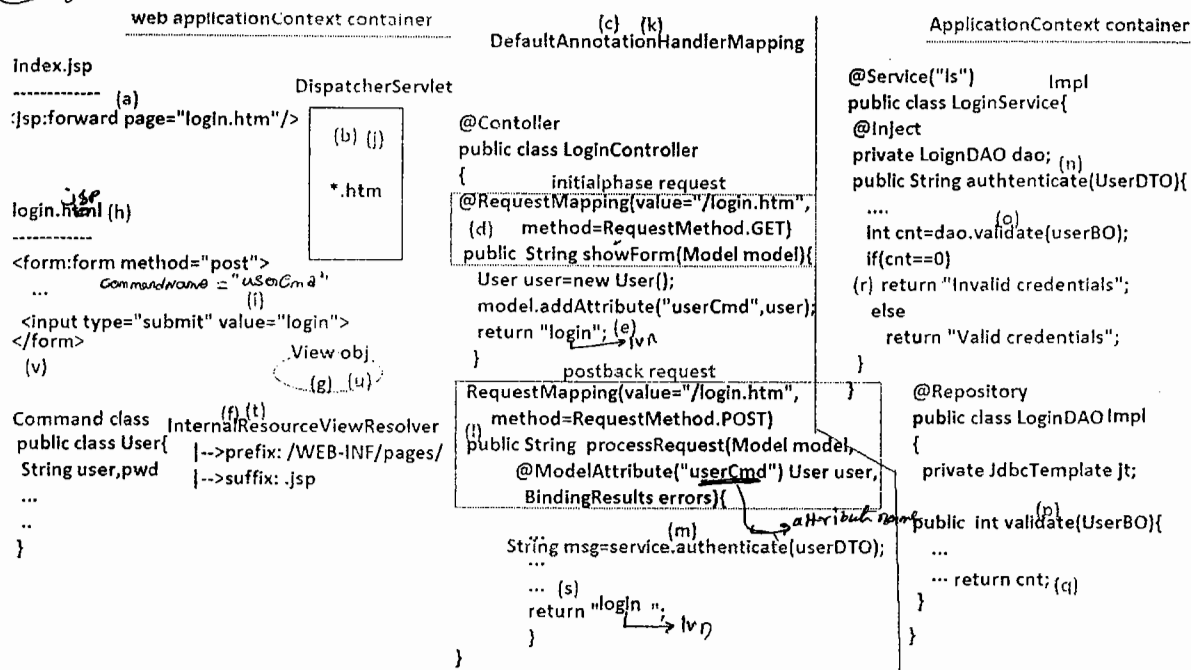
#### PostBack request (req method=POST) (wrt to aml)

- Create/locate command obj
- initBinder ()
- request wrapping
- BindException obj creation
- Validation support (-), validate (-)
- Referencedata
- onSubmit() and etc....

#### index.jsp

```
<jsp:forward Page="login.html">
```

## Diagram



### Sample Annotation based SimpleFormController class

#### DefaultAnnotationHandlerMapping

#### @Controller

#### public class LoginController

{

#### Sample code

#### Initial phase request (comment)

@RequestMapping (value="/login.html", method=RequestMethod.GET)

public String showForm(Model model){

User user=new User();

model.addAttribute("usercmd", user);

return "login";

}

@RequestMapping (value="/login.htm", method=RequestMethod.POST)

public String processRequest(Model model,@ModelAttribute ("usercmd") User user,

BindingResult errors){

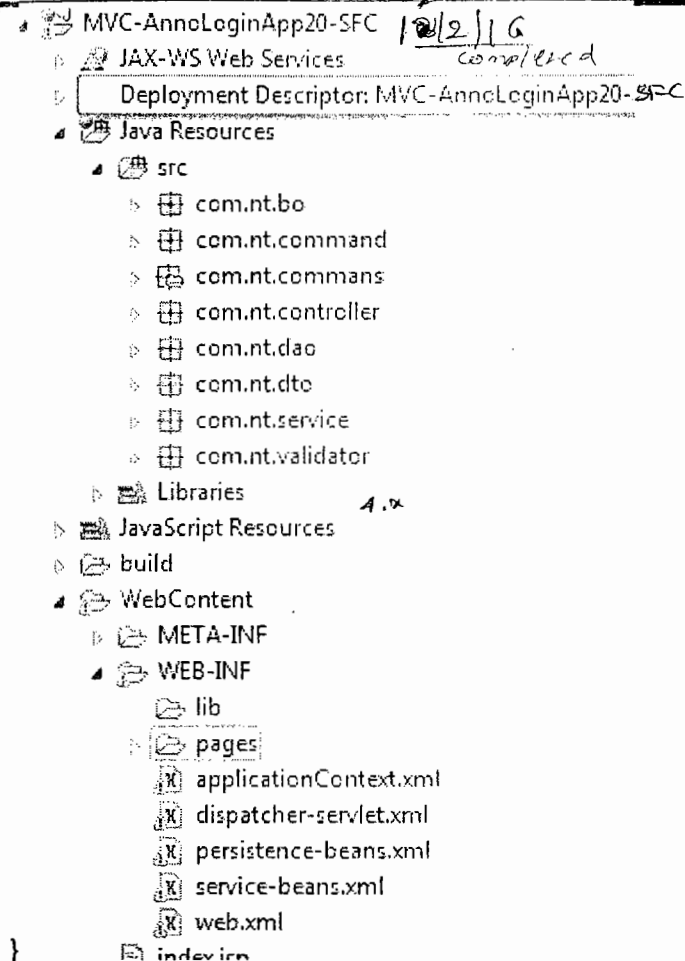
String msg=service.authenticateconfigure:userDTO);

.....

.....

return "login";

}

**UserBO.java**

```

package com.nt.bo;
public class UserBO {
 private String user,pwd;

 public String getUser() {
 return user;
 }
 public void setUser(String user) {
 this.user = user;
 }
 public String getPwd() {
 return pwd;
 }
 public void setPwd(String pwd) {
 this.pwd = pwd;
 }
}

```

**LoginDao.java**

```

package com.nt.dao;
import com.nt.bo.UserBO;
public interface LoginDao {

```

```
public int validate(UserBO ubo);
```

**LoginDaoImpl.java**

```
package com.nt.dao;
import javax.inject.Inject;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;
import com.nt.bo.UserBO;
@Repository
public class LoginDaoImpl implements LoginDao {
 private static final String AUTH_QRY="SELECT COUNT(*) FROM USERLIST WHERE UNAME=?
 AND PWD=? ";
 @Inject
 private JdbcTemplate jt;
 @Override
 public int validate(UserBO ubo) {
 int cnt=jt.queryForInt(AUTH_QRY, ubo.getUser(),ubo.getPwd());
 return cnt;
 }
}
```

**UserDTO.java**

```
package com.nt.dto;
import java.io.Serializable;
public class UserDTO implements Serializable {
 private String user,pwd;
 public String getUser() {
 return user;
 }
 public void setUser(String user) {
 this.user = user;
 }
 public String getPwd() {
 return pwd;
 }
 public void setPwd(String pwd) {
 this.pwd = pwd;
 }
}
```

**LoginService.java**

```
package com.nt.service;
import com.nt.dto.UserDTO;
public interface LoginService {
 public String authenticate(UserDTO udto);
}
```

**LoginServiceImpl.java**

```
package com.nt.service;
import javax.inject.Inject;
```

```
import org.springframework.stereotype.Service;
import com.nt.bo.UserBO; 12/2/16
import com.nt.dao.LoginDao;
import com.nt.dto.UserDTO;
@Service
public class LoginServiceImpl implements LoginService {
 @Inject
 private LoginDao dao;
 @Override
 public String authenticate(UserDTO udto) {
 //convert DTO obj to BO obj
 UserBO ubo=new UserBO();
 ubo.setUser(udto.getUser());
 ubo.setPwd(udto.getPwd());
 //Use DAO class
 int cnt=dao.validate(ubo);
 if(cnt==0)
 return "Invalid Credentials";
 else
 return "Valid Credentials";
 }
}
```

**LoginValidator.java**

```
package com.nt.validator;
import org.springframework.stereotype.Component;
import org.springframework.validation.Errors;
import org.springframework.validation.Validator;
import com.nt.command.User;
@Component
public class LoginValidator implements Validator {
 @Override
 public boolean supports(Class<?> clazz) {
 return clazz.isAssignableFrom(User.class);
 }
 @Override
 public void validate(Object cmd, Errors errors) {
 //type casting
 User ucmd=(User)cmd;
 if(ucmd.getUser()==null || ucmd.getUser().equals("") ||
 ucmd.getUser().length()==0){
 errors.rejectValue("user","user.required");
 }

 if(ucmd.getPwd()==null || ucmd.getPwd().equals("") ||
 ucmd.getPwd().length()==0){
 errors.rejectValue("pwd","pwd.required");
 }
 }
}
```

**Errors.properties**

user.required=Username is mandatory

pwd.required=Password is mandatory

**User.java**

```
package com.nt.command;
import java.util.Date;
public class User {
 private String user,pwd,dmn;
 private Date dor;
 public String getUser() {
 return user;
 }
 public void setUser(String user) {
 this.user = user;
 }
 public String getPwd() {
 return pwd;
 }
 public void setPwd(String pwd) {
 this.pwd = pwd;
 }
 public String getDmn() {
 return dmn;
 }
 public void setDmn(String dmn) {
 this.dmn = dmn;
 }
 public Date getDor() {
 return dor;
 }
 public void setDor(Date dor) {
 this.dor = dor;
 }
}
```

**LoginController.java**

```
package com.nt.controller;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import javax.inject.Inject;
import org.springframework.beans.propertyeditors.CustomDateEditor;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.WebDataBinder;
import org.springframework.web.bind.annotation.InitBinder;
```

```

import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import com.nt.command.User;
import com.nt.dto.UserDTO;
import com.nt.service.LoginService;
import com.nt.validator.LoginValidator;
@Controller
public class LoginController {
 @Inject
 private LoginService service;
 @Inject
 private LoginValidator validator;
 @RequestMapping(value="/login.htm",method=RequestMethod.GET)
 public String showForm(Model model){
 //create command obj and keep it as model attribute
 User user=new User();
 model.addAttribute("userCmd",user);
 //return lvn
 return "login";
 }
 @RequestMapping(value="/login.htm",method=RequestMethod.POST)
 public String processForm(Model model,@ModelAttribute("userCmd")User
 cmd,BindingResult errors){

```

Alternate to Binding exception

```

//convert Command obj to DTO obj
UserDTO udto=new UserDTO();
udto.setUser(cmd.getUser());
udto.setPwd(cmd.getPwd());

```

```

//use validator
if(validator.supports(User.class)){
 validator.validate(cmd, errors);
 if(errors.hasErrors()){
 return "login";
 }
}

```

```

//use service class
String result=service.authenticate(udto);
model.addAttribute("result",result);
//return lvn
return "login";
}

```

```

@ModelAttribute("domains") // alternate for referenceData
public List<String> populateDomains(){
 List<String> domainsList =new ArrayList<String>();
 domainsList.add("Gmail.com");
 domainsList.add("yahoo.com");
 domainsList.add("rediff.com");
}

```

return domainsList;

12/12/16  
13/12/16  
@InitBinder

```
public void myInitBinder(WebDataBinder binder){
 SimpleDateFormat sdf=new SimpleDateFormat("dd-MM-yyyy");
 binder.registerCustomEditor(Date.class,
 new CustomDateEditor(sdf,true));
}
```

Index.jsp

<jsp:forward page="login.htm"/>

web.xml

<web-app>

<welcome-file-list>

<welcome-file>index.jsp</welcome-file>

</welcome-file-list>

<servlet>

<servlet-name>dispatcher</servlet-name>

<servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>

<load-on-startup>2</load-on-startup>

</servlet>

<servlet-mapping>

<servlet-name>dispatcher</servlet-name>

<url-pattern>\*.htm</url-pattern>

</servlet-mapping>

<listener>

<listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>

</listener>

<context-param>

<param-name>contextConfigLocation</param-name>

<param-value>/WEB-INF/applicationContext.xml</param-value>

</context-param>

</web-app>

service-beans.xml

<beans xmlns="http://www.springframework.org/schema/beans"

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xmlns:context="http://www.springframework.org/schema/context"

xsi:schemaLocation="http://www.springframework.org/schema/beans

http://www.springframework.org/schema/beans/spring-beans.xsd

http://www.springframework.org/schema/context

http://www.springframework.org/schema/context/spring-context.xsd">

<context:component-scan base-package="com.nt.service"/>

</beans>

persistence-beans.xml

<beans xmlns="http://www.springframework.org/schema/beans"

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xmlns:context="http://www.springframework.org/schema/context"

xsi:schemaLocation="http://www.springframework.org/schema/beans



<http://www.springframework.org/schema/beans/spring-beans.xsd>  
<http://www.springframework.org/schema/context>  
<http://www.springframework.org/schema/context/spring-context.xsd>>  
 <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">  
   <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>  
   <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>  
   <property name="username" value="scott"/>  
   <property name="password" value="tiger"/>  
 </bean>  
 <bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">  
   <property name="dataSource" ref="dataSource"/>  
 </bean>  
 <context:component-scan base-package="com.nt.dao"/>  
 </beans>

### dispatcher-servlet.xml

<beans xmlns="http://www.springframework.org/schema/beans"  
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
 xmlns:context="http://www.springframework.org/schema/context"  
 xsi:schemaLocation="http://www.springframework.org/schema/beans  
<http://www.springframework.org/schema/beans/spring-beans.xsd>  
<http://www.springframework.org/schema/context>  
<http://www.springframework.org/schema/context/spring-context.xsd>>  
 <bean class="org.springframework.web.servlet.mvc.annotation.Default  
 AnnotationHandlerMapping"/>  
 <bean id="messageSource" class="org.springframework.context.support.ResourceBundle  
 MessageSource">  
   <property name="basename" value="com/nt/commons/errors"/>  
 </bean>  
 <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">  
   <property name="prefix" value="/WEB-INF/pages/">  
   <property name="suffix" value=".jsp"/>  
 </bean>  
 <context:component-scan base-package="com.nt.controller,com.nt.validator"/>  
 </beans>

### applicationContext.xml

<beans xmlns="http://www.springframework.org/schema/beans"  
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
 xmlns:context="http://www.springframework.org/schema/context"  
 xsi:schemaLocation="http://www.springframework.org/schema/beans  
<http://www.springframework.org/schema/beans/spring-beans.xsd>  
<http://www.springframework.org/schema/context>  
<http://www.springframework.org/schema/context/spring-context.xsd>>  
 <import resource="service-beans.xml"/>  
 <import resource="persistence-beans.xml"/>  
 </beans>

login.jsp (here we have to write in spring supplied JSP, because then only it recognizes command classes)  
 <%@taglib uri="http://www.springframework.org/tags/form" prefix="form"%>

```

<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<h1><center> Login Page</center></h1>
<form:form method="post" commandName="userCmd">
 <form:errors path="*" />

 Username: <form:input path="user" />

 Password : <form:password path="pwd" />

 Domains: <form:checkboxes items="${domains}" path="dmn" />

 Date of Registration: <form:input path="dor" /> (dd-MM-yyyy)

 <input type="submit" value="Login" />

</form:form>

<c:if test="${result ne null}">
 <c:out value="${result}" />
</c:if>
<c:out value="${userCmd.dor}" />

```

12/11/16  
 → To perform form validation while working with SimpleFormController with Annotations

1. Prepare properties file having form validation error messages.
2. Configure properties file.
3. Develop validator class implementing Validator (i) and overriding supports(-), validate(-) method
4. Inject validator(-) method in the handler supports(-), validate(-) methods in the handler method, that can process postback request.
5. Place <form:errors> tag in form page to display form validation errors.

→ While working with annotation based SimpleFormController we can use @ModelAttribute annotation method as alternate to referenceData() of xml base SimpleFormController. This holds data that has to be displayed in the form page when we want to configure as checkbox.

In controller class:

```

@ModelAttribute("domains")
public List<String> populateDomains()
{
 List<String> domainList = new ArrayList<String>();
 domainList.add("Gmail.com");
 domainList.add("Yahoo.com");
 domainList.add("rediff.com");
 return domainList;
}

```

→ as additional data as part of initial phase req and post-back req. with validations.

In form Page:

```

Domains : <form:checkboxes items="${domains}" path="dma" />

```

→ In order to register property editors that are required to convert form supplied String values to various types of command class properties we override TnitBinder() in our controller class while working with xml configured SimpleFormController. We can bring the same affect by using @InitBinder while using annotations driven simple form controller.

Ex :

In Controller Class

@InitBinder

```

public void MyInitBinder(WebDataBinder binder)
{

```

```

 SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy");

```

```
Binder.registerCustomEditor(Date.class, new CustomDateEditor(sdf, true)); }
```

### File Uploading and file downloading:

→ The process of selecting file from client machine file system and sending that file to server system file system is called file uploading and reverse is called File downloading.

While developing matrimony application, job portal, video sharing apps the file uploading and downloading are quietly required.

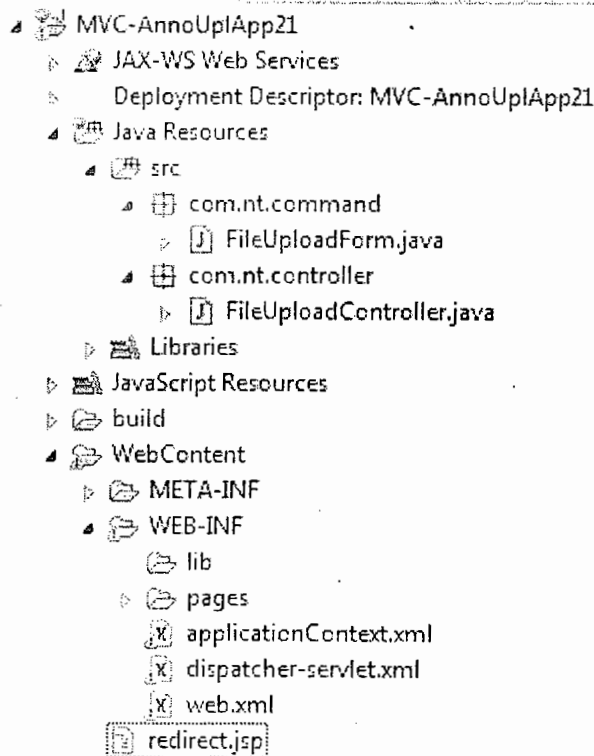
→ Spring MVC gives built-in support for fileuploading.

We need to configure "CommonsMultipartResolver" in dispatcher-servlet.xml to make controller class ready to resolve multi MIME TYPES/parts based form data (form data having uploading files)

→ Using this class we can also configure properties like "maxUploadSize", "maxInMemorySize", "defaultEncoding" properties.

### Perform the following operations for file uploading:

1. Take file uploading component in form page and also specify "enctype" and request method as "POST"



commons-fileupload-1.1.1.jar } Jar: MVC jar, AOP jars  
commons-io-1.3.1.jar } download from internet

2. Redirect.jsp

```
<jsp:forward page="home.htm"/>
```

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<web-app xmlns:web="http://xmlns.jcp.org/xml/ns/javaee">
```

```
<welcome-file-list>
```

```
<welcome-file>redirect.jsp</welcome-file>
```

```
</welcome-file-list>
```

```
<servlet>
```

```
<servlet-name>dispatcher</servlet-name>
```

```
<servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
```

```
<load-on-startup>2</load-on-startup>
```

```
</servlet>
```

```

<servlet-mapping>
 <servlet-name>dispatcher</servlet-name>
 <url-pattern>*.htm</url-pattern>
</servlet-mapping>
<listener>
 <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
<context-param>
 <param-name>contextConfigLocation</param-name>
 <param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>
</web-app>

```

### dispatcher-servlet.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xmlns:context="http://www.springframework.org/schema/context"
 xsi:schemaLocation="http://www.springframework.org/schema/beans
 http://www.springframework.org/schema/beans/spring-beans.xsd
 http://www.springframework.org/schema/context
 http://www.springframework.org/schema/context/spring-context.xsd">
 <bean class="org.springframework.web.servlet.mvc.annotation.DefaultAnnotation
 HandlerMapping"/>
 <bean id="multipartResolver" class="org.springframework.web.multipart.commons.CommonsMultipartResolver"/>
 <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
 <property name="prefix" value="/WEB-INF/pages/" />
 <property name="suffix" value=".jsp" />
 </bean>
 <context:component-scan base-package="com.nt.controller"/>
</beans>

```

### applicationContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xmlns:context="http://www.springframework.org/schema/context"
 xsi:schemaLocation="http://www.springframework.org/schema/beans
 http://www.springframework.org/schema/beans/spring-beans.xsd">
</beans>

```

### file\_upload\_form.jsp

```

<%@ taglib prefix="form" uri="http://www.springframework.org/tags/form"%>

```

```

<h2>File Upload App</h2>

```

```

<form:form method="post" action="save.htm" modelAttribute="uploadForm"
 enctype="multipart/form-data">

```

Please select a file to upload : <input type="file" name="file" /><br/>

```

<input type="submit" value="upload" />

```

```

</form:form>

```

```

</body>

```

```

</html>

```

### file\_upload\_success.jsp

*Files uploaded successfully.*

### **FileUploadController.java**

```

package com.nt.controller;
import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.multipart.MultipartFile;
import com.nt.command.FileUploadForm;
@Controller
public class FileUploadController {
 @RequestMapping(value = "/home.htm",
 method = RequestMethod.GET)
 public String displayForm(Model model) {
 return "file_upload_form"; //returns the page name to ViewResolver ultimately
 }
 @RequestMapping(value = "/save.htm", method = RequestMethod.POST)
 public String upload(Model map, @ModelAttribute("uploadForm")
 FileUploadForm cmd) {
 File dir = new File("E:/springfileuploads/");
 //get Uploaded file
 MultipartFile file = cmd.getFile();
 String fileName = "";
 if (file != null) {
 fileName = file.getOriginalFilename();
 System.out.println("original file name="+fileName);
 //Handle file content - multipartFile.getInputStream()
 InputStream inputStream=null;
 OutputStream outputStream;
 try {
 inputStream = file.getInputStream();
 //If file is not present then only create a file and write the data into
 them
 if(!dir.exists()){
 dir.mkdir();
 }
 File newFile = new File("E:/springfileUploads/"+fileName);
 outputStream = new FileOutputStream(newFile);
 int read = 0;
 byte[] buffer = new byte[1024];
 while ((read = inputStream.read(buffer)) != -1) {

```

```

 outputStream.write(buffer, 0, read);
 }
 outputStream.close();
 inputStream.close();
} catch (IOException e) {
 System.out.println("Exception "+e);
}
}
map.addAttribute("files", fileName);
return "file_upload_success";
}
}

```

### FileUploadForm.java

```

package com.nt.command;
import org.springframework.web.multipart.MultipartFile;
public class FileUploadForm {
 //Can be used to hold the uploaded file
 private MultipartFile file;
 public MultipartFile getFile() {
 return file;
 }
 public void setFile(MultipartFile file) {
 this.file = file;
 }
}

```

*Handwritten notes:*  
 - **R** (circled)  
 - **P** (circled) → creates command class obj and writes uploaded file to a property whose name is file type is multipart

Take a command class having property of type "MultipartFile" to receive and hold uploaded file.

3. Configure "CommonsMultipartResolver" class as bean in dispatcher-servlet.xml ,

4. Write the stream based logic in controller to save the received in server machine file system.

### Downloading:

The process of getting the file belonging to server machine file system to client machine file system is called file downloading. By using "Content-Disposition" header we can give instruction to browser to display the generated output as downloadable or attachable file.

⇒ For example application on file uploading, refer app-18

We can use @RequestParam annotation to store the received request parameter into handler method parameter.

For example app on downloading refer app-19

- MVC-AnnoFileDownloadApp22 (14/2/16) *completed*
  - ▶ JAX-WS Web Services
  - ▶ Deployment Descriptor: MVC-AnnoFileDownload
  - ▶ Java Resources
    - ▶ src
      - ▶ com.nt.controller
        - ▶ FileController.java
    - ▶ Libraries
  - ▶ JavaScript Resources
  - ▶ build
  - ▶ WebContent
    - ▶ META-INF
    - ▶ WEB-INF
      - ▶ lib
      - ▶ pages
        - ▶ applicationContext.xml
        - ▶ dispatcher-servlet.xml
        - ▶ web.xml
      - ▶ redirect.jsp

mvcLib + jars : commons-fileupload-1.1.jar

### ① Index.jsp

<jsp:forward page="home.htm"/>

### web.xml

<?xml version="1.0" encoding="UTF-8"?>

<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
 xmlns:web="http://xmlns.jcp.org/xml/ns/javaee"  
 xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee  
 http://java.sun.com/xml/ns/javaee/web-app\_2\_5.xsd">

<servlet>

<servlet-name>dispatcher</servlet-name>

<servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>

<load-on-startup>1</load-on-startup>

</servlet>

<servlet-mapping>

<servlet-name>dispatcher</servlet-name>

<url-pattern>\*.htm</url-pattern>

</servlet-mapping>

<welcome-file-list>

<welcome-file>index.jsp</welcome-file>

</welcome-file-list>

</web-app>

### dispatcher-servlet.xml

<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"  
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
 xmlns:context="http://www.springframework.org/schema/context"  
 xmlns:mvc="http://www.springframework.org/schema/mvc"  
 xsi:schemaLocation="http://www.springframework.org/schema/beans

<http://www.springframework.org/schema/beans/spring-beans.xsd>

<http://www.springframework.org/schema/context>

⑩③④ → Since no handler mapping is configured the 'default AnnotationHandlerMapping'  
<http://www.springframework.org/schema/context/spring-context.xsd>

<http://www.springframework.org/schema/mvc>

<http://www.springframework.org/schema/mvc/spring-mvc.xsd>

<context:component-scan base-package="com.nat.controller"/><!--

package name where controllers are present -->

<context:annotation-config />

<!-- View Resolver, pointing to WEB-INF/jsp folder -->

<bean id="jspViewResolver"

class="org.springframework.web.servlet.view.InternalResourceViewResolver">

<property name="viewClass"

value="org.springframework.web.servlet.view.JstlView" />

<property name="prefix" value="/WEB-INF/pages/" />

<property name="suffix" value=".jsp" />

</bean>

</beans>

file\_download\_form.jsp ⑦

<%@page import="java.io.File" %>

<table border=1>

<tr><td>FileName</td><td>Download</td></tr>

<% String path = "E:/springfileuploads";

String files;

File folder = new File(path);

File[] listOfFiles = folder.listFiles();

for (int i = 0; i < listOfFiles.length; i++)

{

if (listOfFiles[i].isFile())

{

files = listOfFiles[i].getName();

%>

<tr><td><%= files %></td>

<td><a href="download.htm?file\_name=<%= files %>">Download</a></td></tr>

<% }

}

%>

</table>

</html>

**FileDownloadController.java**

package com.nat.controller;

import java.io.FileInputStream;

import java.io.IOException;

import java.io.InputStream;

import java.io.OutputStream;

import javax.servlet.http.HttpServletResponse;



```

import org.apache.commons.io.IOUtils;
import org.springframework.core.io.FileSystemResource;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RequestParam;
@Controller
public class FileDownloadController {
 @RequestMapping(value = "/home.htm")
 public String displayForm() {
 return "file_download_form"; //returns the page name to
 //ViewResolver ultimately
 }
 @RequestMapping(value = "/download.htm")
 public void download(@RequestParam("file_name") String fileName,
 // (1) stores the received request HttpServletResponse response) {
 // parameter value to handler parameter
 InputStream is=null;
 try {
 System.out.println("Requested File"+fileName+" response="+response);
 String filePath="E:/springfileuploads/"+fileName;
 // get your file as InputStream
 is=new FileInputStream(filePath);

 //Remove the below code, will dispatch the file in view mode like txt, bat file
 response.setHeader("Content-Disposition", "attachment; filename="
 +fileName); // Gives instruction to browser, to display the received content as downloadable
 IOUtils.copy(is, response.getOutputStream());
 response.flushBuffer(); // copies the content of downloadable file to the Response object.
 } catch (IOException ex) {
 //log.info("Error writing file to output stream. Filename was " + fileName + "");
 throw new RuntimeException("IOError writing file to output stream");
 }
 // This controller class gets the content of file to be downloaded through
 // inputstream and writes that content to Response object. And Response
 // object writes the content to browser, but browser displays content as

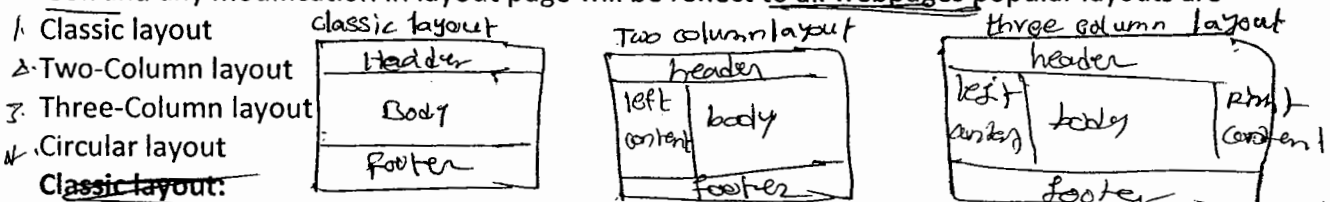
```

**SpringMVC with Tiles:** downloadable file bears "Content-Disposition" header.

A tile is a logical portion/part in a webpage like header tile, footer tile, body tile and etc.....

→ Apache tiles framework/plugin is given to construct webpages based on layout having layout control. This is like working with compositeView design pattern.

→ Layout control means all the web pages will be constructed based on single layout page for uniform look and any modification in layout page will be reflect to all webpages. popular layouts are



**Classic layout:** Spring gives built-in support to work with tiles.

2) For this we need to configure "TileConfigurer" specifying the tile definition file name(xml). The default file name is /WEB-INF/tiles.xml

→ Every tile definition specifies the tile values i.e each webpage based on layout page. We create tile definition on 1 per webpage <sup>by</sup> basic having logical name. we can use "TilesViewResolver" to display webpage based on each file definition.

### Procedure to develop spring MVC with Tiles application:

1) Design layout page for templating webpages and to maintain webpages having layout control. Develop files xml in WEB-INF folder having file definitions based on no of webpage in our website on 1 per <sup>web pages</sup> ~~file definition~~ basis. We can use "tile definition inheritance" concept here.

2) Configure "TilesConfiguration" class in dispatcher-servlet.xml file as spring bean specifying "tile definition configurator" file <sup>non filename</sup>

3) Configure "TilesViewResolver" as ViewResolver <sup>(or)</sup> take Filesview as view class for urlBasedViewResolver. Make a controller class handler methods to return "tile definition name" as logical view

14/2/16 MVC-AnnoTilesMultiActionApp23

completed

- ▶ JAX-WS Web Services
- ▶ Deployment Descriptor: MVC-AnnoTilesMultiAct
- ▶ Java Resources
  - ▶ src
    - ▶ com.nt.controller
      - ▶ HomeController.java
  - ▶ Libraries
  - ▶ JavaScript Resources
  - ▶ build
  - ▶ WebContent
    - ▶ META-INF
    - ▶ WEB-INF
      - ▶ lib
      - ▶ pages
        - ▶ dispatcher-servlet.xml
        - ▶ tiles.xml
        - ▶ web.xml
    - ▶ redirect.jsp

name.

Jar: MVC jar, AOP jars

Tiles-template-2.2.2.jar

Tiles-servlet, tiles-core, tiles-jsp, tiles-api,

814j-api, 814j-log4j12, jstl, antlr.jar

Commons-beanutils, commons-collections, commons-deigester, commons-fileupload, commons-10-jar. commons-io.jar, 814j-log4j12, jstl, jstl-api

### HomeController.java

```
package com.nat.controller;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.servlet.ModelAndView;
@Controller
public class HomeController {
 @RequestMapping("/demo")
 public ModelAndView hello() { page
 return new ModelAndView("hello", "message", "Body Content goes Here !");
 }
}
```

```

@RequestMapping("/link1")
public ModelAndView register() {
 return new ModelAndView("message", "Registration page");
}
@RequestMapping("/link2")
public ModelAndView login() {
 return new ModelAndView("message", "Login Page");
}
@RequestMapping("/link3")
public ModelAndView reset() {
 return new ModelAndView("message", "Login Page");
}
}

```

#### 4. Redirect.jsp

```
<jsp:forward page="demo.htm"/>
```

#### Web.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xmlns:web="http://xmlns.jcp.org/xml/ns/javaee"
 xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
 http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">
 <welcome-file-list>
 <welcome-file>redirect.jsp</welcome-file>
 </welcome-file-list>
 <servlet>
 <servlet-name>dispatcher</servlet-name>
 <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
 <load-on-startup>1</load-on-startup>
 </servlet>
 <servlet-mapping>
 <servlet-name>dispatcher</servlet-name>
 <url-pattern>*.htm</url-pattern>
 </servlet-mapping>
</web-app>

```

#### tiles.xml

```

<!DOCTYPE tiles-definitions PUBLIC
 "-//Apache Software Foundation//DTD Tiles Configuration 2.0//EN"
 "http://tiles.apache.org/dtds/tiles-config_2_0.dtd">
<tiles-definitions>
 <definition name="base.definition" template="/WEB-INF/pages/Layout.jsp">
 <put-attribute name="title" value="" />
 <put-attribute name="header" value="/WEB-INF/pages/Header.jsp" />
 <put-attribute name="menu" value="/WEB-INF/pages/Menu.jsp" />
 <put-attribute name="body" value="" />
 <put-attribute name="footer" value="/WEB-INF/pages/Footer.jsp" />
 </definition>
 <definition name="hello" extends="base.definition">

```

base definition  
from

```

<put-attribute name="title" value="Home Page" />
<put-attribute name="body" value="/WEB-INF/pages/Hello.jsp" />
</definition>
<definition name="crs" extends="base.definition">
 <put-attribute name="title" value="course page" />
 <put-attribute name="body" value="/WEB-INF/pages/courses.jsp" />
</definition>
<definition name="faculty" extends="base.definition">
 <put-attribute name="title" value="faculty page" />
 <put-attribute name="body" value="/WEB-INF/pages/faculties.jsp" />
</definition>
<definition name="address" extends="base.definition">
 <put-attribute name="title" value="address page" />
 <put-attribute name="body" value="/WEB-INF/pages/address.jsp" />
</definition>
</tiles-definitions>

```

page 2 → definition 2 for web page 2  
 page 3 → definition 3 for web page 3  
 page 4 → definition 4 for web page 4

**dispatcher-servlet.xml**

```

<beans xmlns="http://www.springframework.org/schema/beans"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xmlns:p="http://www.springframework.org/schema/p"
 xmlns:context="http://www.springframework.org/schema/context"
 xsi:schemaLocation="http://www.springframework.org/schema/beans
 http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
 http://www.springframework.org/schema/context
 http://www.springframework.org/schema/context/spring-context-3.0.xsd">
 <context:component-scan base-package="com.nat.controller" />
 <bean id="viewResolver"
 class="org.springframework.web.servlet.view.UrlBasedViewResolver">
 <property name="viewClass"
 value="org.springframework.web.servlet.view.tiles2.TilesView"/>
 </bean>
 <bean id="tilesConfigurer"
 class="org.springframework.web.servlet.view.tiles2.TilesConfigurer">
 <property name="definitions">
 <list>
 <value>/WEB-INF/tiles.xml</value>
 </list>
 </property>
 </bean>
 <!-- <bean id="tilesviewResolver"
 class="org.springframework.web.servlet.view.tiles3.TilesViewResolver"/>
 <bean id="tilesConfigurer"
 class="org.springframework.web.servlet.view.tiles3.TilesConfigurer">
 <property name="definitions">
 <list>
 <value>/WEB-INF/tiles.xml</value>
 </list>
 </property>
 </bean>

```

17  
 16  
 1304

viewResolver having TilesView as the view class to launch the  
 web page based on Tile definition  
 since no handlerMapping is configured defaultAnnotationMethodMapping will be taken  
 makes springmvc to recognise and use Tile definition

&lt;/bean&gt;

--&gt;

&lt;/beans&gt;

**Address.jsp**

Naresh IT

Satyam Theater,

Ameerpet

**Courses.jsp**

java

.net

oracle

**faculty.jsp**

Hari Krishna

Satish Gupta

Vijay kumar

**Footer.jsp**

&lt;center&gt;

&lt;p&gt;Copyright &amp;copy; www.nataraj.in&lt;/p&gt;

&lt;/center&gt;

**Header.jsp**

&lt;marquee&gt;&lt;b&gt;Welcome to Tiles Example&lt;/b&gt;&lt;/marquee&gt;

**Hello.jsp**<%@page isELIgnored="false" %>  
<center>\${message}</center>**Layout.jsp**

&lt;%@ taglib uri="http://tiles.apache.org/tags-tiles" prefix="tiles"%&gt;

&lt;html&gt;

&lt;head&gt;

&lt;title&gt;&lt;tiles:insertAttribute name="title" ignore="true" /&gt;

&lt;/title&gt; tile1

&lt;/head&gt;

&lt;body&gt;

&lt;table border="1" cellspacing="2" align="center" width="100%" height="100%"&gt;

&lt;tr bgcolor="cyan"&gt;

&lt;td height="50" colspan="2"&gt;&lt;tiles:insertAttribute name="header" /&gt;

&lt;/td&gt;

tile2

&lt;/tr&gt;

&lt;tr&gt;

&lt;td height="250" width="150"&gt;&lt;tiles:insertAttribute name="menu" /&gt;

&lt;/td&gt;

tile3

&lt;td width="400"&gt;&lt;tiles:insertAttribute name="body" /&gt;

&lt;/td&gt;

tile4

&lt;/tr&gt;

&lt;tr bgcolor="red"&gt;

&lt;td height="30" colspan="2"&gt;&lt;tiles:insertAttribute name="footer" /&gt;

&lt;/td&gt;

tile5

```
</tr>
</table>
</body>
</html>
```

### Menu.jsp

```
Courses

faculties

addresses
```

## Handling exceptions

### HandlerExceptionResolver

Spring HandlerExceptionResolver implementations deal with unexpected exceptions that occur during controller execution. A HandlerExceptionResolver somewhat resembles the exception mappings you can define in the web application descriptor web.xml. However, they provide a more flexible way to do so. For example they provide information about which handler was executing when the exception was thrown. Furthermore, a programmatic way of handling exceptions gives you more options for responding appropriately before the request is forwarded to another URL (the same end result as when you use the Servlet specific exception mappings).

Besides implementing the HandlerExceptionResolver interface, which is only a matter of implementing the resolveException(Exception, Handler) method and returning a ModelAndView, you may also use the provided SimpleMappingExceptionResolver or create @ExceptionHandler methods. The SimpleMappingExceptionResolver enables you to take the class name of any exception that might be thrown and map it to a view name. This is functionally equivalent to the exception mapping feature from the Servlet API, but it is also possible to implement more finely grained mappings of exceptions from different handlers.

The @ExceptionHandler annotation on the other hand can be used on methods that should be invoked to handle an exception. Such methods may be defined locally within an @Controller or may apply to many @Controller classes when defined within an @ControllerAdvice class. The following sections explain this in more detail.

### @ExceptionHandler

The HandlerExceptionResolver interface and the SimpleMappingExceptionResolver implementations allow you to map Exceptions to specific views declaratively along with some optional Java logic before forwarding to those views. However, in some cases, especially when relying on @ResponseBody methods rather than on view resolution, it may be more convenient to directly set the status of the response and optionally write error content to the body of the response.

You can do that with @ExceptionHandler methods. When declared within a controller such methods apply to exceptions raised by @RequestMapping methods of that controller (or any of its sub-classes). You can also declare an @ExceptionHandler method within an @ControllerAdvice class in which case it handles exceptions from @RequestMapping methods from many controllers. Below is an example of a controller-local @ExceptionHandler method:

```
@Controller
public class SimpleController {

 // @RequestMapping methods omitted ...

 @ExceptionHandler(IOException.class)
 public ResponseEntity<String> handleIOException(IOException ex) {
 // prepare responseEntity
 return responseEntity;
 }
}
```

The `@ExceptionHandler` value can be set to an array of Exception types. If an exception is thrown that matches one of the types in the list, then the method annotated with the matching `@ExceptionHandler` will be invoked. If the annotation value is not set then the exception types listed as method arguments are used.

Much like standard controller methods annotated with a `@RequestMapping` annotation, the method arguments and return values of `@ExceptionHandler` methods can be flexible. For example, the `HttpServletRequest` can be accessed in Servlet environments and the `PortletRequest` in Portlet environments. The return type can be a `String`, which is interpreted as a view name, a `ModelAndView` object, a `ResponseEntity`, or you can also add the `@ResponseBody` to have the method return value converted with message converters and written to the response stream.

### Handling Standard Spring MVC Exceptions

Spring MVC may raise a number of exceptions while processing a request.

The `SimpleMappingExceptionHandler` can easily map any exception to a default error view as needed. However, when working with clients that interpret responses in an automated way you will want to set specific status code on the response. Depending on the exception raised the status code may indicate a client error (4xx) or a server error (5xx).

The `DefaultHandlerExceptionHandler` works transparently by setting the status of the response. However, it stops short of writing any error content to the body of the response while your application may need to add developer-friendly content to every error response for example when providing a REST API. You can prepare a `ModelAndView` and render error content through view resolution — i.e. by configuring a `ContentNegotiatingViewResolver`, `MappingJackson2JsonView`, and so on. However, you may prefer to use `@ExceptionHandler` methods instead.

If you prefer to write error content via `@ExceptionHandler` methods you can extend `ResponseEntityExceptionHandler` instead. This is a convenient base for `@ControllerAdvice` classes providing an `@ExceptionHandler` method to handle standard Spring MVC exceptions and return `ResponseEntity`. That allows you to customize the response and write error content with message converters. See the `ResponseEntityExceptionHandler` javadocs for more details.

## Customizing the Default Servlet Container Error Page

When the status of the response is set to an error status code and the body of the response is empty, Servlet containers commonly render an HTML formatted error page. To customize the default error page of the container, you can declare an `<error-page>` element in `web.xml`. Up until Servlet 3, that element had to be mapped to a specific status code or exception type. Starting with Servlet 3 an error page does not need to be mapped, which effectively means the specified location customizes the default Servlet container error page.

```
<error-page>

 <location>/error</location>

</error-page>
```

Note that the actual location for the error page can be a JSP page or some other URL within the container including one handled through an `@Controller` method:

When writing error information, the status code and the error message set on the `HttpServletResponse` can be accessed through request attributes in a controller:

```
@Controller
public class ErrorController {

 @RequestMapping(path = "/error", produces = MediaType.APPLICATION_JSON_UTF8_VALUE)
 @ResponseBody
 public Map<String, Object> handle(HttpServletRequest request) {

 Map<String, Object> map = new HashMap<String, Object>();
 map.put("status", request.getAttribute("javax.servlet.error.status_code"));
 map.put("reason", request.getAttribute("javax.servlet.error.message"));

 return map;
 }
}
```

or in a JSP:

```
<%@ page contentType="application/json" pageEncoding="UTF-8"%>
{
 status:<%=request.getAttribute("javax.servlet.error.status_code") %>,
 reason:<%=request.getAttribute("javax.servlet.error.message") %>
}
```



# Spring Security <sup>20/2/16</sup> <sub>comp/ud</sub>

→ To perform dependency injection on servletFilter comp its class must be maintained by IOC container as spring bean having configuration in spring bean configuration file. Due to this it can't take request from client as filter is not configured as web component in web.xml file. To overcome this problem we configure, special proxy filter in web.xml file to take the request from client to pass the request to filter comp that is there in IOC container as spring bean. The special filter is "org.springframework.web.filter.DelegatingProxyFilter". But we need to match logical name (Given in <file-name>) of DelegatingProxyFilter with bean name of filter comp managed by IOC container.

## Security (authentication + authorization)

- Checking the identity of the user is called authentication (verifying username, password)
- Checking the Access permission of the user to utilize the service is called authorization.
  - Provides authorization/access permission not based on username, provide them based on roles. Roles are like designation.
- Checking the identity of user to use banking application is called authentication.
- Checking access permission of a user based on his roles to the various services is called authorization.

While implementing security we need the following services.

### 1. Authentication provider/Authentication info provider :-

It is the place where we can maintain set of username and password that are required for authentication. We can take....

- File system file (Text file, xml file etc....)
- DB software
  - LDAP server (light weight Directory Access protocol)
  - LDAP server is dedicatedly given to maintain username and password. The special <sup>ity</sup> ~~one~~ of this server is, once you forgot the password you can't get the password but you can reset the password. It is also to maintain username, password that required in single sign on operations (login once use them for multiple apps)

Eg: Login to gmail and use in youtube.com, googleDrive.com, google+.com, gmail.com and etc..

### 2. Authentication Manager:

- It is separate comp that is linked servletfilter to verify to given user, password with username, password that are available in authentication provider. If valid allows the request going to the actually request resource, If not valid shows 401, 403 error pages.
- This has to ignore authentication when request is given some resource like home page and has to enable authentication for other resources.

In non spring environment we can implement security in two ways.

#### 1. Programatic/Custom security model :-

Here we need to write code as Authentication manager that takes the request, render dialog boxes asking username, password and Talks to Authentication provider for verification and either allows the request to go to required resource or blocks the request by throwing 403 or 404 error page. Doing this work manually is very complex. So it is not recommended model.

#### 2. Server Manager declarative Security model:

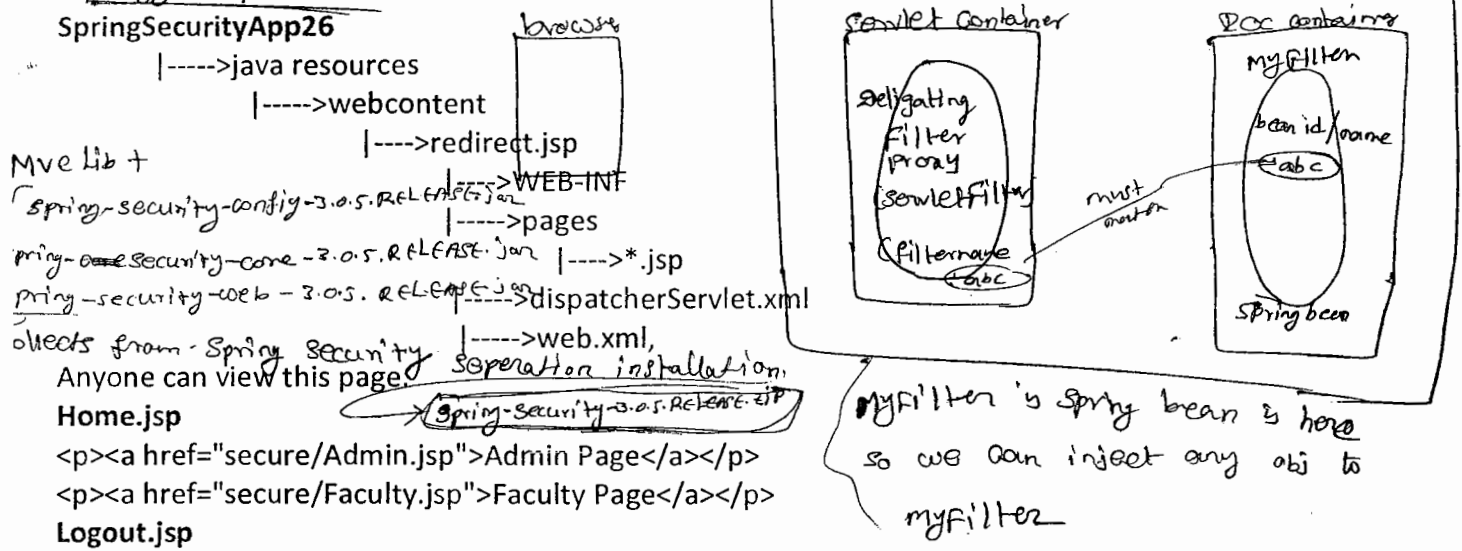
Here authentication manager work, authentication provider <sup>work</sup> will be taken care by server/container based on the configuration <sup>high</sup> done in web.xml to reduce are there in this approach. burden on pgm. but few

1. Web.xml entries are not portable <sup>access</sup> across the multiple servers. limitations are there in this approach
  2. Some web services <sup>ex</sup> don't supports in this model.
  3. Going to commercial Application server only for security model is cost effective.
  4. Some servers donot support LDAP server as authentication provider.
- To over ~~and~~ <sup>come</sup> this problem use spring security whose earlier name is (Spring acegi security)

- It runs in all servers and contains irrespective they support security or not.
- Supports all mode authentication proved.
- It is also declarative security model.
- Can be applied on normal web application and spring mvc web application and etc...

**Note:**

- We can configure spring security definitions in spring bean configura file having security namespace, based on this configured internally one ServletFilter will be generated as spring bean in IOC container having bean name "SpringSecurityFilterChain" so to take request from client and to pass to request the above generated filter, we need to configure "DelegatingProxyfilter" in web.xml with logical name "springSecurityFilterChain".



Mvc Lib +

Spring-security-config-3.0.5.RELEASE.jar

Spring-security-core-3.0.5.RELEASE.jar

Spring-security-web-3.0.5.RELEASE.jar

objects from Spring Security

Anyone can view this page

operation installation

Home.jsp

<p><a href="secure/Admin.jsp">Admin Page</a></p>

<p><a href="secure/Faculty.jsp">Faculty Page</a></p>

Logout.jsp

You have successfully LoggedOut<br>

<a href="home.jsp">start again</a>

Admin.jsp

<h1>Admin Page</h1>

Welcome Mr:<%=request.getUserPrincipal().getName()%><br><br><br>

Wanna goto Faculty Page <a href="Faculty.jsp">click here</a>

<p><a href=" ../j\_spring\_security\_logout">Logout</a>

Faculty.jsp

<h1> Faculty Page </h1>

Welcome Mr:<%=request.getUserPrincipal().getName()%><br><br><br>

Wanna goto Admin Page <a href="Admin.jsp">click here</a>

<p><a href=" ../j\_spring\_security\_logout">Logout</a>

Web.xml

<?xml version="1.0" encoding="UTF-8"?>

<web-app xmlns:web="http://xmlns.jcp.org/xml/ns/javaee">

<listener>

<listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>

</listener>

<context-param>

<param-name>contextConfigLocation</param-name>

<param-value>/WEB-INF/applicationContext.xml</param-value>

</context-param>

```

<welcome-file-list>
 <welcome-file>home.jsp</welcome-file>
</welcome-file-list>
<filter>
 <filter-name>springSecurityFilterChain</filter-name>
 <filter-class>org.springframework.web.filter.DelegatingFilterProxy</filter-class>
</filter>
<filter-mapping>
 <filter-name>springSecurityFilterChain</filter-name>
 <url-pattern>/*</url-pattern> fixed
</filter-mapping>
</web-app>

```

### applicationContext.xml

```

<beans:beans xmlns="http://www.springframework.org/schema/security"
 xmlns:beans="http://www.springframework.org/schema/beans"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://www.springframework.org/schema/beans
 http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
 http://www.springframework.org/schema/security
 http://www.springframework.org/schema/security/spring-security-3.0.3.xsd">

```

```

 <http use-expressions="true">

```

```

 <intercept-url pattern="/home.jsp" access="permitAll" />
 <intercept-url pattern="/secure/Admin.jsp" access="hasRole('admin')" />
 <intercept-url pattern="/secure/Faculty.jsp" access="hasRole('faculty')" />

```

```

 <form-login />

```

```

 <logout />

```

```

 <remember-me />

```

```

 <session-management invalid-session-url="/logout.jsp">

```

```

 <concurrency-control max-sessions="3" error-if-maximum-exceeded="true" />

```

```

 </session-management>

```

```

 </http>

```

```

 <authentication-manager>

```

```

 <authentication-provider>

```

```

 <user-service>

```

```

 <user name="raja" password="rani" authorities="admin,faculty" />

```

```

 <user name="ravi" password="ravi" authorities="faculty" />

```

```

 </user-service>

```

```

 </authentication-provider>

```

```

 </authentication-manager>

```

```

</beans:beans>

```

it after  
 seeing http  
 it understands  
 it's a bean  
 it's a security  
 so it creates  
 servlet in  
 Doc container

authentication  
 provider  
 details

*to show form page asking username and pwd*  
*to end the session when the req is given to url called "j-spring-security-logout"*  
*to launch remember me*

*nothing but  
 roles*

## SPRING ORM

Jdbc Technology allows the programmers to develop DB software dependent persistence logic by using SQL queries due to this jdbc code is not portable across the multiple DB software.

ORM allows to develop objects based DB software independent persistence logic due to this ORMMapping persistence logic is portable across the multiple DB software.

The process of mapping java class with DB table java class properties with DB table cols and making java class objects representing DB table records having synchronization between them is called o-n mapping.

Hibernate, ibatis, jdo, Toplink and etc... ORM softwares.

Spring ORM is not direct ORM software. It is spring module that provides abstraction layer on multiple ORM software like Hibernate, iBatis, JDO, JPA etc.... and simplifies o-r mapping persistence logic development by providing template classes to avoiding boilerplate code problem.

### Spring ORM Advantages:

Common Transaction management : instead of performing each ORM software programmatic Transaction, management we can go for common spring style declarative Trans management.

Nested transaction support: Most of the ORM forward don't support Nested Transaction management directly. But spring supports Nested Transaction management . By using spring ORM we can add nested Transaction management support on ORM persistence logic.

Portability : gives portability to change one ORM software to another ORM software to develop persistence logic with minimum change.

Avoids Boilerplatecode: with the support of multiple template classes we can avoid boiler plate code (Repeated code)

CommonException handling: Instead of Handling each ORM specific checked and unchecked Exceptions spring ORM thows them as common "DataAccessException" hierarchy classes based exceptions and etc....

### Spring Hibernate Integration (Spring with Hibernate):

Spring with Hibernate application means developing Hibernate style object based OR mapping persistence logic by using HibernateTemplate class in the DAO class of spring application.

To create HibernateTemplate object we need HB sessionFactory object. We can use FactoryBeans like "LocalSessionFactory" or "AnnotationSessinFactory" classes as beans to create sessionFactory object by supplying HB configure properties/HB configure file and mapping annotation based Domain class/HLO (Hibernate Language class) and also a Datasource object.

This HibernateTemplate object will be injected to DAO class to perform persistence operation.

HibernateTemplate class important methods

save(-), persist(-), update(-), load(-), get etc for single row operations.

bulkUpdate(-) ---->to perform HQL based non-select operations.

find(-) or findXxx(-) : To perform HQL/NativeSQL/criteris based bulk select operations.

DB Table:

SPEmployee

|---->eno number pk

|---->ename varchar

|---->salary float

ORMProj1(Spring 4withHibernate3)

src

com.nt.cfgs

aop-beans.xml

applicationContext.xml

dispatcher-servlet.xml

persistence-beans.xml

service-beans.xml

com.nt.controller

EmployeeController.java

com.nt.dao

EmployeeDAO.java

EmployeeDAOImpl.java

com.nt.domain

EmployeeHLO.java

com.nt.dto

EmployeeDTO.java

com.nt.service

EmployeeService.java

EmployeeServiceImpl.java

test

clientApp.java

JRE System Library [JaveSE-1.7]

AOPlib

TXlib

Jar IOC: jars + Hibernate 3.6.5 jars+ AOP jar+ ojdbc 14.jar+Apache DPCP jar+Spring-ORM-4+SpjDBC jar+aspects.jar+aopalliance+cglib

**Aop-beans.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:aop="http://www.springframework.org/schema/aop"
xmlns:tx="http://www.springframework.org/schema/tx"
xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/tx
http://www.springframework.org/schema/tx/spring-tx.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd">
```

```
<bean id="txMgmr"
class="org.springframework.orm.hibernate3.HibernateTransactionManager">
<property name="sessionFactory" ref="sesfact"/>
</bean>
<tx:advice id="txAdvice" transaction-manager="txMgmr">
<tx:attributes>
<tx:method name="register" propagation="REQUIRED"
read-only="false"/>
</tx:attributes>
```

```
</tx:advice>
<aop:config>
 <aop:pointcut id="ptc1"
 expression="execution(* com.nt.service.EmployeeService.*(..))"/>
 <aop:advisor advice-ref="txAdvice" pointcut-ref="ptc1"/>
</aop:config>
</beans>
```

**applicationContext.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
 <import resource="service-beans.xml"/>
 <import resource="persistence-beans.xml"/>
 <import resource="aop-beans.xml"/>
 <context:component-scan base-package="com.nt.controller"/>
</beans>
```

**dispatcher-servlet.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
 <bean class="org.springframework.web.servlet.mvc.annotation.DefaultAnnotation
HandlerMapping"/>
 <bean id="messageSource" c
 lass="org.springframework.context.support.ResourceBundleMessageSource">
 <property name="basename" value="com/nt/commons/errors"/>
 </bean>
 <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
 <property name="prefix" value="/WEB-INF/pages/">
 <property name="suffix" value=".jsp"/>
 </bean>
 <context:component-scan base-package="com.nt.controller,com.nt.validator"/>
</beans>
```

**persistence-beans.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
```

```
http://www.springframework.org/schema/context/spring-context.xsd">
<bean id="dbcpds" class="org.apache.commons.dbcp.BasicDataSource">
 <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
 <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
 <property name="username" value="scott"/>
 <property name="password" value="tiger"/>
</bean>
<bean id="sesfact"
class="org.springframework.orm.hibernate3.annotation.Annotation
SessionFactoryBean">
 <property name="dataSource" ref="dbcpds"/>
 <property name="annotatedClasses">
 <list>
 <value>com.nt.domain.EmployeeHLO</value>
 </list>
 </property>
 <property name="hibernateProperties">
 <props>
 <prop key="show_sql">true</prop>
 <prop key="dialect">org.hibernate.dialect.Oracle10gDialect</prop>
 </props>
 </property>
</bean>
<bean id="template" class="org.springframework.orm.hibernate3.
HibernateTemplate">
 <property name="sessionFactory" ref="sesfact"/>
</bean>
<context:component-scan base-package="com.nt.dao"/>
</beans>
```

**service-beans.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
<context:component-scan base-package="com.nt.service"/>
</beans>
```

**EmployeeController.java**

```
package com.nt.controller;
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
import com.nt.dto.EmployeeDTO;
import com.nt.service.EmployeeService;
@Component("controller")
public class EmployeeController {
```

**@Autowired**

```
private EmployeeService service;

 public String registerEmployee(String name,float salary){
 //create DTO class obj
 EmployeeDTO empDTO=new EmployeeDTO();
 empDTO.setEname(name);
 empDTO.setEsalary(salary);
 //use service class
 String result=service.register(empDTO);
 return result;
 }

 public List<EmployeeDTO> listEmployees(){
 //use service class
 List<EmployeeDTO> listEmpDTO=service.listEmps();
 return listEmpDTO;
 }
}
```

**ClientApp.java**

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.controller.EmployeeController;
public class ClientApp {

 public static void main(String[] args) {
 //create IOC container
 ApplicationContext ctx=new
 FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
 //get Bean (Controller)
 EmployeeController controller=ctx.getBean("controller",EmployeeController.class);
 //call b.method
 //System.out.println(controller.registerEmployee("raja",9000));

 System.out.println("Emp Details are"+controller.listEmployees());
 }
}
```

**EmployeeDAO.java**

```
package com.nt.dao;
import java.util.List;
import com.nt.domain.EmployeeHLO;
public interface EmployeeDAO {
 public int insert(EmployeeHLO empHLO);
 public List<EmployeeHLO> getAllEmployees();
}
```



**EmployeeDAOImpl.java**

```
package com.nt.dao;
import java.sql.SQLException;
import java.util.List;
import org.hibernate.HibernateException;
import org.hibernate.SQLQuery;
import org.hibernate.Session;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.orm.hibernate3.HibernateCallback;
import org.springframework.orm.hibernate3.HibernateTemplate;
import org.springframework.stereotype.Repository;
import com.nt.domain.EmployeeHLO;

@Repository
public class EmployeeDAOImpl implements EmployeeDAO{
 @Autowired
 private HibernateTemplate ht;
 @Override
 public int insert(EmployeeHLO empHLO) {
 int id=(Integer)ht.save(empHLO);
 return id;
 }
 @Override
 public List<EmployeeHLO> getAllEmployees() {

 List<EmployeeHLO>list=ht.execute(new HibernateCallback <List<EmployeeHLO>>(){
 @Override
 public List<EmployeeHLO> doInHibernate(Session ses)
 throws HibernateException, SQLException {
 SQLQuery query=ses.createSQLQuery("select * from SpEmployee");
 query.addEntity(EmployeeHLO.class);
 List<EmployeeHLO> list=query.list();
 return list;
 }
 });

 return list;
 }
}
```

**EmployeeService.java**

```
package com.nt.service;
import java.util.List;
import com.nt.dto.EmployeeDTO;
public interface EmployeeService {
 public String register(EmployeeDTO empDTO);
 public List<EmployeeDTO> listEmps();
}
```

**EmployeeServiceImpl.java**

```
package com.nt.service;
import java.util.ArrayList;
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.nt.dao.EmployeeDAO;
import com.nt.domain.EmployeeHLO;
import com.nt.dto.EmployeeDTO;
@Service
public class EmployeeServiceImpl implements EmployeeService {

 @Autowired
 private EmployeeDAO dao;
 @Override
 public String register(EmployeeDTO empDTO) {
 //Covert DTO obj to HLO obj/Domain obj
 EmployeeHLO empHLO=new EmployeeHLO();
 empHLO.setEname(empDTO.getEname());
 empHLO.setEsalary(empDTO.getEsalary());
 //use DAO
 int id=dao.insert(empHLO);
 return "Emp Registred with "+id;
 }
 @Override
 public List<EmployeeDTO> listEmps() {
 //use service DAO class
 List<EmployeeHLO> listEmpHLO=dao.getAllEmployees();
 //convert ListHLO to ListDTO
 List<EmployeeDTO> listEmpDTO=new ArrayList<EmployeeDTO>();
 for(EmployeeHLO hlo:listEmpHLO){
 EmployeeDTO dto=new EmployeeDTO();
 dto.setEno(hlo.getEno());
 dto.setEname(hlo.getEname());
 dto.setEsalary(hlo.getEsalary());
 listEmpDTO.add(dto);
 }

 return listEmpDTO;
 }
}
```

**EmployeeDTO.java**

```
package com.nt.dto;
import java.io.Serializable;

public class EmployeeDTO implements Serializable {
```

```
private int eno;
private String ename;
private float esalary;
public int getEno() {
 return eno;
}
public void setEno(int eno) {
 this.eno = eno;
}
public String getEname() {
 return ename;
}
public void setEname(String ename) {
 this.ename = ename;
}
public float getEsalary() {
 return esalary;
}
public void setEsalary(float esalary) {
 this.esalary = esalary;
}
@Override
public String toString() {
 return "EmployeeDTO [eno=" + eno + ", ename=" + ename + ", esalary="
 + esalary + "]";
}
}
```

**EmployeeHLO.java**

```
package com.nt.domain;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.Id;
import javax.persistence.NamedQuery;
import javax.persistence.Table;
import org.hibernate.annotations.GenericGenerator;
@Table(name="SpEmployee")
@Entity
@NamedQuery(name="GET_EMPS_BY_SAL_RANGE",
 query="from EmployeeHLO where esalary>=:min")
public class EmployeeHLO {
 @Id
 @GenericGenerator(strategy="increment", name="g1")
 @GeneratedValue(generator="g1")
 private int eno;
 private String ename;
 private float esalary;
 public int getEno() {
```

```
 return eno;
 }
 public void setEno(int eno) {
 this.eno = eno;
 }
 public String getEname() {
 return ename;
 }
 public void setEname(String ename) {-
 this.ename = ename;
 }
 public float getEsalary() {
 return esalary;
 }
 public void setEsalary(float esalary) {
 this.esalary = esalary;
 }
}
```

Plain hibernate application create configuration object and call configure() method.

Create service registry object by using serviceRegistryBuilder

Create Sessionfactory object

Open session with DB software Begin Transaction

Perform persistence opern

Commit or Rollback Transaction management

Close session, sessionfactory object

In plain HB application we need to write both common and application specific logics.

**Spring with Hibernate application (Using spring ORM)**

Inject HibernateTemplate object

Perform persistence operation.

**Executing HQL queries with named parameters:**

```
List<EmployeeHLO> List=(List<EmployeeHLO>)h+.findByNameParam ("from EmployeeHLO where salary>=:min and salary<=:max", new String [] {"min","max"}, new Object[] {9000F, 10000F});
```

**Executing HQL query with positional parameters:**

```
List<EmployeeHLO>List=(List<EmployeeHLO>)h+.find("from EmpolyeeHLO where salary>=? and salary<=?", 9000F, 10000F);
```

While working with Hibernate we place Nativesql/HQL query in mapping file or domain class as named queries to make them visible in multiple session objects of one or more DAO class.

**Executing named HQLQuery with positional parameters:**

**In EmployeeHLO.java (Domain Class)**

```
@NamedQuery(name=""GET_EMPS_BY_SAL_RANGE", QUERY="from EmployeeHLO where salary>=? and salary<=?")
```

**In DAO Class Method**

```
List<EmployeeHLO>List=(ListEmployeeHLO)h+.findByNameQuery("GET_EMPS_BY_SAL_RANGE", 9000F,10000F);
```

**Executing Named HQLQuery with Named parameters:**

**In Domain class (EmployeeHLO.java)**

```
@NamedQuery(name="GET_EMPS_BY_SAL_RANG", Query="from EmployeeHLO where salary>=:min")
```

**In DAO class .**

List

```
<EmployeeHLO>List=(List<EmployeeHLO>)h+.findNamedQueryAndNmaedParam("GETEMPS_BY_SAL_RNG", new String[]{"min"} new Object[] {9000f}));
```

If HibernateTemplate class doesn't provide environment to use certain feature of plain Hibernate, then take the support of Hibernate Callback interface, that gives doInHibernate(Session session) method. Using this object we can finish of that operation.

**For example**

We can't use native Sql query using HibernateTemplate, to make it possible we can use this Hibernate Callback Interface support.

Ex:

```
List <EmployeeHLO> list=h+.execute(new HibernateCallback<List<EmployeeHLO>>())
```

```
{
```

```
@Override
```

```
Public List<EmployeeHLO> doInHibernate(Session ses) throws HibernateExc, SQLException
```

```
{
```

```
//Plain Hb code
```

```
SQLQuery query=CreateSQLQuery("Select * from spemployee");
```

```
Query.addEntity(EmployeeHLO.class);
```

```
List<EmployeeHLO>list=query.list();
```

```
return list;
```

```
}
```

```
}
```

## Spring JEE/Context Module

Provides abstraction layer on multiple JEE technologies and provides direct facilities to develop different kinds of applications.

This module provides

ApplicationContext Container

Timer service for scheduling

Provides abstraction layer on RMI to develop spring (Distribution applications)

Provides abstraction layer on EJB to develop EJB client application.

Provides abstraction layer on jms to develop spring jms applications for message based synchronous communication.

Gives HttpInvoker, Burlap and etc... Qs direct distributed technologies to develop distributed.

Provides abstraction layer on jndi to perform jndi operations and etc.....

### **Scheduling:**

Keeping certain task to execute automatically at regular intervals or at certain time is called Scheduling. It is all about enabling timer service on the logic that we want to execute.

While dealing with remainder applications, passing data for marketing from 1 place to another place we need scheduling support

Ex: youtube, facebook operational data--->Broker agency--->Categories the data

Uses then for marketing<-----deals.

1,2,3----> develops to execute based on scheduling at regular intervals to pass the data every day.

While developing time critical jobs we need scheduling....., taking backup, doing date,time adjustment and etc...

In jdk Timer, TimerTask classes are given for scheduling.

TimerTask-----> To define job on which we want to enable scheduling.

Timer----->To specify scheduling

Here we can specify initial delay, period value time gap between two back to back execution

### **Spring based Timer/Scheduling applications:**

Allows to enable scheduling on multiple jobs at a time.

Allows to specify jobs in user-defined methods of user defined classes and we can specify initial delay, period params through xml configures declaratively.

ScheduledTimerTask---->Allows to hold the job that is defined in the subclass of TimerTask class and can also specify initialDelay, period params

### **MethodInvokingTimerTaskFactoryBean:**

Returns TimerTask class object having job defined in user-defined method (not in run method).

TimerFactoryBean---> Start Timer service on the startup of applicationContext container and stops the timer service once Application Context container stops.

## SPRING FAQs

**Q. Explain Dependency injection (DI) or IOC pattern.**

A: Dependency injection (DI) is a programming design pattern and architectural model, sometimes also referred to as inversion of control or IOC, although technically speaking, dependency injection specifically refers to an implementation of a particular form of IOC. Dependency Injection describes the situation where one object uses a second object to provide a particular capacity. For example, being passed a database connection as an argument to the constructor instead of creating one internally. The term "Dependency injection" is a misnomer, since it is not a dependency that is injected, rather it is a provider of some capability or resource that is injected. There are three common forms of dependency injection: setter-, constructor- and interface-based injection. Dependency injection is a way to achieve loose coupling. Inversion of control (IOC) relates to the way in which an object obtains references to its dependencies. This is often done by a lookup method. The advantage of inversion of control is that it decouples objects from specific lookup mechanisms and implementations of the objects it depends on. As a result, more flexibility is obtained for production applications as well as for testing.

**Q. What are the different IOC containers available?**

A. Spring is an IOC container. Other IOC containers are HiveMind, Avalon, PicoContainer.

**Q. What are the different types of dependency injection. Explain with examples.**

A: There are two types of dependency injection: setter injection and constructor injection. Setter Injection: Normally in all the java beans, we will use setter and getter method to set and get the value of property as follows:

```
public class namebean {
 String name;
 public void setName(String a) {
 name = a; }
 public String getName() {
 return name; }
}
```

We will create an instance of the bean 'namebean' (say bean1) and set property as bean1.setName("tom"); Here in setter injection, we will set the property 'name' in spring configuration file as shown below:

```
<bean id="bean1" class="namebean">
 <property name="name">
 <value>tom</value>
 </property>
</bean>
```

The subelement <value> sets the 'name' property by calling the set method as setName("tom"); This process is called setter injection.

To set properties that reference other beans <ref>, subelement of <property> is used as shown below,

```
<bean id="bean1" class="bean1impl">
 <property name="game">
 <ref bean="bean2"/>
 </property>
</bean>
<bean id="bean2" class="bean2impl" />
```

**Constructor Injection:** For constructor injection, we use constructor with parameters as shown below,

```
public class namebean {
 String name;
 public namebean(String a) {
 name = a;
 }
}
```

We will set the property 'name' while creating an instance of the bean 'namebean' as namebean bean1 = new namebean("tom");

Here we use the <constructor-arg> element to set the the property by constructor injection as

```
<bean id="bean1" class="namebean">
 <constructor-arg>
 <value>My Bean Value</value>
 </constructor-arg>
</bean>
```

**Q. What is spring? What are the various parts of spring framework? What are the different persistence frameworks which could be used with spring?**

A. Spring is an open source framework created to address the complexity of enterprise application development. One of the chief advantages of the Spring framework is its layered architecture, which allows you to be selective about which of its components you use while also providing a cohesive framework for J2EE application development. The Spring modules are built on top of the core container, which defines how beans are created, configured, and managed, as shown in the following figure. Each of the modules (or components) that comprise the Spring framework can stand on its own or be implemented jointly with one or more of the others. The functionality of each component is as follows:

**The Core Container:** The core container provides the essential functionality of the Spring framework. A primary component of the core container is the BeanFactory, an implementation of the Factory pattern. The BeanFactory applies the Inversion of Control (IOC) pattern to separate an application's configuration and dependency specification from the actual application code.

**Spring Context:** The Spring context is a configuration file that provides context information to the Spring framework. The Spring context includes enterprise services such as JNDI, EJB, e-mail, internalization, validation, and scheduling functionality.

**Spring AOP:** The Spring AOP module integrates aspect-oriented programming functionality directly into the Spring framework, through its configuration management feature. As a result you can easily AOP-enable any object managed by the Spring framework. The Spring AOP module provides transaction management services for objects in any Spring-based application. With Spring AOP you can incorporate declarative transaction management into your applications without relying on EJB components.



**Spring DAO:** The Spring JDBC DAO abstraction layer offers a meaningful exception hierarchy for managing the exception handling and error messages thrown by different database vendors. The exception hierarchy simplifies error handling and greatly reduces the amount of exception code you need to write, such as opening and closing connections. Spring DAO's JDBC-oriented exceptions comply to its generic DAO exception hierarchy.

**Spring ORM:** The Spring framework plugs into several ORM frameworks to provide its Object Relational tool, including JDO, Hibernate, and iBatis SQL Maps. All of these comply to Spring's generic transaction and DAO exception hierarchies.

**Spring Web Module:** The Web context module builds on top of the application context module, providing contexts for Web-based applications. As a result, the Spring framework supports integration with Jakarta Struts. The Web module also eases the tasks of handling multi-part requests and binding request parameters to domain objects.

**Spring MVC Framework:** The Model-View-Controller (MVC) framework is a full-featured MVC implementation for building Web applications. The MVC framework is highly configurable via strategy interfaces and accommodates numerous view technologies including JSP, Velocity, Tiles, iText, and POI.

**Q. What is AOP? How does it relate with IOC? What are different tools to utilize AOP?**

**A:** Aspect-oriented programming, or AOP, is a programming technique that allows programmers to modularize crosscutting concerns, or behavior that cuts across the typical divisions of responsibility, such as logging and transaction management. The core construct of AOP is the aspect, which encapsulates behaviors affecting multiple classes into reusable modules. AOP and IOC are complementary technologies in that both apply a modular approach to complex problems in enterprise application development. In a typical object-oriented development approach you might implement logging functionality by putting logger statements in all your methods and Java classes. In an AOP approach you would instead modularize the logging services and apply them declaratively to the components that required logging. The advantage, of course, is that the Java class doesn't need to know about the existence of the logging service or concern itself with any related code. As a result, application code written using Spring AOP is loosely coupled. The best tool to utilize AOP to its capability is AspectJ. However AspectJ works at the byte code level and you need to use AspectJ compiler to get the aop features built into your compiled code. Nevertheless AOP functionality is fully integrated into the Spring context for transaction management, logging, and various other features. In general any AOP framework control aspects in three possible ways:

**Joinpoints:** Points in a program's execution. For example, joinpoints could define calls to specific methods in a class

**Pointcuts:** Program constructs to designate joinpoints and collect specific context at those points

**Advices:** Code that runs upon meeting certain conditions. For example, an advice could log a message before executing a joinpoint

**Q. What are the advantages of spring framework?**

**A.**

1. Spring has layered architecture. Use what you need and leave you don't need now.
2. Spring Enables POJO Programming. There is no behind the scene magic here. POJO programming enables continuous integration and testability.
3. Dependency Injection and Inversion of Control Simplifies JDBC (Read the first question.)
4. Open source and no vendor lock-in.

**Q. Can you name a tool which could provide the initial ant files and directory structure for a new spring project.**

A: Appfuse or equinox.

**Q. Explain BeanFactory in spring.**

A: Bean factory is an implementation of the factory design pattern and its function is to create and dispense beans. As the bean factory knows about many objects within an application, it is able to create association between collaborating objects as they are instantiated. This removes the burden of configuration from the bean and the client. There are several implementation of BeanFactory. The most useful one is "org.springframework.beans.factory.xml.XmlBeanFactory" It loads its beans based on the definition contained in an XML file. To create an XmlBeanFactory, pass a InputStream to the constructor. The resource will provide the XML to the factory. `BeanFactory factory = new XmlBeanFactory(new FileInputStream("myBean.xml"));`

This line tells the bean factory to read the bean definition from the XML file. The bean definition includes the description of beans and their properties. But the bean factory doesn't instantiate the bean yet. To retrieve a bean from a 'BeanFactory', the `getBean()` method is called. When `getBean()` method is called, factory will instantiate the bean and begin setting the bean's properties using dependency injection. `myBean bean1 = (myBean)factory.getBean("myBean");`

**Q. Explain the role of ApplicationContext in spring.**

A. While Bean Factory is used for simple applications, the Application Context is spring's more advanced container. Like 'BeanFactory' it can be used to load bean definitions, wire beans together and dispense beans upon request. It also provide

- 1) a means for resolving text messages, including support for internationalization.
- 2) a generic way to load file resources.
- 3) events to beans that are registered as listeners.

Because of additional functionality, 'Application Context' is preferred over a BeanFactory. Only when the resource is scarce like mobile devices, 'BeanFactory' is used. The three commonly used implementation of 'Application Context' are

**1. ClassPathXmlApplicationContext :** It Loads context definition from an XML file located in the classpath, treating context definitions as classpath resources. The application context is loaded from the application's classpath by using the code

`ApplicationContext context = new ClassPathXmlApplicationContext("bean.xml");`

**2. FileSystemXmlApplicationContext :** It loads context definition from an XML file in the filesystem. The application context is loaded from the file system by using the code

`ApplicationContext context = new FileSystemXmlApplicationContext("bean.xml");`

**3. XmlWebApplicationContext :** It loads context definition from an XML file contained within a web application.

**Q. How does Spring supports DAO in hibernate?**

A. Spring's `HibernateDaoSupport` class is a convenient super class for Hibernate DAOs. It has handy methods you can call to get a Hibernate Session, or a SessionFactory. The most convenient method is `getHibernateTemplate()`, which returns a `HibernateTemplate`. This template wraps Hibernate checked exceptions with runtime exceptions, allowing your DAO interfaces to be Hibernate exception-free.

**Example:**

```
public class UserDAOHibernate extends HibernateDaoSupport {

 public User getUser(Long id) {
 return (User) getHibernateTemplate().get(User.class, id);
 }

 public void saveUser(User user) {
 getHibernateTemplate().saveOrUpdate(user);
 if (log.isDebugEnabled()) {
 log.debug("userId set to: " + user.getId());
 }
 }

 public void removeUser(Long id) {
 Object user = getHibernateTemplate().load(User.class, id);
 getHibernateTemplate().delete(user);
 }

}
```

**Q. What are the id generator classes in hibernate?**

**A:** increment: It generates identifiers of type long, short or int that are unique only when no other process is inserting data into the same table. It should not be used in the clustered environment.

**identity:** It supports identity columns in DB2, MySQL, MS SQL Server, Sybase and HypersonicSQL. The returned identifier is of type long, short or int.

**sequence:** The sequence generator uses a sequence in DB2, PostgreSQL, Oracle, SAP DB, McKoi or a generator in Interbase. The returned identifier is of type long, short or int.

**hilo:** The hilo generator uses a hi/lo algorithm to efficiently generate identifiers of type long, short or int, given a table and column (by default hibernate\_unique\_key and next\_hi respectively) as a source of hi values. The hi/lo algorithm generates identifiers that are unique only for a particular database. Do not use this generator with connections enlisted with JTA or with a user-supplied connection.

**seqhilo:** The seqhilo generator uses a hi/lo algorithm to efficiently generate identifiers of type long, short or int, given a named database sequence.

**uuid:** The uuid generator uses a 128-bit UUID algorithm to generate identifiers of type String, unique within a network (the IP address is used). The UUID is encoded as a String of hexadecimal digits of length 32.

**guid:** It uses a database-generated GUID String on MS SQL Server and MySQL.

**native:** It picks identity, sequence or hilo depending upon the capabilities of the underlying database.

**assigned:** lets the application to assign an identifier to the object before save() is called. This is the default strategy if no <generator> element is specified.

**select:** retrieves a primary key assigned by a database trigger by selecting the row by some unique key and retrieving the primary key value.

**foreign:** uses the identifier of another associated object. Usually used in conjunction with a <one-to-one> primary key association.

**Q. How is a typical spring implementation look like?**

A. For a typical Spring Application we need the following files

1. An interface that defines the functions.
2. An Implementation that contains properties, its setter and getter methods, functions etc.,
3. A XML file called Spring configuration file.
4. Client program that uses the function.

**Q. How do you define hibernate mapping file in spring?**

A. Add the hibernate mapping file entry in mapping resource inside Spring's applicationContext.xml file in the web/WEB-INF directory.

```
<property name="mappingResources">
 <list>
 <value>org/appfuse/model/User.hbm.xml</value>
 </list>
</property>
```

**Q. How do you configure spring in a web application?**

A. It is very easy to configure any J2EE-based web application to use Spring. At the very least, you can simply add Spring's ContextLoaderListener to your web.xml file:

```
<listener>
 <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
```

**Q. Can you have xyz.xml file instead of applicationContext.xml?**

A. ContextLoaderListener is a ServletContextListener that initializes when your webapp starts up. By default, it looks for Spring's configuration file at WEB-INF/applicationContext.xml. You can change this default value by specifying a <context-param> element named "contextConfigLocation." Example:

```
<listener>
 <listener-class>org.springframework.web.context.ContextLoaderListener

 <context-param>
 <param-name>contextConfigLocation</param-name>
 <param-value>/WEB-INF/xyz.xml</param-value>
 </context-param>
</listener-class>
</listener>
```

**Q. How do you configure your database driver in spring?**

A. Using datasource "org.springframework.jdbc.datasource.DriverManagerDataSource".

Example:

```
<bean id="dataSource" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
 <property name="driverClassName">
 <value>org.hsqldb.jdbcDriver</value>
 </property>
 <property name="url">
 <value>jdbc:hsqldb:db/appfuse</value>
 </property>
```

```
<property name="username"><value>sa</value></property>
<property name="password"><value></value></property>
</bean>
```

**Q. How can you configure JNDI instead of datasource in spring applicationContext.xml?**

A. Using "org.springframework.jndi.JndiObjectFactoryBean". Example:

```
<bean id="dataSource" class="org.springframework.jndi.JndiObjectFactoryBean">
 <property name="jndiName">
 <value>java:comp/env/jdbc/appfuse</value>
 </property>
</bean>
```

**Q. What are the key benefits of Hibernate?**

A: These are the key benefits of Hibernate:

- Transparent persistence based on POJOs without byte code processing
- Powerful object-oriented hibernate query language
- Descriptive O/R Mapping through mapping file.
- Automatic primary key generation
- Hibernate cache : Session Level, Query and Second level cache.
- Performance: Lazy initialization, Outer join fetching, Batch fetching

**Q. What is hibernate session and session factory? How do you configure sessionFactory in spring configuration file?**

A. Hibernate Session is the main runtime interface between a Java application and Hibernate. SessionFactory allows applications to create hibernate session by reading hibernate configurations file hibernate.configure.xml.

```
// Initialize the Hibernate environment
```

```
Configuration configure = new Configuration().configure();
```

```
// Create the session factory
```

```
SessionFactory factory = configure.buildSessionFactory();
```

```
// Obtain the new session object
```

```
Session session = factory.openSession();
```

The call to Configuration().configure() loads the hibernate.configure.xml configuration file and initializes the Hibernate environment. Once the configuration is initialized, you can make any additional modifications you desire programmatically. However, you must make these modifications prior to creating the SessionFactory instance. An instance of SessionFactory is typically created once and used to create all sessions related to a given context.

The main function of the Session is to offer create, read and delete operations for instances of mapped entity classes. Instances may exist in one of three states:

**transient:** never persistent, not associated with any Session

**persistent:** associated with a unique Session

**detached:** previously persistent, not associated with any Session

A Hibernate Session object represents a single unit-of-work for a given data store and is opened by a SessionFactory instance. You must close Sessions when all work for a transaction is completed.

The following illustrates a typical Hibernate session:

```
Session session = null;
```

```
UserInfo user = null;
```

```
Transaction tx = null;
try {
 session = factory.openSession();
 tx = session.beginTransaction();
 user = (UserInfo)session.load(UserInfo.class, id);
 tx.commit();
} catch (Exception e) {
 if (tx != null) {
 try {
 tx.rollback();
 } catch (HibernateException e1) {
 throw new DAOException(e1.toString());
 }
 throw new DAOException(e.toString());
 }
} finally {
 if (session != null) {
 try {
 session.close();
 } catch (HibernateException e) {}
 }
}
```

**Q. What is the difference between hibernate get and load methods?**

A. The load() method is older; get() was added to Hibernate's API due to user request. The difference is trivial:

The following Hibernate code snippet retrieves a User object from the database: `User user = (User) session.get(User.class, userID);`

The get() method is special because the identifier uniquely identifies a single instance of a class. Hence it's common for applications to use the identifier as a convenient handle to a persistent object. Retrieval by identifier can use the cache when retrieving an object, avoiding a database hit if the object is already cached.

Hibernate also provides a load() method: `User user = (User) session.load(User.class, userID);` If load() can't find the object in the cache or database, an exception is thrown. The load() method never returns null. The get() method returns null if the object can't be found. The load() method may return a proxy instead of a real persistent instance. A proxy is a placeholder instance of a runtime-generated subclass (through cglib or Javassist) of a mapped persistent class, it can initialize itself if any method is called that is not the mapped database identifier getter-method. On the other hand, get() never returns a proxy. Choosing between get() and load() is easy: If you're certain the persistent object exists, and nonexistence would be considered exceptional, load() is a good option. If you aren't certain there is a persistent instance with the given identifier, use get() and test the return value to see if it's null. Using load() has a further implication: The application may retrieve a valid reference (a proxy) to a persistent instance without hitting the database to retrieve its persistent state. So load() might not throw an exception when it doesn't find the persistent object in the cache or database; the exception would be thrown later, when the proxy is accessed.

**Q. What type of transaction management is supported in hibernate?**

A. Hibernate communicates with the database via a JDBC Connection; hence it must support both managed and non-managed transactions.

**non-managed in web containers:**

```
<bean id="transactionManager"
class="org.springframework.orm.hibernate.HibernateTransactionManager">
 <property name="sessionFactory">
 <ref local="sessionFactory"/>
 </property>
</bean>
```

**managed in application server using JTA:**

```
<bean id="transactionManager"
class="org.springframework.transaction.jta.JtaTransactionManager.">
 <property name="sessionFactory">
 <ref local="sessionFactory"/>
 </property>
</bean>
```

**Q. What is lazy loading and how do you achieve that in hibernate?**

A. Lazy setting decides whether to load child objects while loading the Parent Object. You need to specify parent class. Lazy = true in hibernate mapping file. By default the lazy loading of the child objects is true. This make sure that the child objects are not loaded unless they are explicitly invoked in the application by calling getChild() method on parent. In this case hibernate issues a fresh database call to load the child when getChild() is actually called on the Parent object. But in some cases you do need to load the child objects when parent is loaded. Just make the lazy=false and hibernate will load the child when parent is loaded from the database. Examples: Address child of User class can be made lazy if it is not required frequently. But you may need to load the Author object for Book parent whenever you deal with the book for online bookshop. Hibernate does not support lazy initialization for detached objects. Access to a lazy association outside of the context of an open Hibernate session will result in an exception.

**Q. What are the different fetching strategy in Hibernate?**

A. Hibernate3 defines the following fetching strategies:

**Join fetching** - Hibernate retrieves the associated instance or collection in the same SELECT; using an OUTER JOIN.

**Select fetching** - a second SELECT is used to retrieve the associated entity or collection. Unless you explicitly disable lazy fetching by specifying lazy="false", this second select will only be executed when you actually access the association.

**Subselect fetching** - a second SELECT is used to retrieve the associated collections for all entities retrieved in a previous query or fetch. Unless you explicitly disable lazy fetching by specifying lazy="false", this second select will only be executed when you actually access the association.

**Batch fetching** - an optimization strategy for select fetching - Hibernate retrieves a batch of entity instances or collections in a single SELECT, by specifying a list of primary keys or foreign keys.

For more details read short primer on fetching strategy at <http://www.hibernate.org/315.html>

**Q. What are different types of cache hibernate supports ?**

A. Caching is widely used for optimizing database applications. Hibernate uses two different caches for objects: first-level cache and second-level cache. First-level cache is associated with the Session object, while second-level cache is associated with the Session Factory object. By default, Hibernate uses first-level cache on a per-transaction basis. Hibernate uses this cache mainly to reduce the number of SQL queries it needs to generate within a given transaction. For example, if an object is modified several times within the same transaction, Hibernate will generate only one SQL UPDATE statement at the end of the transaction, containing all the modifications. To reduce database traffic, second-level cache keeps loaded objects at the Session Factory level between transactions. These objects are available to the whole application, not just to the user running the query. This way, each time a query returns an object that is already loaded in the cache, one or more database transactions potentially are avoided. In addition, you can use a query-level cache if you need to cache actual query results, rather than just persistent objects. The query cache should always be used in conjunction with the second-level cache. Hibernate supports the following open-source cache implementations out-of-the-box:

- **EHCache** is a fast, lightweight, and easy-to-use in-process cache. It supports read-only and read/write caching, and memory- and disk-based caching. However, it does not support clustering.
- **OSCache** is another open-source caching solution. It is part of a larger package, which also provides caching functionalities for JSP pages or arbitrary objects. It is a powerful and flexible package, which, like EHCache, supports read-only and read/write caching, and memory- and disk-based caching. It also provides basic support for clustering via either JavaGroups or JMS.
- **SwarmCache** is a simple cluster-based caching solution based on JavaGroups. It supports read-only or nonstrict read/write caching (the next section explains this term). This type of cache is appropriate for applications that typically have many more read operations than write operations.
- **JBoss TreeCache** is a powerful replicated (synchronous or asynchronous) and transactional cache. Use this solution if you really need a true transaction-capable caching architecture.
- Commercial Tangosol Coherence cache.

**Q. What are the different caching strategies?**

A. The following four caching strategies are available:

- **Read-only:** This strategy is useful for data that is read frequently but never updated. This is by far the simplest and best-performing cache strategy.
- **Read/write:** Read/write caches may be appropriate if your data needs to be updated. They carry more overhead than read-only caches. In non-JTA environments, each transaction should be completed when `Session.close()` or `Session.disconnect()` is called.
- **Nonstrict read/write:** This strategy does not guarantee that two transactions won't simultaneously modify the same data. Therefore, it may be most appropriate for data that is read often but only occasionally modified.
- **Transactional:** This is a fully transactional cache that may be used only in a JTA environment.

**Q. How do you configure 2nd level cache in hibernate?**

A. To activate second-level caching, you need to define the `hibernate.cache.provider_class` property in the `hibernate.config.xml` file as follows: `<hibernate-configuration>`  
`<session-factory>`



```
<property
name="hibernate.cache.provider_class">org.hibernate.cache.EHCacheProvider</property>
</session-factory>
</hibernate-configuration>
```

By default, the second-level cache is activated and uses the EHCache provider.

To use the query cache you must first enable it by setting the property `hibernate.cache.use_query_cache` to true in `hibernate.properties`.

**Q. What is the difference between sorted and ordered collection in hibernate?**

A. A sorted collection is sorted in-memory using java comparator, while order collection is ordered at the database level using order by clause.

**Q. What are the types of inheritance models and describe how they work like vertical inheritance and horizontal?**

A. There are three types of inheritance mapping in hibernate :

**Example:** Let us take the simple example of 3 java classes. Class Manager and Worker are inherited from Employee Abstract class.

1. Table per concrete class with unions : In this case there will be 2 tables. Tables: Manager, Worker [all common attributes will be duplicated]
2. Table per class hierarchy: Single Table can be mapped to a class hierarchy. There will be only one table in database called 'Employee' that will represent all the attributes required for all 3 classes. But it needs some discriminating column to differentiate between Manager and worker;
3. Table per subclass: In this case there will be 3 tables represent Employee, Manager and Worker

## Additional Questions:

### 1. What is IOC (or Dependency Injection)?

The basic concept of the Inversion of Control pattern (also known as dependency injection) is that you do not create your objects but describe how they should be created. You don't directly connect your components and services together in code but describe which services are needed by which components in a configuration file. A container (in the case of the Spring framework, the IOC container) is then responsible for hooking it all up.

i.e., Applying IoC, objects are given their dependencies at creation time by some external entity that coordinates each object in the system. That is, dependencies are injected into objects. So, IoC means an inversion of responsibility with regard to how an object obtains references to collaborating objects.

### 2. What are the different types of IOC (dependency injection) ?

There are three types of dependency injection:

- Constructor Injection (e.g. Pico container, Spring etc): Dependencies are provided as constructor parameters.
- Setter Injection (e.g. Spring): Dependencies are assigned through JavaBeans properties (ex: setter methods).
- Interface Injection (e.g. Avalon): Injection is done through an interface.

**Note:** Spring supports only Constructor and Setter Injection

### 3. What are the benefits of IOC (Dependency Injection)?

Benefits of IOC (Dependency Injection) are as follows:

- Minimizes the amount of code in your application. With IOC containers you do not care about how services are created and how you get references to the ones you need. You can also easily add additional services by adding a new constructor or a setter method with little or no extra configuration.
- Make your application more testable by not requiring any singletons or JNDI lookup mechanisms in your unit test cases. IOC containers make unit testing and switching implementations very easy by manually allowing you to inject your own objects into the object under test.
- Loose coupling is promoted with minimal effort and least intrusive mechanism. The factory design pattern is more intrusive because components or services need to be requested explicitly whereas in IOC the dependency is injected into requesting piece of code. Also some containers promote the design to interfaces not to implementations design concept by encouraging managed objects to implement a well-defined service interface of your own.
- IOC containers support eager instantiation and lazy loading of services. Containers also provide support for instantiation of managed objects, cyclical dependencies, life cycles management, and dependency resolution between managed objects etc.

### 4. What is Spring ?

Spring is an open source framework created to address the complexity of enterprise application development. One of the chief advantages of the Spring framework is its layered architecture, which allows you to be selective about which of its components you use while also providing a cohesive framework for J2EE application development.

**5. What are the advantages of Spring framework?**

The advantages of Spring are as follows:

- Spring has layered architecture. Use what you need and leave you don't need now.
- Spring Enables POJO Programming. There is no behind the scene magic here. POJO programming enables continuous integration and testability.
- Dependency Injection and Inversion of Control Simplifies JDBC
- Open source and no vendor lock-in.

**6. What are features of Spring ?****Lightweight:**

spring is lightweight when it comes to size and transparency. The basic version of spring framework is around 1MB. And the processing overhead is also very negligible.

**Inversion of control (IOC):**

Loose coupling is achieved in spring using the technique Inversion of Control. The objects give their dependencies instead of creating or looking for dependent objects.

**Aspect oriented (AOP):**

Spring supports Aspect oriented programming and enables cohesive development by separating application business logic from system services.

**Container:**

Spring contains and manages the life cycle and configuration of application objects.

**MVC Framework:**

Spring comes with MVC web application framework, built on core Spring functionality. This framework is highly configurable via strategy interfaces, and accommodates multiple view technologies like JSP, Velocity, Tiles, iText, and POI. But other frameworks can be easily used instead of Spring MVC Framework.

**Transaction Management:**

Spring framework provides a generic abstraction layer for transaction management. This allowing the developer to add the pluggable transaction managers, and making it easy to demarcate transactions without dealing with low-level issues. Spring's transaction support is not tied to J2EE environments and it can be also used in container less environments.

**JDBC Exception Handling:**

The JDBC abstraction layer of the Spring offers a meaningful exception hierarchy, which simplifies the error handling strategy. Integration with Hibernate, JDO, and iBATIS: Spring provides best Integration services with Hibernate, JDO and iBATIS

contexts for Web-based applications. As a result, the Spring framework supports integration with Jakarta Struts. The Web module also eases the tasks of handling multi-part requests and binding request parameters to domain objects.

**7 . What is web module?**

This module is built on the application context module, providing a context that is appropriate for web-based applications. This module also contains support for several web-oriented tasks such as transparently handling multipart requests for file uploads and programmatic binding of request parameters to your business objects. It also contains integration support with Jakarta Struts.

**8. What are the types of Dependency Injection Spring supports?****Setter Injection:**

Setter-based DI is realized by calling setter methods on your beans after invoking a no-argument constructor or no-argument static factory method to instantiate your bean.

**Constructor Injection:**

Constructor-based DI is realized by invoking a constructor with a number of arguments, each representing a collaborator.

**9. What is Bean Factory ?**

A BeanFactory is like a factory class that contains a collection of beans. The BeanFactory holds Bean Definitions of multiple beans within itself and then instantiates the bean whenever asked for by clients.

- BeanFactory is able to create associations between collaborating objects as they are instantiated. This removes the burden of configuration from bean itself and the beans client.
- BeanFactory also takes part in the life cycle of a bean, making calls to custom initialization and destruction methods.

**10. What is Application Context?**

A bean factory is fine to simple applications, but to take advantage of the full power of the Spring framework, you may want to move up to Springs more advanced container, the application context. On the surface, an application context is same as a bean factory. Both load bean definitions, wire beans together, and dispense beans upon request. But it also provides:

- A means for resolving text messages, including support for internationalization.
- A generic way to load file resources.
- Events to beans that are registered as listeners

**11. What is the difference between Bean Factory and Application Context ?**

On the surface, an application context is same as a bean factory. But application context offers much more..

- Application contexts provide a means for resolving text messages, including support for i18n of those messages.
- Application contexts provide a generic way to load file resources, such as images.
- Application contexts can publish events to beans that are registered as listeners.
- Certain operations on the container or beans in the container, which have to be handled in a programmatic fashion with a bean factory, can be handled declaratively in an application context.
- ResourceLoader support: Spring's Resource interface us a flexible generic abstraction for handling low-level resources. An application context itself is a ResourceLoader, Hence provides an application with access to deployment-specific Resource instances.
- MessageSource support: The application context implements MessageSource, an interface used to obtain localized messages, with the actual implementation being pluggable

**12. What are the common implementations of the Application Context ?**

The three commonly used implementation of 'Application Context' are

- ClassPathXmlApplicationContext : It Loads context definition from an XML file located in the classpath, treating context definitions as classpath resources. The application context is loaded from the application's classpath by using the code .
- ApplicationContext context = new ClassPathXmlApplicationContext("bean.xml");
- FileSystemXmlApplicationContext : It loads context definition from an XML file in the filesystem. The application context is loaded from the file system by using the code .
- ApplicationContext context = new FileSystemXmlApplicationContext("bean.xml");

- **XmlWebApplicationContext** : It loads context definition from an XML file contained within a web application.

### 13. How is a typical spring implementation look like ?

For a typical Spring Application we need the following files:

- An interface that defines the functions.
- An Implementation that contains properties, its setter and getter methods, functions etc.,
- Spring AOP (Aspect Oriented Programming)
- A XML file called Spring configuration file.
- Client program that uses the function.

### 14. What is the typical Bean life cycle in Spring Bean Factory Container ?

Bean life cycle in Spring Bean Factory Container is as follows:

- The spring container finds the bean's definition from the XML file and instantiates the bean.
- Using the dependency injection, spring populates all of the properties as specified in the bean definition
- If the bean implements the **BeanNameAware** interface, the factory calls **setBeanName()** passing the bean's ID.
- If the bean implements the **BeanFactoryAware** interface, the factory calls **setBeanFactory()**, passing an instance of itself.
- If there are any **BeanPostProcessors** associated with the bean, their **postProcessBeforeInitialization()** methods will be called.
- If an **init-method** is specified for the bean, it will be called.
- Finally, if there are any **BeanPostProcessors** associated with the bean, their **postProcessAfterInitialization()** methods will be called.

### 15. What do you mean by Bean wiring ?

The act of creating associations between application components (beans) within the Spring container is referred to as Bean wiring.

### 16. What do you mean by Auto Wiring?

The Spring container is able to autowire relationships between collaborating beans. This means that it is possible to automatically let Spring resolve collaborators (other beans) for your bean by inspecting the contents of the **BeanFactory**. The autowiring functionality has five modes.

- no
- **byName**
- **byType**
- **constructor**
- **autodirect**

### 17. What is DelegatingVariableResolver?

Spring provides a custom **JavaServer Faces VariableResolver** implementation that extends the standard **Java Server Faces managed beans** mechanism which lets you use **JSPRINGFRAMEWORK** and Spring together. This variable resolver is called as **DelegatingVariableResolver**

### 18. How to integrate Java Server Faces (JSPRINGFRAMEWORK) with Spring?

**JSPRINGFRAMEWORK** and Spring do share some of the same features, most noticeably in the area of **IOC services**. By declaring **JSPRINGFRAMEWORK managed-beans** in the **faces-config.xml**

configuration file, you allow the FacesServlet to instantiate that bean at startup. Your JSR169 pages have access to these beans and all of their properties. We can integrate JSR169 and Spring in two ways:

- DelegatingVariableResolver: Spring comes with a JSR169 variable resolver that lets you use JSR169 and Spring together.

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN//EN"
```

```
"http://www.springframework.org/dtd/spring-beans.dtd">
```

```
<faces-config>
```

```
<application>
```

```
<variable-resolver>
```

```
org.springframework.web.jsr169.DelegatingVariableResolver
```

```
</variable-resolver>
```

```
</application>
```

```
</faces-config>
```

The DelegatingVariableResolver will first delegate value lookups to the default resolver of the underlying JSR169 implementation, and then to Spring's 'business context' WebApplicationContext. This allows one to easily inject dependencies into one's JSR169-managed beans.

- FacesContextUtils: custom VariableResolver works well when mapping one's properties to beans in faces-config.xml, but at times one may need to grab a bean explicitly. The FacesContextUtils class makes this easy. It is similar to WebApplicationContextUtils, except that it takes a FacesContext parameter rather than a ServletContext parameter.

```
ApplicationContext ctx =
```

```
FacesContextUtils.getWebApplicationContext(FacesContext.getCurrentInstance());
```

## 19. What is Java Server Faces (JSR169) - Spring integration mechanism?

Spring provides a custom JavaServer Faces VariableResolver implementation that extends the standard JavaServer Faces managed beans mechanism. When asked to resolve a variable name, the following algorithm is performed:

- Does a bean with the specified name already exist in some scope (request, session, application)? If so, return it
- Is there a standard JavaServer Faces managed bean definition for this variable name? If so, invoke it in the usual way, and return the bean that was created.
- Is there configuration information for this variable name in the Spring WebApplicationContext for this application? If so, use it to create and configure an instance, and return that instance to the caller.
- If there is no managed bean or Spring definition for this variable name, return null instead.
- BeanFactory also takes part in the life cycle of a bean, making calls to custom initialization and destruction methods.

As a result of this algorithm, you can transparently use either JavaServer Faces or Spring facilities to create beans on demand.

## 20. What is Significance of JSR169- Spring integration ?

Spring - JSR169 integration is useful when an event handler wishes to explicitly invoke the bean factory to create beans on demand, such as a bean that encapsulates the business logic to be performed when a submit button is pressed.

**21. How to integrate your Struts application with Spring?**

To integrate your Struts application with Spring, we have two options:

- Configure Spring to manage your Actions as beans, using the ContextLoaderPlugin, and set their dependencies in a Spring context file.
- Subclass Spring's ActionSupport classes and grab your Spring-managed beans explicitly using a `getWebApplicationContext()` method.

**22. What are ORM's Spring supports ?**

Spring supports the following ORM's :

- Hibernate
- iBatis
- JPA (Java Persistence API)
- TopLink
- JDO (Java Data Objects)
- OJB

**23. What are the ways to access Hibernate using Spring ?**

There are two approaches to Spring's Hibernate integration:

- Inversion of Control with a HibernateTemplate and Callback
- Extending HibernateDaoSupport and Applying an AOP Interceptor

**24. How to integrate Spring and Hibernate using HibernateDaoSupport?**

Spring and Hibernate can integrate using Spring's SessionFactory called LocalSessionFactory. The integration process is of 3 steps.

- Configure the Hibernate SessionFactory
- Extend your DAO Implementation from HibernateDaoSupport
- Wire in Transaction Support with AOP

**25. What are Bean scopes in Spring Framework ?**

The Spring Framework supports exactly five scopes (of which three are available only if you are using a web-aware ApplicationContext). The scopes supported are listed below:

**singleton**

Scopes a single bean definition to a single object instance per Spring IoC container.

**prototype**

Scopes a single bean definition to any number of object instances.

**request**

Scopes a single bean definition to the lifecycle of a single HTTP request; that is each and every HTTP request will have its own instance of a bean created off the back of a single bean definition. Only valid in the context of a web-aware Spring ApplicationContext.

**session**

Scopes a single bean definition to the lifecycle of a HTTP Session. Only valid in the context of a web-aware Spring ApplicationContext.

**global session**

Scopes a single bean definition to the lifecycle of a global HTTP Session. Typically only valid when used in a portlet context. Only valid in the context of a web-aware Spring ApplicationContext.

**26. What is AOP?**

Aspect-oriented programming, or AOP, is a programming technique that allows programmers to modularize crosscutting concerns, or behavior that cuts across the typical divisions of responsibility, such as logging and transaction management. The core construct of AOP is the aspect, which encapsulates behaviors affecting multiple classes into reusable modules.

**27. How the AOP used in Spring?**

AOP is used in the Spring Framework: To provide declarative enterprise services, especially as a replacement for EJB declarative services. The most important such service is declarative transaction management, which builds on the Spring Framework's transaction abstraction. To allow users to implement custom aspects, complementing their use of OOP with AOP.

**28. What do you mean by Aspect ?**

A modularization of a concern that cuts across multiple objects. Transaction management is a good example of a crosscutting concern in J2EE applications. In Spring AOP, aspects are implemented using regular classes (the schema-based approach) or regular classes annotated with the `@Aspect` annotation (`@AspectJ` style).

**29. What do you mean by JointPoint?**

A point during the execution of a program, such as the execution of a method or the handling of an exception. In Spring AOP, a join point always represents a method execution.

**30. What do you mean by Advice?**

Action taken by an aspect at a particular join point. Different types of advice include "around," "before" and "after" advice. Many AOP frameworks, including Spring, model an advice as an interceptor, maintaining a chain of interceptors "around" the join point.

**31. What are the types of Advice?**

Types of advice:

- Before advice: Advice that executes before a join point, but which does not have the ability to prevent execution flow proceeding to the join point (unless it throws an exception).
- After returning advice: Advice to be executed after a join point completes normally: for example, if a method returns without throwing an exception.
- After throwing advice: Advice to be executed if a method exits by throwing an exception.
- After (finally) advice: Advice to be executed regardless of the means by which a join point exits (normal or exceptional return).
- Around advice: Advice that surrounds a join point such as a method invocation. This is the most powerful kind of advice. Around advice can perform custom behavior before and after the method invocation. It is also responsible for choosing whether to proceed to the join point or to shortcut the advised method execution by returning its own return value or throwing an exception

**32. What are the types of the transaction management Spring supports ?**

Spring Framework supports:

- Programmatic transaction management.
- Declarative transaction management.



**33. What are the benefits of the Spring Framework transaction management ?**

The Spring Framework provides a consistent abstraction for transaction management that delivers the following benefits:

- Provides a consistent programming model across different transaction APIs such as JTA, JDBC, Hibernate, JPA, and JDO.
- Supports declarative transaction management.
- Provides a simpler API for programmatic transaction management than a number of complex transaction APIs such as JTA.
- Integrates very well with Spring's various data access abstractions.

**34. Why most users of the Spring Framework choose declarative transaction management ?**

Most users of the Spring Framework choose declarative transaction management because it is the option with the least impact on application code, and hence is most consistent with the ideals of a non-invasive lightweight container.

**35. Explain the similarities and differences between EJB CMT and the Spring Framework's declarative transaction management?**

The basic approach is similar: it is possible to specify transaction behavior (or lack of it) down to individual method level. It is

possible to make a `setRollbackOnly()` call within a transaction context if necessary. The differences are:

- Unlike EJB CMT, which is tied to JTA, the Spring Framework's declarative transaction management works in any environment. It can work with JDBC, JDO, Hibernate or other transactions under the covers, with configuration changes only.
- The Spring Framework enables declarative transaction management to be applied to any class, not merely special classes such as EJBs.
- The Spring Framework offers declarative rollback rules: this is a feature with no EJB equivalent. Both programmatic and declarative support for rollback rules is provided.
- The Spring Framework gives you an opportunity to customize transactional behavior, using AOP. With EJB CMT, you have no way to influence the container's transaction management other than `setRollbackOnly()`.
- The Spring Framework does not support propagation of transaction contexts across remote calls, as do high-end application servers.

**36. What are object/relational mapping integration module?**

Spring also supports for using of an object/relational mapping (ORM) tool over straight JDBC by providing the ORM module. Spring provide support to tie into several popular ORM frameworks, including Hibernate, JDO, and iBATIS SQL Maps. Spring's transaction management supports each of these ORM frameworks as well as JDBC.

**37. When to use programmatic and declarative transaction management ?**

Programmatic transaction management is usually a good idea only if you have a small number of transactional operations. On the other hand, if your application has numerous transactional operations, declarative transaction management is usually worthwhile. It keeps transaction management out of business logic, and is not difficult to configure.

**38. Explain about the Spring DAO support ?**

The Data Access Object (DAO) support in Spring is aimed at making it easy to work with data access technologies like JDBC, Hibernate or JDO in a consistent way. This allows one to switch between the persistence technologies fairly easily and it also allows one to code without worrying about catching exceptions that are specific to each technology.

**39. What are the exceptions thrown by the Spring DAO classes ?**

Spring DAO classes throw exceptions which are subclasses of `org.springframework.dao.DataAccessException`. Spring provides a convenient translation from technology-specific exceptions like `SQLException` to its own exception class hierarchy with the `DataAccessException` as the root exception. These exceptions wrap the original exception.

**40. What is `SQLExceptionTranslator` ?**

`SQLExceptionTranslator`, is an interface to be implemented by classes that can translate between `SQLExceptions` and Spring's own data-access-strategy-agnostic `org.springframework.dao.DataAccessException`.

**41. What is Spring's `JdbcTemplate` ?**

Spring's `JdbcTemplate` is central class to interact with a database through JDBC. `JdbcTemplate` provides many convenience methods for doing things such as converting database data into primitives or objects, executing prepared and callable statements, and providing custom database error handling.

```
JdbcTemplate template = new JdbcTemplate(myDataSource);
```

**42. What is `PreparedStatementCreator` ?**

`PreparedStatementCreator`:

- Is one of the most common used interfaces for writing data to database.
- Has one method – `createPreparedStatement(Connection)`
- Responsible for creating a `PreparedStatement`.
- Does not need to handle `SQLExceptions`.

**43. What is `SQLProvider` ?**

`SQLProvider`:

- Has one method – `getSql()`
- Typically implemented by `PreparedStatementCreator` implementers.
- Useful for debugging.

**44. What is `RowCallbackHandler` ?**

The `RowCallbackHandler` interface extracts values from each row of a `ResultSet`.

- Has one method – `processRow(ResultSet)`
- Called for each row in `ResultSet`.
- Typically stateful.

**45. What are the differences between EJB and Spring ?**

*Transaction management*

**EJB:**

- Must use a JTA transaction manager.

- Supports transactions that span remote method calls.

**Spring:**

- Supports multiple transaction environments through its PlatformTransactionManager interface, including JTA, Hibernate, JDO, and JDBC.
- Does not natively support distributed transactions—it must be used with a JTA transaction manager.

**Declarative transaction support****EJB:**

- Can define transactions declaratively through the deployment descriptor.
- Can define transaction behavior per method or per class by using the wildcard character \*.
- Cannot declaratively define rollback behavior—this must be done programmatically.

**Spring:**

- Can define transactions declaratively through the Spring configuration file or through class metadata.
- Can define which methods to apply transaction behavior explicitly or by using regular expressions.
- Can declaratively define rollback behavior per method and per exception type.

**Persistence****EJB:**

Supports programmatic bean-managed persistence and declarative container managed persistence.

**Spring:**

Provides a framework for integrating with several persistence technologies, including JDBC, Hibernate, JDO, and iBATIS.

**Declarative security****EJB:**

- Supports declarative security through users and roles. The management and implementation of users and roles is container specific.
- Declarative security is configured in the deployment descriptor.

**Spring:**

- No security implementation out-of-the box.
- Acegi, an open source security framework built on top of Spring, provides declarative security through the Spring configuration file or class metadata.

**Distributed computing****EJB:**

Provides container-managed remote method calls.

**Spring:**

Provides proxying for remote calls via RMI, JAX-RPC, and web services.

**46 . Do I need any other SOAP framework to run Spring Web Services?**

You don't need any other SOAP framework to use Spring Web services, though it can use some of the features of Axis 1 and 2.

**47 . I get NAMESPACE\_ERR exceptions when using Spring-WS. What can I do about it?**

**If you get the following Exception:**

NAMESPACE\_ERR: An attempt is made to create or change an object in a way which is incorrect with regard to namespaces.

Most often, this exception is related to an older version of Xalan being used. Make sure to upgrade to 2.7.0.

#### 48 .Does Spring-WS run under Java 1.3?

Spring Web Services requires Java 1.4 or higher.

#### 49. Does Spring-WS work under Java 1.4?

Spring Web Services works under Java 1.4, but it requires some effort to make it work. Java 1.4 is bundled with the older XML parser Crimson, which does not handle namespaces correctly. Additionally, it is bundled with an older version of Xalan, which also has problems. Unfortunately, placing newer versions of these on the class path does not override them. .

The only solution that works is to add newer versions of Xerces and Xalan in the lib/endorsed directory of your JDK, as explained in those FAQs (i.e.\$JAVA\_HOME/lib/endorsed). The following libraries are known to work with Java 1.4.2:

Library	Version
Xerces	2.8.1
Xalan	2.7.0
XML-APIs	1.3.04
SAAJ	1.2

If you want to use WS-Security, note that the XwsSecurityInterceptor requires Java 5, because an underlying library (XWSS) requires it. Instead, you can use the Wss4jSecurityInterceptor.

#### 50 .Does Spring-WS work under Java 1.6?

Java 1.6 ships with SAAJ 1.3, JAXB 2.0, and JAXP 1.4 (a custom version of Xerces and Xalan).

Overriding these libraries by putting different version on the classpath will result in various classloading issues, or exceptions in org.apache.xml.serializer.ToXMLSAXHandler. The only option for using more recent versions is to put the newer version in the endorsed directory (see above).

#### 51 . Why do the Spring-WS unit tests fail under Mac OS X?

For some reason, Apple decided to include a Java 1.4 compatibility jar with their JDK 1.5. This jar includes the XML parsers which were included in Java 1.4. No other JDK distribution does this, so it is unclear what the purpose of this compatibility jar is.

The jar can be found at

/System/Library/Frameworks/JavaVM.framework/Versions/1.5.0/Classes/.compatibility/14compatibility.jar. You can safely remove or rename it, and the tests will run again.

#### 52 . What is SAAJ?

SAAJ is the SOAP with Attachments API for Java. Like most Java EE libraries, it consists of a set of interfaces (saaj-api.jar), and implementations (saaj-impl.jar). When running in a Application Server, the implementation is typically provided by the application server. Previously, SAAJ has been part of JAXM, but it has been released as a separate API as part of the , and also as part of J2EE 1.4. SAAJ is generally known as the package javax.xml.soap.

Spring-WS uses this standard SAAJ library to create representations of SOAP messages.

Alternatively, it can use

#### 59. Why does Spring-WS only support contract-first?

Note that Spring-WS only requires you to write the XSD; the WSDL can be generated from that.

**60 . How do I retrieve the WSDL from a Service?**

The &WSDL query parameter does not work.

The &WSDL query parameter is a way to get a WSDL of a class. In SWS, a service is generally not implemented as a single class, but as a collection of endpoints.

There are two ways to expose a WSDL:

- Simply add the WSDL to the root of the WAR, and the file is served normally. This has the disadvantage that the "location" attribute in the WSDL is static, i.e. it does not necessarily reflect the host name of the server. You can transpringframeworkorm locations by using a WsdlDefinitionHandlerAdapter.
- Use theMessageDispatcherServlet, which is done in the samples. Every WsdlDefinition listed in the \*-servlet.xml will be exposed under the bean name. So if you define a WsdlDefinition named echo, it will be exposed as echo.wsdl (i.e. http://localhost:8080/echo/echo.wsdl).

**61 What is web module?**

Spring comes with a full-featured MVC framework for building web applications. Although Spring can easily be integrated with other MVC frameworks, such as Struts, Spring's MVC framework uses IoC to provide for a clean separation of controller logic from business objects. It also allows you to declaratively bind request parameters to your business objects. It also can take advantage of any of Spring's other services, such as I18N messaging and validation.

**62 What is a BeanFactory?**

A BeanFactory is an implementation of the factory pattern that applies Inversion of Control to separate the application's configuration and dependencies from the actual application code.

**63 What is AOP Alliance?**

AOP Alliance is an open-source project whose goal is to promote adoption of AOP and interoperability among different AOP implementations by defining a common set of interfaces and components.

**64 What is Spring configuration file?**

Spring configuration file is an XML file. This file contains the classes information and describes how these classes are configured and introduced to each other.

**65 .What does a simple spring application contain?**

These applications are like any Java application. They are made up of several classes, each performing a specific purpose within the application. But these classes are configured and introduced to each other through an XML file. This XML file describes how to configure the classes, known as the Spring configuration file.

**66. What is XmlBeanFactory?**

BeanFactory has many implementations in Spring. But one of the most useful one is org.springframework.beans.factory.xml.XmlBeanFactory, which loads its beans based on the definitions contained in an XML file. To create an XmlBeanFactory, pass a java.io.InputStream to the constructor. The InputStream will provide the XML to the factory. For example, the following code snippet uses a java.io.FileInputStream to provide a bean definition XML file to XmlBeanFactory.

```
BeanFactory factory = new XmlBeanFactory(new FileInputStream("beans.xml"));
```

To retrieve the bean from a BeanFactory, call the getBean() method by passing the name of the bean you want to retrieve.

```
MyBean myBean = (MyBean) factory.getBean("myBean")
```

**67 . What are important ApplicationContext implementations in spring framework?**

- ClassPathXmlApplicationContext – This context loads a context definition from an XML file located in the class path, treating context definition files as class path resources.
- FileSystemXmlApplicationContext – This context loads a context definition from an XML file in the filesystem.
- XmlWebApplicationContext – This context loads the context definitions from an XML file contained within a web application.

**68 . Explain Bean lifecycle in Spring framework?**

- The spring container finds the bean's definition from the XML file and instantiates the bean.
- Using the dependency injection, spring populates all of the properties as specified in the bean definition.
- If the bean implements the BeanNameAware interface, the factory calls setBeanName() passing the bean's ID.
- If the bean implements the BeanFactoryAware interface, the factory calls setBeanFactory(), passing an instance of itself.
- If there are any BeanPostProcessors associated with the bean, their post-ProcessBeforeInitialization() methods will be called.
- If an init-method is specified for the bean, it will be called.
- Finally, if there are any BeanPostProcessors associated with the bean, their postProcessAfterInitialization() methods will be called.

**69 What is bean wiring?**

Combining together beans within the Spring container is known as bean wiring or wiring. When wiring beans, you should tell the container what beans are needed and how the container should use dependency injection to tie them together.

**70. How do add a bean in spring application?**

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN//EN"
"http://www.springframework.org/dtd/spring-beans.dtd">
<beans>
 <bean id="foo" class="com.act.Foo"/>
 <bean id="bar" class="com.act.Bar"/>
</beans>
```

**71. What are singleton beans and how can you create prototype beans?**

Beans defined in spring framework are singleton beans. There is an attribute in bean tag named 'singleton' if specified true then bean becomes singleton and if set to false then the bean becomes a prototype bean. By default it is set to true. So, all the beans in spring framework are by default singleton beans.

```
<beans>
 <bean id="bar" class="com.act.Foo" singleton="false"/>
</beans>
```

**72 . What are the important beans lifecycle methods?**

There are two important bean lifecycle methods. The first one is setup which is called when the bean is loaded in to the container. The second method is the teardown method which is called when the bean is unloaded from the container.

**73 . How can you override beans default lifecycle methods?**

The bean tag has two more important attributes with which you can define your own custom initialization and destroy methods. Here I have shown a small demonstration. Two new methods fooSetup and fooTeardown are to be added to your Foo class.

```
<beans>
 <bean id="bar" class="com.act.Foo" init-method="fooSetup" destroy="fooTeardown"/>
</beans>
```

**74 . What are Inner Beans?**

When wiring beans, if a bean element is embedded to a property tag directly, then that bean is said to the Inner Bean. The drawback of this bean is that it cannot be reused anywhere else.

**75. What are the different types of bean injections?**

There are two types of bean injections.

- By setter
- By constructor

**76. What is Auto wiring?**

You can wire the beans as you wish. But spring framework also does this work for you. It can auto wire the related beans together. All you have to do is just set the autowire attribute of bean tag to an autowire type.

```
<beans>
 <bean id="bar" class="com.act.Foo" Autowire="autowire type"/>
</beans>
```

**77 . What are different types of Autowire types?**

There are four different types by which autowiring can be done.

?byName

?byType

?constructor

?autodetect

**78. What are the different types of events related to Listeners?**

There are a lot of events related to ApplicationContext of spring framework. All the events are subclasses of org.springframework.context.Application-Event. They are

- ContextClosedEvent – This is fired when the context is closed.
- ContextRefreshedEvent – This is fired when the context is initialized or refreshed.
- RequestHandledEvent – This is fired when the web context handles any request.

**79. What is an Aspect?**

An aspect is the cross-cutting functionality that you are implementing. It is the aspect of your application you are modularizing. An example of an aspect is logging. Logging is something that is required throughout an application. However, because applications tend to be broken down into

layers based on functionality, reusing a logging module through inheritance does not make sense. However, you can create a logging aspect and apply it throughout your application using AOP.

#### 80 . What is a Joinpoint?

A joinpoint is a point in the execution of the application where an aspect can be plugged in. This point could be a method being called, an exception being thrown, or even a field being modified. These are the points where your aspect's code can be inserted into the normal flow of your application to add new behavior.

#### 81. What is an Advice?

Advice is the implementation of an aspect. It is something like telling your application of a new behavior. Generally, an advice is inserted into an application at joinpoints.

#### 82. What is a Pointcut?

A pointcut is something that defines at what joinpoints an advice should be applied. Advices can be applied at any joinpoint that is supported by the AOP framework. These Pointcuts allow you to specify where the advice can be applied.

#### 83. What is an Introduction in AOP?

An introduction allows the user to add new methods or attributes to an existing class. This can then be introduced to an existing class without having to change the structure of the class, but give them the new behavior and state.

#### 84 What is a Target?

A target is the class that is being advised. The class can be a third party class or your own class to which you want to add your own custom behavior. By using the concepts of AOP, the target class is free to center on its major concern, unaware to any advice that is being applied.

#### 85 . What is a Proxy?

A proxy is an object that is created after applying advice to a target object. When you think of client objects the target object and the proxy object are the same.

#### 86 .What is meant by Weaving?

The process of applying aspects to a target object to create a new proxy object is called as Weaving. The aspects are woven into the target object at the specified joinpoints.

#### 87 What are the different points where weaving can be applied?

- Compile Time
- Classload Time
- Runtime

#### 88 . What are the different advice types in spring?

- Around : Intercepts the calls to the target method
- Before : This is called before the target method is invoked
- After : This is called after the target method is returned
- Throws : This is called when the target method throws an exception
- Around : `org.aopalliance.intercept.MethodInterceptor`



- Before : org.springframework.aop.BeforeAdvice
- After : org.springframework.aop.AfterReturningAdvice
- Throws : org.springframework.aop.ThrowsAdvice

**89. What are the different types of AutoProxying?**

- BeanNameAutoProxyCreator
- DefaultAdvisorAutoProxyCreator
- Metadata autoproxing

**90. What kind of exceptions those spring DAO classes throw?**

The spring DAO class does not throw any technology related exceptions such as SQLException. They throw exceptions which are subclasses of DataAccessException.

**91. What is DataAccessException?**

DataAccessException is a RuntimeException. This is an Unchecked Exception. The user is not forced to handle these kinds of exceptions.

**92. How can you configure a bean to get DataSource from JNDI?**

```
<bean id="dataSource" class="org.springframework.jndi.JndiObjectFactoryBean">
 <property name="jndiName">
 <value>java:comp/env/jdbc/myDatasource</value>
 </property>
</bean>
```

**93. How can you create a DataSource connection pool?**

```
<bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
 <property name="driver">
 <value>${db.driver}</value>
 </property>
 <property name="url">
 <value>${db.url}</value>
 </property>
 <property name="username">
 <value>${db.username}</value>
 </property>
 <property name="password">
 <value>${db.password}</value>
 </property>
</bean>
```

**94. How JDBC can be used more efficiently in spring framework?**

JDBC can be used more efficiently with the help of a template class provided by spring framework called as JdbcTemplate.

**95. How JdbcTemplate can be used?**

With use of Spring JDBC framework the burden of resource management and error handling is reduced a lot. So it leaves developers to write the statements and queries to get the data to and from the database.

```
JdbcTemplate template = new JdbcTemplate(myDataSource);
```

A simple DAO class looks like this.

```
public class StudentDaoJdbc implements StudentDao {
 private JdbcTemplate jdbcTemplate;

 public void setJdbcTemplate(JdbcTemplate jdbcTemplate) {
 this.jdbcTemplate = jdbcTemplate;
 }
 more..
}
```

The configuration is shown below.

```
<bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">
 <property name="dataSource">
 <ref bean="dataSource"/>
 </property>
</bean>
<bean id="studentDao" class="StudentDaoJdbc">
 <property name="jdbcTemplate">
 <ref bean="jdbcTemplate"/>
 </property>
</bean>
<bean id="courseDao" class="CourseDaoJdbc">
 <property name="jdbcTemplate">
 <ref bean="jdbcTemplate"/>
 </property>
</bean>
```

#### 96. How do you write data to backend in spring using JdbcTemplate?

The JdbcTemplate uses several of these callbacks when writing data to the database. The usefulness you will find in each of these interfaces will vary. There are two simple interfaces. One is PreparedStatementCreator and the other interface is BatchPreparedStatementSetter.

#### 97. Explain about PreparedStatementCreator?

PreparedStatementCreator is one of the most common used interfaces for writing data to database. The interface has one method createPreparedStatement().

```
PreparedStatement createPreparedStatement(Connection conn)
throws SQLException;
```

When this interface is implemented, we should create and return a PreparedStatement from the Connection argument, and the exception handling is automatically taken care off. When this interface is implemented, another interface SqlProvider is also implemented which has a method called getSql() which is used to provide sql Strings to JdbcTemplate.

#### 98. Explain about BatchPreparedStatementSetter?

If the user what to update more than one row at a shot then he can go for BatchPreparedStatementSetter. This interface provides two methods

```
setValues(PreparedStatement ps, int i) throws SQLException;
int getBatchSize();
```

The `getBatchSize()` tells the `JdbcTemplate` class how many statements to create. And this also determines how many times `setValues()` will be called.

### 99. Explain about `RowCallbackHandler` and why it is used?

In order to navigate through the records we generally go for `ResultSet`. But spring provides an interface that handles this entire burden and leaves the user to decide what to do with each row. The interface provided by spring is `RowCallbackHandler`. There is a method `processRow()` which needs to be implemented so that it is applicable for each and every row.

```
void processRow(java.sql.ResultSet rs);
```

### 100. What is JDBC abstraction and DAO module?

Using this module we can keep up the database code clean and simple, and prevent problems that result from a failure to close database resources. A new layer of meaningful exceptions on top of the error messages given by several database servers is bought in this module. In addition, this module uses Spring's AOP module to provide transaction management services for objects in a Spring application.

Flow of execution with the AOP project page: 181 :-

Target class:-

```
public class IntrAmtCalculator {
 public float calcIntrAmt(float pamt, float rate) {
 return (pamt * rate * time) / 100.0;
 }
}
```

AOP aspect class:-

```
@Override
public Object invoke(MethodInvocation invocation) throws Throwable {
 long start = System.currentTimeMillis();
 Object retVal = invocation.proceed();
 long end = System.currentTimeMillis();
 return retVal;
}
```

Generated proxy class:-

```
public class IntrAmtCalculator$Proxy extends IntrAmtCalculator {
 public float calcIntrAmt(float pamt, float rate, float time) {
 MethodInvocation invocation = new MethodInvocation();
 invocation.setTarget("IntrAmtCalculator");
 invocation.setMethod("calcIntrAmt");
 Object args[] = new Object[3];
 args[0] = pamt; args[1] = rate; args[2] = time;
 invocation.setArguments(args);
 }
}
```

